DunesForge Reference

Back to README - New here? Read QUICKSTART.md first - Installing? INSTALL.md

Every control in DunesForge, organised the way the interface is organised, so you can find something by remembering where you saw it.

How to read an entry

> **Control Name** - `technical_id` *Type* - *default* - *range* What it does.

The `technical_id` is what you use in Python and in drivers.

Three things worth knowing before you start looking things up:

1. Almost everything here is off or neutral by default. A control at its default value does not change the simulation - it costs nothing and means nothing until you move it.

2. The interface hides what you do not need. If you cannot find a control described here, switch **Interface** to **Standard** or **Advanced** at the top of the panel. Nothing is ever disabled, only hidden, and hidden controls keep their values.

3. Ranges are guidance, not walls. Where a control has a soft range, you can usually type a value outside it. Whether that is a good idea depends on the control.

Contents

- DunesForge - 85 controls
- Presets - 4 controls
- Wind - 78 controls
- Objects / Masks - 66 controls
- Physics - 69 controls
- Water / Moisture - 131 controls
- Snow - 13 controls
- Vegetation - 42 controls
- Interactive Brush - 23 controls
- Surface Shader - 144 controls
- Surface Detail - 43 controls
- Sand Haze - 15 controls
- Dust Devils - 29 controls
- Crest Backlight - 5 controls
- Multi-Fraction (Megaripples / Pavement) - 13 controls
- Field Maps (Load / Save) - 7 controls
- Export / Bake - 50 controls
- Performance - 21 controls
- Debug / State - 4 controls
- Extra Phenomena - 48 controls
- Demo scenes - 66 scenes

DunesForge

The main panel is the simulation itself: build a terrain, set the wind, press a button, get dunes. If you only ever open one panel, it is this one.

When you need it: Always. Everything else modifies what happens here.

Interface

`ui_mode` - Enum - default **SIMPLE

How much of the interface to draw. NOTHING IS DISABLED, ONLY HIDDEN: a control you cannot see keeps its value and the solver keeps reading it, so switching mode cannot change a single number in the simulation and switching back shows the same scene. Simple is the default. Standard adds scene authoring, masks and export; Advanced adds detailed numerical controls and diagnostics. Important hidden active features are listed above the terrain controls

Option	Meaning
Simple	Start with demos, terrain, wind, basic weather and appearance. Every offered action includes its required inputs
Standard	The working set: physics, objects and masks, water, plants, surface detail, export. Solver internals, diagnostics and experiments stay out of the way
Advanced	Everything, including the pressure solver, the planetary constants, the tracers and the experimental block

Guide Language

`help_language` - Enum - default **PL

Language of the offline QuickStart and feature guide; UI labels stay in English

Option	Meaning
PL	Polski
EN	English

Disable GPU Acceleration

`force_cpu` - Bool - default **off

Run the solver on the CPU even when a CUDA GPU is present. NEEDS A BLENDER RESTART, and that is not a shortcut: the array library is bound at import time in 14 modules and 966 call sites, so nothing can switch it mid-session. The choice is written to a marker file in your home folder, so it survives the restart. THE COST IS LARGE AND MEASURED: 20x slower on a 128 grid (0.37 ms against 7.53 ms per step), 138x at 256, and 1723x at 512 -- 0.56 ms against 965 ms. This is for comparing results or working around a driver problem, not for work. The two paths agree closely but are NOT bit-identical: measured field correlation 0.9999787 and total sand within 0.16%, because the reductions run in a different order

Demo Scene

`demo_scene` - Enum - default **DESERT

Which ready-made demonstration setup Create Demo Scene builds. Each demo is a complete scene: terrain, preset, helper objects, camera, light and materials, pre-simulated so it looks good immediately. Best used in a fresh empty scene

Option	Meaning
Grand Scene: Desert Expedition	A 400 m landscape with dunes, grains, rock shelters, sources, tracks and airborne dust

Option	Meaning
Grand Scene: Living Oasis	A 400 m rain-fed basin combining wet shores, plant species, growth and surface weathering
Grand Scene: Winter Storm	A 400 m snowfield with layered drifts, density, slabs, shelter and snow-fed vortices
Grand Scene: Dynamic Coast	A 400 m coast with Flow 2D wind, shallow water, waves, sediment, rain and vegetation
Desert: Dunes + Obstacles	Classic sand dune field with rock obstacles casting wind shadows, pebble scatter, camera, sun and a sand material. Pre-runs the simulation so dunes are already formed
Snow: Drifts + Fence	Wind-packed snow drifting behind a slatted fence, with a snow material and cool daylight. Shows the snow solver and obstacle wind shadows building lee tails
Oasis: Water + Valley Masks	Sand placed in valleys around a shallow water table with a translucent water plane: wet shores lock in place while dry sand keeps migrating around them
Water: Pools That Flow	The SAME oasis as above, with the water given a depth field instead of one global level -- so every hollow settles to its own height, water runs downhill between them and the shore is wherever the ground says. Built as a direct A/B against the Oasis demo: same terrain, same warmup, one switch different
Water: Wind On The Lake	The flowing field with a 30 m/s wind gripping the surface. The water piles up on the DOWNWIND shore at a slope of exactly tau/(rho g h), so the shoreline moves and the pools lean. MEASURED on this terrain at 30 m/s against no wind, after 3 s of water time: 7699 cells cross between wet and dry -- 4010 flooded and 3689 drained -- and the surface stands 13.7 mm apart at most. The wind here is the ANALYTIC one, so it is the same everywhere; switch Wind Model to Flow 2D and give the flow an Air Layer to have crests and lee pockets push the water differently
Snow: Slab Avalanche	A weak layer buried under a wind slab lets go on a 38 degree face and the debris RUNS OUT onto the flat below. This is the one scene where Slab Dynamics is on: without it the released mass is only ROUTED downhill, which moves it DOWN THE SLOPE rather than out across the flat. MEASURED on this scene, the two models differ by 10.7 % of the released mass: turning Dynamics on shifts 266 units off the upper face and lays them on the runout, concentrated around 75-87 % of the way across. Both models DO reach the valley floor and both stop at the same 95th-percentile distance -- the difference is WHERE the debris piles up, not how far it gets, and in a wide shot it is only 0.12 % of pixels. Look at the runout zone, not at the whole slope. MEASURED, and it is why the scene sculpts its own mountainside rather than using noise: on the stock demo terrain the 95th-percentile slope is 0.103 against a release threshold of tan(34) = 0.675 -- six times too flat, and 220 steps released nothing at all. Try Slab Dynamics 1 -> 0 and watch the pile move UP the runout toward the foot of the face
Snow: Avalanche That Scours Its Path	The same 38 degree face and the same storm as Snow: Slab Avalanche, with Entrainment ON (0.007 m per m of path) and Turbulent Friction set to 400. A real avalanche does not carry only the slab that let go: it scrapes the snow it runs over and grows. MEASURED on this scene, over the same storm: the flow picked up 35 033 units of snow off its path against 9 137 released by the slabs themselves -- 3.8 times the slab -- and 54 342 with the turbulent term on top, because a slower flow lives longer inside its step budget and scrapes further. What you see is the SCOURED TRACKS down the face and a fuller pile at the foot: 5.0 % of the pixels change against Slab Avalanche at the 1 % level, where the two release models of that scene differ by 1.7 %. The turbulent term is here for the SPEED the entrainment law reads, not for the runout: on this 90 m flat it does NOT move the deposit -- 0.432 of the debris ends on the flat at xi 400, 0.432 at 0 and 0.437 at 10, MEASURED -- while the mass-weighted speed drops from 70 to 54 m/s. Try Entrainment 0.007 -> 0 and watch the tracks on the face fill back in

Option	Meaning
Snow: Avalanche Into A Valley	The same 38 degree face and the same storm as Snow: Slab Avalanche, but the flat is SHORT (30 m) and a 22 degree counter-slope rises behind it -- and all four momentum controls (Carry Momentum, Momentum Rides Along, Sharper Transport, Momentum Crosses Lanes) are ON. Momentum is what lets debris CLIMB the far side; with the four at 0 the flow is hydrostatic and stops at the foot. MEASURED on this valley (512 cells, a 0.5 m slab off the face, friction 0.12): with the four on the front reaches the counter-slope after about 7 s, climbs to 174 m along it (22 m up) by about 11 s, then part slides back and 11 % of the debris settles on the counter-slope; with the four off the flow stops on the flat after about 10 s and never touches it. TIME IS THE TRICK: at 1500 substeps the run is cut after about 3 s and both variants deposit mid-flight on the flat with no run-up at all, so this scene runs 12000 substeps with Slab Flow Rest Speed 0.05 m/s. Try Carry Momentum 1 -> 0: the other three go dead with it and the debris stays on the flat
Sand: Slide Under Gravity	No wind. A rock face sculpted to 45 degrees, steeper than the Repose Angle (34), with a 0.6 m strip of sand and two thicker patches painted on it. The repose relaxation runs every step whether or not the wind blows, so the sand slides on its own and stops where its surface reaches 34 degrees. MEASURED on this scene (512 cells): 91 % on the face when the demo opens, 73 % after 20 steps and at rest from 40 steps on at 69.5 %, with 30.5 % at the foot as talus, mass kept to the last digit; with Avalanche Passes 0 nothing moves at all. Try Repose Angle 34 -> 25 (rest at 49 %) or 40 (85 % and still creeping)
Snow: Sluffs Off A Rock Face	The same rock face at 50 degrees, no wind, and a storm falling straight down. Snow landing above the repose limit sluffs to the foot as it falls; with Avalanche Trigger 6 it holds a few degrees past repose and lets go in events. MEASURED: 30-40 % of what fell on the face is still there when the storm ends, the drops come in steps of up to 0.18 points per step against 0.04 with continuous relaxation. Snow Repose Offset does NOT change this -- it belongs to the slab model (measured 0 / +12 / +20 identical). Try Avalanche Trigger 6 -> 0
Snow: Slab Avalanches Held Back By Gates	The Slab Avalanche face with three plates standing across it at 30, 40 and 50 % of the way down; the storm keeps falling while you Play, slabs keep letting go and the debris is ROUTED downhill (Slab Runs Out 0 on purpose: only the routing sees obstacles). The plates are KEYFRAMED to lift at frames 30, 70 and 110; each lift hands what it held to the next gate. MEASURED on a fixture: without a plate 89 % of the debris ends below its line, with it 45.5 % and the upper half stays put; with Slab Runs Out 1 the plate is ignored (85 % passes). Loose snow would not do: a uniform layer does not slide in the local repose rule, it only retreats from the top edge
Water: Reservoir Behind A Sluice Gate	A rock plateau with a 6 m basin, a dam with a 16 m gap and a valley below. The basin holds 1.5 m of water above the sill; a red gate in the gap is an ordinary obstacle that, with Water Sees the Rocks on, is part of the water bed and holds the water back. It is KEYFRAMED to lift 24 m from frame 20: press Play and the reservoir pours down the valley as a shallow-water flood. MEASURED on a fixture of this geometry: without the gate 7.8 % of the water is below the dam after 20 s and 34.8 % after 120 s, with the gate 0.0 % for 120 s, gate removed at 40 s -> the same curve shifted by 40 s; mass 1.0000. Try Water Sees the Rocks off: the flood starts at once
Water: Ripples Carried By The Current	The Pools That Flow basin with a DEAD SWELL pumping it, so the current never settles -- and that is the whole condition of this scene. Flow Ripples makes the fine chop read the FLOW FIELD instead of an independent noise: the pattern stretches along the current and twists where it turns. MEASURED on this scene: straight after the warm-up the basin is in EQUILIBRIUM -- peak speed 0.0 and 0.0 % of cells moving -- so the knob changes 0.000 % of the pixels, because there is no flow to read. With the swell on: peak 36.15, mean 0.218, 70.4 % of cells alive, and the same knob changes 4.69 % of the pixels on the built scene (4.84 % on the gate fixture) against 0.000 % render noise. Try Flow Ripples 2.5 -> 0: the chop goes back to an independent noise that does not know where the water is going. Try Swell Height 0: the basin settles, the current dies, and the knob stops mattering -- which is the same trap the Ice: Pack scene documents

Option	Meaning
Ice: Pack, Ridges and Leads	The ice mechanics of 1.3.134-1.3.138 in one scene: a frozen basin kept ALIVE by a steady inflow, so the pack never settles. The current carries the sheet, piles it against the shore into RIDGES where it converges and pulls it open into LEADS where it diverges -- all three out of one conservative flux, with no extra rule. MEASURED, and this is why the demo exists: on the Pools That Flow terrain the water reaches equilibrium and the ice sampled a LIVE current in only 2 of 20 steps, so none of it could be seen. With the inflow on it is 20 of 20. Try Ice Yield Strength 20 -> 0 and watch the ridge turn into an even thickening
Plants: Two Species Side By Side	A succulent and a grass on the same dune, which is the commonest arrangement on a real one. Drought Tolerance used to be ONE number for the whole scene, so the two could not stand together; above 0 Second Species turns it into a FIELD of patches, deterministic in Seed. MEASURED on the gate fixture, cover of the tolerant plant against the sensitive one: 0.4973 / 0.7078 with one species (a +0.2105 gap that is just the two thresholds) against 0.5087 / 0.7811 with the species map on -- the gap widens to +0.2725 because each plant now sits where it can. The two colours are what makes it readable: Vegetation Colour is the grass, Vegetation Colour B the succulent. Try setting Tolerance (Species B) equal to Drought Tolerance: the map stops mattering and the result goes back to being identical to the bit
Snow: A Thaw That Leaves One Patch	The drift field and the same fence, run through a THAW. Snow lies at Initialize (Initial Snow Depth 0.5), the winter builds the drifts, and then the air goes to +8 C with Melt Rate on -- and the field loses ALL of it. MEASURED over 30 steps: mass of the fresh layer 20 584 with no thaw against 0 with it, cover 98.2 % against 0.0 %. One patch survives, because a snow EMITTER keeps feeding it: Snow Source Rate 0.05 over a box, feathered by Snow Source Smoothing, holds 2 842 of mass and 4.9 % cover when everything around it is bare. MEASURED and the reason this scene keeps the DunesForge surface shader instead of the flat snow material the other snow demos wear: on flat white paint the same thaw changed frame brightness from 0.7232 to 0.7200 and the renders looked identically white -- only the drift relief moved. With the snow mask driving the surface, the thaw repaints 57.1 % of the pixels and the emitter patch another 4.4 %. Try Melt Rate 0.01 -> 0 with the air still warm: melting is Melt Rate PLUS the temperature term, so the warm air alone still takes it, just more slowly. Try Snow Source Rate 0.05 -> 0: the last white patch goes with it
Another Planet: Mars	The same ORGANIC dune field as the other scenes with ONE switch different: Other Planet, set to Mars. Martian air is sixty times thinner than ours (0.020 against 1.20 kg/m3) and the saturation length divides by air density, so the same physics builds dunes far larger on the same ground. Turn Other Planet off and the scene rebuilds itself at Earth scale, which is the whole point of the comparison; scale the terrain object up if the Martian dunes outgrow the default 200 m domain
Snow: Fresh, Packed and Drifted	The same fence and the same wind as Snow: Drifts + Fence, with Snow Density switched ON -- so the pack stops being a depth and starts being a MASS. A depth-only solver cannot tell 10 cm of powder from 10 cm of settled snow; MEASURED on the gate fixture those are 10.00 and 50.00 kg/m2, five times apart at the same depth, and across this field the mass runs 0.37 to 17.42 kg/m2. Drifted snow is denser than the snow that fell, because the grains shatter in saltation and pack tighter, so the drift behind the fence settles SHALLOWER and HARDER than the fresh fall around it -- measured mean hardness 0.1204 at Drifted Density 350 against 0.8125 at 100. Try Drifted Snow Density 350 -> 100: the mechanism switches itself off bit for bit and the pack deepens again
Rock + Sand Source	Start Terrain = Rock workflow: a rocky FBM terrain frozen as bedrock, with sand fed only from an upwind source zone so drifts grow from nothing over bare rock
Rain: Wet Sand + Plants	Storm cells sweep a formed dune field: the dunes are built DRY first, then the rain arrives, so you can see what it changes. Damp patches go dark and matte and stop moving, the lee of the rocks stays dry in the rain shadow, and vegetation greens up where the root zone holds water and then pins the sand down
Rain: Monsoon Shutdown	The heavy end of the rain model: near-total coverage, slow cells and strong infiltration, so almost the whole field goes wet and almost all of it stops moving. Shows what OUTLIVES the rain -- a drying crust plus the vegetation it fed, which keep holding the sand after the weather has gone

Option	Meaning
Rain: Wet Oasis	Water table AND rain at the same time: two independent sources feeding one wetness mask, each with its own hold strength. Plants ring the standing water where the root zone never dries out. Good for checking that the two systems combine rather than fight
Rain: After The Storm	The field is soaked and then the rain STOPS. Wind Drying makes exposed crests dry first while sheltered lee slopes stay damp, so the dunes wake up unevenly instead of all at once -- the clearest demonstration of the wind/moisture coupling
Rain: Drops On Water	A close-up of the rain IMPACT layer, framed on a shoreline so you can see both behaviours at once: rings spreading where a drop lands in standing water, and splash craters with dark wet spots where it lands on the shore. The camera sits low and near on purpose -- an impact is about 5 cm across, so from the usual demo distance every one of them would be a single pixel
Terrain: Mountain Range	Ridged fractal peaks with sand collecting in the valleys -- the classic case where the rock is the scenery and the dunes are what moves through it
Terrain: Canyon	A terraced, deeply-cut canyon frozen as bedrock with sand fed along the floor, so drifts snake between the walls and pile against them
Terrain: Cellular Mesas	Turbulence terracing gives flat-topped mesas with steep sides; sand streams around them and builds lee tails behind each
Terrain: Ready Dune Field	Wind-aligned ridges as the starting field, so a mature dune sea exists from frame 0 instead of forming over hundreds of steps
Terrain: Beach + Buildings	A shallow shore with a water table, a row of buildings behind the dune line, and sand drifting up against them
Terrain: Buried Town	A block of buildings on an open plain being swallowed by sand: each one casts a wind shadow and grows its own lee drift
Terrain: Crater Basin	A steep-walled basin that traps everything blown into it -- a sand trap, and a good stress test for avalanching on rock
Terrain: Rolling Hills	Soft FBM hills with a full sand cover: gentle relief where the dunes and the ground shape each other
Terrain: Stepped Plateau	Hard terraces frozen as bedrock, with sand only on the steps -- shows the non-erodible rock path and how sand cascades between levels
Terrain: Islands + Water	Scattered high ground in a shallow sea: wet locked shores, dry migrating crests, and rain to tie the two together
Showcase: Object Masks	Three object slots doing three different jobs in one scene: a source box feeding fresh sand, a wind-gain box acting as a gust lane, and rock obstacles casting wind shadows. Every one of these masks can also be painted with the Interactive Brush
Showcase: What Objects Tell The Sand	Three slabs on one moving sand sheet, each holding the ground to a different rule, with every number below measured on the scene itself. BEDROCK is a floor the sand cannot sink below; this one sits above the sheet, so it stands as a rock sill the sand flows around, 1.43 m proud of the field. ERODE is the quiet one: the ground there stops moving altogether, roughness 0.0025 against 0.30 for the sheet beside it, so dunes travel past a patch that never changes. NO-DEPOSIT is the one people mix up with it: nothing new lands, while whatever sand is there is carried off as usual, so the footprint SINKS -- 0.12 m after the warm-up and 0.38 m after 300 steps more. Erode holding still while No-Deposit digs itself out is the whole point
Showcase: Grain Systems	The grain-scale models together: Multi-Fraction winnowing fines out into an armouring coarse-lag pavement, Strata painting cross-bedding on cut faces, Sand Age reading fresh on scoured slip faces, and the physical ripple solver at megaripple scale. Ships with the Surface Shader so all of it is visible
Phenomena: Playa Crust	A dried lake bed: the crust sets and cracks into plates, and the wind deflates along the cracks while the plates stand
Phenomena: Wind Erases Tracks	A rut imprinted across the field, fading faster where the wind is strongest and surviving in the lee

Option	Meaning
Vegetation: Nabkha Hummocks	Scattered plant clumps on a sand-fed plain, each one trapping a mound with a downwind tail -- the nabkha (coppice dune) landform. Built with the patch-shape controls open, so the clumps are ragged, come in a range of sizes and grow in stands instead of being identical circles on a uniform scatter, and with the clumps taking their own darker colour so a hummock reads as a bush and not as a green stain
Vegetation: Rain-Fed (nothing grows on its own)	The only scene where NOTHING grows by itself: Veg Growth is set to zero, so every plant you see was put there by rain. Growth From Rain is a multiplier and cannot help at zero growth, so Rain Seeds Growth -- which adds its own rate from the root zone -- is the only term that can raise a single stem, which makes the scene its own control. Spread sits at 1.0 so growth happens only where plants already are: clumps thicken and creep at their edges while two thirds of the ground stays bare, and that bare sand is what lets you see the second half of the loop, Plants Shade The Ground, holding moisture under the cover that the rain grew
Plants: Valleys Only (Slope and Height Windows)	The blowout field again, with the vegetation kept OUT of the top 40 % of the relief by Height Max 0.6, Limits Apply to Seeding ON and Edge Noise 0.5. The window is what keeps a real field from greening over: plants hold the hollows and the crests stay bare sand. MEASURED on this scene after the 150 step warm-up, cover on the top third of the relief: 0.992 with no window against 0.040 with it, and 0.000 at frame 0 because Limits Apply to Seeding never lays it there; without that switch seeding covers the crests first and burial has to clear them. The relief is 8.0 here, ten times the Blowouts scene, MEASURED: at 0.8 the dunes rebuild the height field during the warm-up, the crests at the end are not the crests the seeding obeyed, and the window keeps only 0.676 of them bare (2.0 -> 0.292, 4.0 -> 0.098, 8.0 -> 0.040). Burial is 0.5, not 2.0, because at 2.0 the crests are bare with or without a window and there would be nothing to show. MEASURED and fixed in this version: Edge Noise used to jitter the OPEN limits too, so a height-only window refused 45.3 % of a perfectly flat floor; it now frays only the limit you set. Try Invert Limits: the crests green and the hollows go bare. Try Edge Noise 0.5 -> 0 and the tree-line turns into a contour
Vegetation: Blowout Hollows	The opposite starting point: a fully vegetated coastal grassland with bare gaps punched in it. The wind scours the unprotected gaps into deflation hollows while the anchored surroundings hold, which is how a blowout -- and eventually a parabolic dune -- gets going
Vegetation: Green Retreats	A field that starts GREEN EVERYWHERE and loses it: drought dieback takes the cover off the dry ridges while the plants ringing the water hold, so the green pulls back to the shoreline instead of simply fading. Built the way round it is for a measured reason -- with the usual near-bare start the same physics moves 41% of the dry-zone cover and changes 1% of the rendered pixels, because 0.068 and 0.040 both read as bare sand. Starting green makes the loss an AREA that shrinks. MEASURED on this scene after 90 steps against the same scene with Drought Dieback at 0, counting only VISIBLE LAND (ground above the waterline -- cover under an opaque water surface is not something you can see): the strongly green share of the land falls from 78.4% to 11.0%, and mean cover on land from 0.549 to 0.165. The pool itself is half a percent of the frame while the wetness reaches 77% of it, so the green ring sits on LAND rather than under water. Turn Drought Dieback to 0 and the green stays put
Temperature: Cold Front	One cloud over a warming scene: the same storm lays snow while the air is below zero and wets the ground once it is above
Temperature: Spring Thaw	A snowfield under a rising sun -- melt on degree-days, so it stays put until the air crosses zero and then goes quickly
Rain on Snow: Crust	Warm rain falling on fresh snow: it takes depth off the pack and leaves what is left bonded into a crust
Rain on Snow: Loaded Slab	The same crust with fresh snow dumped on top of it -- the layered setup a slab releases over
Freeze: Locked Shoreline	Wet sand along a water table, frozen solid: the damp band stops eroding entirely while the dry dunes behind it keep migrating
Freeze: Thawed Shoreline	The same shoreline above freezing -- the control that shows the lock is the ice and not the water
Frost: Cold Dawn	Surface frost whitening the damp hollows while the dry crests stay bare sand

Option	Meaning
Frost: Hoar on Snow	A cold, sheltered snowfield growing surface hoar -- the weak layer a later slab will release over
Flow 2D: Canyon Jet	Two rock walls with a gap. The solved wind speeds up through the gap because the same air has to pass a narrower opening -- a result of mass conservation, not of a correction term
Flow 2D: Recirculating Wake	A single bluff rock. Behind it the flow REVERSES and turns back on itself, which the analytic wind cannot do at all -- it has a lee shadow but no momentum
Flow 2D: Wind Off a Wall	An angled barrier that turns the wind sideways. Tracers and the direction overlay are on, so the bounce is visible as motion and as colour
Phenomena: Snow Fence	A POROUS blocker: the long downwind drift a real snow fence builds, next to the short steep pile a solid wall makes
Phenomena: Snow Layer Stack	A season of storms as dated layers, with a buried weak layer that decides where a slab releases
Phenomena: Sublimation + Hoar	Snow leaving as vapour on the exposed crests while surface hoar grows in the sheltered hollows
Phenomena: Tidal Coast	A moving shoreline: the damp band sweeps up and down the beach and the dunes behind it advance in steps
Showcase: Surface Detail	The layer that lives ABOVE the dune, shot from close enough to see it -- from the usual demo distance it is smaller than a pixel. The Micro-Detail pass has been run once already, freezing the dunes as bedrock and running transport again at a finer scale over them, with Crispness, Crests and Slump Allowance set; those three do nothing until the pass is RUN, so press the button again after changing them. De-facet Crests takes out the straight grid zigzag that a one-direction wind leaves along the crest lines, Edge Rounding takes the corner off the brink but not off the flats, and Detail Feeds Sim hands a little of the detail back to the solver so the ripples are sand rather than a decal the wind ignores
Showcase: Surface Shader	How the surface texture is PROJECTED, rather than what it is. A flat top-down projection stretches the grain down every slope and a dune is mostly slope, so this scene switches the scanned sand and the bedrock onto a triplanar projection and turns up the four map-side controls: the scan follows the wind heading and the local downslope, the procedural/scan balance becomes a map instead of one number, and the terrain sits in its own ambient shade. Note the trade it has to make: triplanar and Hex-Tiling cannot both run on the sand, so the scene uses Texture Bombing -- put Hex back and the triplanar tick silently stops doing anything
Showcase: Airborne + Dressing	Everything that is not the heightfield: swept sand-haze particles emitted off exposed crests and settling in the lee, wandering dust devils, a curve imprinted as a rut with berms, and pebble scatter driven by the simulation masks

Preset

`preset` - Enum - default **ORGANIC

A complete starting point: physics, wind, look and surface detail together. Applying one RESETS the shared knobs so presets cannot leak into each other. The three named REPLACES TERRAIN also replace a terrain you loaded; every other preset leaves it alone

Option	Meaning
Organic Field	Soft varied barchanoid field
Big Rolling	Larger smoother forms
Fine Rippled	Smaller sharper dunes
Calm / Smooth	Low wind, polished smooth field
Coastal Source	Source-fed field with stronger wind
Star Dunes	Three-way wind regime

Option	Meaning
Linear / Seif	Alternating oblique winds. NOTE: this preset sets the wind regime but NOT 3D Wind, which is what actually aligns the ridges -- it ships EXPERIMENTAL and off for production. MEASURED at 512 over 1000 steps: without it the ridge angle lands 36.6 deg off the bisector and anisotropy DECAYS to 0.143; with it the angle converges to 0.8 deg and anisotropy RISES to 0.576. Turn 3D Wind on for true seifs, and expect about 6.6x the step cost
Reversing Ridges	Wind that flips 180 degrees. The net transport cancels, so the ridge does not travel -- it grows in place, with a slip face on BOTH sides. The only regime the classifier knows that no preset reached before
Barchans Open	Open boundary with low sand supply
Sharp Ridges	Large coherent sharp-crested ridges over bare interdune plateaus - the sculpted terrain-tool look (steady wind, low supply, strong settle, crisp lee)
Snow / Sastrugi	Cohesive wind-carved snow surface
Snow Blizzard	Active snowfall with strong wind
Snow: Thaw	Melting, softened snow forms
Snow: Drifts	Hard wind-packed slabs and lee tails
Parabolic Vegetated	Vegetation-stabilized parabolic dunes
Sand + Avalanches	Metastable lee faces with episodic sand slides
Snow + Avalanches	Snow slabs with cohesive avalanches
Snow over Sand (mixed)	BOTH materials in one scene, at TWO different angles. The sand settles at its own 34 degrees while the snow slab on top of it holds to about 45 before it lets go, which is what Snow Repose Offset exists for -- until it did, one angle had to serve both. Snowfall is light and the crust is hard, so the cover is patchy and there is a real snowline for things to walk across
MASK: Sand in Valleys	Combined placement: normalized low height band
MASK: Hollows + Patches	Combined placement: hollows with organic patches
MASK: Scattered Patches	Combined placement: isolated sand patches
MASK: Flats Only	Combined placement: sand on flat slopes
MASK: Plateau Top	Combined placement: sand on high plateau zones
SNOW v2: Powder Blizzard	Two-layer powder snow with active haze
SNOW v2: Slab Avalanches	Hard slabs with episodic releases. Since 1.3.35 it also arms the SLAB AVALANCHE on a buried weak layer -- the layer stack, the snowfall that fills it and the release criterion, because the switch alone reads a weak-layer hardness of 1.0 and a slab depth of 0.0 and does nothing at all. IT NEEDS A SLOPE TO RELEASE ON, and that is the one thing the preset cannot supply: the driving stress is depth times sin(slope) times cos(slope), and this preset leaves your terrain alone by design. On the flat RANDOM starting terrain the measured slope is 0.02-0.10 against a repose tangent of 0.675, so nothing will let go until there is real relief under it
SNOW v2: Icy Wind Crust	Hard wind crust and carved sastrugi
REPLACES TERRAIN: FBM	Generates an FBM starting terrain. This preset OVERWRITES a loaded heightmap or a sculpted mesh -- that is what it is for. The other 24 presets leave your terrain alone (measured)
REPLACES TERRAIN: Ridged	Generates a ridged starting terrain. OVERWRITES a loaded heightmap or a sculpted mesh -- that is what it is for
REPLACES TERRAIN: Dune Ridges	Generates wind-ridge starting terrain. OVERWRITES a loaded heightmap or a sculpted mesh -- that is what it is for

Terrain

`terrain_object` - Pointer

The grid mesh the simulation runs on -- created by Create Terrain, or pick your own dense grid. Its world size and position define the sim domain (scale the OBJECT to resize the desert); the solver rewrites its Z heights every update

Resolution

`solve_res` - Int - default **128** - 32 to 8192

Grid vertex resolution; changing this preserves existing terrain size. COST grows with the square: 2048 is heavy, 4096 needs a strong CUDA GPU (enable Fast 32-bit) and builds a 16M-vertex terrain mesh, 8192 is extreme (67M vertices, several GB of RAM/VRAM, slow viewport -- best driven via Bake rather than live playback)

Render Subdivision

`render_subdiv` - Int - default **1** - 1 to 4

Bicubically upsample the solver grid to a finer render mesh (Mitchell-Netravali). 1 = render at solver resolution; 2-4 = smoother high-detail render from a coarser/faster solver. Rebuilds the terrain mesh to solver_res x subdiv (heavy at high resolution)

Res-Independent Dune Size

`dune_res_independent` - Bool - default **off

Keep the dune SIZE the same when you change Solver Resolution, so the same scene reads identically at 256, 512, 1024 or 2048 and you can block out fast at low res, then raise it for the final. ON: Dune Size (Ls) is rescaled with the resolution -- it is a world length, so it has to shrink as cells get smaller to keep the dune the same number of cells wide -- and the render-detail ripple lengths (Ripple Length + physical ripple wavelength), which are measured in CELLS, are scaled the opposite way so ripples keep their world size instead of shrinking to sub-pixel at high res. Anchored at 1024, so scenes tuned there are unchanged. Presets switch this ON -- leave it on unless you want the raw behaviour. OFF: every length stays as typed, so a lower resolution gives much SMALLER dunes (only a few cells wide, which reads as noise and spikes) and a higher one gives larger ones

Tile Grid

`tile_grid_n` - Int - default **3** - 2 to 8

Size N of the NxN tiled preview built by the Tile button: instanced copies of the terrain arranged in a grid so you can check that the dune field repeats seamlessly (the sim is periodic by default, so it tiles). Purely a viewport check -- no effect on the simulation. Pointless with Open X or Edge Clamp, where the edges do not wrap

Dune Scale

`dune_scale` - Float - default **25** - 0.01 to 1000

Overall dune size on the terrain: scales the dune wavelength (Ls) up or down. 50 leaves Ls exactly as the preset sets it; the shipped default of 25 therefore HALVES it, which is the calibrated starting point, not a neutral one. LOWER = smaller and MORE dunes, higher = bigger and fewer. The terrain keeps its own size -- scale the terrain object in Blender to resize it. CAUTION: very low values combined with Sand Supply near 1 and deep sand pile unnaturally steep peaks (deposition outruns the slope relaxation) -- keep it 30+ when the sand budget is high. The slider stops at 0.1 and 300, which is where the model is calibrated; type a value or ctrl-drag to go out to 0.01 / 1000 for stylised work. Below about 1 the wavelength closes on the cell size and what you get is grid-scale ripple, not dunes; above about 300 a single dune is wider than most terrains

Height Scale

`height_scale` - Float - default **1** - 0.1 to 50

Vertical DISPLAY scale: solver height units are multiplied by this when written to the mesh / displacement preview (the physics itself is unaffected). THE SCALE RULE: dunes grow to about 1.5-2.5 solver units, so their world height is roughly 2 x this value in meters -- pick it as desired dune height / 2. Note the angle of repose lives in SOLVER units: sand cannot rest on world slopes steeper than atan(HS x tan 34deg), so tiny values also flatten every sandy slope. For imported terrain prefer ~1 and set Image Amplitude to the real height in meters. Default 1 = physically TRUE angles (slip faces render at the real 34 deg repose) and ~2 m dunes on the generated 400 m playground

Seed

`seed` - Int - default **42** - 0 to 99999

Random seed for everything procedural: the start terrain generators, the supply-variation field, haze/dust particles and scatter stamps. Same seed = identical run. Applied at Initialize (change it, then Reset + Initialize to get a different landscape)

Initial Heights

`init_mode` - Enum - default **RANDOM

What the terrain looks like BEFORE the first step. Frame-0 only: these reshape the mesh live while you drag them, but once Initialize has run you have to Reset to re-seed. Current Mesh keeps a terrain you sculpted or imported

Option	Meaning
Natural Random Sand	Resolution-stable fBm seed matching the original natural dune start
White Random Legacy	Old per-cell random seed; high resolutions create many tiny proto-dunes
Current Mesh	Use the terrain mesh Z heights as the starting field
Flat	Start from a flat heightfield
Simplex FBM	Generate a layered simplex-noise terrain
Fractal FBM	Generate a layered value-noise terrain
Ridged FBM	Generate ridged mountain-like terrain
Turbulence	Generate folded absolute-value noise
Dune Ridges	Generate wind-aligned procedural ridges

Heightmap File

`terrain_image_path` - String - default

Bitmap to load as the START terrain: PNG / JPG / EXR / TIF (any format Blender can open; luminance is used) or a TXT/PGM heightfield. It is stretched to fit the whole terrain grid. Before Initialize it writes onto the mesh and switches Initial Heights to "Current Mesh"; after Initialize it REPLACES the live sim heights and the sim continues from the image

Edge Clamp - open ALL edges (non-tiling)

`edge_clamp` - Bool - default **off

THIS IS THE ONE FOR A LOADED HEIGHTMAP, and it OVERRIDES Boundary Mode -- with it on, Boundary Mode does nothing at all. Measured on a terrain whose opposite edges disagree, movement at the border after 20 steps: Periodic 6.13, Open X 4.73, this 1.96 -- and the far edge itself 4.97 down to 0.13. Treat the terrain as NON-TILING: clamp ALL edges so the wind shadow, sand transport AND avalanches never wrap across the seam (no false cliff or shadow bleeding from the opposite edge). Sand blows in at the upwind edge (Input Flux) and off the downwind edge. Leave OFF for a seamless tileable result; turn ON for a bounded real-world heightmap. Note: on CPU/EXACT this uses the open-boundary transport (a touch less saturated at the very edges) instead of the periodic fast scan

Image Amplitude

`terrain_image_amp` - Float - default **5** - min 0

Height range of the loaded bitmap in SOLVER units: image black maps to 0, white maps to this value. The slider stops at 500, but you can type any value above that. THE IMPORT RULE: amplitude = real terrain height in meters / Height Scale (the panel shows the resulting dune height live). Avoid a huge amplitude compensated by a tiny Height Scale -- the repose physics reads SOLVER units, so those slopes exceed the angle of repose and sand slides off them (no dunes on the flanks). Set Solver Resolution BEFORE loading (the mesh is rebuilt to it)

Image Offset

`terrain_image_offset` - Float - default **0

Constant height added AFTER the amplitude mapping (solver units). Use it to raise the whole imported terrain, e.g. to keep it above Water Level or to add a sand reserve above bedrock

De-band 8-bit Heightmap

`import_deband` - Bool - default **off

Remove CONTOUR TERRACING from a low-bit-depth (8-bit PNG) heightmap on load: an 8-bit image has only ~256 height levels, so smooth slopes import as flat stair-steps (visible as lines following the contours). This smooths the steps away while KEEPING real features (ridges, crater rims) sharp -- it is feature-preserving and clamped to one quantum. Auto no-op on a smooth 16-bit image or EXR, so it is safe to leave on. Best fix of all: export the heightmap as 16-bit or EXR

Remove Spikes

`import_despike` - Float - default **6** - 0 to 50

Replace isolated bad pixels in an imported heightmap with their local median. A scanned or generated map can carry single hot pixels that import as needles metres tall -- measured on a real 8192 map: 103 cells standing up to 50.5 on a terrain whose whole range was 51.5. A MEDIAN is used, not a blur, so a genuine ridge (a whole LINE of high cells, and therefore its own median) passes through untouched while a lone pixel is replaced. The number is how many times the local spread a cell must exceed before it counts as bad: lower catches more, 0 turns the filter off

Noise Amp

`init_noise_amplitude` - Float - default **0.5** - 0 to 100

Height range of the generated START terrain in solver units (with Normalize on, the terrain spans 0..this value). Frame-0 only -- it live-previews on the mesh before Initialize and is baked in at Initialize. Around 5 gives dune-scale relief on the default 50 m terrain; large values make mountains the sand then has to climb

Noise Offset

`init_noise_offset` - Float - default **0** - -100 to 100

Constant height added to the generated start terrain (solver units), applied after Noise Amp. Use it to lift the whole landscape above Water Level or to sink it for lake-filled hollows. Frame-0 only, previewed live before Initialize

Noise Scale

`init_noise_scale` - Float - default **4** - 0.05 to 128

How many large noise features fit across the terrain (frequency): low values = a few broad hills, high values = many small bumps. The single biggest "shape" control of the generated start terrain. Frame-0 only, previews live before Initialize

Octaves

`init_noise_octaves` - Int - default **5** - 1 to 12

Number of fractal detail layers stacked on the base noise: 1-2 = smooth rolling forms, 5+ = rich rugged detail down to small scales. More octaves cost a little more at Initialize only. Frame-0 generator setting, previews live

Lacunarity

`init_noise_lacunarity` - Float - default **2** - 1.01 to 8

How much finer each successive octave gets (frequency multiplier per layer). 2 = classic natural fractal; higher values leave a visible gap between the big forms and the fine grain (sparser, more "designed" look). Frame-0 generator setting

Gain

`init_noise_gain` - Float - default **0.5** - 0.01 to 1

How strongly each finer octave contributes (amplitude multiplier per layer): low = smooth terrain dominated by the big forms, high (toward 1) = rough, noisy terrain where the fine detail is as tall as the hills. Frame-0 generator setting

Domain Warp

`init_noise_warp` - Float - default **0** - 0 to 8

Domain warp: bend the noise coordinates before sampling so the generated terrain gets swirled, flow-like, less "computer noise" shapes. 0 = off; 1-3 = organic drift; high values can look marbled. Frame-0 generator setting, previews live

Ridge Sharpness

`init_noise_ridge_sharpness` - Float - default **1** - 0.25 to 8

Crest sharpening for the RIDGED / DUNE RIDGES start modes: low = soft rounded crests, high = knife-edge mountain ridges with wide gentle flanks. Ignored by the other generator modes. Frame-0 generator setting

Terraces

`init_noise_terraces` - Int - default **0** - 0 to 64

Slice the generated start terrain into N flat terrace levels (stepped mesa / canyon look); 0 = off. The steps are start geometry only -- the sim will soften them over time unless the terrain is rock (Start Terrain = Rock keeps the mesas crisp while sand drifts around them)

Normalize

`init_noise_normalize` - Bool - default **on

Stretch the generated terrain to exactly the 0..Noise Amp range before Noise Offset is added -- predictable height no matter the octaves/gain settings (recommended). OFF = raw noise range, so the actual relief varies with the other settings and between modes

Random Start Height

`supply` - Float - default **0.5** - 0 to 1

Height amplitude of Natural Random Sand and White Random Legacy at initialization, in solver height units. Other start modes use their own heights. This is not a coverage percentage or an ongoing sand source. Use Initial Sand Depth for an added layer, Sand Placement for coverage, and Source Rate to add sand during simulation

Terrain = Sand

`terrain_as_sand` - Bool - default **off

Treat the imported / sculpted start terrain itself as LOOSE SAND the wind can carve away, instead of solid bedrock that only collects sand on top. ON: hills deflate, migrate and can vanish entirely. OFF: the landscape keeps its shape and only the added sand moves. Opposite intent of Start Terrain = Rock; applied at Initialize

Start Terrain = Rock

`start_terrain_rock` - Bool - default **off

Treat the WHOLE starting terrain as solid ROCK (bedrock), no matter how it was made -- noise generators, a loaded heightmap or the current mesh. The wind then cannot erode the landscape itself: sand exists ONLY where you add it (Initial Sand Depth, Sand Source objects, Influx, or painting). Perfect for "bare rocky terrain that slowly collects drifting sand" setups. OFF (default) keeps the classic behaviour: a generated start terrain IS loose sand ready to migrate

Initial Sand Depth

`initial_sand_depth` - Float - default **0** - 0 to 100

Even blanket of loose sand (solver height units) laid over the whole start terrain at Initialize -- the sand reserve the wind can immediately sculpt into dunes. Dunes need about 2 units of sand to fully form: below ~1 on Start Terrain = Rock you get drifting SAND SHEETS and deflation patches on bare rock (a real desert look, not a bug); 2.5-4 gives a fully dune-covered field. Deep sand + Supply near 1 + a small Dune Scale can pile unnaturally steep peaks -- lower one of the three. Frame-0 only

Starting Sand from a Map

`use_sand_map` - Bool - default **off

Start from a PAINTED or IMPORTED sand map instead of one Initial Sand Depth everywhere. Load a field of barchans, a scan, or something you painted, and the simulation begins with that sand already on the ground. IT IS READ AS A THICKNESS ABOVE THE FLOOR, not as terrain, and that is the point: added to the HEIGHT those dunes would be ROCK and would never move. As sand they start being reworked on the first step, exactly like generated ones. It replaces Initial Sand Depth while it is on

Sand Map Depth (m)

`sand_map_depth` - Float - default **2** - 0 to 1000

What WHITE in the map means, in metres of sand. The image is a 0..1 ratio and this is its scale. Explicit on purpose: reading the image as absolute heights would silently change what the map means the moment you touched Height Scale

Fit

`sand_map_fit` - Enum - default **STRETCH

How the image is laid onto a terrain of a different shape

Option	Meaning
Stretch	Fill the terrain with the whole image, distorting the aspect if they differ. Right when the map was made FOR this terrain
Fit (keep aspect)	Crop the image to a square before sampling, so round dunes stay round. A 2:1 map stretched onto a square grid turns barchans into ellipses, which is a different landform, not a fit
Tile	Repeat the image across the terrain at its own pixel scale. For a small tileable patch covering a large field, where stretching would smear the dunes into stripes

Offset (m)

`sand_map_offset` - Float - default **0** - -100 to 100

Raise or lower the WHOLE map, in metres, after it is scaled. Positive puts a sheet of sand everywhere including the black areas; negative eats into the thin parts first and is the quick way to turn a full map into scattered patches -- the result is clamped at zero, so the lowest ground simply goes bare rather than digging into the rock

Contrast (gamma)

`sand_map_gamma` - Float - default **1** - 0.05 to 8

Reshape the 0..1 image before it becomes metres. Below 1 lifts the mid-tones -- thin sand spreads and the dunes read as a fuller sheet; above 1 pushes them down, so only the brightest parts keep their depth and the map breaks into separate dunes with bare ground between them. 1.0 is the image exactly as painted

Invert

`sand_map_invert` - Bool - default **off

Read black as full depth and white as bare ground. For maps painted the other way round, and for using a mask of where sand may NOT be as one of where it may

Flip X

`sand_map_flip_x` - Bool - default **off

Mirror the map left-right before sampling

Flip Y

`sand_map_flip_y` - Bool - default **off

Mirror the map top-bottom. Worth trying first if an imported map looks upside down: image rows run the opposite way to grid rows in several tools

Rotate

`sand_map_rotate` - Enum - default **0

Turn the map in quarter steps, so a map made for a different wind direction can be lined up without repainting it

Option	Meaning
0 deg	No rotation
90 deg	Quarter turn clockwise
180 deg	Half turn
270 deg	Quarter turn anticlockwise

Fine Rotate

`sand_map_rotate_fine` - Float - default **0** - -6.28319 to 6.28319

Turn the map by ANY angle, on top of the quarter turns above. Positive turns clockwise, the same way the quarter steps do. The canvas does not grow, so the corners rotate out of frame and bare ground (no sand) rotates in -- growing it would change the SIZE of the dunes as well as their heading. At 0 the map is untouched, bit for bit

Edge Feather

`sand_map_feather` - Float - default **0** - 0 to 0.5

Fade the map to nothing towards the edge of the domain, as a fraction of the terrain width. An imported patch otherwise ends in a wall of sand at the border, which the first step then avalanches into a ring you did not paint. 0.1 is usually enough to hide the join

Combine

`sand_map_blend` - Enum - default **REPLACE

How the map combines with the sand the terrain already has. Every mode works on THICKNESS ABOVE THE FLOOR, never on the terrain height -- sand added to the height would be rock. Modes that read the existing sand need a bedrock to measure it from, which means Start From = Mesh with Terrain Is Sand off

Option	Meaning
Replace	The map IS the sand layer: whatever sand the terrain already had is replaced by what the map says. The right one when you exported the sand, painted on it and brought it back, because it is the only mode where painting DOWN removes sand
Max (keep the deeper)	Keep whichever is deeper, the existing sand or the map. What every build before this one did. Painting sand AWAY has no effect in this mode -- the existing depth wins
Add	Pile the map on top of the sand already there. For adding a field of dunes to a bed that already has a layer
Subtract	Take the map OUT of the existing sand. A map of where sand should not be, used as a scoop
Multiply	Modulate the existing sand by the image read as 0..1: white keeps full depth, black scrapes to the rock, grey thins. Needs sand to already be there -- it multiplies, it cannot create
Blend	Mix existing sand and the map by Blend Amount. For easing an imported field into a generated one instead of cutting between them
Difference	The absolute difference between the two. Not a landform tool -- it is for SEEING where a map and the current sand disagree

Blend Amount

`sand_map_mix` - Float - default **0.5** - 0 to 1

How far towards the map the Blend mode goes. 0 keeps the existing sand, 1 is the map alone (the same as Replace)

Preview As

`sand_map_preview_mode` - Enum - default **MESH

Two ways to look at the map before committing to it

Option	Meaning
Shape (separate object)	A throwaway duplicate of the terrain with the sand displaced into it. Shows the actual landform. It is a SEPARATE object on purpose: writing it onto the terrain would make Initialize add the sand a second time
Overlay on the terrain	No new mesh. Paints the map onto the REAL terrain as a colour attribute, so you can see where the sand will land against the features you already have. Nothing about the simulation changes, and Clear Preview takes it off again

Paint

`brush_field` - Enum - default **SAND

Which solver field the brush edits

Option	Meaning
Sand Height	Sculpt the sand surface up/down
Bedrock	Raise/lower the solid non-erodible floor
Erodibility	Paint how easily sand erodes (0 = protected, 1 = free)
Non-Erodible Rock	Paint bare rock that never erodes but can still be buried (paint toward 1 = rock)
Wind Gain	Paint local wind strength (gust / lull zones)
Vegetation	Paint vegetation cover that anchors the sand
Snow Depth	Paint the depth of FRESH loose snow, in metres, exactly as Sand Height paints sand. Painting it arms the Snow model, and before Initialize it is staged on the mesh and loaded automatically. This is the powder that drifts and can be blown off crests; for the wind-packed crust use Snow Crust
Snow Crust	Paint how HARD the snow is, 0 = loose powder, 1 = wind slab or glaze ice. Separate from Snow Depth on purpose: the crust is what decides whether a slab releases and which end of the snow colour ramp the surface takes, so a sastrugi field can be tuned without also moving the fresh powder
Slab Weakness	Paint a WEAK LAYER in the snow. The painted cells release their slab at a lower angle -- by up to Slab Weakness Drop -- so you can put the fracture line where you want it instead of waiting for one. Like the Avalanche Trigger it lowers a THRESHOLD and adds no snow, and it needs fresh snow and a crust to act on
Avalanche Trigger	Paint where slopes let go EARLY. The painted cells drop their repose angle by up to Avalanche Trigger Drop, so a slope that was standing quietly starts to slide -- for forcing a cascade where you want one instead of waiting for the wind to build it. It lowers the THRESHOLD, it never adds sand: an avalanche here moves material, it does not create it. At 0 everywhere the simulation is bit-for-bit what it was
Snow Source	Paint where snow FALLS. Works like Sand Source but feeds fresh powder: the painted cells get Snow Source Rate of snow every step, on top of the global Snowfall. Needs Snow Source Rate above 0 to do anything
Sand Source	Paint where fresh sand blows in each step
No-Deposit	Paint ground where sand may NOT settle. Deposition only: whatever sand is already there still blows away, so a painted slab or road clears itself instead of freezing. Paint = block, Shift = allow again
Water / Wet	Paint wet / submerged ground (locked from wind erosion, deposition still allowed; enables the Water system)
Freeze Mask	Paint areas that stop evolving (locked)
Flow Strength	Paint how fast the SOLVED wind runs here. Needs the Flow 2D wind model. 1.0 leaves the solve alone, below 1 slows the air, above speeds it up -- and Flow Paint Mode decides whether the paint SCALES the solve or REPLACES it

Option	Meaning
Flow Direction	Paint WHICH WAY the wind goes, by dragging: the direction of your stroke is what gets written into the cells under the brush. Needs the Flow 2D wind model and Flow Paint Mode = Painted Speed + Direction, otherwise nothing reads it. Direction is stored as a vector, not an angle, so overlapping strokes average the way you expect instead of flipping to the opposite heading; an untouched cell keeps the solved flow. Shift+drag erases back to unpainted
Wind Gizmo	Drag in the viewport to aim the global wind from the drag vector (paints no field)

Mode

`brush_mode` - Enum - default **ADD

What a stroke does to the field under the cursor. Add, Smooth and Set are the everyday three; Grab, Slope and Erase are drag-aware and read the direction you move the mouse

Option	Meaning
Add	Add to the field (Shift to subtract)
Smooth	Blur/relax the field under the brush
Set	Flatten the field toward the Target Value
Flatten	Level toward the average height under the brush
Roughen	Add procedural bumps (break up smooth areas)
Sharpen	Push values away from the local average (crisp ridges)
Grab	Drag the sand mass along with the cursor -- pull dunes around (height fields)
Slope	Paint a linear ramp tilted along the drag direction (height fields)
Erase to Bedrock	Scrape the sand down to the bedrock floor, exposing bare rock

Radius

`brush_radius` - Float - default **6** - 0.1 to 1000

Brush radius in world units (mouse wheel to resize while painting)

Strength

`brush_strength` - Float - default **0.5** - 0 to 1

Stroke intensity 0..1. For ADD/height modes it scales the height moved per dab; for mask fields a full deliberate pass reaches about half the field range at 0.5. Shift inverts the direction (subtract / un-paint)

Target Value

`brush_value` - Float - default **1** - 0 to 10

Level the SET brush mode pushes the field toward under the stroke: for height fields it is a height in solver units (flatten a plateau at an exact level), for mask fields use 0..1 (e.g. Set 0 on Erodibility = paint full protection)

Sand Placement

`placement_mode` - Enum - default **FULL

WHERE sand may sit at the start. Full covers everything; the band modes restrict it by height or slope so sand collects in valleys, on flats or along a coast and the rest stays bare rock

Option	Meaning
Full Coverage	Place sand everywhere
Height Band	Place sand only within a height band
Slope Band	Place sand only within a slope-angle band
Noise Patches	Place sand in smooth procedural patches

Option	Meaning
Hollows	Place sand preferentially in local depressions
Combined Filters	Combine height, slope, cavity and noise placement filters

Height Low %

`place_height_low` - Float - default **0** - 0 to 100

Lower bound of the band that receives initial sand, as a PERCENT of the terrain own vertical range: 0 = the lowest point of this terrain, 100 = the highest. Local on purpose -- "sand from 20 to 60 percent up" survives rescaling the terrain, changing Image Amplitude or swapping the heightmap, which an absolute unit does not. Before 1.2.10 this was in raw solver units in HEIGHT BAND and a 0..1 fraction in COMBINED, so the same two numbers meant different things and the shipped 0..1 selected the WHOLE terrain. Used by the HEIGHT BAND and COMBINED placement modes

Height High %

`place_height_high` - Float - default **100** - 0 to 100

Upper bound of the initial-sand band, as a PERCENT of the terrain own vertical range (0 = lowest point, 100 = highest). Leave at 100 and raise Height Low to sand only the upper slopes; lower this and leave Low at 0 to sand only the valleys. Used by HEIGHT BAND / COMBINED placement

Mask Softness

`place_softness` - Float - default **0.25** - 0 to 50

Feather width of the placement mask edges: 0 = hard cut-off at the band borders, larger = gradual fade of the initial sand depth into the bare zones. UNITS FOLLOW THE MODE -- PERCENT of the terrain height range in HEIGHT BAND (since 1.2.10, matching Height Low/High) and DEGREES in SLOPE BAND. COMBINED derives its own edge from the band width and ignores this

Slope Low

`place_slope_low` - Float - default **0** - 0 to 90

Lower slope bound in DEGREES of the start-sand band (0 = flat ground). Only used by SLOPE BAND / COMBINED placement -- e.g. 0..20 puts sand on plains and keeps cliffs bare

Slope High

`place_slope_high` - Float - default **20** - 0 to 90

Upper slope bound in degrees of the start-sand band (SLOPE BAND / COMBINED placement)

Noise Scale

`place_noise_scale` - Float - default **12** - 2 to 128

Feature size (in solver cells) of the NOISE PATCHES placement pattern -- larger = bigger, fewer sand islands

Noise Threshold

`place_noise_threshold` - Float - default **0.5** - 0 to 1

Coverage cut-off 0..1 for NOISE PATCHES placement: higher = less of the terrain starts sandy (only the strongest noise peaks get sand)

PlaceCavity

`place_cavity` - Float - default **0** - -1 to 1

Cavity filter of the COMBINED placement mode: positive values put the starting sand in hollows and valley floors (natural wind-sheltered accumulation), negative values put it on ridges and crests instead. 0 = filter off. Magnitude sets how strict the selection is; feather the result with Mask Softness

PlaceNoise

`place_noise_coverage` - Float - default **0** - 0 to 1

Noise filter of the COMBINED placement mode: approximate fraction 0..1 of the terrain covered by organic sand patches (0 = filter off, 0.3 = about a third starts sandy). Patch size comes from PlaceCells, patch character from Noise Type. Combine with the height/slope/cavity filters via AND (default) or Combine with OR

PlaceCells

`place_noise_cells` - Int - default **60** - 4 to 512

Feature size of the COMBINED-placement noise patches, in solver cells: larger = few big continuous sand fields, smaller = many scattered islands. Scale it with resolution (60 cells reads very differently at 256 vs 2048)

Noise Type

`place_noise_type` - Enum - default **FRACTAL

The noise that breaks up the placement mask, so the edge of a sand field reads as patchy ground instead of a drawn boundary

Option	Meaning
Fractal Blobs	Smooth organic fBm blobs
Turbulence Strings	Sharper turbulent/stringy patches

Combine with OR

`place_combine_or` - Bool - default **off

How the COMBINED placement filters merge. OFF = AND: sand only where EVERY active filter agrees (strict -- e.g. low AND flat AND in a patch; can easily produce almost no sand). ON = OR: sand wherever ANY filter passes (generous coverage). If a Combined start looks empty, try OR or loosen the bands

Mask As Erodibility

`placement_as_erodibility` - Bool - default **off

Reuse the initial sand-placement mask as a live ERODIBILITY mask during the run: the bare zones stay protected from erosion, so the sand pattern you placed keeps its footprint instead of smearing over the whole terrain. Combine with Place Hold to also keep deposition inside the pattern; OFF = the mask only affects frame 0

Place Hold

`place_hold` - Float - default **0** - 0 to 1

How strongly the initial placement mask keeps CONFINING the sand during the simulation (0 = off, sand is free to migrate anywhere; 1 = the placed pattern is strictly held -- sand that tries to settle outside it is suppressed). Mid values let the pattern slowly bleed outward. Use for "sand only in the valleys" looks that must survive a long run

SHOW Placement

`place_show` - Bool - default **off

DEBUG overlay: tint the terrain with the frozen initial-placement mask so you can see exactly which areas received start sand (and what Place Hold is confining). Viewport visualisation only -- no effect on the sim or renders; turn off before final output

Shader Variant

`shader_variant` - Enum - default **FULL

Which BLOCKS this build of the surface shader contains. NOTHING IS REMOVED FROM THE ADD-ON -- every feature stays and switching variant simply rebuilds the material, so you can move between them freely. Choose the blocks needed by your scene to control material complexity; Full includes rain and displacement together

Option	Meaning
Full (everything)	Procedural detail, scanned sets, rain impacts and displacement together. The heaviest graph. This combination rendered in the tested Blender 5.3 demo; a heavier material or another renderer version may still hit shader limits. Test your final scene, or choose a smaller variant for features you actually need
1 - Procedural only	Procedural detail alone: no scanned sets, no rain, no displacement. ZERO image samples, so it is the cheapest way to see the simulation's own masks and shape shading. Good for look-dev on the physics
2 - Rain + scans	Rain impacts and everything rain-related, plus the scanned material sets. This is the variant for wet scenes: it leaves displacement out to reduce graph complexity. The procedural noise layers are turned down to ZERO rather than left out of the build, so they cost their node but contribute nothing -- your own settings for them are remembered and come back when you pick another variant
3 - Procedural + scans	Procedural detail and the scanned sets, without displacement and without rain. The everyday dry-desert build
4 - Procedural + scans + displacement	Adds real geometric displacement from the scanned height maps to variant 3. Use for a displaced dry surface; rain is left out

Reset to Original: Masks

`reset_original_masks` - Enum - default **KEEP

What Reset to Original does to the painted masks. The terrain and the masks are separate stores, so this is a separate choice -- a committed run writes SIMULATED masks into the same attributes hand-painting uses, and after that the two cannot be told apart

Option	Meaning
Keep Masks	Reset to Original puts the TERRAIN back and leaves every staged mask alone -- what you painted, and anything a commit filed, stays on the mesh
Original Masks	Put the masks back as they were BEFORE the first commit: the ones you painted by hand, without anything the simulation contributed. If there were none then, there are none now. This is the closest thing to "the scene as it came in"
No Masks	Remove every staged mask, so the next Initialize starts from clean fields on the original terrain

Undo Memory (MB)

`undo_memory_mb` - Float - default **256** - 0 to 8192

How much RAM the DunesForge undo history may hold, across undo AND redo together. Older levels are dropped first; one level always survives. MEASURED cost of one level: 1 MB at 256 cells, 4 MB at 512, 16 MB at 1024 (heights + bedrock; masks add more only when they exist). So 256 MB is roughly 16 levels at 1024, 64 at 512. The snapshots live in system RAM, never in video memory

Undo History

`undo_enabled` - Bool - default **on

Keep a bounded history of DunesForge actions on the SOLVER -- heights, bedrock and the masks -- so they can be undone. Blender's own Ctrl+Z cannot see these: they live outside bpy.data, in the solver's arrays. Turn it off to spend no memory on history at all

Steps/Frame

`iters` - Int - default **5** - 1 to 100

PLAYBACK pacing, not physics: how many solver steps run per timeline frame. Higher = the dunes evolve faster per frame of animation (and each frame costs proportionally more). It does not change HOW the dunes form, only how much simulated time one frame covers -- for smoother motion in renders lower this; to fast-forward big evolution raise it or use the Run Steps batch button

Freeze Dunes

`freeze_dunes` - Bool - default **off

PLAYBACK control: hold the terrain perfectly still while the wind clock keeps running -- wind direction schedules, haze and dust keep animating but the dunes stop evolving. Great for hero shots where the camera moves over a finished dune field with live blowing sand. Turn OFF to resume evolution exactly where it paused

Batch Steps

`run_steps_count` - Int - default **100** - 1 to 200000

How many solver steps the "Run Steps" button simulates in one go with NO viewport updates in between -- the terrain is written once at the end. The fastest way to fast-forward the simulation (e.g. let a dune field settle), and handy for reproducible lookdev

Commit With Render Detail

`commit_include_detail` - Bool - default **off

What Bake State as New Start freezes. OFF (the default) commits the SOLVER heightfield -- the physical state, which is what the next run should build on. ON commits the surface AS DRAWN, with the ripples, dunelets, grain and micro-detail baked in as real geometry. That is a ONE-WAY trip: the next run draws a fresh detail layer on top of the one you just made permanent, so committing repeatedly with this on piles roughness up. Turn it on when you want the render relief to become something the wind can then rework

Snapshot Layout

`snapshot_layout` - Enum - default **ROW_X

Where "Snapshot Terrain" parks each copy of the current terrain state: a horizontal row, a vertical row, or a grid -- an evolution contact sheet you build up while the sim keeps running

Option	Meaning
Row (X)	Place each snapshot to the +X side of the previous one
Row (Y)	Place each snapshot to the +Y side of the previous one
Grid	Fill a grid: N columns along +X, new rows toward -Y

Snapshot Gap

`snapshot_gap` - Float - default **5** - min 0

Empty space (world units) between parked snapshots

Grid Columns

`snapshot_grid_cols` - Int - default **4** - 1 to 32

Snapshots per row when Snapshot Layout is Grid

Edge Width

`edge_falloff_width` - Float - default **25** - 1 to 100

How much of the terrain the ramp occupies, as a percentage of the distance from the middle to the edge -- 25 fades the outer quarter and leaves the middle half exactly as it was. A percentage, not a distance, so it means the same thing at any Solver Resolution or terrain size

Edge Shape

`edge_falloff_shape` - Enum - default **SQUARE

The shape the lowered border follows. The terrain is square, so a square ramp keeps that silhouette visible in the result; the circular one hides it

Option	Meaning
Square	Follow the terrain outline: the ramp runs parallel to the four edges. Right for butting tiles together in a grid
Circle	A round island: the corners go all the way down, so tiles OVERLAP into one another instead of meeting at a line. Use this when a square ramp on a square terrain still reads as a square

Edge Curve

`edge_falloff_curve` - Float - default **1** - 0.25 to 8

Bias of the ramp. 1.0 is a plain smoothstep -- flat at both ends, which is what lets two tiles meet without a crease. ABOVE 1 keeps more terrain: the fade waits and then drops close to the border. BELOW 1 starts fading earlier and takes more of the terrain with it

Edge Level

`edge_falloff_level_mode` - Enum - default **MIN

What height the border is pulled down TO. Terrain Minimum is the quick answer for a single terrain; for a set of tiles that must join, give every one of them the same Custom Height

Option	Meaning
Terrain Minimum	Blend down to this terrain's own lowest point
Custom Height	Blend down to a height you type -- use the SAME value on every tile so they all meet at one level

Level

`edge_falloff_level` - Float - default **0

The height the border is blended down to, in solver units, when Edge Level is set to Custom Height

---

Presets

Presets are complete starting points -- physics, wind, look and surface detail together, chosen so they work as a set. Demo Scenes go further: they build an entire finished scene with terrain, lights, camera and materials.

When you need it: At the start of anything. Applying a preset RESETS the shared controls, so presets cannot leak into each other.

Preset

`preset` - Enum - default **ORGANIC

A complete starting point: physics, wind, look and surface detail together. Applying one RESETS the shared knobs so presets cannot leak into each other. The three named REPLACES TERRAIN also replace a terrain you loaded; every other preset leaves it alone

Option	Meaning
Organic Field	Soft varied barchanoid field
Big Rolling	Larger smoother forms
Fine Rippled	Smaller sharper dunes
Calm / Smooth	Low wind, polished smooth field
Coastal Source	Source-fed field with stronger wind
Star Dunes	Three-way wind regime

Option	Meaning
Linear / Seif	Alternating oblique winds. NOTE: this preset sets the wind regime but NOT 3D Wind, which is what actually aligns the ridges -- it ships EXPERIMENTAL and off for production. MEASURED at 512 over 1000 steps: without it the ridge angle lands 36.6 deg off the bisector and anisotropy DECAYS to 0.143; with it the angle converges to 0.8 deg and anisotropy RISES to 0.576. Turn 3D Wind on for true seifs, and expect about 6.6x the step cost
Reversing Ridges	Wind that flips 180 degrees. The net transport cancels, so the ridge does not travel -- it grows in place, with a slip face on BOTH sides. The only regime the classifier knows that no preset reached before
Barchans Open	Open boundary with low sand supply
Sharp Ridges	Large coherent sharp-crested ridges over bare interdune plateaus - the sculpted terrain-tool look (steady wind, low supply, strong settle, crisp lee)
Snow / Sastrugi	Cohesive wind-carved snow surface
Snow Blizzard	Active snowfall with strong wind
Snow: Thaw	Melting, softened snow forms
Snow: Drifts	Hard wind-packed slabs and lee tails
Parabolic Vegetated	Vegetation-stabilized parabolic dunes
Sand + Avalanches	Metastable lee faces with episodic sand slides
Snow + Avalanches	Snow slabs with cohesive avalanches
Snow over Sand (mixed)	BOTH materials in one scene, at TWO different angles. The sand settles at its own 34 degrees while the snow slab on top of it holds to about 45 before it lets go, which is what Snow Repose Offset exists for -- until it did, one angle had to serve both. Snowfall is light and the crust is hard, so the cover is patchy and there is a real snowline for things to walk across
MASK: Sand in Valleys	Combined placement: normalized low height band
MASK: Hollows + Patches	Combined placement: hollows with organic patches
MASK: Scattered Patches	Combined placement: isolated sand patches
MASK: Flats Only	Combined placement: sand on flat slopes
MASK: Plateau Top	Combined placement: sand on high plateau zones
SNOW v2: Powder Blizzard	Two-layer powder snow with active haze
SNOW v2: Slab Avalanches	Hard slabs with episodic releases. Since 1.3.35 it also arms the SLAB AVALANCHE on a buried weak layer -- the layer stack, the snowfall that fills it and the release criterion, because the switch alone reads a weak-layer hardness of 1.0 and a slab depth of 0.0 and does nothing at all. IT NEEDS A SLOPE TO RELEASE ON, and that is the one thing the preset cannot supply: the driving stress is depth times sin(slope) times cos(slope), and this preset leaves your terrain alone by design. On the flat RANDOM starting terrain the measured slope is 0.02-0.10 against a repose tangent of 0.675, so nothing will let go until there is real relief under it
SNOW v2: Icy Wind Crust	Hard wind crust and carved sastrugi
REPLACES TERRAIN: FBM	Generates an FBM starting terrain. This preset OVERWRITES a loaded heightmap or a sculpted mesh -- that is what it is for. The other 24 presets leave your terrain alone (measured)
REPLACES TERRAIN: Ridged	Generates a ridged starting terrain. OVERWRITES a loaded heightmap or a sculpted mesh -- that is what it is for
REPLACES TERRAIN: Dune Ridges	Generates wind-ridge starting terrain. OVERWRITES a loaded heightmap or a sculpted mesh -- that is what it is for

Preset JSON

`user_preset_path` - String - default

JSON file used by the Save/Load Settings buttons: store your own tuned setups and share them between scenes or machines (the built-in presets in the dropdown above do not use this path)

Demo Scene

`demo_scene` - Enum - default **DESERT

Which ready-made demonstration setup Create Demo Scene builds. Each demo is a complete scene: terrain, preset, helper objects, camera, light and materials, pre-simulated so it looks good immediately. Best used in a fresh empty scene

Option	Meaning
Grand Scene: Desert Expedition	A 400 m landscape with dunes, grains, rock shelters, sources, tracks and airborne dust
Grand Scene: Living Oasis	A 400 m rain-fed basin combining wet shores, plant species, growth and surface weathering
Grand Scene: Winter Storm	A 400 m snowfield with layered drifts, density, slabs, shelter and snow-fed vortices
Grand Scene: Dynamic Coast	A 400 m coast with Flow 2D wind, shallow water, waves, sediment, rain and vegetation
Desert: Dunes + Obstacles	Classic sand dune field with rock obstacles casting wind shadows, pebble scatter, camera, sun and a sand material. Pre-runs the simulation so dunes are already formed
Snow: Drifts + Fence	Wind-packed snow drifting behind a slatted fence, with a snow material and cool daylight. Shows the snow solver and obstacle wind shadows building lee tails
Oasis: Water + Valley Masks	Sand placed in valleys around a shallow water table with a translucent water plane: wet shores lock in place while dry sand keeps migrating around them
Water: Pools That Flow	The SAME oasis as above, with the water given a depth field instead of one global level -- so every hollow settles to its own height, water runs downhill between them and the shore is wherever the ground says. Built as a direct A/B against the Oasis demo: same terrain, same warmup, one switch different
Water: Wind On The Lake	The flowing field with a 30 m/s wind gripping the surface. The water piles up on the DOWNWIND shore at a slope of exactly tau/(rho g h), so the shoreline moves and the pools lean. MEASURED on this terrain at 30 m/s against no wind, after 3 s of water time: 7699 cells cross between wet and dry -- 4010 flooded and 3689 drained -- and the surface stands 13.7 mm apart at most. The wind here is the ANALYTIC one, so it is the same everywhere; switch Wind Model to Flow 2D and give the flow an Air Layer to have crests and lee pockets push the water differently
Snow: Slab Avalanche	A weak layer buried under a wind slab lets go on a 38 degree face and the debris RUNS OUT onto the flat below. This is the one scene where Slab Dynamics is on: without it the released mass is only ROUTED downhill, which moves it DOWN THE SLOPE rather than out across the flat. MEASURED on this scene, the two models differ by 10.7 % of the released mass: turning Dynamics on shifts 266 units off the upper face and lays them on the runout, concentrated around 75-87 % of the way across. Both models DO reach the valley floor and both stop at the same 95th-percentile distance -- the difference is WHERE the debris piles up, not how far it gets, and in a wide shot it is only 0.12 % of pixels. Look at the runout zone, not at the whole slope. MEASURED, and it is why the scene sculpts its own mountainside rather than using noise: on the stock demo terrain the 95th-percentile slope is 0.103 against a release threshold of tan(34) = 0.675 -- six times too flat, and 220 steps released nothing at all. Try Slab Dynamics 1 -> 0 and watch the pile move UP the runout toward the foot of the face

Option	Meaning
Snow: Avalanche That Scours Its Path	The same 38 degree face and the same storm as Snow: Slab Avalanche, with Entrainment ON (0.007 m per m of path) and Turbulent Friction set to 400. A real avalanche does not carry only the slab that let go: it scrapes the snow it runs over and grows. MEASURED on this scene, over the same storm: the flow picked up 35 033 units of snow off its path against 9 137 released by the slabs themselves -- 3.8 times the slab -- and 54 342 with the turbulent term on top, because a slower flow lives longer inside its step budget and scrapes further. What you see is the SCOURED TRACKS down the face and a fuller pile at the foot: 5.0 % of the pixels change against Slab Avalanche at the 1 % level, where the two release models of that scene differ by 1.7 %. The turbulent term is here for the SPEED the entrainment law reads, not for the runout: on this 90 m flat it does NOT move the deposit -- 0.432 of the debris ends on the flat at xi 400, 0.432 at 0 and 0.437 at 10, MEASURED -- while the mass-weighted speed drops from 70 to 54 m/s. Try Entrainment 0.007 -> 0 and watch the tracks on the face fill back in
Snow: Avalanche Into A Valley	The same 38 degree face and the same storm as Snow: Slab Avalanche, but the flat is SHORT (30 m) and a 22 degree counter-slope rises behind it -- and all four momentum controls (Carry Momentum, Momentum Rides Along, Sharper Transport, Momentum Crosses Lanes) are ON. Momentum is what lets debris CLIMB the far side; with the four at 0 the flow is hydrostatic and stops at the foot. MEASURED on this valley (512 cells, a 0.5 m slab off the face, friction 0.12): with the four on the front reaches the counter-slope after about 7 s, climbs to 174 m along it (22 m up) by about 11 s, then part slides back and 11 % of the debris settles on the counter-slope; with the four off the flow stops on the flat after about 10 s and never touches it. TIME IS THE TRICK: at 1500 substeps the run is cut after about 3 s and both variants deposit mid-flight on the flat with no run-up at all, so this scene runs 12000 substeps with Slab Flow Rest Speed 0.05 m/s. Try Carry Momentum 1 -> 0: the other three go dead with it and the debris stays on the flat
Sand: Slide Under Gravity	No wind. A rock face sculpted to 45 degrees, steeper than the Repose Angle (34), with a 0.6 m strip of sand and two thicker patches painted on it. The repose relaxation runs every step whether or not the wind blows, so the sand slides on its own and stops where its surface reaches 34 degrees. MEASURED on this scene (512 cells): 91 % on the face when the demo opens, 73 % after 20 steps and at rest from 40 steps on at 69.5 %, with 30.5 % at the foot as talus, mass kept to the last digit; with Avalanche Passes 0 nothing moves at all. Try Repose Angle 34 -> 25 (rest at 49 %) or 40 (85 % and still creeping)
Snow: Sluffs Off A Rock Face	The same rock face at 50 degrees, no wind, and a storm falling straight down. Snow landing above the repose limit sluffs to the foot as it falls; with Avalanche Trigger 6 it holds a few degrees past repose and lets go in events. MEASURED: 30-40 % of what fell on the face is still there when the storm ends, the drops come in steps of up to 0.18 points per step against 0.04 with continuous relaxation. Snow Repose Offset does NOT change this -- it belongs to the slab model (measured 0 / +12 / +20 identical). Try Avalanche Trigger 6 -> 0
Snow: Slab Avalanches Held Back By Gates	The Slab Avalanche face with three plates standing across it at 30, 40 and 50 % of the way down; the storm keeps falling while you Play, slabs keep letting go and the debris is ROUTED downhill (Slab Runs Out 0 on purpose: only the routing sees obstacles). The plates are KEYFRAMED to lift at frames 30, 70 and 110; each lift hands what it held to the next gate. MEASURED on a fixture: without a plate 89 % of the debris ends below its line, with it 45.5 % and the upper half stays put; with Slab Runs Out 1 the plate is ignored (85 % passes). Loose snow would not do: a uniform layer does not slide in the local repose rule, it only retreats from the top edge
Water: Reservoir Behind A Sluice Gate	A rock plateau with a 6 m basin, a dam with a 16 m gap and a valley below. The basin holds 1.5 m of water above the sill; a red gate in the gap is an ordinary obstacle that, with Water Sees the Rocks on, is part of the water bed and holds the water back. It is KEYFRAMED to lift 24 m from frame 20: press Play and the reservoir pours down the valley as a shallow-water flood. MEASURED on a fixture of this geometry: without the gate 7.8 % of the water is below the dam after 20 s and 34.8 % after 120 s, with the gate 0.0 % for 120 s, gate removed at 40 s -> the same curve shifted by 40 s; mass 1.0000. Try Water Sees the Rocks off: the flood starts at once

Option	Meaning
Water: Ripples Carried By The Current	The Pools That Flow basin with a DEAD SWELL pumping it, so the current never settles -- and that is the whole condition of this scene. Flow Ripples makes the fine chop read the FLOW FIELD instead of an independent noise: the pattern stretches along the current and twists where it turns. MEASURED on this scene: straight after the warm-up the basin is in EQUILIBRIUM -- peak speed 0.0 and 0.0 % of cells moving -- so the knob changes 0.000 % of the pixels, because there is no flow to read. With the swell on: peak 36.15, mean 0.218, 70.4 % of cells alive, and the same knob changes 4.69 % of the pixels on the built scene (4.84 % on the gate fixture) against 0.000 % render noise. Try Flow Ripples 2.5 -> 0: the chop goes back to an independent noise that does not know where the water is going. Try Swell Height 0: the basin settles, the current dies, and the knob stops mattering -- which is the same trap the Ice: Pack scene documents
Ice: Pack, Ridges and Leads	The ice mechanics of 1.3.134-1.3.138 in one scene: a frozen basin kept ALIVE by a steady inflow, so the pack never settles. The current carries the sheet, piles it against the shore into RIDGES where it converges and pulls it open into LEADS where it diverges -- all three out of one conservative flux, with no extra rule. MEASURED, and this is why the demo exists: on the Pools That Flow terrain the water reaches equilibrium and the ice sampled a LIVE current in only 2 of 20 steps, so none of it could be seen. With the inflow on it is 20 of 20. Try Ice Yield Strength 20 -> 0 and watch the ridge turn into an even thickening
Plants: Two Species Side By Side	A succulent and a grass on the same dune, which is the commonest arrangement on a real one. Drought Tolerance used to be ONE number for the whole scene, so the two could not stand together; above 0 Second Species turns it into a FIELD of patches, deterministic in Seed. MEASURED on the gate fixture, cover of the tolerant plant against the sensitive one: 0.4973 / 0.7078 with one species (a +0.2105 gap that is just the two thresholds) against 0.5087 / 0.7811 with the species map on -- the gap widens to +0.2725 because each plant now sits where it can. The two colours are what makes it readable: Vegetation Colour is the grass, Vegetation Colour B the succulent. Try setting Tolerance (Species B) equal to Drought Tolerance: the map stops mattering and the result goes back to being identical to the bit
Snow: A Thaw That Leaves One Patch	The drift field and the same fence, run through a THAW. Snow lies at Initialize (Initial Snow Depth 0.5), the winter builds the drifts, and then the air goes to +8 C with Melt Rate on -- and the field loses ALL of it. MEASURED over 30 steps: mass of the fresh layer 20 584 with no thaw against 0 with it, cover 98.2 % against 0.0 %. One patch survives, because a snow EMITTER keeps feeding it: Snow Source Rate 0.05 over a box, feathered by Snow Source Smoothing, holds 2 842 of mass and 4.9 % cover when everything around it is bare. MEASURED and the reason this scene keeps the DunesForge surface shader instead of the flat snow material the other snow demos wear: on flat white paint the same thaw changed frame brightness from 0.7232 to 0.7200 and the renders looked identically white -- only the drift relief moved. With the snow mask driving the surface, the thaw repaints 57.1 % of the pixels and the emitter patch another 4.4 %. Try Melt Rate 0.01 -> 0 with the air still warm: melting is Melt Rate PLUS the temperature term, so the warm air alone still takes it, just more slowly. Try Snow Source Rate 0.05 -> 0: the last white patch goes with it
Another Planet: Mars	The same ORGANIC dune field as the other scenes with ONE switch different: Other Planet, set to Mars. Martian air is sixty times thinner than ours (0.020 against 1.20 kg/m3) and the saturation length divides by air density, so the same physics builds dunes far larger on the same ground. Turn Other Planet off and the scene rebuilds itself at Earth scale, which is the whole point of the comparison; scale the terrain object up if the Martian dunes outgrow the default 200 m domain
Snow: Fresh, Packed and Drifted	The same fence and the same wind as Snow: Drifts + Fence, with Snow Density switched ON -- so the pack stops being a depth and starts being a MASS. A depth-only solver cannot tell 10 cm of powder from 10 cm of settled snow; MEASURED on the gate fixture those are 10.00 and 50.00 kg/m2, five times apart at the same depth, and across this field the mass runs 0.37 to 17.42 kg/m2. Drifted snow is denser than the snow that fell, because the grains shatter in saltation and pack tighter, so the drift behind the fence settles SHALLOWER and HARDER than the fresh fall around it -- measured mean hardness 0.1204 at Drifted Density 350 against 0.8125 at 100. Try Drifted Snow Density 350 -> 100: the mechanism switches itself off bit for bit and the pack deepens again

Option	Meaning
Rock + Sand Source	Start Terrain = Rock workflow: a rocky FBM terrain frozen as bedrock, with sand fed only from an upwind source zone so drifts grow from nothing over bare rock
Rain: Wet Sand + Plants	Storm cells sweep a formed dune field: the dunes are built DRY first, then the rain arrives, so you can see what it changes. Damp patches go dark and matte and stop moving, the lee of the rocks stays dry in the rain shadow, and vegetation greens up where the root zone holds water and then pins the sand down
Rain: Monsoon Shutdown	The heavy end of the rain model: near-total coverage, slow cells and strong infiltration, so almost the whole field goes wet and almost all of it stops moving. Shows what OUTLIVES the rain -- a drying crust plus the vegetation it fed, which keep holding the sand after the weather has gone
Rain: Wet Oasis	Water table AND rain at the same time: two independent sources feeding one wetness mask, each with its own hold strength. Plants ring the standing water where the root zone never dries out. Good for checking that the two systems combine rather than fight
Rain: After The Storm	The field is soaked and then the rain STOPS. Wind Drying makes exposed crests dry first while sheltered lee slopes stay damp, so the dunes wake up unevenly instead of all at once -- the clearest demonstration of the wind/moisture coupling
Rain: Drops On Water	A close-up of the rain IMPACT layer, framed on a shoreline so you can see both behaviours at once: rings spreading where a drop lands in standing water, and splash craters with dark wet spots where it lands on the shore. The camera sits low and near on purpose -- an impact is about 5 cm across, so from the usual demo distance every one of them would be a single pixel
Terrain: Mountain Range	Ridged fractal peaks with sand collecting in the valleys -- the classic case where the rock is the scenery and the dunes are what moves through it
Terrain: Canyon	A terraced, deeply-cut canyon frozen as bedrock with sand fed along the floor, so drifts snake between the walls and pile against them
Terrain: Cellular Mesas	Turbulence terracing gives flat-topped mesas with steep sides; sand streams around them and builds lee tails behind each
Terrain: Ready Dune Field	Wind-aligned ridges as the starting field, so a mature dune sea exists from frame 0 instead of forming over hundreds of steps
Terrain: Beach + Buildings	A shallow shore with a water table, a row of buildings behind the dune line, and sand drifting up against them
Terrain: Buried Town	A block of buildings on an open plain being swallowed by sand: each one casts a wind shadow and grows its own lee drift
Terrain: Crater Basin	A steep-walled basin that traps everything blown into it -- a sand trap, and a good stress test for avalanching on rock
Terrain: Rolling Hills	Soft FBM hills with a full sand cover: gentle relief where the dunes and the ground shape each other
Terrain: Stepped Plateau	Hard terraces frozen as bedrock, with sand only on the steps -- shows the non-erodible rock path and how sand cascades between levels
Terrain: Islands + Water	Scattered high ground in a shallow sea: wet locked shores, dry migrating crests, and rain to tie the two together
Showcase: Object Masks	Three object slots doing three different jobs in one scene: a source box feeding fresh sand, a wind-gain box acting as a gust lane, and rock obstacles casting wind shadows. Every one of these masks can also be painted with the Interactive Brush
Showcase: What Objects Tell The Sand	Three slabs on one moving sand sheet, each holding the ground to a different rule, with every number below measured on the scene itself. BEDROCK is a floor the sand cannot sink below; this one sits above the sheet, so it stands as a rock sill the sand flows around, 1.43 m proud of the field. ERODE is the quiet one: the ground there stops moving altogether, roughness 0.0025 against 0.30 for the sheet beside it, so dunes travel past a patch that never changes. NO-DEPOSIT is the one people mix up with it: nothing new lands, while whatever sand is there is carried off as usual, so the footprint SINKS -- 0.12 m after the warm-up and 0.38 m after 300 steps more. Erode holding still while No-Deposit digs itself out is the whole point

Option	Meaning
Showcase: Grain Systems	The grain-scale models together: Multi-Fraction winnowing fines out into an armouring coarse-lag pavement, Strata painting cross-bedding on cut faces, Sand Age reading fresh on scoured slip faces, and the physical ripple solver at megaripple scale. Ships with the Surface Shader so all of it is visible
Phenomena: Playa Crust	A dried lake bed: the crust sets and cracks into plates, and the wind deflates along the cracks while the plates stand
Phenomena: Wind Erases Tracks	A rut imprinted across the field, fading faster where the wind is strongest and surviving in the lee
Vegetation: Nabkha Hummocks	Scattered plant clumps on a sand-fed plain, each one trapping a mound with a downwind tail -- the nabkha (coppice dune) landform. Built with the patch-shape controls open, so the clumps are ragged, come in a range of sizes and grow in stands instead of being identical circles on a uniform scatter, and with the clumps taking their own darker colour so a hummock reads as a bush and not as a green stain
Vegetation: Rain-Fed (nothing grows on its own)	The only scene where NOTHING grows by itself: Veg Growth is set to zero, so every plant you see was put there by rain. Growth From Rain is a multiplier and cannot help at zero growth, so Rain Seeds Growth -- which adds its own rate from the root zone -- is the only term that can raise a single stem, which makes the scene its own control. Spread sits at 1.0 so growth happens only where plants already are: clumps thicken and creep at their edges while two thirds of the ground stays bare, and that bare sand is what lets you see the second half of the loop, Plants Shade The Ground, holding moisture under the cover that the rain grew
Plants: Valleys Only (Slope and Height Windows)	The blowout field again, with the vegetation kept OUT of the top 40 % of the relief by Height Max 0.6, Limits Apply to Seeding ON and Edge Noise 0.5. The window is what keeps a real field from greening over: plants hold the hollows and the crests stay bare sand. MEASURED on this scene after the 150 step warm-up, cover on the top third of the relief: 0.992 with no window against 0.040 with it, and 0.000 at frame 0 because Limits Apply to Seeding never lays it there; without that switch seeding covers the crests first and burial has to clear them. The relief is 8.0 here, ten times the Blowouts scene, MEASURED: at 0.8 the dunes rebuild the height field during the warm-up, the crests at the end are not the crests the seeding obeyed, and the window keeps only 0.676 of them bare (2.0 -> 0.292, 4.0 -> 0.098, 8.0 -> 0.040). Burial is 0.5, not 2.0, because at 2.0 the crests are bare with or without a window and there would be nothing to show. MEASURED and fixed in this version: Edge Noise used to jitter the OPEN limits too, so a height-only window refused 45.3 % of a perfectly flat floor; it now frays only the limit you set. Try Invert Limits: the crests green and the hollows go bare. Try Edge Noise 0.5 -> 0 and the tree-line turns into a contour
Vegetation: Blowout Hollows	The opposite starting point: a fully vegetated coastal grassland with bare gaps punched in it. The wind scours the unprotected gaps into deflation hollows while the anchored surroundings hold, which is how a blowout -- and eventually a parabolic dune -- gets going
Vegetation: Green Retreats	A field that starts GREEN EVERYWHERE and loses it: drought dieback takes the cover off the dry ridges while the plants ringing the water hold, so the green pulls back to the shoreline instead of simply fading. Built the way round it is for a measured reason -- with the usual near-bare start the same physics moves 41% of the dry-zone cover and changes 1% of the rendered pixels, because 0.068 and 0.040 both read as bare sand. Starting green makes the loss an AREA that shrinks. MEASURED on this scene after 90 steps against the same scene with Drought Dieback at 0, counting only VISIBLE LAND (ground above the waterline -- cover under an opaque water surface is not something you can see): the strongly green share of the land falls from 78.4% to 11.0%, and mean cover on land from 0.549 to 0.165. The pool itself is half a percent of the frame while the wetness reaches 77% of it, so the green ring sits on LAND rather than under water. Turn Drought Dieback to 0 and the green stays put
Temperature: Cold Front	One cloud over a warming scene: the same storm lays snow while the air is below zero and wets the ground once it is above
Temperature: Spring Thaw	A snowfield under a rising sun -- melt on degree-days, so it stays put until the air crosses zero and then goes quickly
Rain on Snow: Crust	Warm rain falling on fresh snow: it takes depth off the pack and leaves what is left bonded into a crust

Option	Meaning
Rain on Snow: Loaded Slab	The same crust with fresh snow dumped on top of it -- the layered setup a slab releases over
Freeze: Locked Shoreline	Wet sand along a water table, frozen solid: the damp band stops eroding entirely while the dry dunes behind it keep migrating
Freeze: Thawed Shoreline	The same shoreline above freezing -- the control that shows the lock is the ice and not the water
Frost: Cold Dawn	Surface frost whitening the damp hollows while the dry crests stay bare sand
Frost: Hoar on Snow	A cold, sheltered snowfield growing surface hoar -- the weak layer a later slab will release over
Flow 2D: Canyon Jet	Two rock walls with a gap. The solved wind speeds up through the gap because the same air has to pass a narrower opening -- a result of mass conservation, not of a correction term
Flow 2D: Recirculating Wake	A single bluff rock. Behind it the flow REVERSES and turns back on itself, which the analytic wind cannot do at all -- it has a lee shadow but no momentum
Flow 2D: Wind Off a Wall	An angled barrier that turns the wind sideways. Tracers and the direction overlay are on, so the bounce is visible as motion and as colour
Phenomena: Snow Fence	A POROUS blocker: the long downwind drift a real snow fence builds, next to the short steep pile a solid wall makes
Phenomena: Snow Layer Stack	A season of storms as dated layers, with a buried weak layer that decides where a slab releases
Phenomena: Sublimation + Hoar	Snow leaving as vapour on the exposed crests while surface hoar grows in the sheltered hollows
Phenomena: Tidal Coast	A moving shoreline: the damp band sweeps up and down the beach and the dunes behind it advance in steps
Showcase: Surface Detail	The layer that lives ABOVE the dune, shot from close enough to see it -- from the usual demo distance it is smaller than a pixel. The Micro-Detail pass has been run once already, freezing the dunes as bedrock and running transport again at a finer scale over them, with Crispness, Crests and Slump Allowance set; those three do nothing until the pass is RUN, so press the button again after changing them. De-facet Crests takes out the straight grid zigzag that a one-direction wind leaves along the crest lines, Edge Rounding takes the corner off the brink but not off the flats, and Detail Feeds Sim hands a little of the detail back to the solver so the ripples are sand rather than a decal the wind ignores
Showcase: Surface Shader	How the surface texture is PROJECTED, rather than what it is. A flat top-down projection stretches the grain down every slope and a dune is mostly slope, so this scene switches the scanned sand and the bedrock onto a triplanar projection and turns up the four map-side controls: the scan follows the wind heading and the local downslope, the procedural/scan balance becomes a map instead of one number, and the terrain sits in its own ambient shade. Note the trade it has to make: triplanar and Hex-Tiling cannot both run on the sand, so the scene uses Texture Bombing -- put Hex back and the triplanar tick silently stops doing anything
Showcase: Airborne + Dressing	Everything that is not the heightfield: swept sand-haze particles emitted off exposed crests and settling in the lee, wandering dust devils, a curve imprinted as a rut with berms, and pebble scatter driven by the simulation masks

Warmup Steps

`demo_warmup` - Int - default **150** - 0 to 5000

Simulation steps to pre-run right after building the demo scene so dunes or drifts are already formed when it opens. 0 = build the scene only, no pre-run. Higher = a more evolved field but a longer wait on click (GPU recommended above ~300)

Wind

Everything about how the air moves. The default wind is an analytical model -- fast, stable, and the one the dune literature is built on. The heavier models (2D flow, 3D pressure solver) exist for cases where air has to go AROUND things rather than over them.

When you need it: Wind Direction and Wind Strength on day one. The rest when you need variation over time, terrain channelling, or flow around architecture.

Wind Strength

`wind_str` - Float - default **1** - 0 to 20

Overall wind strength (0..5, nominal 1). Scales how much sand the wind moves everywhere; local variation comes from the Wind Gain object/mask. MEASURED at 1.0 / 2.0 / 5.0 over 20 steps on a flat bed: the relief it builds goes 4.91 -> 8.01 -> 11.66 m and the sand it shifts 0.54 -> 1.31 -> 2.07 m per cell. UP TO 1.2.89 THIS SLIDER DID NOTHING AT ALL -- the entrainment threshold was written as a fraction of this very number, so it cancelled out and 0.25 and 4.0 gave bit-identical sand. The threshold is now a property of the grain, as Bagnold has it, and 1.0 still reproduces every earlier build exactly. Values >2 move a LOT of sand per step and can look jumpy -- consider more Steps/Frame instead

Wind Mode

`wind_mode` - Enum - default **NORMAL

How the wind DIRECTION behaves over time. It is the biggest single control over what shape of field you get, because a dune records the wind that built it: one prevailing wind gives crescents and transverse ridges, alternating winds give long seif ridges, and winds from several sides give star dunes

Option	Meaning
Normal	One prevailing wind (plus Wobble/Jitter sway) -- classic crescent barchans and transverse ridges migrating downwind
Linear / Seif	Two oblique winds alternate every Phase Steps. Seif ridges line up along the AVERAGE wind direction ONLY with 3D Wind on: the ridge is built by a HELICAL cell turning about a HORIZONTAL axis, and a plan-view solver has no vertical component to hold one. MEASURED over 1000 steps at 512, ridge angle against the bisector (0 deg = seif) and anisotropy: Analytic -36.6 deg / 0.143, Flow 2D -73.9 deg / 0.156, 3D Wind -0.8 deg / 0.576. Without it the field is transverse and the direction DECAYS
Star	Three winds rotate on the Phase Steps schedule -- sand converges into tall pointy star dunes with radiating arms; needs long runs to develop
Reversing	The wind flips 180 degrees every Phase Steps -- dunes stay in place, grow tall and sharp-crested with small reversing caps at the brink. The sudden flip can spike steep terrain at bare-CFL pacing: raise Min Substeps (Performance) or keep Settle Slopes on

Phase Steps

`wind_phase_steps` - Int - default **50** - 1 to 10000

How long each scheduled wind holds its direction, in solver STEPS, for the Linear / Star / Reversing wind modes (ignored in Normal). Short phases = the winds blend and forms stay soft; long phases = each wind fully reworks the field before the next takes over, giving crisper seif ridges / star arms

Wind Direction

`wind_dir` - Float - default **0** - 0 to 360

Base wind heading in DEGREES (0 = +X, 90 = +Y; compass-style around the terrain). Wobble/Jitter and the wind modes sway AROUND this base. Two-way with the wind gizmo: rotate the spawned arrow while paused and this value follows; during playback the arrow shows the live (swaying) heading instead

Use Wind Object

`use_wind_object` - Bool - default **off

Drive the wind direction from a scene object: the chosen local axis of the Wind Object (projected onto the ground) becomes the live wind heading every frame. Rotate or ANIMATE the object to art-direct the wind over time -- overrides the numeric Wind Direction while enabled (Wobble/Jitter still sway around it)

Wind Object

`wind_object` - Pointer

Scene object that steers the wind when Use Wind Object is ON: its selected local axis (projected to the ground plane) becomes the live wind heading. Any object works -- an Empty arrow is ideal; keyframe its rotation to animate the wind

Wind Axis

`wind_object_axis` - Enum - default **Z

Which local axis of the wind object points downwind. Rotate the object and the simulation follows, which is what makes the wind keyframable by hand

Option	Meaning
+Z	Use local +Z axis
+X	Use local +X axis
+Y	Use local +Y axis
-Z	Use local -Z axis
-X	Use local -X axis
-Y	Use local -Y axis

Wind Wobble

`wobble` - Float - default **0.15** - 0 to 1

Wind sway: the heading gently oscillates back and forth around the base direction (0 = perfectly straight; 1 ~= +/-90 deg sway). Bounded -- the wind never rotates away, it always returns

Wind Jitter

`jitter` - Float - default **0.05** - 0 to 0.5

Random wind gusts: a bounded random walk on the heading around the base direction (0 = none, higher = gustier). Stays near the base -- it does not accumulate into a full rotation

Gust Strength

`wind_gust_strength` - Float - default **0** - 0 to 1.5

How hard the wind GUSTS -- variation in the SPEED, as a fraction. The existing gust controls walk the HEADING, so a "gusty" wind changed where it blew and never how hard. This one changes the strength, and it reaches both wind models: the analytic shear scales with it, and the 2D flow solves with it as its free stream (whose speed used to be hard-coded, so gusts could not reach the flow at all). Bear the physics in mind -- shear goes as the SQUARE of the speed, so a 1.4x gust is roughly DOUBLE the transport, and 0.6 here is already a violent wind. Held for Gust Min/Max steps when those are set, otherwise a per-step walk. 0 = off, and the speed factor is then exactly 1.0

Turn Time (steps)

`wind_turn_steps` - Float - default **10** - 0 to 5000

How many STEPS the wind takes to swing round to a new heading. Everything that decides direction -- a scheduled phase in a Star or Seif regime, the rotation rate, a gust, the jitter walk -- sets a TARGET, and without this the wind adopted it inside one step: a phase change snapped from one angle to the next the moment its counter ran out, which puts a fresh slip face on the wrong side of every dune in a single frame. Real wind has inertia; a front takes hours to turn a desert. The heading eases toward the target with this many steps as its time constant, so about two thirds of a turn is done after that many steps and it is essentially complete after three times as many. Larger = more sluggish. 0 = OFF and the wind turns instantly, which is what every build before this one did

Gust Hold Min

`wind_gust_min` - Int - default **0** - 0 to 2000

GUST DURATION, lower bound (in solver STEPS): with Gust Hold Max > 0, Wind Jitter switches from a per-step random walk to distinct GUSTS -- each gust picks a random heading offset within the Jitter range and HOLDS it for a random number of steps between Min and Max (easing smoothly between gusts). 0/0 = classic per-step jitter. Example: Min 20 / Max 80 gives gusts lasting roughly 1-4 seconds of 24 fps playback at 1 step/frame

Gust Hold Max

`wind_gust_max` - Int - default **0** - 0 to 2000

GUST DURATION, upper bound (in solver STEPS). Set > 0 to enable gust-hold mode: the wind holds each random gust direction for Min..Max steps instead of wandering every step. Longer holds = calmer, weather-like direction changes; shorter = nervous, turbulent wind. Requires Wind Jitter > 0 (jitter sets the gust ANGLE range, this sets the gust DURATION)

Use Wind Gain Curve

`use_wind_curve` - Bool - default **off

Boost (or damp) the wind along a corridor around a drawn CURVE object -- wind funneled through a canyon, a valley breeze, a localized gust lane. The corridor multiplies with the Wind Gain object/attribute channels. Sand moves faster and dunes stretch along the corridor; refreshes when you toggle or reassign

Wind Curve

`wind_curve` - Pointer

Curve object defining the wind corridor path (drawn in top view over the terrain works best). Needs Use Wind Gain Curve ON

Corridor Width

`wind_curve_width` - Float - default **8** - min 0.1

Full width of the wind corridor in WORLD units (meters on the default 50 m terrain). The boost fades smoothly to zero at the corridor edges

Corridor Gain

`wind_curve_gain` - Float - default **1

Wind strength boost inside the corridor: +1 doubles the shear on the curve centreline, negative values SHELTER the corridor instead (a calm lane where sand settles). 0 disables the effect

Show Corridor

`wind_curve_show` - Bool - default **off

Draw a translucent ribbon draped over the terrain showing the EXACT corridor the Wind Gain Curve affects (Corridor Width wide, centred on the curve). Updates live when you move the curve or change the width, and re-drapes as the dunes evolve. Viewport only, never renders

Arrow Grid

`wind_field_density` - Int - default **32** - 8 to 96

Arrows per side of the wind-field overlay grid (32 = 1024 arrows). Higher shows finer shear detail, costs a slightly bigger overlay mesh update per displayed frame

Use Temperature

`temp_enabled` - Bool - default **off

Let one air temperature drive the weather instead of four independent sliders. WHAT IT FIXES: rain, snowfall, melt and drying ship as unrelated controls, so a scene can rain at -10 C, melt while it snows, and evaporate a puddle at freezing

point -- three states that cannot coexist. With this on, the same cloud drops rain or snow depending on the air, melt runs on degree-days so nothing melts below zero, evaporation follows the temperature, wet ground FREEZES and stops eroding, plants stop growing in the cold, and hoar grows only where it can. OFF is bit-identical to every scene made before it existed

Air Temperature (C)

`air_temp` - Float - default **15** - -40 to 50

Degrees Celsius. Zero is the hinge: below it precipitation falls as snow, melt stops, evaporation nearly stops and wet ground freezes solid. Keyframe it for a thaw, a cold front or a day/night cycle -- everything downstream follows from this one number

Sleet Band

`temp_freeze_band` - Float - default **1.5** - 0 to 10

Width in degrees of the mixed rain-and-snow band around zero. Real sleet is a mixture, and a hard switch exactly at 0 C makes a slowly warming front SNAP from snow to rain in one frame. 0 gives that hard switch if you want it

Melt per Degree

`temp_melt_per_deg` - Float - default **0.004** - 0 to 0.1

How much snow leaves per step for each degree above zero -- the degree-day model every snow-hydrology text starts from. Added to the plain Melt Rate slider, which stays available as a constant background melt

Drying per Degree

`temp_dry_per_deg` - Float - default **0.04** - 0 to 1

How much faster the ground gives up water per degree above zero. At or below freezing the rate drops to 15% instead of zero, because ice still sublimates -- slowly

Rain on Snow

`temp_rain_on_snow` - Float - default **0** - 0 to 4

Warm rain falling on snow melts it and leaves the rest HARDER. This is the main way a snowpack gains a crust in nature, and the commonest avalanche trigger there is: the meltwater percolates, refreezes and turns loose snow into a slab sitting on whatever was underneath. Needs Rain and a temperature above zero -- rain that falls as snow cannot melt anything. 0 = off

Freeze Wet Ground

`temp_ground_freeze` - Float - default **0** - 0 to 1

Below zero, WET ground freezes and stops eroding. Damp sand is already harder to lift than dry sand (that model ships), but damp and FROZEN is a different thing: it is bonded, and the wind takes nothing off it at all. Scales how completely -- 1.0 makes frozen wet cells fully non-erodible. Dry cells are unaffected however cold it gets, which is the whole point: it is the water that freezes. 0 = off

Growth Stops Below

`temp_veg_min` - Float - default **5** - -20 to 25

Plants stop growing below this temperature. Vegetation currently grows at the same rate in a heatwave and under snow, which is why a winter scene keeps sprouting. Only the GROWTH is gated -- what has already grown stays, and burial still removes it

Day/Night Swing (K)

`temp_day_amp` - Float - default **0** - 0 to 40

Half the daily temperature range: the air rises this far above Air Temperature at noon and falls this far below it before dawn. 0 keeps the air flat, which is what shipped. Deserts swing hardest -- 20 K between afternoon and dawn is ordinary -- and this is the slider that makes frost, melt, evaporation, ice and plant growth BREATHE TOGETHER

instead of each sitting at one value. It feeds the scalar every part of the model reads, not just the temperature field

Steps Per Day

`temp_day_period` - Float - default **100** - 1 to 10000

How many solver steps make one day. This solver has no astronomical clock -- it has a morphological step -- so a day is worth exactly what you say it is worth. Short periods make the scene flicker between frost and thaw; long ones let each half of the cycle actually do something before it reverses

Temperature Varies Over The Terrain

`temp_field_enabled` - Bool - default **off

Air Temperature is ONE NUMBER for the whole scene: snow melts as fast on a ridge crest as in the shaded hollow below it, and the ground freezes everywhere at once. Turn this on and the temperature becomes a FIELD -- colder with height, warmer on the slopes the sun actually reaches. Off by default, and then the solver runs exactly as before

Lapse Rate

`temp_lapse_rate` - Float - default **6.5** - 0 to 100

How much colder the air gets with height, in kelvin per kilometre. The troposphere averages 6.5, which over a 40 m dune is only 0.26 K -- but over 800 m of relief it is 5.2 K, the difference between snow that lies and snow that melts. Raise it past the physical value if you want the effect to read on a small terrain

Sunlit Slope Warming

`temp_slope_gain` - Float - default **0** - 0 to 30

How much warmer a fully sunlit slope is than the scene average, in kelvin. Irradiance on a tilted surface follows the cosine of the incidence angle, so a slope facing the sun collects more than flat ground and one turned away collects none -- this is why snow survives on the shaded side of a ridge. 0 keeps the temperature independent of aspect

Sun Azimuth

`temp_sun_azimuth` - Float - default **180** - 0 to 360

Compass direction the sun comes FROM, in degrees, for the slope warming above. 180 is due south, which is where the midday sun sits in the northern hemisphere. Only used when Sunlit Slope Warming is above 0

Sun Elevation

`temp_sun_elevation` - Float - default **45** - 0 to 90

How high the sun stands above the horizon, in degrees. A low sun exaggerates the difference between slopes because the incidence angle changes faster; overhead sun at 90 lights every slope almost equally. Only used when Sunlit Slope Warming is above 0

Surface Frost

`frost_enabled` - Bool - default **on

Grow the FROST field at all. Frost is a surface finish, not a layer with mass -- it whitens and roughens what is underneath without ever avalanching. Switch it off to keep a cold, damp scene bare. This is the physical switch; Build Frost in the Surface Shader panel decides whether the render reads the field

Initial Frost

`frost_initial` - Float - default **0** - 0 to 1

Frost already on the ground at Initialize, 0..1. THIS IS ALSO HOW YOU GET FROST WITHOUT THE TEMPERATURE MODEL: with Use Temperature off the field is seeded to this value and then left alone, so it behaves as a static art control. With Use Temperature on it is a starting point that the air then grows or burns off. 0 = start bare and let the weather decide

Max Frost

`frost_max` - Float - default **1** - 0 to 1

Ceiling on how heavy the frost can get. This is the knob that makes frost CONTROLLABLE rather than binary: the growth term saturates almost immediately at any real frost temperature, so without a ceiling every cold damp scene ends up at full hoar. Set it to 0.3 for a light rime, 1.0 for a hard white morning

Frost Growth

`frost_rate` - Float - default **1** - 0 to 8

How fast frost forms on cold, damp ground. The underlying rate reaches its ceiling within a handful of steps at any real frost temperature (measured: 1.0 within 10 steps at -5 C and below), so lower this when you want to watch it creep in over a sequence rather than appear on frame two

Patchy Thaw

`frost_thaw_patchy` - Float - default **0** - 0 to 1

How UNEVENLY frost burns off above zero. At 0 the whole surface thaws at one rate, which is what it always did. Turn it up and the same total loss is spread unevenly: ground that is exposed to the wind clears first while the lee keeps its hoar, so the field breaks into patches instead of fading as one sheet. It is a WIND shelter, not a sun angle -- sublimation scales with the wind at the surface. It changes the CONTRAST rather than the rate: over a single step the average loss is exactly what the even thaw would take, and on ground with no shelter at all this does nothing, because there is nothing to contrast against. Over a whole sequence the sheltered ground does keep more, which is the point of it

Wind Model

`wind_model` - Enum - default **ANALYTIC

How the wind field is produced. Analytic is the shipped model and the default; Flow 2D actually solves the air

Option	Meaning
Analytic (Jackson-Hunt)	The classic model this plugin has always used: a linear perturbation of a free stream over the separation envelope, computed by FFT. Fast, matches the 3ds Max original and grows correct dunes -- but it is a one-way map from terrain to shear. It has no momentum, so it can produce NO recirculation, no wall bounce and no true channelling, whatever the terrain looks like
Flow 2D (solved)	Solve incompressible Navier-Stokes in plan view: advection, a pressure projection and real solid boundaries. This is what gives a canyon a genuine Venturi speed-up (from mass conservation, not a correction term), a reversing wake behind a rock, and wind that turns when it meets an angled wall. Costs a second grid and a Poisson solve -- see Flow Resolution and Solve Every

Flow Quality

`flow_mode` - Enum - default **PERFORMANCE

Accuracy against cost for the 2D flow solver. Both modes solve the same equations and both bend the wind round walls; they differ in how much of a vortex survives being carried across the grid, and in how strictly the pressure solve enforces incompressibility (i.e. how little the flow leaks through a wall)

Option	Meaning
Performance	Semi-Lagrangian advection and a fixed number of red-black SOR sweeps. Still incompressible enough to bend the flow round walls and shed a wake, just more diffusive -- vortices smear out sooner. Measured at 128x128: about 93 ms per flow step
Reference	BFECC (error-compensated) advection so vortices survive instead of dissolving into numerical viscosity, and the pressure solved by conjugate gradient to a residual tolerance. MEASURED against Performance: a test feature carried 256 steps keeps 97.0% of its peak instead of 60.0%, and the divergence lands at 2.8e-06 instead of 5.4e-05. Costs about 6x more (588 ms per flow step at 128x128), so raise Solve Every with it

Flow Resolution

`flow_res` - Int - default **128** - 32 to 1024

The flow runs on its OWN square grid, spanning the same terrain. It does not need the sand resolution: the wind field is smooth on the scale of the obstacles that shape it, while the sand grid exists to resolve ripples. Keep this at or below 256 unless the walls in your scene are only a few cells wide -- the Poisson solve is the dominant cost and it scales with the cell count

Solve Every

`flow_every` - Int - default **10** - 1 to 200

Re-solve the flow once every N simulation steps and hold it in between. WHY THIS IS NOT 1: at 128x128 a Performance flow step costs about 93 ms and a Reference one about 588, against roughly 290 ms for an entire 1024x1024 sand step -- solving every step would dominate the simulation. The air reaches a quasi-steady state in a few tens of its own steps while the terrain changes far more slowly, so holding it is a real and deliberate approximation. Lower it if your obstacles MOVE during the run

Flow Influence

`flow_blend` - Float - default **0.5** - 0 to 1

Mix between the analytic shear and the solved one. 0 returns the analytic field EXACTLY (verified bit-identical), 1 uses the flow alone. IT DEFAULTS TO A BLEND FOR A PHYSICAL REASON, not as a compromise: the 2D flow is a PLAN VIEW and has no vertical dimension, so it cannot speed the wind up OVER a crest -- and that speed-up is what erodes a pile and drives dune migration. The analytic Jackson-Hunt model exists precisely for it, and knows nothing about walls. MEASURED with a sand source running 80 steps, the pile left sitting on the source: 3.99 at Influence 0, 4.33 at 0.25, 4.68 at 0.50, 4.98 at 1.00 -- push it to 1 and sand stops being swept off its own heap. Use 1.0 only when the walls are the whole point and the terrain is flat

Flow Drift

`flow_drift` - Float - default **0** - 0 to 4

Carry sand ACROSS the mean wind where the flow turns. HONEST SCOPE, because it explains why this is a separate control: the transport sweep takes ONE wind direction for the whole grid (that is the 3ds Max design), so the solved flow reaches the sand in two ways. Its SPEED sets the shear, which already gives you the canyon speed-up, the stagnation upwind of a rock and the dead wake behind it. This adds the part speed cannot express: material moving sideways where the wind bends. It shifts the sand layer by the DEVIATION of the local flow from the domain mean, so a uniform flow moves nothing at all, and the total is rescaled so no sand is created or lost (measured drift 0.0000%). 0 = off

Air Layer

`flow_air_layer` - Float - default **0** - 0 to 200

Depth of the moving air above the LOWEST ground, in solver units. This is what gives the flow a TOPOGRAPHY: a plan-view solve has no vertical dimension, so on its own it cannot speed the wind up over a crest -- and that speed-up is what strips a dune and keeps sand from piling on its own heap (measured: a sand source left 4.98 at full flow influence against 3.99 on the analytic wind). Give the air a thickness and a hill thins it, so by mass conservation the wind must accelerate: the same Venturi as a canyon, turned on its side. MEASURED: a hill filling 30% of the layer runs 1.18x the free stream over its crest, 60% runs 1.49x, 85% runs 2.05x. Set it a few times your dune height; 0 disables it and the flow goes back to an infinitely deep layer

Wall Height

`flow_wall_height` - Float - default **0** - 0 to 1000

Terrain standing more than this far above its own surroundings acts as a WALL for the wind. This is what a canyon carved into a heightmap needs: it owns no obstacle object, so without this the flow solver sees an empty domain and there is nothing to bounce off. It is 0 (off) by default because turning terrain into walls is a modelling choice rather than a fact -- this is a PLAN-VIEW flow, so a wall means "tall enough that the wind goes around it rather than over it", and only you know that height for your scene. IT NOW APPLIES TO OBSTACLE OBJECTS TOO, and that is the point: an

object used to become a wall the instant it stood above the sand AT ALL, so a 20 cm kerb blocked the wind exactly as a 20 m cliff did, and a fence drawn across the terrain stopped it as if it reached the sky. With a height set, only what is taller than this diverts the flow and the wind passes over the rest. A body the water has closed over is never a wall, whatever its height -- the wind blows over the water surface, not through it. 0 = the old behaviour: every standing object blocks

Wall Scale

`flow_wall_scale` - Int - default **12** - 2 to 128

Over how many cells "above its surroundings" is measured for Wall Height. Roughly half the width of the landform you want treated as a wall: too small and a wall becomes its own surroundings so nothing qualifies, too large and gentle regional slopes start to count

Flow Paint Mode

`flow_paint_mode` - Enum - default **MULTIPLY

What the painted Flow Strength mask does to the solve

Option	Meaning
Multiply the Solve	The solve still decides WHERE the air goes; the paint only says where it should run faster or slower. Direction is untouched, so the field stays divergence free and the shear, the tracers, the haze and the precipitation shelter all keep agreeing with it. This is the one to use
Painted Only (no solve)	THROW THE SOLVE AWAY and use the painted value as the speed, along the global wind direction. Be clear about what this is: it is no longer a flow. It cannot bounce off a wall, because there is no momentum left to bounce -- an obstacle will not deflect it and a canyon will not accelerate it. It is for art direction, when you want to paint the exact corridor a shot needs, and it is honest about being a cheat
Painted Speed + Direction	Speed from Flow Strength AND direction from Flow Direction -- but ONLY WHERE YOU PAINTED. Every cell you did not touch keeps the solved flow, which is the whole difference from Painted Only: you overrule the air in one corridor and the rest of the scene still solves properly around it. Paint the direction with the Flow Direction brush -- the stroke DIRECTION is what gets written, so you draw the path the wind should take. Leave Project Painted Flow on and the result is a real flow rather than a drawing

Project Painted Flow

`flow_paint_project` - Bool - default **on

Push the painted field through the SAME pressure projection the flow solver applies to itself, so what you drew becomes a real, divergence-free flow. This matters more than it sounds: a hand-drawn field invents and destroys air, and the shear, the tracers, the haze and the precipitation shelter all read that as sand appearing and vanishing out of nowhere. Projected, the painted corridor also ACCELERATES through a gap on its own, because the mass has nowhere else to go -- which is exactly the behaviour no amount of painting can fake. Turn it off only when you want literally what you drew, divergence and all. Costs one pressure solve per step and does nothing outside Painted Speed + Direction

Free Slip Walls

`flow_free_slip` - Bool - default **on

Let the wind slide along a wall instead of sticking to it. Free slip is the right model at these scales -- one cell is metres across, so the real no-slip boundary layer is far below the grid, and it is what lets a canyon accelerate rather than stall. Turn it off for a thicker, earlier separation and a broader wake; that is a one-cell wall model, not a resolved boundary layer

Viscosity

`flow_viscosity` - Float - default **0** - 0 to 1

Explicit diffusion of momentum. 0 leaves only the scheme's own numerical viscosity, which is already substantial in Performance mode. Raise it to calm a flow that flickers between frames; too much and the wake stops shedding and

turns into a smooth blob

Multigrid Pressure

`flow_use_multigrid` - Bool - default **on

Performance mode only (Reference solves the pressure by conjugate gradient, so this has no effect there). Solve the pressure with a multigrid V-cycle instead of plain relaxation sweeps. WHY IT IS THE DEFAULT: a relaxation sweep moves information exactly ONE CELL, and the pressure around a large obstacle is a long-wavelength field, so sweeps spend nearly all their work crawling it across the grid. Multigrid solves the smooth part on a coarse grid where one cell is a long distance. MEASURED at 256x256: 4 V-cycles reach 4.6e-06 divergence in 210 ms, where 400 SOR sweeps need 1006 ms for 1.2e-06 and 120 sweeps give only 8.0e-05 in 314 ms -- about 4.8x faster at equal accuracy, and the gap grows with resolution. Turn it off only to compare

V-Cycles

`flow_mg_cycles` - Int - default **3** - 1 to 12

Performance mode only, and only with Multigrid Pressure on. How many multigrid V-cycles per pressure solve. Each one costs about the same and cuts the residual by roughly an order of magnitude, so this is a real quality/speed dial -- unlike CG Iterations, where too few is worse than useless

Pressure Sweeps

`flow_sor_iters` - Int - default **40** - 1 to 400

Performance mode only: how many red-black SOR sweeps the pressure solve gets. More sweeps means less residual divergence, i.e. flow that leaks less through walls, at a proportional cost. 40 measured a peak divergence of 5.4e-05 against a free stream of 1.0

CG Iterations

`flow_cg_iters` - Int - default **200** - 10 to 2000

Reference mode only: the CEILING on conjugate-gradient iterations per pressure solve. DO NOT TREAT THIS AS A QUALITY/SPEED DIAL -- lowering it makes Reference WORSE THAN PERFORMANCE while still costing more. MEASURED on a bluff-body scene, peak divergence against a free stream of 1.0: Performance at 40 sweeps gives 7.1e-05; Reference at 100 iterations gives 5.3e-04 (WORSE), at 150 it merely ties at 4.2e-05, and only at 200 does it pull ahead at 4.5e-06. The reason is that CG has to travel the whole domain before the pressure around a large obstacle is resolved, and a half-finished solve leaves exactly the long-range error that matters. If Reference is too slow, raise Solve Every or lower Flow Resolution -- do not cut this

CG Tolerance

`flow_cg_tol` - Float - default **0.0001** - 1e-08 to 0.1

Reference mode only: stop the pressure solve once the residual falls to this fraction of the right-hand side. Smaller is more strictly incompressible and slower. Checked every 10 iterations rather than every one, because each check is a GPU stall

3D Wind (EXPERIMENTAL)

`flow3d_enabled` - Bool - default **off

EXPERIMENTAL, and shipped that way on purpose: the physics is measured and gated, but the COST is not yet where it needs to be and the feature is being finished in stages. Leave it off for production work -- a scene that never switches it on behaves exactly as it did before, to the bit. Solve the wind in THREE dimensions, so it climbs over terrain and objects instead of only flowing around them. What it changes is the WIND SHADOW: the flat models lay a plane descending at a fixed 14 degrees behind every crest, which gives a shadow exactly 3.98 times the height for a dune brink, a rock, a wall and a building alike. Solved in 3D the shadow comes from momentum instead, and measured it runs 5 to 11 body heights and depends on the shape -- the band the literature reports for bluff bodies. It does NOT replace the analytic shear: the upwind shift of the stress peak, which is what makes a dune MIGRATE rather than grow in place, comes from the Jackson-Hunt kernel and is kept. COSTS REAL TIME -- see 3D Steps per Sand Step. Off by default, and a scene that leaves it off behaves exactly as before

3D Plan Resolution

`flow3d_res` - Int - default **256** - 32 to 2048

Plan-view grid the 3D wind is solved on, independent of the sand resolution. Memory and time both grow with its SQUARE times the layer count, so this is the main cost dial. 256 is a good working value; the solver refuses outright anything that would not leave the graphics card usable, and prints what would fit. IT IS ALSO THE ANSWER TO THE SQUARE ARTEFACTS, and that is worth saying plainly because two other dials look like they should be. The wake is SOLVED here and then carried onto the sand grid, so structure one cell of THIS grid wide is genuine output of the coarse solve -- not a resampling artefact, and nothing downstream can invent detail that was never computed. What matters is the RATIO to Solver Resolution. MEASURED as spectral power at the flow-cell frequency, against Flow 2D on the same scene: 16:1 reads 7.2x (Flow 2D 0.99), 8:1 reads 5.0x (0.90), 4:1 reads 3.0x (0.87) -- Flow 2D is flat because it never goes through this path. So at Solver 1024 the shipped 256 is 4:1, and 512 halves the squares again at four times the cost. Eddy Viscosity does NOT help: this is not a viscous phenomenon

3D Layers (0 = auto)

`flow3d_layers` - Int - default **0** - 0 to 512

How many cells the wind is stacked in vertically. 0 derives it from the tallest thing in the scene, which is what you want: MEASURED, the lid has to sit about FOUR obstacle heights up before it stops changing the answer -- at 1.5x the error is 9.3%, at 2x 4.0%, at 3x 1.3% and at 4x 0.57%. Setting it by hand also risks a number like 20, which factors as 4x5 and leaves the pressure solver a hierarchy two levels deep instead of four; that measured EIGHT times the work for a worse answer. The automatic value is always a multiple of 16 for exactly that reason. On 0 the depth GROWS WITH THE DUNES and never shrinks inside a run, with 25% of headroom each time it grows: rebuilding the 3-D solver throws away the wake it has spent hundreds of steps developing, and without that hysteresis a 400-step run rebuilt it 39 times, twice in the shrinking direction. Pin a value only if you need the memory to be constant -- and pick a multiple of 16

3D Steps per Sand Step

`flow3d_steps` - Int - default **2** - 1 to 16

How many wind steps are marched for every sand step. THE WIND IS NEVER RE-SOLVED FROM SCRATCH, because there is nothing to converge to: a real wake SHEDS, and measured over 800 steps its length oscillated between 4.5 and 15 body heights and never settled. The time AVERAGE does settle, so the solver runs alongside the sand and the shadow is read from its running average. Measured cost: about 26 ms per 3D step at 512 plan resolution against a sand step of roughly 81 ms, so 2 is about +64% on the whole simulation. Raise it if the wind lags a fast-changing terrain; lower it if the frame rate matters more than the lag

3D Warm-up Steps

`flow3d_warmup` - Int - default **200** - 0 to 2000

Steps marched ONCE, at the start, to let the wake form before the sand starts reading it. A wake is a HISTORY -- it does not exist in the first frame and cannot be solved for -- and measured it takes on the order of 200 steps to develop. Too few and the first part of the run uses a shadow that is still growing. SPREAD ACROSS THE FIRST FEW SAND STEPS, not run in one: doing all of them in a single step froze Blender for 2.5 s at 3D Plan Resolution 256 and 48 SECONDS at 512, which reads as a hang because the UI cannot repaint. The total number of steps is exactly this value either way -- only when they run has changed

3D Wall Friction

`flow3d_no_slip` - Float - default **0.3** - 0 to 1

How much tangential wind a surface takes away per step. THIS IS THE KNOB THAT MAKES A WAKE AT ALL: a frictionless wall generates no vorticity, so at 0 the solve is potential flow and has NO wake whatever else is set -- measured, zero reversed cells behind a wall at every height. 0.3 is the calibrated value; it puts the reattachment length at about 10 body heights, inside the 5-15 the literature reports, while keeping the vertical motion over dunes

3D Eddy Viscosity

`flow3d_eddy` - Float - default **0** - 0 to 3

Extra turbulent smoothing. DEFAULTS TO ZERO ON PURPOSE, and that is a measurement rather than an omission: every non-zero value tried cost more vertical motion than it bought wake. At 0.05 the air climbing a dune drops from 32% of the horizontal wind to 11% for eight percent more shadow length. The advection scheme already supplies enough dissipation to stay stable -- 800 steps stayed bounded. Raise it only if a scene shows grid-scale noise

3D Shadow Threshold

`flow3d_frac` - Float - default **0.35** - 0.05 to 0.9

How slow the near-surface wind has to be for a cell to count as sheltered, as a fraction of the undisturbed wind AT THE SAME HEIGHT. Comparing against the free stream instead would mark the whole scene sheltered, because the lowest cell of every column is slowed by the ground -- measured, that read 16384 cells of 16384 on FLAT terrain. Lower values give longer shadows

Show Wind Field

`wind_show` - Bool - default **off

Paint the SOLVED wind onto the terrain as a vertex colour, and switch the viewport to vertex-colour shading. Only the Flow 2D model has a field worth showing -- the analytic wind is one global direction with a shear magnitude, so a direction map of it would be a flat colour. With the analytic model selected this draws nothing and says so

Wind Display

`wind_show_mode` - Enum - default **SPEED

What the wind overlay encodes. Speed answers "where is the air fast", Direction answers "where does it turn" -- and Direction is the one that makes a wall bounce visible, because a deflection is a change of heading and barely changes the speed at all

Option	Meaning
Speed	WHAT THE COLOURS MEAN: dark BLUE = still or sheltered air, GREY = roughly the free-stream speed, RED = clearly faster than the free stream (a canyon jet, a gap, a corner). Normalised by the field MEAN, not its maximum, so one very fast cell in a narrow gap cannot push the whole rest of the terrain to black -- which also means the scale is RELATIVE to this scene, not absolute
Direction	WHAT THE COLOURS MEAN: hue is the compass heading of the flow, going RED (blowing +X) - YELLOW/GREEN (+Y) - CYAN (-X) - BLUE/MAGENTA (-Y) and back. A uniform wind is therefore ONE FLAT COLOUR, and every change of colour is the flow turning: this is the mode that shows a wall bouncing the wind, and the one where a recirculation appears as the opposite hue to its surroundings
Direction + Speed	Hue is the heading (see Direction) and BRIGHTNESS is the speed (see Speed): dark means slow whatever way it is going. The whole field at once, at the cost of being harder to read precisely

Wind Tracers

`wind_tracers` - Bool - default **off

Spawn massless marker particles that ride the SOLVED wind, so the flow is visible as motion rather than as a colour. Different job from the haze: a haze grain is SAND -- it appears where the wind is actually lifting material and dies in the lee, so its density is a statement about erosion. A tracer carries nothing and is seeded UNIFORMLY, including in the dead air behind a rock, which is exactly where you want to watch a recirculation turn. Needs the Flow 2D wind model; with the analytic model there is no field to trace and this does nothing

Tracer Count

`wind_tracer_count` - Int - default **3102** - 0 to 200000

How many markers. This is a viewport aid, so keep it far below the haze count -- a few thousand is already more than the eye can follow, and every one costs a bilinear sample and an instance

Tracer Life

`wind_tracer_life` - Float - default **60** - 2 to 2000

Steps a marker lives before it is recycled to a fresh random spot. WHY THEY MUST EXPIRE: without a life the whole pool drains into the stagnation zones and the recirculation you wanted to see ends up drawn by a handful of survivors. Short lives keep the picture refreshing; long ones draw longer streamlines

Tracer Speed

`wind_tracer_step` - Float - default **1** - 0.05 to 8

How far a marker travels per simulation step, as a multiple of the true flow velocity. This is a viewing speed, not physics: raise it to read a slow field at a glance, lower it if the markers jump so far per frame that the eye cannot track them

Tracer Size

`wind_tracer_size` - Float - default **1.02** - 0.001 to 50

Radius of a marker in WORLD UNITS. Markers fade in and out over their life, so this is the size at mid-life, not at spawn. IT SHIPPED AT 0.06 UNTIL 1.3.12, AND THAT IS WHY THE ARROW LOOKED LIKE A BLOB: on a 200 m terrain a 6 cm marker is a couple of pixels, and a 12-sided shaft-and-head at two pixels is a dot whatever shape it really is. MEASURED -- the shape picker does reach the render (2.07% of the frame changes between Blob and Arrow) and at 0.35 on the same scene the arrows are unmistakable. Because this is in world units it does NOT follow the terrain size: on a much larger or smaller domain it is the first thing to retune, and the soft maximum was raised to 6 to make that possible

Tracer Streak

`wind_tracer_streak` - Float - default **4.27** - 0 to 20

Stretch each marker along its own velocity by this multiple, so a fast marker draws a streak and a stalled one a dot. 0 gives round markers everywhere, which reads the direction less well but the position more precisely

Tracer Shape

`wind_tracer_shape` - Enum - default **ARROW

What a single marker looks like. This is a LOOK, not a physics setting: both shapes sit at the same place, are the same size and are aimed the same way -- only the silhouette differs

Option	Meaning
Blob	A stretched sphere: fast markers draw streaks, stalled ones dots. Reads the SPEED of the flow best, because the eye compares lengths easily -- but it is symmetric, so it cannot show which end is the front. Use it when you are reading how hard the wind is blowing rather than where it goes. This is what every build before 1.2.96 drew, unchanged
Arrow	A real arrow -- shaft plus head -- with the point downwind. Reads the DIRECTION at a glance, including the two places a blob is genuinely ambiguous: a recirculation behind an obstacle, where the air comes back UPWIND and a symmetric marker looks identical either way, and a converging flow, where two streams meeting head-on look the same as one stream passing through. It is exactly as LONG as the blob it replaces, but slimmer: an arrow is scaled by its own length so head, shaft and total stay in proportion at every speed, which is what stops a slow marker turning into a thumbtack and a fast one into a needle. The price is that length no longer reads as speed -- that is what the blob is for. Pair it with Tracer Ground Tilt so the arrows lie along the slope instead of stabbing through it

Arrow Thickness

`wind_tracer_thickness` - Float - default **0.55** - 0.05 to 5

How fat the ARROW marker is, as a multiple of its reference build. It scales the shaft and the head TOGETHER, so the head keeps its 3:1 lead over the shaft and the silhouette stays an arrow at every setting -- a single radius would turn into a thumbtack at one end of the range and a needle at the other. 1.0 is the shape 1.2.96 introduced; the shipped 0.55 is slimmer, because at 1.0 a field of markers reads as a field of blobs from any distance that shows more than a

few of them, which is exactly the report this setting exists to answer. Length is NOT affected: that is Tracer Size and Tracer Streak. No effect on the Blob shape

Tracer Colour

`wind_tracer_colour` - FloatVector - default **0.25, 0.55, 1.0, 1.0** - 0 to 1

Colour of the wind tracers, with alpha. They default to BLUE rather than sand: a tracer is a measuring instrument, not sand, and one that looks like blowing grains is easy to mistake for the haze it sits next to. Alpha 1.0 is also deliberate -- transparent particles cost the viewport a sorted blend pass per frame, which is most of what makes a few thousand of them feel slow

Tracer Ground Tilt

`wind_tracer_tilt` - Float - default **0.48** - 0 to 3

Lays the tracer arrows ALONG the ground instead of keeping them dead level. They were level because the flow model is two-dimensional: the markers were handed a velocity with X and Y filled in and Z left at exactly zero, so the node group that aims them had nothing vertical to point at, and an arrow crossing a slip face pointed straight through it. This adds the vertical part the ground implies -- the slope measured along each marker's own direction of travel, times its speed, which is what air following a hill actually does. 1.0 is the true slope; below that reads as a flatter, calmer flow and above exaggerates it for legibility. 0 is the level marker of every earlier build, exactly

Tracer Object

`wind_tracer_object` - Pointer

Carrier mesh the tracers are written to (created automatically). One loose vertex per marker; the Geometry Nodes modifier on it instances the visible particle

---

Objects / Masks

How the things in your scene talk to the sand. An object can block the wind, feed sand into the scene, hold the ground down, press its shape into the surface, or steer the wind -- each of those is a different slot, because they are genuinely different jobs.

When you need it: The moment your shot contains anything that is not terrain: a building, a rock, a vehicle, a fence, a body.

Objects Leave Tracks

`use_imprint_objects` - Bool - default **off

Let real objects press themselves into the surface -- boots, wheels, paws, a sled, a landing skid -- instead of authoring one rut along a curve by hand. Only the part of an object that is actually TOUCHING the ground stamps (see Contact Gap), so an animated walk leaves prints where a foot is down and nothing where it is in the air. What the mark LOOKS like depends on what is under it: see the two depth pairs below

Track Object

`imprint_object` - Pointer

One object that leaves marks. Animate it and it leaves a TRAIL, one press per frame, which the simulation then reworks like any other disturbance

Track Collection

`imprint_collection` - Pointer

A whole collection of objects that leave marks -- a herd, a set of wheels, both boots of a rig. They are merged into one footprint, so overlapping feet give one outline rather than a seam

Contact Gap (m)

`imprint_contact` - Float - default **0.15** - 0.001 to 1000

How close an object's UNDERSIDE has to be to the ground to count as touching it. This is what turns a moving object into a track rather than a stripe: a raised foot is further than this and leaves nothing. Too large and the whole body prints even in the air; too small and a foot that is a centimetre off never marks. Roughly the depth you expect the print to be is a good start

Edge Softness

`imprint_smooth` - Float - default **0** - 0 to 64

Feather the footprint by this many GRID CELLS. 0 is a hard outline, which at a coarse solver resolution reads as a staircase; one or two cells softens it without losing the shape

Depth in Snow

`imprint_depth_snow` - Float - default **0.25** - 0 to 100

How deep the print goes where the ground is SNOW (solver height units). Powder gives way further than sand does, and the print comes out of the powder itself -- the ground under it does not move until the snow runs out

Depth in Sand

`imprint_depth_sand` - Float - default **0.12** - 0 to 100

How deep the print goes where the ground is SAND. Shallower than snow on purpose: dry sand carries load at the angle of repose and then collapses, so a deep hole does not survive the next step of the simulation anyway

Rim in Snow

`imprint_berm_snow` - Float - default **0.1** - 0 to 100

How high the rim of displaced material stands around a print in SNOW. Cohesive snow holds a sharp lip -- that lip is most of what makes a footprint read as a footprint

Rim in Sand

`imprint_berm_sand` - Float - default **0.03** - 0 to 100

How high the rim stands around a print in SAND. Low, because dry sand cannot hold one: the avalanche relaxation flattens anything past the repose angle within a few steps, which is the correct behaviour and not something to fight

Imprint Every Frame

`imprint_every_frame` - Bool - default **on

Press the objects in on every simulated frame, so an ANIMATED object leaves a trail behind it. Off, only the Imprint Objects Now button stamps -- which is what you want for a single pose, and what you want while you are still placing things

Max Float Gap (m)

`mask_max_gap` - Float - default **0** - 0 to 1000

Ignore the part of an object that HOVERS further than this above the terrain. The footprint of every object slot is found by firing rays straight down, and a ray does not care about the gap underneath -- so a rock parked ten metres in the air still stamped its full outline, casting a wind shadow, blocking erosion and seeding sand from something that is not touching the ground. With a limit set, the object's UNDERSIDE is measured against the terrain surface and cells that hover higher are dropped, which is also what lets a bridge, an overhang or a canopy stop imprinting the ground beneath it while its legs still do. Applies to Obstacles, Sand Source and Wind Gain. 0 = OFF and the footprint is exactly what it always was; that is the default, because turning it on changes existing scenes

Use Obstacle

`use_obstacle_object` - Bool - default **off

Turn the Obstacle object into a solid wall for the sim: it casts a wind shadow, blocks sand transport and dunes bank up against it (add Echo Strength for an upwind echo dune). Toggle or swap the object MID-simulation -- the mask refreshes immediately. See Fill Obstacle Footprint for the cosmetic well it leaves

Obstacle

`obstacle_object` - Pointer

Mesh sampled as a solid obstacle heightfield. Can be assigned or swapped MID-simulation -- the mask refreshes immediately

Obstacle Collection

`obstacle_collection` - Pointer

All meshes in this collection act as obstacles TOGETHER with the single Obstacle object (union of solids) -- drop a whole rock field or fence in one go. Updates live when assigned or when its objects move

Obstacle Smoothing

`obstacle_smooth` - Float - default **0** - 0 to 64

Feather the RIM of the blocking objects -- the group that cuts the vertical scarps into a dune field by this many GRID CELLS, so the stamp ramps into the terrain instead of leaving a vertical step the simulation then has to live with. The flat top of an object is unaffected -- only the edge slopes -- and nothing is added, so the object never grows. Applied after the whole group is merged, so touching objects in one collection get a single outline rather than a seam where they overlap. Each group has its own value; 0 leaves the footprint exactly as it was

Echo Strength

`echo_strength` - Float - default **0** - 0 to 1

Build an ECHO DUNE on the UPWIND side of obstacles: the wind slows as it approaches the wall, so sand drops out and a separate ridge forms a short distance in front (a classic desert feature). 0 = off; at 1.0 it can pile a steep bank directly against the obstacle wall -- combine with Settle Slopes to keep the bank at a natural angle. Needs an Obstacle object; Echo Distance sets how far upwind the effect reaches

Echo Distance

`echo_dist` - Int - default **6** - 1 to 64

How far UPWIND of an obstacle (in solver cells) the echo-dune slowdown reaches: larger = the echo ridge forms further from the wall and the gap between ridge and obstacle is wider; small = sand banks up close to the wall. Only matters with Echo Strength above 0

Fill Obstacle Footprint (render)

`obstacle_fill_render` - Bool - default **off

RENDER-ONLY: raise the terrain surface INSIDE solid-obstacle footprints up to the surrounding sand level. Physically sand never enters the obstacle, so the mesh there stays at its start height -- as dunes bank up around a tall object that leaves a deep pixel-edged well and an ugly jagged curtain climbing the object side. The object itself hides the filled cells, so this is purely cosmetic; physics and exports are untouched

Use Bedrock

`use_bedrock_object` - Bool - default **off

Use the Bedrock object as the non-erodible FLOOR under the sand: the wind can strip sand down to it but never below -- exposed patches read as bare rock pavement. Toggle or swap mid-simulation; the mask refreshes immediately

Bedrock

`bedrock_object` - Pointer

Mesh sampled as the non-erodible floor. Can be assigned mid-simulation

Bedrock Collection

`bedrock_collection` - Pointer

All meshes in this collection form the non-erodible floor together with the single Bedrock object (union). Updates live

Bedrock Smoothing

`bedrock_smooth` - Float - default **0** - 0 to 64

Feather the RIM of the rock objects stamped in as the floor by this many GRID CELLS, so the stamp ramps into the terrain instead of leaving a vertical step the simulation then has to live with. The flat top of an object is unaffected -- only the edge slopes -- and nothing is added, so the object never grows. Applied after the whole group is merged, so touching objects in one collection get a single outline rather than a seam where they overlap. Each group has its own value; 0 leaves the footprint exactly as it was

Use Snow Source Object

`use_snow_source_object` - Bool - default **off

Take the snow emitter footprint from Snow Source Object and Snow Source Collection. Off leaves only what you painted with the Snow Source brush channel

Snow Source Object

`snow_source_object` - Pointer

Object whose footprint SNOWS. Everything under it gets Snow Source Rate of fresh powder every step, on top of whatever the global Snowfall is doing -- so you can have a drift building under a roof edge or a cornice feeding one slope while the rest of the scene stays clear. Add more emitters with Snow Source Collection

Snow Source Collection

`snow_source_collection` - Pointer

Every object in this collection snows as well, exactly like Snow Source Object. Use it when the emitter is a group of objects rather than one

Snow Source Smooth

`snow_source_smooth` - Float - default **0** - 0 to 32

Feather the edge of the snow emitter footprint, in cells. 0 is a hard silhouette; raise it so the drift fades out instead of ending on a straight line

Snow Source Rate

`snow_source_rate` - Float - default **0** - 0 to 5

Metres of fresh powder the snow emitters drop each step. ADDED to the global Snowfall, never instead of it: at 0 the whole feature is skipped and the scene runs exactly as it did before. Drives the painted Snow Source channel as well as the objects

Use Source

`use_source_object` - Bool - default **off

Use the Source object footprint as a sand EMITTER: fresh sand appears there every step at the Source Rate -- a beach, wash or supply corridor feeding the field. Toggle or move mid-simulation (set ObsEvery to track a moving source). With Open X boundaries this is what keeps the field from emptying

Source

`source_object` - Pointer

Mesh sampled as a sand source mask. Can be assigned or moved mid-simulation (set ObsEvery to track a MOVING source automatically)

Source Collection

`source_collection` - Pointer

All meshes in this collection emit sand together with the single Source object (union of footprints). Updates live

Sand Source Smoothing

`source_smooth` - Float - default **0** - 0 to 64

Feather the RIM of the emitter footprint by this many GRID CELLS, so the stamp ramps into the terrain instead of leaving a vertical step the simulation then has to live with. The flat top of an object is unaffected -- only the edge slopes -- and nothing is added, so the object never grows. Applied after the whole group is merged, so touching objects in one collection get a single outline rather than a seam where they overlap. Each group has its own value; 0 leaves the footprint exactly as it was

Use Source Map

`use_source_map` - Bool - default **off

Drive the sand SOURCE from an image: white = sand arrives here, black = it does not. A third layer next to the Source object and the brush -- the three take the MAXIMUM, so turning the map on never silences what you painted. Point it at an image SEQUENCE and the supply MOVES over time, which is the animated map this solver can honour: a source is an INPUT to the simulation, while an animated Sand Map would overwrite the terrain the wind just built

Source Map

`source_map_image` - Pointer

Image whose brightness says where sand arrives. Load an image SEQUENCE (Source = Image Sequence) to animate it -- the frame handler re-reads the matching numbered file every evaluated frame

Source Map Strength

`source_map_strength` - Float - default **1** - 0 to 1

Scales the map before it joins the other source layers. At 0 the map contributes nothing and the object/brush layers are untouched

Follow Frame

`source_map_follow_frame` - Bool - default **on

For an image SEQUENCE, read the file whose number matches the current frame -- this is what makes the source map animated. Off freezes it on whatever frame the image datablock holds

Source Rate

`source_rate` - Float - default **0** - -10 to 10

Sand (or snow) EMITTED each solver step inside the Source mask / object footprint -- a continuous feed, like a beach supplying a coastal dune field. Needs a Source object or painted Source mask. Fast feeding over-steepens the pile: enable Settle Slopes Every N to hold it at the repose angle cheaply. NEGATIVE VALUES MAKE IT A SINK: the painted area REMOVES sand each step instead of supplying it, down to the bedrock floor and no further. Not physical -- nothing in a desert deletes sand at a fixed rate -- but it is the artistic control a blocker cannot give you, because a blocker only stops sand ARRIVING while a sink keeps an area clear while the simulation runs. The slider stops at 0; type a negative value or ctrl-drag past it

Painted Source Gain

`source_paint_gain` - Float - default **1** - 0 to 20

Multiplies the PAINTED sand-source mask after the fact, so a stroke that turned out too weak can be turned up without repainting it. 1 is the mask exactly as you painted it. It scales the painted layer ONLY -- a Source object footprint keeps its own strength, so turning this up cannot quietly change a setup that was driven by an object. The emitted amount is this times Source Rate, so either knob gets you there; this one keeps the SHAPE you painted and only changes how much comes out of it

Source Hold

`source_hold` - Float - default **0** - 0 to 1

Keeps the sand a source emits WHERE IT LANDS instead of letting the wind carry it off immediately. 0 = the shipped behaviour, the pile is fair game the moment it appears. 1 = the source footprint does not erode at all, so the pile only grows. Mid values let it bleed downwind slowly. This is the Sand Source equivalent of Place Hold, which confines the INITIAL placement -- they are separate masks and separate settings, so holding a source does not confine the start sand and the other way round. MEASURED, AND SMALL FOR NOW: it holds the footprint against EROSION, which moves the sand kept inside it by about 0.1%, because a source pile mostly leaves by SLUMPING past the repose angle rather than by being eroded -- and the avalanche does not read this yet. Turn it up if you want the effect it has; do not expect it to pin a tall pile in place

Use Wind Gain

`use_wind_gain_object` - Bool - default **off

Use the Wind Gain object footprint to locally BOOST the wind (strength set by the Wind Gain slider): scour corridors, active funnels between rocks, wind-blasted passes. Toggle or swap mid-simulation; the mask refreshes immediately

Wind Gain

`wind_gain_object` - Pointer

Mesh sampled as a local wind-strength boost mask. Can be assigned mid-simulation

Wind Gain Collection

`wind_gain_collection` - Pointer

All meshes in this collection boost the local wind together with the single Wind Gain object (union of footprints). Updates live

Wind Gain Smoothing

`wind_gain_smooth` - Float - default **0** - 0 to 64

Feather the RIM of the wind-gain footprint by this many GRID CELLS, so the stamp ramps into the terrain instead of leaving a vertical step the simulation then has to live with. The flat top of an object is unaffected -- only the edge slopes -- and nothing is added, so the object never grows. Applied after the whole group is merged, so touching objects in one collection get a single outline rather than a seam where they overlap. Each group has its own value; 0 leaves the footprint exactly as it was

Wind Gain

`wind_gain_strength` - Float - default **0** - 0 to 2

How strongly the Wind Gain object/attribute mask boosts the LOCAL wind (0 = mask ignored, 1 = up to double shear inside the mask, 2 = triple). Use it to carve wind corridors, scour a pass clean or keep a valley active while the rest of the field calms down. Needs a Wind Gain object or painted Wind Gain mask to have any effect

Use Erodibility Attr

`use_erodibility_attribute` - Bool - default **off

Drive the erodibility mask from a vertex attribute / color layer on the terrain mesh (name below): 0 = protected ground, 1 = erodes freely. Paint it in Vertex Paint or with the DunesForge Erodibility brush -- pin roads, ruins or set dressing in place while the sand moves around them

Erodibility Attr

`erodibility_attribute` - String - default **tydunes_erodibility

FLOAT or color attribute name; 0 protects, 1 erodes freely

Use Erode Objects

`use_erode_objects` - Bool - default **off

Protect the ground under MANY objects at once: every mesh in the Erode Collection (or the name list) stamps its footprint into the erodibility mask -- e.g. a whole set of ruins or props that must not be undermined. ErodeIn sets the protection level, ErodeFall feathers the edge

Erode Collection

`erode_collection` - Pointer

All mesh objects in this collection contribute erode footprints

Erode Obj Names

`erode_object_names` - String - default

Comma-separated list of scene object names whose footprints are stamped into the erode mask (used with Use Erode Objects; the Erode Collection is the easier way to manage many objects)

ErodeIn

`erode_in` - Float - default **0** - 0 to 1

Erodibility INSIDE the erode-object footprints: 0 = fully protected (the ground under the objects never erodes), 1 = no protection at all. Mid values slow erosion there instead of stopping it -- half-buried set dressing that sinks slowly

ErodeFall

`erode_fall` - Float - default **0** - 0 to 100000

Feather width in WORLD units around each erode-object footprint: protection fades from ErodeIn at the object edge back to normal erodibility over this distance. 0 = hard edge, which can leave a visible pedestal outline as the surroundings deflate

ObsEvery

`obs_refresh_every` - Int - default **0** - 0 to 10000

Re-rasterise ALL object masks (obstacle, source, bedrock, wind gain, erode objects) every N evaluated frames -- required for ANIMATED/moving objects, e.g. a source that follows a path or a vehicle acting as a moving obstacle. 0 = masks are static (cheapest; assignment changes still refresh instantly)

Use No-Deposit Objects

`use_nodeposit_objects` - Bool - default **off

Stop sand SETTLING under the objects in the No-Deposit Collection -- a slab, a road, a swept rock pavement, or any blocker that must stay clear. THIS IS NOT THE SAME AS PROTECTING THEM. Erode Objects make ground hard to pick up, and a patch that resists erosion but accepts deposition MUST grow a mound -- measured, +1.4543 m over 200 steps. This one is the other way round: nothing new lands, and whatever sand is already there is picked up and carried off as usual, so the footprint clears itself. The avalanche obeys it too, so a neighbouring dune cannot slump onto the footprint either

No-Deposit Collection

`nodeposit_collection` - Pointer

All mesh objects in this collection block sand from settling on their footprint

No-Deposit Obj Names

`nodeposit_object_names` - String - default

Comma-separated scene object names whose footprints block deposition (used with Use No-Deposit Objects; the collection is the easier way to manage many objects)

BlockIn

`nodeposit_in` - Float - default **0** - 0 to 1

How much sand may still settle INSIDE the footprints: 0 = none at all, 1 = no blocking. Mid values let a footprint silt up slowly instead of staying bare -- measured on a blocked patch after 60 steps, sand depth 0.0026 at 0, 0.0133 at 0.1, 0.0321 at 0.5 and 0.0481 at 1 (which is exactly the unblocked scene)

BlockFall

`nodeposit_fall` - Float - default **0** - 0 to 100000

Feather width in WORLD units around each no-deposit footprint: the block fades from BlockIn at the object edge back to free deposition over this distance, so sand is let through gradually at the rim instead of stopping at a hard line. 0 = hard edge

Use Placement Attr

`use_placement_attribute` - Bool - default **off

Multiply the initial sand placement by a painted vertex attribute / color layer (name below): bright = full start sand, dark = bare. Combines WITH the procedural placement mode, so you can hand-paint exceptions on top of it. Frame-0 only

Placement Attr

`placement_attribute` - String - default **tydunes_placement

FLOAT or color attribute name used as initial placement weight

Use Source Attr

`use_source_attribute` - Bool - default **off

Drive the sand-source mask from a vertex attribute / color layer on the terrain (name below): bright areas emit fresh sand every step at the Source Rate. A painted alternative to the Source object -- ideal for irregular supply zones like a shoreline

Source Attr

`source_attribute` - String - default **tydunes_source

FLOAT or color attribute name used as source emission weight

Use Wind Gain Attr

`use_wind_gain_attribute` - Bool - default **off

Drive the wind-gain mask from a vertex attribute / color layer on the terrain (name below): bright areas get locally stronger wind, scaled by the Wind Gain slider. A painted alternative to the Wind Gain object for freeform wind corridors

Wind Gain Attr

`wind_gain_attribute` - String - default **tydunes_wind_gain

FLOAT or color attribute name; 0 neutral/low, 1 strong boost

SHOW Erode Mask

`mask_show` - Bool - default **off

DEBUG overlay: tint the terrain red where the ACTIVE erodibility mask protects the ground (painted masks, erode objects, placement-as-erodibility all combined). Quick way to verify a mask actually landed where you think. Viewport only -- no effect on the sim

Mask Export Dir

`mask_export_dir` - String - default **//tydunes_masks

Folder where Export All Masks writes every simulated field -- sand depth, moisture, frost, hardness, vegetation, wind flow, snow layers, grain size and the rest -- one image per mask plus a JSON listing each map's range. // means next to the .blend file

Mask Format

`mask_export_format` - Enum - default **EXR

File format for Export All Masks

Option	Meaning
EXR (32-bit float)	Real values, signed and unbounded. This is the one to use if you are building your own material: height stays in metres, temperature keeps its sign and flow keeps its direction. A 16-bit PNG cannot hold any of that without rescaling
PNG (16-bit)	Smaller and openable anywhere, but the format is 0..1 only -- so anything outside that range has to be normalised, and the scale lives in the exported JSON instead of in the file

Raw Values

`mask_export_raw` - Bool - default **on

Write the numbers the solver actually holds instead of stretching each map to 0..1. NORMALISING THROWS THE SCALE AWAY: moisture and height both come back as 0..1 and nothing in the file says whether 0.5 means half-wet or six metres. Either way the exported tydunes_masks.json carries every map's min, max and mean, so a normalised map can still be rescaled -- value * (max - min) + min. Turn this OFF only when you want maps to LOOK at rather than compute with (PNG cannot hold negatives at all)

Physics

The material model: how steep a slope loose grains hold, how they avalanche when they cannot, how much the wind has to do before anything moves at all.

When you need it: When the sand behaves but does not behave like YOUR material -- damp sand, coarse gravel, light snow. Also where you go to make avalanches happen.

Physics Preset

`physics_preset` - Enum - default **EARTH_TUNED

Ready-made sets of the PHYSICAL numbers -- threshold, saturation length, repose and separation angles, and the planetary constants -- with the source for each in its own description. They touch only the physics; the look of a scene (dune scale, supply, vegetation, shaders) is left to the sand presets. Pick one and press Apply Physics

Option	Meaning
Earth - as shipped	The tuned Earth numbers every preset and demo was built on: threshold 0.05, repose 34 deg, separation 14 deg. MEASURED: a threshold at 0.05 of the reference shear means the wind sits at u*/u*t = 4.47 and 100% of the field is above threshold at all times -- a permanent gale. That is a deliberate tuning choice (sand creeps everywhere, which reads well), not a physical wind. Use this to reproduce existing work
Earth - literature threshold	The same Earth, with the threshold raised to 0.30 so the wind sits at u*/u*t = 1.83. Sand seas work between about 1.2 and 2 (Fryberger & Dean drift-potential surveys), so transport becomes INTERMITTENT: crests and exposed flanks move, hollows do not. Slower and patchier than the shipped tuning, and closer to a real desert. Raise Steps/Frame to compensate
Earth - sand storm	A strong wind over the literature threshold: Wind Strength 2.0 against threshold 0.20 (u*/u*t = 2.24). MEASURED at Wind Strength 2.0 over 20 steps: it builds 8.01 m of relief against 4.91 m at 1.0, and shifts 1.31 m of sand per cell against 0.54. Note this dial did nothing at all before 1.2.90
Earth - damp coastal sand	A beach or a sabkha: the water table on with a capillary fringe, and wet sand holding hard against the wind (Hold 0.9). MEASURED: wet ground erodes at 0.82 of the dry rate at moisture 0.8, and that ratio does NOT fall as the wind rises -- the model couples moisture as an erodibility multiplier, not as a raised threshold, so wet sand can never be blown clear. Real damp sand has a threshold a strong enough gust does overcome

Option	Meaning
Mars	Thin CO2 at 0.020 kg/m3 and basaltic sand near 100 um. DERIVED, not typed in: saturation length x26.3, threshold x9.9, flux x0.044. That first number is the headline -- Martian dunes are huge because the drag length scales with rho_grain/rho_air (Andreotti & Claudin), and the last is why they crawl
Titan	A cold dense nitrogen atmosphere at 5.4 kg/m3 -- four times Earth -- and sand that is organic haze aggregate at about 800 kg/m3. DERIVED: saturation length x0.067, threshold x0.009, flux x32.7. Titan's dunes are enormous, but NOT because of this length: it comes from supply and the wind regime, so raise Dune Scale yourself rather than expecting the planet to do it
Venus	A 65 kg/m3 near-supercritical CO2 atmosphere. DERIVED: saturation length x0.020, threshold x0.018, flux x59.9 -- transport starts at a crawl of a wind and the pattern is tiny and dense. The Magellan radar "microdunes" are the reference
Snow - cold and dry	Deep winter at -12 C: powder that does not bond, so the wind moves it freely and builds drifts and cornices. Slow sintering (0.004), no melt, and a snow repose offset of +8 deg because a cohesive slab bridges its own weight well past the sand angle. MEASURED: crust grows +0.014 over 25 steps at -25 C
Snow - near melting	Spring snow at -1 C: it welds. MEASURED, same 25 steps, the crust gains +0.433 against +0.014 in deep cold -- thirty times faster, which is what the literature describes as snow near the melting point setting into a slab in hours rather than weeks. Melt is on the temperature model at 0.004 m per degree per step, the standard degree-day figure for snow (3-6 mm/C/day)

Dune Size (Ls)

`ls` - Float - default **3** - 0.5 to 50

Saturation length = DUNE SIZE, in solver CELLS (3ds Max default 3): the distance the sand flux needs to saturate, which sets the dune wavelength (~5-10x this value). Dune Scale and Res-Independent Dune Size multiply it under the hood, so at high resolution prefer those over raw Ls

Shear Smooth

`smooth` - Float - default **0.4** - 0 to 20

FFT shear kernel smoothing width (sigma = smooth x cell size). 3ds Max uses 0.4 for crisp dunes; large values (3+) blur the shear and make the dunes soft and blobby. READ THE UNIT AGAIN, BECAUSE IT HAS A CONSEQUENCE: sigma is in CELLS, so it is a different PHYSICAL length at every Solve Res, and changing the resolution therefore changes the result slightly even though nothing physical moved. That is deliberate -- this is a regulariser against the Nyquist limit and it SHOULD follow the cell (at Nyquist it damps to 45%). MEASURED on a 1 m x 10 m bump in a 140 m domain from 128 to 1024 cells: the peak stress spreads 0.56%, and freezing sigma in METRES instead brings that to 0.18% -- so about 69% of what little resolution-dependence the shear has comes from this knob. Nothing to fix, but worth knowing before you blame a re-render on the solver

Two-Scale

`two_scale` - Bool - default **off

Grow the dune field in TWO stages: first big primary dunes (Ls Big), then after Scale Switch steps the solver drops to Ls Fine so smaller secondary dunes develop on and between the big ones -- a natural hierarchy of scales instead of one uniform wavelength. Let the run pass the Scale Switch step or you only ever see the big phase

Ls Big

`ls_big` - Float - default **5** - 0.5 to 80

Saturation length (dune size, in solver cells) of the FIRST phase of the Two-Scale schedule -- the big primary dunes formed before Scale Switch. Only used with Two-Scale enabled

Ls Fine

`ls_fine` - Float - default **2.5** - 0.5 to 80

Saturation length (solver cells) of the SECOND Two-Scale phase -- the finer secondary dunes grown after Scale Switch steps. Only used with Two-Scale enabled

Scale Switch

`scale_switch` - Int - default **1500** - 1 to 1e+06

Solver step at which Two-Scale switches from Ls Big to Ls Fine. Earlier switch = the big dunes are less developed when the fine detail starts; later = mature primary dunes first. Compare with Steps/Frame to know which timeline frame the switch lands on (step = frame x Steps/Frame)

Angle of Repose

`repose_deg` - Float - default **34** - 20 to 45

Steepest slope loose material can hold, in DEGREES -- above it sand avalanches down until the face relaxes back to this angle. Dry sand ~34 (default); lower = softer, rounder dunes; higher = steeper, more dramatic slip faces. Snow uses it too, modified by crust hardness

Separation Angle

`sep_angle` - Float - default **14** - 5 to 30

Angle in degrees at which the wind SHADOW descends behind a crest or obstacle (3ds Max default 14). It sets how far downwind the sheltered zone reaches: smaller angle = a longer shadow, longer lee tails and wider calm zones behind obstacles; larger = short shadows and more active lee sides

Lee Recirculation

`lee_recirc` - Float - default **0** - 0 to 1

EXPERIMENTAL lee realism (opt-in). Models the recirculation eddy in the wind shadow: a weak near-surface REVERSE flow that creeps loose sand back UPWIND toward the slip face, sharpening the upper lee and feeding the brink -- detail the plain shadow-trap misses. Works behind dunes AND obstacles (same shadow). 0 = off; small values (0.2-0.4) give a subtle, crisper lee. Mass-conserving; higher values move more (can over-feed the crest, then it avalanches back). Costs a light extra pass. KNOWN ISSUE: in some scenes it produces odd lee-side patterns -- use sparingly, keep values small, and turn it OFF if the lee looks wrong

Avalanche Passes

`avalanche_passes` - Int - default **6** - 0 to 32

How many continuous slope-relaxation sweeps run per solver step to keep faces near the angle of repose. The default 6 keeps normal dunes tidy; if constantly-fed piles (Sand Source against an obstacle) still over-steepen, prefer Settle Slopes Every N -- it is far cheaper than raising this. 0 = no relaxation (unnaturally spiky sand)

Isotropic Avalanche

`avalanche_iso` - Bool - default **off

Relax slopes in ALL 8 directions instead of only the 4 grid axes. The default 4-neighbour sweep can only move sand along X or Y, so slip faces snap to the grid and a unidirectional near-axis wind shows the grid as straight zigzag crest segments. This adds the 4 diagonal neighbours (repose limit scaled by sqrt2) so avalanches are grid-neutral -- it softens the grid faceting at NO relief cost and keeps the exact repose angle (measured: ~24 pct less faceting, relief 99 pct, slope 34.7 vs 34.9). It is a partial cure (the crest POSITION is set by transport, so pair it with De-facet Crests for the rest). COST: it runs in the fused GPU kernel, but it doubles the avalanche directions and the avalanche runs on every substep -- measured +16 ms/step (about +27 pct) at 1024 f64 with 6 substeps. If you want the grid-line cure for free instead, use Cross-wind Shear Smooth alone (0.1 ms and a bigger effect on its own). Off = the Max-faithful sweep (byte-identical)

Cross-wind Shear Smooth

`shear_cross_smooth` - Int - default **0** - 0 to 8

De-facet the DEPOSITION pattern: smooth the wind shear this many passes PERPENDICULAR to the wind each step. The grid anisotropy that makes crests snap into straight zigzag lines comes mostly from where sand DEPOSITS (set by the shear), not the slip face; this de-staircases the crest line at NO relief cost and keeps the repose angle. It really is nearly free: measured 0.14 ms/step at 1024 (a fixed 4-tap stencil once per step), and on its own it cuts more faceting than Isotropic Avalanche does. STACKS with iso (together about half the faceting gone). 2-4 is plenty. 0 = off (byte-identical). Pair with De-facet Crests for the rest

Settle Slopes Every N

`settle_every` - Int - default **0** - 0 to 64

Keep piles at the natural angle of repose CHEAPLY: every N solver steps run a burst of extra slope-relaxation rounds. While sand is constantly fed (Sand Source, streams against an obstacle) the continuous relaxation alone reaches a dynamic equilibrium far ABOVE the repose angle (~60 deg cones at repose 34) -- this fixes that for near-zero cost (measured ~0 ms at 1024 with N=4). Avalanche Trigger fixes it too but costs ~7x the whole step. 0 = off. Presets use N=4

Settle Rounds

`settle_rounds` - Int - default **10** - 1 to 64

How many relaxation rounds each settle burst runs (each round = Avalanche Passes sweeps). 10 with N=4 holds source piles at ~36 deg; raise it if very fast feeding still over-steepens

De-facet Crests (in sim)

`defacet_sim` - Float - default **0** - 0 to 1

IN-SIMULATION cure for the straight grid-line look: a unidirectional near-axis wind makes the solver grid show through as zigzag crest segments. Every N steps the sand crests are smoothed ALONG their length so that zigzag never builds up in the actual MESH geometry (unlike De-facet Crests in Surface Detail, which is a render-only post-process). Gated to sand so bare rock stays sharp; sand mass is conserved. 0 = off (simulation byte-identical). 0.4-0.6 works. MEASURED at Every 30 / 0.5 on a 200 m domain: the crest zigzag drops 85.6 pct, and it costs dune height because it also gently opposes dune growth -- raise Every N to keep taller dunes. THAT COST DEPENDS ON YOUR GRID, and the reason is that the smoothing reaches a fixed number of CELLS rather than a fixed number of metres: at 512 it is 12.7 pct of the relief, at 256 it is 23.6, and at 128 it is 27.6, because the same reach is 0.63 m, 1.26 m and 2.52 m of ground. Coarse grids therefore pay several times more for the same setting. Free alternative: Star or Reversing wind, or more Wobble / Jitter

De-facet Every N

`defacet_sim_every` - Int - default **30** - 1 to 200

Run the in-sim De-facet every N solver steps. Higher = gentler (keeps more dune height) but lets more zigzag build between passes; lower = smoother crests but flatter dunes. 20-50 is a good range. Only used when De-facet Crests (in sim) is above 0

Edge Rounding

`edge_round` - Float - default **0** - 0 to 1

Take the corner off SHARP CONVEX edges -- a cornice lip, a drift brink, a plough ridge, the rim of a footprint -- while leaving the flats, the gentle slopes and the broad dunes alone. This is the opposite job to Polish / Smooth Edge: that pair is a BILATERAL filter whose whole purpose is to remove noise WITHOUT touching edges (measured: a hard step survives it at 100 percent, and a sharp crease gets slightly SHARPER), so it can never soften a rim. This one is weighted by CONVEXITY instead, which is what makes it selective -- a plain blur would flatten the dunes with the edge. Physically it is what wind polish, settling and sintering do to a snow edge within hours. MEASURED on a sharp ridge beside a broad dune: 0.6 strength, radius 2, every step, cut the ridge curvature 3.2x while the dune kept 99.95 percent of its amplitude and the mass drift stayed 0.0000 percent. 0 = off and the sim is bit-identical. Affects the real geometry, so bakes and exports see it

Rounding Radius

`edge_round_radius` - Int - default **1** - 1 to 16

How wide an edge counts as sharp, in CELLS. 1 rounds only single-cell creases; larger radii soften broader rims and act faster (measured: radius 2 cut the test ridge 3.2x against 2.1x at radius 1 for the same strength and passes). Raise it if the edge you want to soften is several cells across

Rounding Threshold

`edge_round_threshold` - Float - default **0** - 0 to 5

Curvature below which an edge is left alone entirely. 0 = relative mode: the sharpest edge present is rounded fully and everything else in proportion, so the control keeps working whatever the scale of the scene. Set a value to make it ABSOLUTE instead -- useful when a shot must round a hard rim but keep every softer feature exactly as it is

Rounding Every

`edge_round_every` - Int - default **1** - 1 to 1000

Apply the rounding every N simulation steps. 1 = every step (a continuously polished surface, right for snow under wind); larger values let edges build up between passes, which reads as weather that only occasionally works the surface. Also the cheap way to keep the cost negligible on big grids

Avalanche Trigger

`ava_trigger` - Float - default **0** - 0 to 30

EPISODIC slab collapses: slopes are allowed to build this many degrees ABOVE repose, then the whole over-steepened face lets go at once -- dramatic slides racing down the slip face instead of smooth continuous creep. 0 = off. EXPENSIVE: costs about 7x the entire solver step at 1024, so keep it for hero shots of collapsing faces. If you only need piles held at the repose angle, use Settle Slopes instead (near-zero cost)

Avalanche Run

`ava_run` - Int - default **3** - 1 to 16

How far each episodic avalanche cascades once triggered (extra relaxation passes per event). Higher = the slide runs further down the face in a single burst; lower = shorter, choppier slides. Only used while Avalanche Trigger is above 0

Avalanche Trigger Drop

`ava_paint_drop` - Float - default **15** - 0 to 40

How many DEGREES the painted Avalanche Trigger takes off the repose angle where the map reads 1.0. At 15 a slope standing at 34 deg starts sliding at 19 -- which is what forces a cascade you want to see now rather than in two hundred steps. Only the painted cells are affected, and with nothing painted this does nothing at all

Other Planet

`planet_enabled` - Bool - default **off

Scale the transport physics to a different world. OFF is Earth and is bit-identical to every build before this one -- the scaling is a MULTIPLIER on the numbers you already have, so Earth is exactly 1.0 and no preset moves. The solver has no gravity term to switch on: it works with a saturation length, a threshold and a flux scale that were tuned for Earth, and this derives what those three become somewhere else. The big one is air density -- the saturation length goes as grain density over AIR density, which is why Martian dunes are tens of times larger than ours for the same physics

World

`planet_preset` - Enum - default **EARTH

Pick a world, or Custom to enter the values by hand

Option	Meaning
Earth	The reference: every multiplier is 1.0

Option	Meaning
Mars	Thin CO2 (~1/60 of Earth air), basaltic sand around 100 um. Long saturation length and a high threshold: big dunes, and a gale needed to move them
Titan	A DENSE cold nitrogen atmosphere -- about four times Earth's -- and light organic sand. Both shorten the saturation length, so the elementary wavelength is small; Titan's giant dunes get their size from supply and wind regime, not from this
Venus	Near-supercritical CO2 at 65 kg/m3. Sand moves at very low wind speeds and the saturation length collapses
Moon (no atmosphere)	There is no aeolian transport without air. Selecting this says so rather than producing a division by zero
Custom	Type the four numbers yourself

Gravity (m/s^2)

`planet_gravity` - Float - default **9.80665** - 0 to 100

Surface gravity. It enters the THRESHOLD (a heavier grain needs more shear to lift) and the flux, but NOT the saturation length. Earth 9.807, Mars 3.71, Titan 1.35, Moon 1.62

Air Density (kg/m^3)

`planet_air_density` - Float - default **1.2** - 0 to 200

THE ONE THAT MATTERS MOST. It divides the saturation length, so thin air means a long drag length and therefore large dunes, and it divides the threshold too. Earth 1.20, Mars 0.020, Titan 5.40, Venus 65.0, Moon 0

Liquid Density (kg/m^3)

`planet_liquid_density` - Float - default **1000** - 1 to 3000

Density of the LIQUID the water solver is moving. Until now the model knew exactly one liquid -- Earth water -- and that was the only reason Titan refused to erode: its grains are 800 kg/m3, lighter than water, so they FLOATED and the sediment model, which describes grains that settle, declined to run. Titan lakes are liquid methane at about 450, where the same grain sinks normally. It also sets how hard the wind pushes the surface: a lighter liquid accelerates more under the same stress. Water 1000, liquid methane 450, brine up to 1250

Liquid Viscosity (m^2/s)

`planet_liquid_viscosity` - Float - default **1e-06** - 1e-09 to 0.001

Kinematic viscosity of the liquid. It goes WITH the density and not instead of it: viscosity sets how fast a grain falls (Ferguson-Church) and the van Rijn grain number, so a liquid given the mass of methane and the drag of water would be a half-wired material rather than a liquid. Water at 20 C is 1.0e-6; liquid methane is about 2.6e-7, four times thinner, so grains settle FASTER there

Solid Density (kg/m^3)

`planet_ice_density` - Float - default **917** - 1 to 3000

Density of the liquid FROZEN. Water is the exception in nature: its solid phase is LIGHTER than the liquid (917 against 1000), which is the only reason ice floats. Solid methane is about 500 against liquid methane at 450, so on Titan the ice SINKS -- it lies on the bed instead of forming a sheet, and it does not lift the surface. It also sets the mass exchange: one metre of ice costs this many kilograms of liquid per square metre

Grain Density (kg/m^3)

`planet_grain_density` - Float - default **2650** - 1 to 20000

What the sand is made of. Quartz 2650, basalt ~2900, and Titan's organic haze aggregate is far lighter at roughly 800

Grain Size (um)

`planet_grain_size` - Float - default **250** - 1 to 5000

Grain diameter in MICROMETRES. Medium sand is 250; Mars is usually modelled around 100. It multiplies both the saturation length and the threshold

Repose from Gravity

`planet_apply_repose` - Float - default **0** - 0 to 2

How much of the reduced-gravity repose effect to apply. 0 = none, which is the default ON PURPOSE. THE COMMON ASSUMPTION IS WRONG: the angle of repose does NOT follow gravity. For dry cohesionless grains it is set by inter-particle friction, and weight appears on both sides of the Coulomb criterion and cancels -- a dry pile stands at the same angle on the Moon as here. What reduced-gravity experiments do show is a MODEST increase as gravity falls, and the mechanism is cohesion rather than gravity: van der Waals attraction between grains does not get lighter, so at a sixth of a g the same cohesion holds a steeper pile. It is a few degrees, it depends on how fine and dusty the material is, and it is therefore a suggestion you opt into rather than something applied behind your back. 1 = the magnitude the literature reports

Shear Response A

`phys_a` - Float - default **1.5** - 0.1 to 20

Jackson-Hunt pressure coefficient: how strongly the wind speeds up over bumps (3ds Max default 1.5 -- keep it unless you know why). Higher = the shear over-reacts to relief, dunes grow and evolve too fast and drift from the reference look; lower = sluggish, slow-growing dunes

Windward Shift B

`phys_b` - Float - default **1.2** - 0.1 to 20

Jackson-Hunt upwind shift: how far the peak wind stress sits UPWIND of a crest (3ds Max default 1.2 -- keep it). This offset is what makes dunes migrate and keep their asymmetric windward/lee shape; wrong values give dunes that stall, sharpen oddly or run away. IT IS NOT A LINEAR DISTANCE AND IT HAS A CEILING: the peak sits atan(B/A) of a WAVELENGTH upwind, so it can never pass a QUARTER wavelength however far you push this. MEASURED against the A = 1.5 default: B = 0.1 shifts 0.011 of a wavelength, the 1.2 default 0.107, B = 5 gives 0.204 and B = 20 only 0.238 -- the top three quarters of the slider buy 14% more shift but more than DOUBLE the stress amplitude (1.53 -> 4.21). Past about B = 5 this knob stops shifting the peak and starts acting as a second A

Grain Diameter

`grain_diameter_um` - Float - default **250** - 100 to 2000

Mean grain diameter in MICRONS. 250 -- medium sand -- is the reference and reproduces the shipped behaviour exactly. This is the number the solver never had: its own planet module says it "contains no gravity, no air density and no grain diameter -- those were folded into hand-tuned numbers for Earth". ONE diameter drives TWO things, by two different laws that happen to share a power: the SATURATION LENGTH (L_sat ~ 2.2 rho_s/rho_f d) and the ENTRAINMENT THRESHOLD (u*t ~ sqrt(g d), so tau_t ~ d). The saturation length is what sets the SMALLEST DUNE THAT CAN EXIST, so with the old fixed value the finest form in a scene was an arbitrary number. Coarse sand at 500 um doubles both: bigger minimum dune, and more wind needed to start it. Clamped at 100 um because below that cohesion takes over and the threshold rises again -- a branch this deliberately does not model rather than extrapolate into

Surface Roughness z0

`surface_z0` - Float - default **0.001** - 1e-06 to 1

Aerodynamic roughness length of the ground, in metres -- how much grip the surface has on the wind. This is the log law and nothing else: the wind profile is U(z) = (u*/kappa) ln(z/z0), so inverting it at the 10 m reference height gives the friction velocity and the bed stress is its square. ROUGHER GROUND TAKES MORE STRESS FROM THE SAME WIND, which is the direction people expect backwards. Worked multipliers on the shear: fine sand 0.0001 = 0.64x, sand 0.001 = 1.00x (the default and the shipped behaviour), gravel 0.003 = 1.25x, fresh snow 0.0002 = 0.70x, grass 0.03 = 2.51x. It scales the SHEAR only -- the entrainment threshold is a property of the GRAIN and deliberately does not move when the ground gets rougher, so raising this makes more sand move rather than redefining what "enough wind" means. Feeds all three wind models identically (analytic kernel, Flow 2D, Flow 3D), so switching model never switches the material under it

Atmospheric Stability

`air_stability` - Float - default **0** - -2 to 1

How the air layer over the ground is stirred, as the Monin-Obukhov parameter z/L: NEGATIVE is unstable (a hot afternoon, ground far warmer than the air, convection dragging momentum down to the sand), 0 is NEUTRAL, POSITIVE is stable (a clear night, the surface colder than the air, the layer capping itself off). The solver has always assumed neutral, and over a desert neutral is the exception -- this is why a dune field does most of its work in the afternoon and goes quiet after dark. It is the same log law that Surface Roughness uses, with the standard Businger-Dyer correction added inside the logarithm: u* = kappa U / [ln(z/z0) - psi_m(z/L)]. There is nothing to calibrate -- kappa cancels and psi_m is exactly zero at neutral, so 0 reproduces every existing scene bit for bit. Worked multipliers on the shear over sand: -2.0 = 1.42x, -0.5 = 1.20x, 0 = 1.00x, +0.3 = 0.74x, +1.0 = 0.42x. It multiplies the SHEAR only, in the same place Surface Roughness does, so all three wind models get it identically and the entrainment threshold -- a property of the grain -- does not move. NOT driven from Air Temperature on purpose: stability is set by the GROUND-minus-air difference, and a hot desert night is still a stable one, so reading it off the air temperature alone would get the sign wrong exactly when it matters most. Clamped to -2..+1 because outside that band the similarity theory stops describing anything: the stable layer goes intermittent and decouples, and free convection takes over on the other side

Saltation Feedback (Owen)

`owen_feedback` - Float - default **0.7** - 0 to 1

How strongly the flying sand SLOWS THE WIND DOWN. This is the only negative feedback in the solver, and 0 -- the shipped default -- means the loop is open: the wind is computed, the sand reads it, and the sand never answers back. Owen (1964) says a saltating layer self-adjusts so the air stress reaching the bed stays pinned near the entrainment threshold, because everything above it is being carried by the grains instead. At 1.0 a cell carrying all the sand it can lands exactly on the threshold; at 0.5 it goes half way. WHAT YOU SEE: windward slopes stop eroding harder just because the wind rose, so dunes reach a size and STAY there instead of needing a cap to hold them back, and long fetches saturate instead of running away. It acts on cells that are actually carrying sand -- somewhere with no transport is untouched, which is why this is not the same as simply lowering Wind Strength. Feeds all three wind models identically

Cloud Roughness (Owen)

`owen_rough` - Float - default **0** - 0 to 8

Owen's FIRST hypothesis, the companion to Saltation Feedback: a dense saltation cloud is itself a roughness, so the outer wind gives up MORE total stress over sand that is already moving. It runs BEFORE the feedback -- roughness raises the total, the feedback then takes the grains' share out of it. HONEST SCOPE: the physical form is z0_eff = z0 + a u*^2/(2g), which is a LENGTH and needs a friction velocity in metres per second; this solver carries no wind speed at all, so writing metres here would be inventing a calibration. It is expressed in the solver's own units instead -- the roughness thickens in proportion to how dense the cloud is -- which makes this a SHAPE parameter, not a physical constant. 0 = off

Slope Affects Threshold

`slope_threshold` - Float - default **1** - 0 to 1

How much the local slope helps or hinders grains leaving the bed. 0 = off and the shipped behaviour, where a windward slope and a lee slope are equally easy to erode -- which gravity says they are not. The closure is f = cos a + sin a / tan(repose), with a the slope ALONG THE WIND. It was chosen over a fitted ramp because it is EXACTLY ZERO at the downhill repose angle: a face already standing at repose needs no wind at all to shed grains, which is both true and checkable. At 1.0 the full effect applies -- windward faces resist, lee faces give way, and slip faces sharpen because the brink is where the two meet

Moisture Raises Threshold

`moisture_threshold` - Float - default **1** - 0 to 8

How many times the dry threshold a fully WET cell reaches (Belly): water bridges between grains hold them down. 0 = off. This is DELIBERATELY NOT the same as Water Holds Sand, which scales erodibility -- a capacity cut moves less

sand everywhere, while a threshold rise means a weak wind moves NONE. Damp sand is either still or blowing, not slowly seeping, and only a threshold can say that. Needs a water table or a painted water mask; in a dry scene it is an exact no-op

Body Wake Length / Height

`body_shadow_ratio` - Float - default **6** - 0 to 20

How long a wind shadow a SOLID BODY casts, in body heights. 0 = off and the shipped behaviour. THE DEFECT IT FIXES is measured and it is this solver's own: the 14-degree separation cone gives a shadow of EXACTLY 3.98 times the height for a dune brink, a rock, a wall and a building alike -- 7.55 m behind 2 m, 23.90 behind 6 m, 71.70 behind 18 m -- while a solved 3D wake gives 5.4 to 10.8 depending on the body, and the bluff-body band in the literature is 5 to 15. One angle cannot describe a dune AND a building. Set this and only OBSTACLES get the longer wake; dune brinks keep their 14 degrees, which is the case that number was calibrated for. Needs no 3D solver: it is a second pass of the same shadow scan. A buried obstacle stops casting a body wake exactly when it stops being a wall

Upwind Stagnation / Height

`stagnation_ratio` - Float - default **0.7** - 0 to 6

How far UPWIND of a solid body the wind is slowed, in body heights. 0 = off and the shipped behaviour, which has no upwind stagnation at all: MEASURED, the analytic shadow covers 0.50 of the ring in front of a block where a solved 3D field covers 1.00, so sand piles against the windward face when it should be sheltered before it gets there. Real air stagnates roughly one body height ahead of a bluff face, so 1.0 is the physical starting point. Independent of the downwind wake -- setting this leaves the lee shadow bit-identical, and vice versa

Crest Shear Ceiling

`crest_cap` - Float - default **0.5** - 0 to 4

Ceiling on how far the ANALYTIC wind kernel may over-speed a crest. 0 -- the default -- is off and bit-identical. HONEST SCOPE, because this is a ceiling and not a correction: the kernel is Jackson-Hunt, a LINEARISED theory derived for gentle slopes, and it returns a speed-up proportional to terrain amplitude with nothing bounding it. Measured on a cosine ridge fed through the separation envelope exactly as the solver does, the perturbation reaches 0.75 at a 30-degree slope -- far outside the range the theory is valid in, where a steep crest is simply handed more and more stress. This caps it: the value IS the ceiling, so 0.5 means a crest can carry at most 1.5x the background shear however steep it gets, while gentle slopes are untouched to first order and the lee is left entirely to the separation envelope. THE NUMBER IS A CHOICE, not a measurement -- the 3D solver was tested as a reference for it and has no stationary near-surface state to calibrate against. Start at 0.5 and judge by eye

Transport Capacity

`qsat` - Float - default **1** - 0.01 to 10

Saturated transport capacity Q_sat: how much material (sand, or blowing snow in SNOW mode) the wind carries at full strength. Higher = more in transit -> taller dunes/drifts, faster fill. 3ds Max = 1.0

Threshold Shear

`tau_t` - Float - default **0.05** - 0 to 5

Shear below which the surface (sand or snow) does not move -- the grain's own resistance, quoted against the NOMINAL wind (Wind Strength 1.0), not against whatever wind you happen to have set. That distinction is what makes Wind Strength work: a threshold that rose with the wind cancelled it out completely, and until 1.2.89 it did. 3ds Max uses 0.05 (material creeps almost everywhere the wind blows); a high value like 1.0 lets it move ONLY on the strongest crests, giving patchy, unnatural transport -- and now a strong enough wind can genuinely overcome it, as a real threshold should

Time Scale

`time_scale` - Float - default **0.3** - 0.01 to 10

Material (sand or snow) moved per simulation step. The 3ds Max original clamps this to 0.15-0.45 (nominal 0.30); higher values make each frame jump instead of evolving smoothly. Keep near 0.30 for Max-like motion; raise

Steps/Frame to advance faster while staying smooth

Wind Speed at 10 m

`wind_speed_10m` - Float - default **8** - 0 to 100

The real wind speed, in m/s at the standard 10 m height, that this scene's Wind Strength stands for. PURELY A READOUT: it feeds nothing, changes no result and leaves every scene bit-identical -- what it buys is a CALENDAR. Set it to 0 and the panel says nothing. The default 8 m/s is a typical sand-moving desert wind, and it is the value at which the model's own migration rates land inside the surveyed band (13.7 m/year for a 10 m dune against roughly 5-15 measured in the field). It puts one step at about 18 hours of sand-moving wind. This is the one physical input the solver never had. Every rate in it is "per step" and Time Scale is a tuning number, so "this dune migrates 12 m per year" was not a statement the plugin could make. With Surface Roughness giving the friction velocity through the log law and Grain Diameter giving the entrainment threshold, the saturated flux has a published closed form (Lettau & Lettau) in m2/s, and the solver carries the same quantity in m2/step -- so their ratio IS the length of a step, derived rather than fitted. Reference winds: 6 m/s barely moves sand, 8 m/s is a typical sand-moving desert wind, 12-15 m/s is a storm. Below about 5.4 m/s over sand nothing should move at all, and the panel says so if you go there. ONE STEP IS A STEP OF SAND-MOVING WIND, not of calendar: real deserts blow above the threshold only part of the time, so a model year equals a calendar year only in a scene driven that way

Wet Sand Holds

`rain_threshold` - Float - default **0** - 0 to 200

How strongly damp ground RAISES THE THRESHOLD at which the wind can lift a grain, after Fecan, Marticorena & Bergametti (1999): u*t(w) = u*t(0) *sqrt(1 + 1.21* (w - w')^0.68). THE NUMBER IS HOW MANY TIMES THE BAR RISES AT FULL WETNESS, and it is a plain multiple rather than the paper's constant for a measured reason: the dry threshold here is 0.050 while a default wind delivers a shear of 1.600, so the wind sits 32x above the bar. The literature scale tops out near 3x and was measured to cut exactly 0.00% of cells. MEASURED ON A HALF-WET GRID: 10 and 30 change nothing (100% and 99.7% of the dry rate), 60 brings the damp half down to 45.2% while the dry half stays within 0.11% of untouched -- so this is a switch that lives near 50-80, not near 1. The CURVE between dry and soaked is still the paper's. THIS IS NOT "Wet Sand Resists". That one multiplies erodibility, so damp ground moves LESS -- this one can make it move NOT AT ALL, because below a threshold nothing is entrained at any wind. The difference is what separates a beach from a dune field: the damp strip by the water stays put while the dry crest beside it walks. They stack, and both are off at 0. Applied through erodibility rather than through the threshold itself, because the transport kernels take a SCALAR threshold -- a cell whose shear does not reach the raised bar gets erodibility 0, which is the same thing where it matters

Residual Moisture

`rain_threshold_residual` - Float - default **0.02** - 0 to 0.5

Moisture below which water sits in the pores without bridging grains, so it holds nothing -- the w' of the Fecan model. In real ground it rises with clay content; on clean sand it is very small, which is why the default is 2%. Raise it if a light dew should not stop the field

Boundary

`boundary_mode` - Enum - default **PERIODIC

What happens at the terrain edges. PERIODIC wraps: sand leaving one side re-enters on the opposite side (endless desert, tileable). OPEN X lets sand really LEAVE the domain downwind and enter as Influx upwind -- without a Sand Source or Influx the field slowly empties, which is the physics, not a bug. Combine Open X with Edge Clamp for a fully non-tiling terrain

Option	Meaning
Periodic	Sand leaving one edge re-enters on the opposite edge: an endless, seamlessly tileable dune field where total sand is conserved (default)
Open X	The X edges are open: sand blows in upwind at the Input Flux rate and GENUINELY LEAVES the domain downwind. Without a Sand Source or enough Influx the field slowly empties -- that is the physics, not a bug. Ideal for barchan fields on limited sand supply

Edge Clamp - open ALL edges (non-tiling)

`edge_clamp` - Bool - default **off

THIS IS THE ONE FOR A LOADED HEIGHTMAP, and it OVERRIDES Boundary Mode -- with it on, Boundary Mode does nothing at all. Measured on a terrain whose opposite edges disagree, movement at the border after 20 steps: Periodic 6.13, Open X 4.73, this 1.96 -- and the far edge itself 4.97 down to 0.13. Treat the terrain as NON-TILING: clamp ALL edges so the wind shadow, sand transport AND avalanches never wrap across the seam (no false cliff or shadow bleeding from the opposite edge). Sand blows in at the upwind edge (Input Flux) and off the downwind edge. Leave OFF for a seamless tileable result; turn ON for a bounded real-world heightmap. Note: on CPU/EXACT this uses the open-boundary transport (a touch less saturated at the very edges) instead of the periodic fast scan

Wrap X Avalanche

`wrap_x` - Bool - default **on

Wrap the avalanche relaxation across the X edges. Only meaningful on a PERIODIC boundary -- with Open X or Edge Clamp the X seam never wraps (the two edges are unrelated surfaces), so this toggle is ignored there

Terrain Wind Shadow

`terrain_shadow` - Float - default **1** - 0 to 1

How much of the terrain OWN wind shadow the generator takes into account. The solver fills the lee of every crest up to a descending plane and the shear is computed from THAT surface, so a cell downwind of a crest sees less wind -- which is what builds slip faces and keeps dunes migrating. Until now only its ANGLE (Separation Angle) and reach were adjustable, never its strength. 1 = the full model (default, and skipped entirely so it is bit-identical); lower values let the wind reach into the lee, giving rounder, less shouldered forms; 0 removes the wind shadow completely and the shear reads the bare sand surface. It scales the DEPTH of the shadow, exactly (0.6 gives 0.6x) -- WHERE the lee falls is the Separation Angle job, so the shadowed area is the same at any setting above zero and empty only at zero. This is the GENERATOR shadow, not shading -- solid obstacles keep their own height, and Shadow Trap is the separate transport-side knob

Shadow Trap

`obstacle_shadow` - Float - default **1** - 0 to 1

How completely the wind shadow behind crests and obstacles traps passing sand (1 = full trap, sand entering the shadow drops there -- crisp slip faces and long lee drifts; 0 = the shadow does nothing and lee sides stay active). Lower it slightly for softer, more weathered lee sides

Shadow Offset (m)

`obstacle_shadow_offset` - Float - default **0** - 0 to 30

Grows the wind shadow OUTWARD from the body, so the sheltered ground reaches around its flanks and a little upwind of it instead of starting hard against the wall. That gap is the part a plan-view separation surface cannot produce on its own: the envelope only descends from the TOP and runs downwind, while real flow separates at the sides of a bluff body as well, sheltering ground the model otherwise leaves fully exposed right up to the object. Use it when sand piles tight against a wall or a rock and you want it held off. Applied BEFORE Shadow Edge Feather, so a feathered shadow ramps at the GROWN edge -- feathering first and growing after would put a hard step back on the outside of the ramp. 0 is the shadow of every earlier build, exactly

Shadow Edge Feather (m)

`obstacle_shadow_feather` - Float - default **0** - 0 to 20

Softens the EDGE of the wind shadow, so it stops printing into the sand as a hard line. MEASURED on a sharp-sided block: the shadow is a yes/no mask with ZERO in-between cells at every Obstacle Smoothing setting, and a rim step of exactly 1.000 -- which is why smoothing the object never smoothed its shadow. This is the control that does. The shadow then fades in over this many metres at its border -- the flanks, the far end, all of it -- while its INTERIOR is left at full strength, so this changes where the edge is soft and not how much the shadow traps. Use Obstacle Shadow for that. The metres are real: a 6 m setting measures 5.95-6.22 m of ramp across a 2.7x range of cell sizes, so it survives

a resolution change. Below about one cell it is floored by the grid. Try 1-3 m. A feather wider than the shadow itself has no interior left to protect and does start weakening it. It softens EVERY wind shadow, a dune lee included, not only the ones cast by objects. 0 is the hard mask of every earlier build and is bit-identical. Above 0 the solver leaves its axis-aligned CUDA sweep for the general one, because that kernel takes the shadow as a yes/no flag and would drop this setting silently. MEASURED per step, off then on: 4.7 -> 4.2 ms at 512, 10.7 -> 10.9 at 1024 and 84.2 -> 43.2 at 2048, so it is a wash on small grids and about twice as FAST on large ones -- the axis kernel walks a whole row per thread and starves a big grid

Sand Can Bury Objects

`obstacle_buriable` - Bool - default **on

Let sand climb ONTO an object once it has piled up to the top of that object, instead of stopping dead against its side. WHAT IS BROKEN WITHOUT THIS, and it is one fact wearing three faces: a cell where the object stands proud is frozen from both sides -- transport forces no height change there, and the avalanche calls it a wall -- so the ground under an object has no route upwards at all. MEASURED on a 3.0 block in a strong flux over 600 steps: the surface on its footprint sat at 0.5000 the whole run, drift exactly 0.0, while the pile upwind of it reached 5.16, well over the height of the object itself. So the object could never be buried however deep the sand got; its wind shadow therefore never went away, because the shadow model honours burial correctly but never got the chance; and sand stacked against the side at 77.8 degrees, because the relaxation was not allowed to carry it over the top. With this on, sand ramps up the side at the repose angle, spills across the top, and the shadow fades by itself as the object stops standing proud. Scour it off again and the old behaviour re-arms, so a re-exposed object goes back to blocking. NOTE ON THE TERRAIN MESH: a covered cell adopts the top of the object as its own surface, so the terrain follows that silhouette once sand reaches it. That is what burial looks like, but it does mean the terrain and your object mesh coincide there. ON BY DEFAULT since 1.3.35. Turn it OFF to get the behaviour of every build up to 1.3.34, where an object could never be covered at all

Re-expose Below

`obstacle_unbury_depth` - Float - default **0.1** - 0 to 20

How far the sand AROUND an object has to drop below its top before the object comes back out of the sand. Only matters with Sand Can Bury Objects on. WHAT IT STOPS: a covered cell takes the object top as its floor, so without a way back it stays covered FOR EVER -- and when the wind then scours the surroundings, they sink while that cell stands, and the terrain carries the SHAPE OF THE OBJECT. MEASURED before this existed, 96 grid, a 3 m block, 1200 steps: cover 0.0000, surface on the object pinned at 3.0000 and the ring around it down at about 1.0 -- a 1.99 m silhouette standing in the sand, coincident with your own object mesh. It is deliberately a HYSTERESIS: burial needs the sand beside the object to REACH its top, re-exposure needs it to fall this far BELOW, so a cell cannot flicker between the two. Raise it to keep objects buried through a windy spell; 0 falls back to a small value derived from the cell size and the repose angle

Object Wake Length

`obstacle_wake_scale` - Float - default **1** - 0.2 to 5

How far the wind shadow behind a solid OBJECT reaches, as a multiple of what it has always been. MEASURED at 1.0: the shadow runs 7.55 m behind a 2 m block, 23.90 m behind a 6 m one and 71.70 m behind an 18 m one -- a constant 3.98 times the height, which is a separation surface leaving the top at 14 degrees. That 14 is the figure for a DUNE BRINK, and it is the only separation angle in the solver, so a rock, a wall and a building all shed the wake of a dune. A bluff body does not: flow leaving a sharp edge reattaches somewhere around 5-15 body heights downwind, so 1.5-3.0 here is closer to a real object and is the dial to reach for when dunes crowd up against a rock instead of being held off it. IT ALSO GOES THE OTHER WAY, AND THAT HALF WAS NOT SAID ANYWHERE. Below 1.0 it SHORTENS the wake, which is the answer to a tall object shadowing the whole terrain in one step: at 1.0 the shadow is 4.011 times the height for every body, so anything taller than about a quarter of the domain covers all of it. MEASURED, a 24 m object in a 96 m domain, cells of that row left in shadow: 88 of 96 at 1.0 (the whole domain), 69 at 0.75, 46 at 0.5 and 18 at 0.2. It scales linearly, so 0.2 gives a wake of 0.80 times the height instead of 4.011. It changes the SHADOW only -- the object still stands exactly as tall as it is for blocking, transport and avalanches -- and it costs nothing: one masked multiply on the obstacle array, and only when the value is not 1.0. 1.0 is bit-identical to every earlier build

Input Flux

`q_in` - Float - default **0.15** - 0 to 5

Incoming material flux at open (non-periodic) boundaries -- sand (or blowing snow) that drifts in from upwind. 3ds Max default 0.15

Conserve Erosion

`conserve_erosion` - Bool - default **on

Never erode more sand than actually exists above the bedrock (leave ON). The wind then strips patches down to bare rock and stops -- deflation hollows, exposed interdune floors. OFF = the legacy behaviour where erosion conjures material. MEASURED on a closed scene: nothing happens until the wind scrapes a patch down to rock, then the field gains +50% mass in 120 steps and +811% in 1920, with the tallest dune growing 30x. A deep bed sits at 0.0000% for the first ~480 steps, so a short preview shows nothing. OldDunes forces this mode ON regardless of this checkbox

Strict Sand Budget

`strict_sand_budget` - Bool - default **off

Close the sand budget when the wind runs ALONG the crests. The flux scan solves an integer-step recurrence while the upwind flux is sampled bilinearly; off the axis the two disagree and the solver CREATES sand. MEASURED at 256 cells, thin sand on rock, 480 steps: wind ACROSS the ridge +0.6% mass, wind ALONG it +14.0%. The phantom sand does not build dunes -- it spreads as a ~7 cm veneer that buries the rock (bare rock 16.4% -> 7.4%) and frays the sand/rock edge into filaments. Matters ONLY for THIN sand over bedrock: a deep bed drifts 0.0% either way. COSTS about 3.5x per solver step. Does NOT fully cure along-crest wind (14.0% -> 1.4% at 256, 17.0% -> 3.0% at 512), so treat it as a large reduction, not a fix

Mask Blocks Deposit

`mask_blocks_deposit` - Bool - default **off

FREEZE semantics for erodibility masks: protected cells (erodibility 0) block DEPOSITION too, so their shape is fully locked -- sand can neither erode from nor settle onto them (blocked sand rides on downwind). OFF = masks only stop erosion; deposition can still bury protected ground

Rock Erode

`rock_erode` - Float - default **0** - 0 to 1

Let the wind slowly cut into the BEDROCK itself once the sand cover is stripped away (0 = rock is indestructible, higher = faster abrasion). Adds yardang-like wind sculpting of exposed rock over long runs; keep small -- rock should erode orders of magnitude slower than sand

Edge Flat

`edge_flat` - Float - default **0** - 0 to 256

Width in solver cells of a border band PINNED to the initial terrain -- nothing evolves there. Use it to keep the edges of a real-world heightmap untouched so the sim tile blends into surrounding terrain, or to stop dunes crawling off a bounded set. 0 = off; pair with Edge Falloff for a soft transition

Edge Falloff

`edge_fall` - Float - default **0** - 0 to 256

Width in solver cells of the blend zone between the pinned Edge Flat border and the fully live interior: the sim fades in gradually instead of starting at a visible line. 0 = hard transition (can read as a seam where dunes abruptly begin)

Water / Moisture

Water in three quite different senses, and it is worth knowing which one you want. A WATER TABLE is a level: everything below it is submerged and does not blow away. FLOWING WATER is a real shallow-water field that runs

downhill and can carve. RAIN is surface moisture -- weather that wets the ground without simulating a drop of it.

When you need it: Coastlines and oases (table), rivers and floods (flow), weather and vegetation (rain).

Water / Moisture

`water_enabled` - Bool - default **off

OPT-IN water table: surfaces at or below Water Level are SUBMERGED (the wind cannot pick material up there, but it can still settle -> shores and sand spits build into the lake), and a damp capillary fringe above the waterline resists erosion. Works for sand (interdune lakes, beaches, oases) AND snow (meltwater ponds). Render ripples fade out automatically on wet ground. Switching it OFF also hides the water plane -- it is hidden, not deleted, so nothing you did to that object is lost; use Remove Water Plane to be rid of it

Water Preset

`water_preset` - Enum - default **STILL

Ready-made sets of the FLOWING-WATER numbers: the field, its clock, its drag, rain feeding and the wind on the surface. They set how the water BEHAVES, never WHERE IT IS -- Water Level stays yours, because it is a height in your scene and no preset can know it. Pick one and press Apply Water

Option	Meaning
Still: pools find their level	The field on, nothing pushing it: every hollow settles to its OWN level and the shoreline is wherever the terrain says. MEASURED on two basins: -0.467 and +0.029 m, both surfaces dead flat, and a lake at rest on a 5 % slope stays put BIT FOR BIT
Draining: water runs downhill	A fast water clock (1.0 s per step) so a scene drains while you watch instead of over a hundred steps. MEASURED: rain over a dome collects 88.6 % of the water in the lowest quarter of the ground, against 25 % if it stayed where it fell
Rain-fed pools	Rain feeds the field, so pools grow where the storm actually is and drain where the ground tilts. Needs Rain switched on as well -- on its own this changes nothing, and the wet-share readout under the field will say so
Breeze on the water	A 12 m/s wind on the surface. Deliberately gentle: the tilt goes as the SQUARE of the speed and inversely with depth, so on a 30 m pool 0.18 m deep this is about 4 mm of lean -- you see it in the shoreline and in the drift, not as a slope
Storm wind on the water	A 40 m/s wind, the loud end. MEASURED against the closed form S = tau/(rho g h) to within 0.02 %: on 0.10 m of water it tilts 509 mm across the 200 m domain, on 0.50 m it is 102 mm and on 2 m it is 26 mm. Local wind field on, so a crest speeds it up and a lee pocket goes calm
Puddles: finest grid	The 2048 field, where a 0.5 m puddle is 5.1 cells across instead of 1.3 and therefore exists at all. MEASURED cost: 1.58x the step of the 1024 grid -- everything below 1024 costs the SAME, because the ceiling there is the number of GPU calls
Dead calm: mirror surface	Heavy drag (2.0) and a slow clock, for a still hero shot. MEASURED: at drag 0.5 a disturbed surface goes dead flat and still keeps 72 % of its wave response; at 2.0 it settles sooner and answers less. The EQUILIBRIUM is untouched either way -- drag changes the road, not the destination

Water Model

`water_mode` - Enum

Which water this scene uses. The two are not degrees of the same thing -- they answer different questions, and the shader one is the default because it is what most scenes want

Option	Meaning
Shader Water	ONE GLOBAL LEVEL, shaded. The plane sits at Water Level and the material does the rest: absorption with depth, fog, foam along the shoreline, waves and ripples as noise. Cheap, stable, and right for a lake, a sea horizon or a flooded pan -- anything where the water does not need to KNOW anything. This is the default and it is unchanged

Option	Meaning
Shallow-Water Simulation	A REAL 2D SOLVER: depth per cell, velocity, mass conservation. Two ponds can sit at different heights, water runs downhill, fills a hollow the dunes just dug and answers rain; a swell refracts and shortens over a bar because its speed is sqrt(g x depth); and with erosion on it carves the sand. Costs a solve per step and needs enough cells for what you are asking to exist -- the panel says how many

Level Given As

`water_level_mode` - Enum - default **SOLVER

Where the starting water surface comes from. Split out in 1.3.107 so the water can be driven without the dune generator having an opinion about it

Option	Meaning
Solver Height x Height Scale	Water Level is a SOLVER height and is multiplied by Height Scale, so it follows the dune terrain when that is rescaled. Unchanged behaviour
World Z	Water Level (World) is a plain Z in the scene, independent of Height Scale and of the dune solver. Use this when the bed is a mesh object

Water Level (World)

`water_level_z` - Float - default **0** - -10000 to 10000

The world Z the surface starts at when Level Given As is World Z. A plain height in the scene: no Height Scale, no solver

Water Level

`water_level` - Float - default **0.5** - -1000 to 1000

Height of the water table in SOLVER height units (same units as the heightfield, before Height Scale). Cells below it are submerged; use Create Water Plane to see it in the viewport. Changing it MOVES an existing plane and re-bakes the depth under every vertex, so what you see and what the solver submerges are the same surface. Dragging the plane by hand is the other direction: press Refresh Water Level and whatever Z you put it at becomes this

Water Detail

`water_field_quality` - Enum - default **MED

Grid the water is solved on. MEASURED on an RTX 3090: the step costs the same at 512 and at 1024 (2.70 against 2.72 ms), because below 1024 the ceiling is the NUMBER OF GPU CALLS and not the amount of data -- so Medium is free. High doubles the grid again for 1.58x, and that is what a 0.5 m puddle needs to exist at all (5.1 cells across instead of 1.3)

Option	Meaning
Low (512)	Cell 0.39 m -- smallest pool that holds its shape is about 2 m
Medium (1024)	Cell 0.20 m -- about 1 m. Costs the same as Low, see the tooltip
High (2048)	Cell 0.10 m -- puddles down to 0.5 m. Costs 1.58x

Water Steps

`water_steps_count` - Int - default **10** - 1 to 2000

How many water steps Step Water runs. This is the WATER clock and it is not the dune clock: one water step is Water Seconds per Step of model time, while one dune step stands for hours of migration. Running the water alone is the point -- a pool can settle, drain or be blown across the scene without the sand moving at all

Rain

`rain_enabled` - Bool - default **off

Weather that wets the ground. This is a SURFACE MOISTURE model, not a water simulation: rain raises the moisture of the sand, and damp sand needs more wind before a grain will move -- which is where nearly all of the visible effect of rain on dunes lives. Wet ground still ACCEPTS blown sand, so damp patches keep building. It costs about 2.5% of a simulation step, plus roughly 8% for switching on the erosion-mask path -- which any scene already using Water, Vegetation, painted Erodibility, Rock or Multi-Fraction is paying anyway. There is no runoff or gullying; that would cost 20-50% of a step for far less of the look

Rain Rate

`rain_rate` - Float - default **0.05** - 0 to 1

How fast the ground wets under a storm cell, per simulation step. Moisture is capped at 1 (saturated), so a high rate mostly means the ground saturates sooner and then stays wet until it dries

Coverage

`rain_coverage` - Float - default **0.35** - 0 to 1

What fraction of the field is under rain at any moment. This is a THRESHOLD on the weather pattern, not a strength: at 0.35 the wettest 35% is raining and the rest is genuinely dry. A strength multiplier would drizzle everywhere instead, which is not what weather does. 1.0 = it rains on everything at once

Shallow Water

`water_shallow_color` - FloatVector - default **0.42, 0.68, 0.62** - 0 to 1

Colour of water thin enough to see straight through. The render walks from this to Deep Water along a real Beer-Lambert absorption curve driven by the baked depth, so the transition is physical rather than a painted gradient

Deep Water

`water_deep_color` - FloatVector - default **0.02, 0.09, 0.13** - 0 to 1

Colour light comes back as after crossing a lot of water. Reached exponentially with depth, not linearly -- which is why real water goes dark so much faster than a linear ramp suggests

Murkiness

`water_murkiness` - Float - default **0.35** - 0 to 8

How fast light is lost per unit of depth (the extinction coefficient). 0.35 puts the half-way colour at about 2 m, which is clear coastal water; 2.0 is a silty lagoon; 0 is distilled water in a glass. This is the knob that makes deep water read as deep instead of merely dark

Opacity Depth

`water_opacity_depth` - Float - default **0.6** - 0.01 to 20

Depth at which the surface stops showing the sand beneath it. Kept separate from Murkiness on purpose: a clear but DEEP sea is an opaque blue, not a sheet of glass, and one number cannot say both

Water Roughness

`water_roughness` - Float - default **0.1** - 0 to 1

Base surface roughness. Foam overrides it to 0.62 locally. Do not push this to a mirror finish at any distance: measured on a shoreline shot, 0.03 broke the sun glitter into individual white pixels over 1.10% of the frame -- a sharp specular sampled over a normal that changes every pixel aliases -- while 0.12 rendered a clean glint path with 0.00% speckle. The ripples a bump map cannot resolve at that distance ARE roughness

Wave Height

`water_wave_height` - Float - default **0.035** - 0 to 1

Strength of the swell in the surface normal. This is a BUMP, not displacement -- the silhouette stays flat, which is the right trade at any distance where you can see the whole shoreline

Wave Size (m)

`water_wave_size` - Float - default **1.6** - 0.05 to 60

Wavelength of the long swell, in world metres

Wave Speed

`water_wave_speed` - Float - default **0.35** - 0 to 5

How fast the wave field drifts, driven off the scene frame. Both wave layers read ONE Time socket, so the whole surface animates from a single driver

Foam Amount

`water_foam_amount` - Float - default **0.75** - 0 to 1

How white the foam gets. 0 = no foam at all

Foam Width (m)

`water_foam_width` - Float - default **0.3** - 0 to 5

How far up the shore the foam reaches, measured as a water DEPTH rather than a distance -- so it follows the real shoreline around every spit and hollow instead of a contour drawn at one height

Wind on Water

`water_wind_speed` - Float - default **0** - 0 to 60

Wind speed in m/s that pushes the water surface. The wind grips the water as a shear stress tau = rho_air *1.3e-3* U^2, so it goes as the SQUARE of the speed, and the surface tilts until gravity cancels it -- at a slope of exactly tau / (rho_water *g* depth). MEASURED against that closed form: 1.0000 to 1.0002. That means it is inversely proportional to DEPTH, so it moves a pool and barely leans a lake: over the 200 m domain a 20 m/s wind tilts 10 cm of water by 127 mm, 50 cm by 25 mm and 2 m by 6 mm. It does NOT make wind waves -- those are shorter than a cell and belong to the shader. COSTS TIME while the surface is still settling: moving water needs smaller sub-steps, and the Water demo took 84 sub-steps at rest against 274 under a 30 m/s wind. At 0 nothing here runs at all

Moist Fringe

`water_moist_band` - Float - default **0.5** - 0.001 to 50

Capillary fringe: how far ABOVE the waterline the sand/snow stays damp (solver height units). Wetness ramps from full at the waterline to dry at the top of the fringe -- the damp beach band

Water Bed

`water_bed_source` - Enum - default **SOLVER

What the flowing water runs OVER. The shallow-water solver is not tied to the dune simulation -- it is plain height plus depth, so any height field will do

Option	Meaning
Dune Solver	The bed is the dune simulation, re-read every step, so pools follow the sand as it migrates. This is what the water did before 1.3.107 and it is unchanged
Mesh Object	The bed is a MESH you pick -- a sculpt, an imported heightfield, a landscape from another add-on. The dune solver is then not needed at all: the water runs on its own, with its own level and its own step. The mesh must be a square grid, the same shape the water plane uses

Bed Mesh

`water_bed_object` - Pointer

The height field the water flows over when Water Bed is Mesh Object. Read in WORLD space, so moving or scaling the object moves the water with it. Must be a square vertex grid

Water Seconds per Step

`water_field_dt` - Float - default **0.25** - 0 to 10

How many seconds of WATER time pass in one simulation step. The two clocks are genuinely different: a dune step is MORPHOLOGICAL and stands for hours of migration, while the water needs about 0.036 s per sub-step to stay stable at this cell size. Raise it for faster waves and quicker draining, lower it for a calm pool. At 0 the field holds still

Water Drag

`water_field_friction` - Float - default **0.5** - 0 to 5

Dissipation per second of water time. NOT an optional polish: with no drag the scheme conserves energy and a pool sloshes for ever. MEASURED, two basins still had 0.19 and 0.41 m of slope across their surfaces after 60 s at drag 0; at 0.5 the surface goes DEAD FLAT and still keeps 72 % of the wave response

Bed Roughness (Manning n)

`water_field_manning` - Float - default **0** - 0 to 0.5

REAL bed friction instead of the flat damping above. Water Drag slows every cell by the same fraction whatever its depth or speed, so a thin film has nothing to stop it and runs away -- MEASURED, a wind-blown film reached tens of m/s and dragged the time step down with it. Manning is the textbook law: the drag goes as speed SQUARED and as depth to the -4/3, so shallow water is held hard and deep water is barely touched. Steady flow then settles at exactly u = h^(2/3) * sqrt(slope) / n, and MEASURED against that closed form this lands within 0.3 % at 20 cm depth. Typical values: 0.011 smooth concrete, 0.025 clean sand, 0.035 gravel, 0.07 dense vegetation. At 0 nothing here runs and the water behaves exactly as before

Time-step Depth Floor (m)

`water_field_dt_floor` - Float - default **0.001** - 0.001 to 0.1

Raises the depth floor the CFL time step reads velocity through, and it buys speed without changing the answer. The velocity a cell reports is flux divided by depth, floored at 1 mm so the shader has a number along the shore -- but the time step reads the same velocity, so one millimetre-deep cell can dictate the step for the whole field. MEASURED at 512 cells on a slope: the cell setting the step held 1.04 mm of water and reported 753.8 m/s against a wave speed of 0.10 m/s, forcing 1206 substeps per 0.25 s where the wave alone needs about four. Raising the floor to 1 cm cut that to 556 substeps (5.1x faster) while mass stayed conserved EXACTLY and the depth field moved by 0.02 %; 3 cm gives 14.6x for 0.03 %. Cross-checked that the floor is not doing the physics: 1 cm against 3 cm differs by 0.03 %. At the default 1 mm nothing here runs and the water is computed bit for bit as before

Evaporation (mm/s)

`water_evaporation` - Float - default **0** - 0 to 50

How fast the surface loses water, in MILLIMETRES PER SECOND OF WATER TIME. Until now this could not be set at all: the solver has always implemented evaporation and nothing ever passed it a value, so a lake in this plug-in could never dry out. The unit is a STAGING number, not a weather one -- a water step is a fraction of a second, so Earth-like 5 mm/day would remove a micrometre over an entire scene. With the Temperature model on it doubles roughly every 10 C and nearly stops at freezing (a floor, not a switch: below zero what continues is sublimation), and with the temperature FIELD on it acts cell by cell -- a pool in a warm hollow shrinks while one on a cold ridge holds

Shields Threshold

`water_shields` - Float - default **0.047** - 0.001 to 0.5

Dimensionless bed shear a grain needs before it moves at all. 0.047 is the Meyer-Peter/Mueller value for sand and is what every published number in this solver was tuned against -- it is a MATERIAL constant, not a taste setting, and moving it moves the boundary between a river that carves and one that does nothing. Raise it for armoured or cohesive beds, lower it for loose fresh deposits. Below the threshold erosion is EXACTLY zero, so a small change here can switch a channel on or off

Ice Cover

`water_field_ice` - Float - default **0** - 0 to 1

Freeze the water surface. A fully frozen cell moves NO water, and that is the whole coupling -- MEASURED, ice 1.0 gives exactly 0.000000 m/s. THE SHALLOWS FREEZE FIRST, which is the part worth having: a lake ices over from the shore because a thin layer loses heat faster than a deep one, so you get a rim of ice with open water in the middle instead of a flat multiplier. With Temperature on, the scene's air temperature scales this; without it the slider works on its own. At 0 nothing here runs

Ice Reaches Down To

`water_ice_depth` - Float - default **0.3** - 0.01 to 20

Water shallower than this freezes; deeper water stays open. The cover ramps LINEARLY from full at zero depth to none at this depth -- a hard threshold would draw a visible edge on the surface where the ground only slopes. Raise it to ice the whole pond over, lower it for a thin shore rim

Freezing Point (C)

`water_freeze_temp` - Float - default **0** - -200 to 40

The temperature the LIQUID freezes at. 0 is water and keeps every existing scene identical; sea water is about -1.8, and liquid methane -- which the Liquid Density slider can now describe -- freezes at -182. Needs the Temperature model: with it off, the ice cover is shaped by DEPTH alone, exactly as before. With the temperature FIELD on, the decision is made cell by cell, so a lake can freeze in a cold hollow and stay open where it is warmer -- the two causes MULTIPLY, because ice needs both shallow water and cold air

Ice Thickness (m)

`water_ice_thickness` - Float - default **0** - 0 to 3

Thickness at which the ice cover is COMPLETE, in metres. 0 keeps the shipped behaviour, where ice is stateless: the cover is recomputed from depth and frost every step, so it appears and vanishes the instant the air crosses freezing and a 20 cm sheet behaves exactly like hoar frost. Above 0 the ice becomes a THICKNESS that accumulates: it grows while the air is below the freezing point, melts while it is above, and carries over between steps. Needs the Temperature model -- without one there is nothing for it to grow or melt from

Ice Growth (mm/s per K)

`water_ice_rate` - Float - default **2** - 0 to 100

How fast ice grows or melts, in millimetres per second of WATER time per kelvin below or above the freezing point -- the same staging clock as Evaporation, not a weather rate. Growth is SLOWED BY THE ICE ALREADY THERE, because a thickening sheet insulates the water from the cold (at the full thickness above, growth runs at half speed); melting is not, because heat attacks the surface instead of conducting through the slab. That one asymmetry is the whole inertia of the model -- thick ice takes time to form AND time to lose, with no time constant invented for the purpose

Ice Drift

`water_ice_drift` - Float - default **0** - 0 to 1

Let the ice SHEET TRAVEL with the current instead of growing and melting where it formed. This is the fraction of the water velocity the ice takes: 1.0 rides the current exactly, 0.3 lags behind it. The transport is CONSERVATIVE -- fluxes across cell faces, not a resampled field -- so two things fall out of it without a rule for either: where the current converges the ice PILES UP, and where it pulls apart the sheet OPENS. Ice cannot flow onto dry ground, so it ridges against the shore. At 0 the sheet is fixed in place, exactly as before

Ice Sheet Strength

`water_ice_strength` - Float - default **0** - 0 to 20

How much THICK ice resists being carried. A sheet is not a dye in the water: a metre of pack does not run with the current the way fresh skim does. Drift is divided by (1 + Strength x thickness), so thick floes lag and thin ice and open leads move freely. NOT a plate model: this damps cell by cell, so it will not give you rigid floes moving as one body or failure along a line. At 0 the ice is carried passively. Only used when Ice Drift is above 0

Floes Move As Bodies

`water_ice_rigid` - Float - default **0** - 0 to 1

Make each FLOE travel as one piece instead of every cell of it following its own bit of current. At 0 the pack drifts the way it always did: MEASURED on the Ice: Pack scene over twelve steps, the largest floe lost 10 % of its area (564 592 -> 507 155 cells) while its raggedness -- perimeter over the square root of area -- grew by 52 % (9.07 -> 13.75) and its elongation barely moved. That is a sheet being EATEN AT THE EDGES, which is what a fluid does; a body keeps its shape. Raise it and every cell of a floe takes the MEAN velocity of that floe, so there is no shearing inside it. MID VALUES ARE THE USEFUL ONES, and that is measured, not a guess: at 0.5 the fine crumbling stops (raggedness 13.75 -> 6.79) while the big structural leads stay open and the pack is still 3 622 pieces. At 1 it goes further than you may want -- leads open from velocity DIVERGENCE, and averaging inside a floe zeroes that, so no new lead forms, the mask stays in one piece, and the whole basin becomes a SINGLE body: 6 122 floes -> 2, ice cover 0.69 -> 0.90. That is landfast ice, not a drifting pack. Costs one extra floe labelling per step: MEASURED on a 1024x1024 field on the GPU, the ice drift step goes 6.7 ms -> 36.6 ms, so that member alone is 5.5x its old cost. Off by default. Try 0 -> 0.5 with Ice Drift on and watch the floe edges stop crumbling while the leads survive

Ice Yield Strength

`water_ice_yield` - Float - default **0** - 0 to 200

How hard it is to push ice INTO ice that is already thick. This is NOT Ice Sheet Strength: that one slows a thick floe down everywhere, this one only resists PILING UP. Flow into a cell is divided by (1 + Yield x thickness there), so open water takes ice for free and a ridge refuses more. Without it the pack thickens a little everywhere instead of failing in one place and moving as a body. MEASURED on a shore fixture: peak thickness 12.80 m with no yield, 0.65 m at 40. Mass is conserved either way -- what does not flow stays on the source side. At 0 the drift is what it was before, to the bit. Only used when Ice Drift is above 0

Ice Crack Threshold

`water_ice_crack` - Float - default **0** - 0 to 10

Divergence, in 1/s, that fully cracks the sheet in one second. Where the current PULLS APART the ice is stretched and opens leads; where it converges the ice ridges instead. The measure is the divergence of VELOCITY, not of flux: flux scales with thickness, so thin ice -- which cracks most easily -- would never reach a flux threshold. A crack is a STATE, not an event: once open it stays and travels with the floe until it heals, so a render can actually catch it. Lower values crack sooner. At 0 nothing cracks and the result is bit-for-bit the old drift

Ice Crack Healing

`water_ice_heal` - Float - default **0** - 0 to 5

How fast open leads freeze shut again, in crack fraction per second, once the stretching stops. At 0 a lead stays open for good, which is what you want for a single dramatic break-up; turn it up for a pack that keeps re-forming. MEASURED: crack fraction in the ice 0.368 after break-up, 0.000 after 30 quiet steps at 0.05, and still 0.368 after the same 30 steps at 0. Only used when Ice Crack Threshold is above 0

Ice Crack Propagation

`water_ice_crack_prop` - Float - default **0** - 0 to 40

Let a crack RUN instead of spreading. Damage computed cell by cell from the local divergence weakens the whole stretched band evenly, so the sheet gets weaker everywhere and severs NOWHERE. MEASURED on the Ice Pack demo without this: 205 connected ice bodies, but a MEDIAN of ONE CELL and a largest body of 570480 -- one huge sheet plus 198 single pixels, against 2 bodies in the no-crack control. This multiplies the growth rate at cells that already touch an open crack AND have at most two cracked neighbours, so a lead extends at its TIP rather than fattening into a blob. It also gives the ice a fixed pattern of material flaws, without which a smooth swell stretches every cell equally and localisation has nothing to pick. At 0 the crack field is bit-for-bit what it was. Only used when Ice Crack Threshold is above 0

Count Ice Floes

`water_ice_floes` - Float - default **0** - 0 to 1

Label the ice as separate BODIES, not just a field with cracks in it. Since 1.3.142 the cracks genuinely cut the sheet -- MEASURED on the field itself with the converged, closed-boundary labelling of 1.3.148, the largest connected piece drops from 0.708 of the ice with crack propagation off to 0.332 with it on, at the same threshold -- but nothing NUMBERED those pieces, so the plug-in could not say how many floes there are, how big they are, or give two neighbours different colours. This flood-fills the ice that cracks do not cut, using a minimum-index propagation: the label is a function of SHAPE, not of visiting order, so it is identical on CPU and GPU and stable between steps. It costs a few dozen passes over the field, which is why it is off by default. Feeds Ice Floe Tint in the material and the floe readout in this panel

Ice Floe Tint

`water_ice_floe_tint` - Float - default **0** - 0 to 1

Give neighbouring floes DIFFERENT shades so the eye separates them. Since 1.3.142 the cracks genuinely cut the pack, but every piece rendered in the same colour, so it read as one wrinkled sheet. The shade comes from the floe LABEL, so a piece keeps its own tone between steps instead of flickering. Applied around the mid grey rather than from zero, which leaves the average brightness of the ice alone and changes only the contrast BETWEEN neighbours. Needs Count Ice Floes on -- without labels there is nothing to tint. At 0 the material is unchanged

Dark Floe Bias

`water_ice_floe_dark` - Float - default **0** - 0 to 1

Push the DARK half of the floe tint harder, which is the half that actually reaches the picture. Ice Colour sits at (0.86, 0.91, 0.95), so a bright floe has only 0.04 of headroom left toward white while a dark one has 0.12 to fall -- the tint is lopsided by construction. MEASURED on the Ice: Pack scene, same frame for every option, as the difference between tint 0 and tint 0.55: shipped 0.0251 peak with 10.5 % of pixels moving a visible step, this at 1.0 gives 0.0789 peak and 15.9 %. Two things it is NOT: lowering Ice Colour to make room (measured WORSE -- 0.0230 peak and only 1.7 % of pixels, because dropping the colour takes more from the dark half than it gives the bright one), and pushing the roughness contrast instead (1.03x looking straight down, where a specular has no way to reach the lens at all, and 1.18x from a grazing camera -- real, but small next to this). At 0 the shader is what it was: the multiplier works out to exactly -1.0, the constant that was hard-coded there before. Needs Ice Floe Tint above 0

Ice Look

`water_ice_look` - Float - default **0** - 0 to 1

Let the ice LOOK like ice instead of like water. Until 1.3.135 the sheet had a thickness the geometry knew about -- it lifts the surface -- but the material had no ice input at all, so a metre of pack rendered exactly like open water. MEASURED on a current scene: switching drift on raised peak thickness 0.183 to 1.961 m, a factor of TEN, and changed 0.14% of the pixels. At 0 the material is what it was before, to the bit

Ice Colour

`water_ice_color` - FloatVector - default **0.86, 0.91, 0.95** - 0 to 1

Colour of full-thickness ice. Kept separate from the terrain Ice Colour on purpose: frozen ground and a floating sheet are lit differently and are rarely the same white

Ice Full Depth

`water_ice_full` - Float - default **0.3** - 0.005 to 20

Thickness at which the ice reads as SOLID -- fully coloured, matt and opaque. Below it the sheet is translucent and the water shows through, which is what thin new ice actually looks like. Ridges run far past this value, so set it near the thickness the sheet grows to and let the pressure ridges saturate

Ice Roughness

`water_ice_roughness` - Float - default **0.35** - 0 to 1

Surface roughness of solid ice. Higher than water on purpose: a mirror-smooth sheet reads as wet asphalt, not as ice. Blends from the water roughness up to this value as the sheet thickens

Ice Crack Depth

`water_ice_crack_look` - Float - default **1** - 0 to 20

How much a crack breaks the ICE LOOK, showing the water below. At 1 a fully cracked cell renders as open water even though the thickness is still there; at 0 the leads are simulated but invisible. This knob does not need an identity default: with Ice Crack Threshold at 0 the crack field is zero everywhere, so the material is unchanged whatever this is set to. Only used when Ice Look is above 0

Ice Crack Sharpness

`water_ice_crack_sharp` - Float - default **1** - 1 to 8

Turn a lead from a WASH into a LINE. The crack fraction subtracts the ice look in proportion, so a crack of 0.3 used to mist a wide band by 30% instead of cutting a dark line -- a fill, not a crack. This raises the fraction to a power: at 3 a value of 0.3 drops to 0.027 while 0.9 stays at 0.729, so only the CORE of the lead survives and its blurred edges go. Raise it when leads look like smudges; lower it when they look like scratches. At 1 it is the first power, which is the field unchanged

Ice Ridge

`water_ice_ridge` - Float - default **0** - 0 to 20

Make PILED-UP ice look piled up. Until 1.3.141 the material normalised thickness by Ice Full Depth and CLAMPED it, so a ridge and ordinary pack rendered identically -- the pile-up was not weak, it had no way out of the shader. MEASURED on the Ice Pack demo: peak-to-mean thickness 1.765 with Ice Yield Strength on against 2.426 with it off, and the render could not tell them apart at all. This reads the excess ABOVE full ice, so the knob behaves the same on a thin scene and a thick one, and paints it with Ice Ridge Colour plus a rougher surface. At 0 nothing is mixed and the material is what it was

Ice Ridge Colour

`water_ice_ridge_color` - FloatVector - default **0.97, 0.98, 1.0, 1.0** - 0 to 1

Colour of ice that has been RAFTED AND BROKEN rather than frozen flat. Rubble scatters light from every angle, so a ridge reads whiter and flatter than the sheet around it -- which is what makes it legible from above. Only used when Ice Ridge is above 0

Water Thermal Inertia (s)

`water_temp_inertia` - Float - default **0** - 0 to 600

How long the WATER takes to follow the air temperature, in seconds of water time -- the time in which the difference falls e-fold. 0 means the water is always exactly at air temperature, which is what shipped and what every existing scene computes. Above 0 the water has heat capacity: a metre-deep lake no longer reacts to an overnight frost as fast as a puddle does, and the ice that forms lags the weather instead of tracking it

Latent Heat (K per m of ice)

`water_latent_heat` - Float - default **0** - 0 to 200

How much freezing WARMS the water, in kelvin per metre of ice formed -- and how much melting cools it, by the same number. This is the feedback that makes a lake DEFEND ITSELF against freezing: the moment a sheet starts to form it releases heat, the water warms, and further freezing slows down. Needs Water Thermal Inertia above 0, because water at air temperature by definition cannot be warmed by anything. Given in kelvin per metre rather than joules on purpose: this model has a temperature but no mass and no specific heat, so quoting an energy would be pretending to a thermodynamics the rest of it does not have

Ice Insulates (per m)

`water_ice_insulation` - Float - default **0** - 0 to 100

How much an ice sheet SLOWS the water down in following the air temperature, per metre of ice. Heat has to cross the sheet, so water under a thick cover reacts to the weather far more slowly than open water does; at 0 the cover is thermally transparent, which is what shipped. It multiplies the Thermal Inertia TIME rather than the temperature:

insulation changes how FAST the water approaches the air, never what it approaches -- adding it to the temperature itself would leave water warmer than the air for ever. Needs Water Thermal Inertia above 0

Inertia Depth Reference (m)

`water_temp_depth_ref` - Float - default **0** - 0 to 20

Depth at which Water Thermal Inertia means exactly what the slider says. 0 turns the scaling off and every cell gets the same time constant, which is what shipped. Above 0 the inertia scales with the water column, because heat capacity grows with VOLUME: at reference 1 m a two-metre pool takes twice as long to follow the air as the slider says, and a five-centimetre puddle a twentieth as long. Without this a puddle and a lake cool at the same rate, which is the same simplification as one air temperature for the whole scene was

Rain Feeds the Water

`water_field_rain` - Bool - default **on

Let Rain add water to the field, so pools grow where it falls and drain where the ground tilts. Needs Rain to be switched on as well -- on its own this changes nothing

Water Sees the Rocks

`water_bed_obstacles` - Bool - default **off

Let the flowing water run AROUND obstacles instead of straight through them. The dune solver keeps rocks in their own array and reads the surface as max(sand, rock) for the wind -- but the water bed was plain sand height, so a boulder only diverted the stream as far as the sand piled against it had grown. With this on the rock is part of the bed: the current splits, an eddy forms behind it and, with erosion on, a scour hole opens in front. Off by default because switching it on changes the flow in every existing scene that has obstacles

Water Leaves at Open X

`water_open_x` - Bool - default **off

Let the water field really LEAVE the terrain through the X edges when Boundary is Open X. The water grid is periodic on its own: water reaching the bottom of a valley sees the far edge of the terrain wrapped around behind it and piles up against it (a sluice flood kept its mass to the last digit for 100 frames while the valley was supposedly open). With this on, water that reaches either X edge is removed every substep and counted as outflow. Only acts when Boundary is Open X; off by default, so every existing scene keeps its water

Water Leaves at Y Edges

`water_open_y` - Bool - default **off

The same as Water Leaves at Open X, but for the Y edges: water reaching the top or bottom edge of the terrain is removed every substep and counted as outflow. ! THE SAND STILL WRAPS IN Y. Boundary has only Periodic and Open X -- there is no Open Y for the sand -- so this option cannot be tied to the sand boundary the way the X one is, and it leaves the two asymmetric on purpose: the water leaves, the dunes do not. Use it for a valley that drains sideways, or for a scene where water piling against the wrapped Y edge is the thing you are fighting. Off by default

Water Wets the Sand

`water_field_moisture` - Bool - default **off

Let the flowing water tell the DUNE simulation where it actually is. Until now that conversation ran one way: the sand told the water where the ground was and how the wind blew, and the water told it nothing -- damp sand came from ONE GLOBAL LEVEL, so a puddle the rain cut into a crest did not exist for the wind, while a dry hollow below sea level counted as wet. With this on the wetness is a MAP: wet sand resists wind erosion exactly where water is standing, so a stream cuts its own armoured banks and a drying pool releases its floor again. Combines with the painted Water mask rather than replacing it. Off by default

Swell Height (field)

`water_wave_field` - Float - default **0** - 0 to 5

Make a REAL wave in the water field, not a bump map on the surface. The difference is everything a shader cannot know: the wave slows down and steepens as it runs into the shallows, BENDS to face the shore, splits around a rock and dies in its lee -- all of it because the celerity is sqrt(g x depth) and depth is a field. This is the AMPLITUDE at the generator in metres (half the crest-to-trough height); MEASURED, the field delivers 1.0001 of what is asked. Long waves only -- see Swell Length. At 0 nothing here runs and the water behaves exactly as it did before

Swell Length

`water_wave_length` - Float - default **8** - 0.5 to 200

Crest-to-crest distance in metres. TWO HARD LIMITS, and the panel reports both. First, a grid cannot carry a wave shorter than about EIGHT CELLS -- at Medium detail that is 1.57 m, and asking for less does not give a small wave, it gives a two-cell staircase with the wrong speed; that band belongs to Ripple Size in the shader. Second, the field WRAPS, so only wavelengths that fit a whole number of times across the scene come back to themselves at the seam -- the length is snapped to the nearest one that does, and the panel prints what it became. The PERIOD is not a separate slider: shallow water is non-dispersive, so it is length divided by sqrt(g x depth) and nothing else

Swell Direction

`water_wave_dir` - Float - default **0** - -360 to 360

Where the swell travels, in degrees: 0 is +X, 90 is +Y -- the same convention as Wind Direction. Like the length, the direction is QUANTISED by the wrapping grid (the wave vector has to land on whole numbers of wavelengths across each axis), so the panel shows the angle you asked for next to the one you got. The difference shrinks as the wave gets shorter relative to the scene

Swell Components

`water_wave_count` - Int - default **1** - 1 to 9

How many wave trains make the swell. ONE is a pure sinusoid and it always reads as corrugated metal -- infinite parallel crests, all the same height, marching in step. A real sea is a SPECTRUM: several trains at slightly different lengths and headings, which beat against each other so crests END instead of crossing the whole scene, and the odd big one stands out. MEASURED, the peak-to-RMS ratio of the surface goes 1.88 at one component, 3.35 at three and 4.14 at five, while the ENERGY stays put -- each train carries 1/sqrt(N) of the amplitude, so this breaks the wave up rather than piling it higher. Costs one extra term per component

Swell Spread

`water_wave_spread` - Float - default **0** - 0 to 60

How far the outer components fan out from Swell Direction, in degrees. 0 is a laser-straight swell from one heading; 15-25 is open water with a steady wind; past 40 the trains cross enough to read as confused chop. Does nothing at one component

Swell Length Spread

`water_wave_spread_len` - Float - default **0** - 0 to 0.6

How much the components differ in LENGTH, as a fraction of Swell Length. This is what makes the group beat: two trains a few per cent apart drift in and out of phase, so the sea has calm stretches and sets of bigger waves instead of one endless rhythm. 0.15 to 0.30 is a natural sea. Does nothing at one component

Swell Comes From

`water_wave_origin` - Float - default **0.15** - 0 to 1

Where across the scene the wave generator sits, as a fraction of the domain -- 0.15 means one-seventh of the way in, so the swell has the rest of the scene to travel across. Move it to put the generator behind the camera, or to the far side to send the swell the other way without changing Swell Direction

Generator Width

`water_wave_band` - Float - default **0.5** - 0.15 to 2

Width of the generator band, in WAVELENGTHS. It is a real trade-off, not a tuning knob: a wide band picks the wavelength cleanly and radiates in one direction, but eats scene you wanted for the wave to travel across. MEASURED with a band of 2 m on a 16 m wave, the dominant mode came out 3 instead of the requested 4 and the far-field amplitude was 0.34 of what was asked. Half a wavelength is the compromise that ships

Water Erodes the Sand

`water_erosion` - Float - default **0** - 0 to 20

Let the flowing water CARVE the bed instead of only running over it: a channel cut into the slope, a fan where it opens out, a silt plume, and a sand streak left behind once the plume settles. Three published laws, none of them a tuning knob: a grain moves only above the SHIELDS threshold (0.047 for sand, and below it the erosion is EXACTLY zero, not small), it is picked up at VAN RIJN pickup rate, and the bed follows EXNER. This slider MULTIPLIES the real pickup rate, which for 250 um sand at twice the critical stress is 3.65e-05 m/s -- so 1.0 is the speed nature runs at and higher values buy a river worth of carving in a few frames. Needs Flowing Water (2D field). At 0 nothing here runs and the water behaves exactly as it did in 1.3.108, bit for bit

Grain Size

`water_grain_size` - Float - default **250** - 4 to 2000

Diameter of the grain the water is moving, in MICROMETRES. It is the single most consequential number here because it enters three places at once: the Shields threshold rises with it (coarse sand needs a faster current to move at all), the settling speed rises with its SQUARE (250 um sand falls 3.18 cm/s and clears 20 cm of water in 6.3 s, while 25 um mud falls 50 times slower and stays suspended long enough to be carried out of frame), and the bed roughness goes as its sixth root. 250 is medium sand, 60 is silt, 1000 is coarse sand

Bed Porosity

`water_bed_porosity` - Float - default **0.4** - 0 to 0.75

Fraction of the settled bed that is holes rather than grain. It is what converts grain VOLUME into bed HEIGHT: at 0.40 a millimetre of grain raises the bed by 1.67 mm, so a looser bed both erodes into a deeper channel and builds a taller bar from the same amount of sand. 0.40 is loose sand, 0.25 is well-packed

Transport Capacity

`water_transport_capacity` - Float - default **0.2** - 0.01 to 0.6

The most sand the water may carry, as a VOLUME FRACTION -- and in an energetic scene this, not Erodibility, is the knob that steers. MEASURED on the Water demo: Erodibility 6.0 and 0.6 gave the same result to three digits (97.577 against 97.579 % of the bed moved) because the flow was already at capacity everywhere, so the pickup rate had stopped mattering. 0.20 by volume is about 40 % by weight, the accepted boundary above which the mixture stops being water carrying sand and becomes a debris flow -- the shallow-water equations use clear-water density and do not describe that. A river in flood carries 0.1 to 1 %. Lower this for a clear stream that only nibbles; raise it for a flash flood

Carved Bed Goes To

`water_erosion_target` - Enum - default **FIELD

Who gets to see the channel the water cut. The change is tracked as its own layer either way -- this only decides whether it is handed over to the dune simulation

Option	Meaning
Water Only	The dune solver never sees it. The water flows over its own carved bed and the shading follows (deeper where it scoured, shallower over a bar), but the sand simulation, avalanches and the terrain mesh are untouched. Reversible: switch the erosion off and the terrain is exactly as it was
The Dune Terrain	The change is added into the dune heightfield itself, so wind, avalanches and the rendered mesh all see the channel and can rework it. This is the one that lets a wadi survive after the water is gone -- and it is PERMANENT, because the sand solver now owns the change. Note the two grids differ, so the hand-over is resampled and is not exact to the gram

Local Wind Field

`water_wind_local` - Bool - default **on

Take the wind on the water from the SOLVED flow field, so the water feels the speed-up over a crest and the dead air behind a rock instead of one direction for the whole scene. Needs Wind Model set to Flow 2D -- with the analytic wind there is no local field to read and this falls back to uniform. MEASURED on a wind that varies across the domain: the lee shore then stands at nine different millimetres across its width, against exactly zero variation for the same wind averaged to one direction

Water Bodies (several levels)

`use_water_bodies` - Bool - default **off

ADD ponds at their OWN heights on top of the single Water Level. Terraces are the case this exists for: a stepped landscape wants a different waterline on each step, and one global number cannot say that. Water Level stays the BASE -- it is what every cell no body covers still uses -- so switching this off returns the scene to exactly the model it had. Each object's TOP SURFACE, sampled straight down, is its water level: a plane sets one height across its footprint, a box sets the height of its lid. The object you place to SEE the water is therefore the one that defines it. Where two bodies overlap the LOWER level wins, because water from a higher terrace would spill into the lower one -- so the result never depends on which order the objects are in

Water Body

`water_body_object` - Pointer

One object whose top surface is a water level

Water Bodies

`water_body_collection` - Pointer

A collection of objects, one per pond. They are combined by taking the LOWEST level wherever they overlap

Wet Hold

`water_hold` - Float - default **0.9** - 0 to 1

How strongly wet material resists WIND erosion (1 = fully submerged ground is untouchable, 0 = water has no holding effect). Deposition is never blocked -- sand still settles into the water

Storm Cell Size

`rain_cell_size` - Float - default **60** - 1 to 10000

Size of a rain cell in WORLD units, so it keeps its meaning when you change solver resolution or scale the terrain. Small cells give scattered showers that mottle the field; cells larger than the terrain give whole-field weather that comes and goes

Cell Drift

`rain_speed` - Float - default **2.5** - 0 to 20

How fast storm cells ride the wind, in grid cells per step. They follow the same smoothed wind direction the ripples and the sand haze use, so the weather agrees with the wind. KEEP IT ABOVE ABOUT 0.3: a dune migrates at roughly 0.1 cells per step (measured on a single dune, 150 steps), so a slower drift means the damp patches fall BEHIND the sand and appear to crawl upwind even though both are moving downwind. Real storm cells cross a dune field in minutes and the dunes take years, so faster is also the physical answer. 0 = the pattern stands still (useful only for a still frame)

Rain Shadow

`rain_shadow` - Float - default **0.7** - 0 to 1

How dry it stays in the lee of obstacles and dune crests. Uses the wind-shadow mask the solver already computes, so it costs one multiply. 1 = the lee gets no rain at all

Surface Dries In

`rain_dry_surface` - Float - default **1** - 0.1 to 100000

Time constant of the TOP layer, in simulation steps -- the layer that decides whether the wind can pick a grain up. After this many steps about 63% of the moisture is gone. Drying is exponential, so any value is numerically safe. THE DEFAULT IS THE MODEL'S RESOLUTION LIMIT, NOT A PHYSICAL NUMBER, and it is worth knowing why. A step is worth roughly a DAY here (see Wind Speed at 10 m), while wet sand surface-dries in about an hour -- so the true value is about 1/25 of a step, and a process faster than the time step cannot be represented. Setting it there was measured to leave the moisture field identically ZERO even while it is pouring, because rain wets and evaporation dries within the same step -- which would silently take the Wetness mask, the moisture threshold, the drying crust and the frozen-ground coupling with it. 1.0 means "gone by the next step", the closest this clock can come. It shipped at 40 until 1.3.11, i.e. forty DAYS of wet sand

Root Zone Dries In

`rain_dry_deep` - Float - default **14** - 1 to 1e+06

Time constant of the DEEP bucket, in simulation steps. This is the water plants live on, and it should be far longer than the surface: days against an hour. It is what makes a field stay green -- and therefore stable -- long after the rain stopped. With a step worth roughly a day (see Wind Speed at 10 m), the default 14 is a fortnight, which is what a real root zone takes to drain. It shipped at 1500 until 1.3.11 -- over four YEARS, and measured: sixty dry steps left the root zone still at 0.980 of full, i.e. it never emptied at all. At 14 the same run drains to 0.008 while the plants still end up green (0.9988 against 0.9998), so this costs the vegetation loop essentially nothing

Infiltration

`rain_infiltration` - Float - default **0.05** - 0 to 1

Fraction of the surface moisture that soaks down into the root zone each step. It is capacity-limited, so already-saturated ground stops absorbing. 0 = nothing soaks in, and rain then only ever affects the wind threshold, never the vegetation

Wind Drying

`rain_wind_drying` - Float - default **0.6** - 0 to 2

How much wind exposure speeds up drying. Exposed crests dry first and start moving again while sheltered lee slopes stay damp, so this is what makes a drying field come back to life unevenly rather than all at once. 0 = everything dries at the same rate

Wet Hold (Rain)

`rain_hold` - Float - default **0.9** - 0 to 1

How strongly damp sand resists WIND erosion, exactly like Wet Hold does for the water table. Transport dies out at only a few percent of moisture in reality, so this is deliberately strong. Deposition is never blocked -- sand still lands on wet ground

Drying Crust

`rain_crust` - Float - default **0.4** - 0 to 1

Ground that got wet and then dried leaves a cohesive skin behind. The crust SURVIVES the rain that made it, so a rained-on field stays harder to erode long after it looks dry. Shares the same crust field the snow model uses. 0 = no crust

Growth From Rain

`rain_veg_growth` - Float - default **1** - 0 to 8

How much the root-zone moisture accelerates plant growth (1 = up to double in a fully wet root zone). Plants then shield the sand, so this single number is the strongest long-term loop rain has: a field visibly settles down after a wet season and starts moving again once it dries out. Needs Vegetation enabled

Splash Particles

`splash_enabled` - Bool - default **off

Throw real droplets up where raindrops land. They are spawned from the SAME instantaneous rainfall field the shader impact layer uses -- the storm-cell field with the rain shadow already applied -- so the particles and the rings on the ground always agree about where it is raining. Needs Rain enabled

Max Droplets

`splash_max` - Int - default **8000** - 0 to 400000

Size of the droplet pool. It is a FIXED pool: the mesh keeps the same vertex count every frame, which is what lets Cycles deformation motion blur work and saves Blender rebuilding the draw buffer. Dead droplets collapse to a zero-area triangle

Spawn Rate

`splash_rate` - Float - default **0.35** - 0 to 1

How much of the free pool is filled each step, weighted by where it is raining. Capped internally so the pool ramps in rather than bursting on the first raining frame, which reads as a pop

Kick Height

`splash_speed` - Float - default **0.09** - 0 to 2

Upward launch speed of a droplet, in world units per step. Varied per droplet so the crown is not a uniform fountain

Spread

`splash_spread` - Float - default **0.35** - 0 to 5

Sideways scatter of a droplet, in grid cells per step. 0 = they go straight up and fall back into their own crater

Droplet Life

`splash_life` - Float - default **9** - 1 to 200

How many simulation steps a droplet lives before it expires. It also dies the moment it lands, so this is mostly a ceiling

Droplet Size

`splash_size` - Float - default **0.3** - 0.01 to 20

Droplet size, as a multiple of one grid cell. Randomised per droplet and tapered with age so they fade out instead of vanishing mid-air

Droplet Stretch

`splash_stretch` - Float - default **2.6** - 0.2 to 12

How far each droplet is drawn out vertically. A falling drop is not a ball. The droplets are real instanced spheres placed on the particle pool by a Geometry Nodes modifier -- the pool itself still writes one small triangle per droplet, because a FIXED vertex count is what lets deformation motion blur work, and the modifier draws spheres on those triangles rather than the triangles themselves

Droplet Detail

`splash_detail` - Int - default **2** - 1 to 4

Ico-sphere subdivisions per droplet. 1 is an octahedron and is plenty at a distance; 3-4 only pays off in a close-up, and the cost is per LIVE droplet times Max Droplets

Teardrop Shape

`splash_teardrop` - Float - default **0.65** - 0 to 1

How far the droplet is drawn to a point. 0 is a ball, 1 is a long tapered tear. It shapes the instanced mesh itself (the top half of a sphere pulled to a point), so it costs nothing per droplet -- the shape is built once and instanced

Impact Squash

`splash_impact_squash` - Float - default **0** - 0 to 1

How hard a droplet pancakes as it lands. 0 = off and the shape is exactly the shipped one; 1 = flattened to about a sixth of its length and spread near double sideways. It squashes ALONG its own flight direction, which is the axis a drop really deforms on, so a droplet coming down steeply reads as a disc on the sand while one thrown sideways stays a tear. Costs nothing per droplet -- it scales the instance, it does not build new geometry. Pair it with Impact Distance, which sets how early it begins

Impact Distance

`splash_impact_dist` - Float - default **0.025** - 0 to 50

How close to the ground, in world units, a droplet starts to flatten. Only a FALLING droplet counts -- one still on its way up is not about to hit anything even though it is right above the surface it launched from. Geometry Nodes cannot know this on its own: the particle pool measures the distance against the simulated heightfield and carries it on the droplet mesh. MEASURED at the shipped Kick Height of 0.09: a falling droplet sits between 0.006 and 0.062 units above the ground (median 0.029), so 0.025 squashes roughly the last half of the descent and anything above ~0.07 squashes every droplet in the air. Raise Kick Height and the arc grows, so raise this with it. Does nothing while Impact Squash is 0

Show Airborne Droplets

`splash_show_drops` - Bool - default **on

Draw the flying droplets. Switch it OFF to keep the impacts on the ground -- the rings, the craters and the wet spots, which live in the shader -- without any particles in the air. The pool keeps running, so turning it back on does not restart from empty

Impact Crown

`splash_crown_enabled` - Bool - default **off

Add a SECOND, independent droplet population: the crown that leaves the impact itself. The first population is spray thrown up out of a drop -- fewer, larger, longer-lived, drifting and falling back; the crown is the rim that jumps outward the instant a drop lands -- many, tiny, brief and low. They run as separate pools with their own object and their own instancer, so neither can eat the other slots and you can shade or hide them apart. Costs a second mesh write per frame

Crown Droplets

`splash_crown_max` - Int - default **4000** - 0 to 400000

Size of the crown pool, counted separately from Max Droplets. A crown is many small droplets, so this usually wants to be as high as the spray pool or higher even though each droplet is smaller

Crown Spawn Rate

`splash_crown_rate` - Float - default **0.55** - 0 to 1

How much of the free crown pool is filled each step. Higher than the spray by default because crown droplets die within a few steps, so the pool has to be refilled that much faster to keep a steady population

Crown Kick Height

`splash_crown_speed` - Float - default **0.05** - 0 to 2

Upward launch speed of a crown droplet. Deliberately lower than the spray: a crown stays close to the ground and is gone quickly, it does not arc away

Crown Spread

`splash_crown_spread` - Float - default **1.2** - 0 to 8

Sideways speed of a crown droplet, in grid cells per step. Much wider than the spray -- that outward throw is what makes a crown read as a crown rather than a puff

Crown Ring

`splash_crown_ring` - Float - default **0.8** - 0 to 1

How tightly the outward speeds cluster at the rim. 0 scatters droplets through a filled DISC, which is right for spray; 1 sends nearly all of them out at the same speed, so they stay on a common expanding circle -- an actual crown. Only this population has it: the spray keeps the disc

Crown Life

`splash_crown_life` - Float - default **4** - 1 to 200

How many simulation steps a crown droplet lives. Short on purpose: a real crown collapses back into the surface almost at once. It also dies the moment it lands, so this is a ceiling

Crown Droplet Size

`splash_crown_size` - Float - default **0.15** - 0.01 to 20

Crown droplet size as a multiple of one grid cell. Half the spray default, because the rim of a splash is finer than the drops that arc up out of it

Crown Stretch

`splash_crown_stretch` - Float - default **1.4** - 0.2 to 12

How far a crown droplet is drawn out along its travel. Lower than the spray: a droplet that has just been flung sideways has not had time to be pulled into a tear yet

Crown Teardrop

`splash_crown_teardrop` - Float - default **0.35** - 0 to 1

How far a crown droplet is drawn to a point. 0 is a ball, 1 a long tapered tear. Rounder than the spray for the same reason as Crown Stretch. Like the spray it shapes the instanced mesh once, so it costs nothing per droplet

Match Terrain Grid

`water_res_match` - Bool - default **on

Build the water plane at the SAME grid as the terrain, one water vertex per terrain vertex. That is the right number, not a taste: the depth under each vertex is read off the terrain grid, and at a coarser water grid it is POINT-SAMPLED, never averaged. MEASURED on the Oasis demo (terrain 512x512): at Water Grid 128 only 6.25 % of the terrain reaches the render, at 256 it is 25 %, at 512 it is 100 %, and at 1024 it is 400 % -- the same terrain sampled twice, which buys nothing. It became affordable in 1.3.106: the plane used the slow per-element vertex API, which cost 10.7 ms of terrain read plus 32.5 ms of vertex write per re-level REGARDLESS of this setting. Off, Water Grid below is used as typed

Water Grid

`water_res` - Int - default **128** - 2 to 2048

Vertex resolution of the water plane. It is a GRID and not a quad because every vertex carries the water depth beneath it -- that field is what drives the depth colour, the underwater fog, the opacity and the shoreline foam. Higher = a finer SHORELINE; the cost is vertices, not shader time. IT DOES NOT AFFECT THE WAVES: those are a shader bump, not geometry, so they are as fine as the render regardless of this -- and they only appear in Rendered or Material Preview shading, never in Solid

Water Follows Tide

`water_follow_tide` - Bool - default **off

Re-level the water surface every frame to the TIDE level instead of pinning it to Water Level, and re-bake the depth with it, so the shore floods and drains. Without this the plane sits at the mean level while the damp band migrates underneath it -- the simulation has a tide, the picture does not. Needs Tides on in Extra Phenomena

Silt Clouds the Water

`water_silt_murkiness` - Float - default **0** - 0 to 200

Add the SUSPENDED SEDIMENT to the extinction, so the water is murky WHERE the sand is in it and clear a metre away. This is the thing a shader cannot fake: Murkiness above is one number for the whole scene, so silty water is silty everywhere, including where nothing stirred it. THE SCALE: the extinction gains this times the VOLUME FRACTION of suspended sand, so read the peak concentration off the Water panel first -- a river in flood carries 0.1 to 1 %, where 100 here adds 0.1 to 1 per metre and reads as proper muddy water, while a scene already at 20 % needs about 5 or the surface goes to milk. MEASURED: at 40 with a 29 % peak the extinction hit 11.9 per metre and 20 cm of water rendered fully opaque. Needs Water Erodes the Sand on -- with no suspended load the field reads zero and the extinction is EXACTLY the Murkiness above. At 0 nothing here runs

Underwater Fog

`water_fog` - Float - default **0.45** - 0 to 1

How much murky water SCATTERS rather than absorbs -- milky, not just dark. A true volume needs a closed mesh; this is the same exponential used as scattering, which reads the same at these depths and works in EEVEE as well as Cycles

Choppiness

`water_choppiness` - Float - default **0.55** - 0 to 1

Stretches the wave lookup along one axis. Wind waves are long across the wind and short along it, and squashing the lookup is what produces that -- a second noise layer just gives a bumpier carpet

Ripple Size (m)

`water_ripple_size` - Float - default **0.28** - 0.02 to 5

Wavelength of the short chop riding on the swell

Ripple Height

`water_ripple_height` - Float - default **0.012** - 0 to 0.5

Strength of the short chop. It also breaks up the foam line

Shore Calm

`water_shore_calm` - Float - default **0.6** - 0 to 1

Waves die in the shallows because the bottom drags on them. Without this the foam line sits on top of a full-height swell, which is the most obvious tell of a fake shoreline. 0 = the same wave height right up to the sand

Ripples Follow the Flow

`water_flow_ripples` - Float - default **0** - 0 to 4

Let the FLOWING WATER FIELD drive the fine ripples instead of an independent noise. Three things the field knows and a noise cannot: the ripples DRIFT with the current instead of along one fixed diagonal, they stretch into STREAKS ALONG the flow instead of being squashed on a fixed axis, and they SWIRL where the current curls -- that last one comes straight from rot(u) in the solver, and it is what separates an eddy from a fast straight current. Amplitude rises with speed as well. Needs Flowing Water (2D field) on; with no field the flow reads zero and every term above collapses to EXACTLY the old wave, not to a frozen one. At 0 nothing here runs

Flow Streaks

`water_flow_stretch` - Float - default **1.2** - 0 to 4

How far the ripples are drawn out ALONG the current. The component of the texture lookup along the flow is squeezed by 1/(1+this), so the pattern changes more slowly downstream and reads as streaks. 0 = round ripples that merely drift. Scaled by Ripples Follow the Flow, so it does nothing on still water

Reference Speed (m/s)

`water_flow_reference` - Float - default **0.35** - 0.01 to 10

The speed that counts as "full effect". Flow faster than this does not keep stretching without limit -- the ratio is clamped at 3x, so one fast cell at a wet/dry front cannot smear the whole surface. MEASURED on the shipped water field: a wind-driven thin film saturates at about 0.2 m/s and a draining pool peaks near 0.5, so 0.35 puts a typical scene in the middle of the range

Eddy Swirl

`water_eddy_swirl` - Float - default **0.6** - 0 to 4

How hard the ripple pattern is TWISTED where the current turns. The angle comes from rot(u) -- the vorticity the shallow-water solver computes -- so it marks real shear lines and real boils, not a noise pretending to be one. Straight flow has zero vorticity and is left alone. Scaled by Ripples Follow the Flow

Foam Break-up

`water_foam_break` - Float - default **0.65** - 0 to 1

Ragged the foam line with the chop. At 0 it is a clean contour, which reads as a decal

Foam in the Eddies

`water_foam_vorticity` - Float - default **0** - 0 to 4

Put foam where the current CURLS, not only where the water is shallow. Shoreline foam is a depth contour and it cannot know about the boil below a drop or the line where two currents shear past each other -- but rot(u) does, and the solver already computes it. Needs Flowing Water (2D field); with no field the vorticity reads zero and the foam is EXACTLY the shoreline band it was. At 0 nothing here runs

Water IOR

`water_ior` - Float - default **1.333** - 1 to 2

Index of refraction. 1.333 is water; 1.0 is air

Sparkle

`water_sparkle` - Float - default **0.25** - 0 to 1

High-frequency roughness break-up, so a calm surface still glitters where the sun catches it instead of reading as polished glass

---

Snow

Switches the material model from sand to snow. Snow drifts like sand but also SINTERS -- it hardens with age -- so old drifts resist the wind while fresh powder keeps moving. That one difference is what makes snow read as snow.

When you need it: Winter. Also the gateway to the avalanche machinery.

Snow

`snow_enabled` - Bool - default **off

Switch the material model to SNOW: a loose fresh-powder layer that drifts like sand, plus a hardness field where still snow sinters into wind crust (sastrugi, slabs). Enables Snowfall / Melt / Sinter / Slab settings below and the snow material channels. The snow presets configure a full look in one click

Initial Snow Depth

`initial_snow_depth` - Float - default **0** - 0 to 50

Snow already lying on the ground at Initialize, in height units -- the snow counterpart of Initial Sand Depth. Without it the only way to get a snowpack was to let it SNOW for a few hundred steps, which is why a Thaw scene had nothing to melt (measured: the Snow Thaw preset finished its 150-step warmup with a snow depth of exactly 0.0). Raises the surface by the same amount, the way a real fall would

Snowfall

`snowfall_rate` - Float - default **0** - 0 to 1

Fresh snow added to the whole surface each step (height units). Feeds the loose blowing layer that drifts and forms snow dunes; 0 = no snowfall. Works whether or not the Snow checkbox is on -- any of Snow / Snowfall / Sinter arms the model

Melt

`melt_rate` - Float - default **0** - 0 to 1

Snow removed from the surface each step (height units), floored at the bedrock. Also softens the wind crust so it can rework; 0 = none

Sinter

`sinter_rate` - Float - default **0** - 0 to 1

How fast still (undisturbed) snow consolidates into a hard wind crust that resists erosion. Higher = crust locks in faster, giving sharper sastrugi; 0 = powder never hardens (SNOW mode)

Sinter Follows Temperature

`sinter_temp` - Float - default **0** - 0 to 1

Let the AIR decide how fast snow bonds into a crust. Sintering is vapour diffusion between grains and it climbs steeply toward the melting point: snow near 0 C welds into a slab in hours, while at -30 C it stays loose faceted crystals for weeks -- which is how depth hoar forms. Without this the crust grows at the same rate at any temperature (measured: exactly 0.26930 at both -30 C and -1 C). At 1.0 the rate is multiplied by 1.00 at 0 C, 0.49 at -5, 0.24 at -10, 0.058 at -20 and 0.014 at -30. Capped above freezing on purpose -- there the pack is melting, and melt already softens the crust, so letting it climb further would count one process twice. Needs Use Temperature. 0 = off, and off is an exact no-op

Sinter Shield

`sinter_shield` - Float - default **0.9** - 0 to 1

How strongly a hard crust shields the snow beneath it from erosion (0 = no protection, 1 = fully shielded). Loose fresh powder on top cancels the shield. Also used as the avalanche crust hold for snow. Note: if you paint or load a Hardness field in SAND mode, the same shield protects those sand crusts too

Initial Hardness

`initial_hardness` - Float - default **0** - 0 to 1

Starting crust hardness of the snow field at frame 0 (0 = fresh powder everywhere that immediately drifts, 1 = fully crusted old surface the wind must first break down). Frame-0 only, applied at Initialize; needs Snow enabled

Slab Trigger

`slab_trigger` - Float - default **0** - 0 to 30

Snow SLAB avalanches: fresh snow releases this many degrees BELOW the sand repose angle (snow lets go on gentler slopes than dry sand). 0 = off. Needs Snow enabled AND real fresh-snow depth on the slopes (from Snowfall or the initial layer) -- nothing releases on bare crust. Combine with Episodic Slabs for burst-style releases instead of continuous creep

Episodic Slabs

`slab_episodic` - Bool - default **off

Slab release style. OFF (default) = smooth continuous CREEP: fresh snow eases downslope every step. ON = real EPISODIC slabs: the fresh-snow slab holds metastably a few degrees above the release angle, then the whole connected slab lets go in a BURST and settles -- quiet between events, sudden avalanches when it goes (needs a hard crust below as the weak layer). Opt-in; does not change the creep default

Slab Weakness Drop

`slab_weak_drop` - Float - default **12** - 0 to 40

How many DEGREES the painted Slab Weakness takes off the slab release angle where the map reads 1.0. Paint the line you want to fracture and the slab lets go there first. With nothing painted this does nothing at all

Snow Repose Offset

`snow_repose_offset` - Float - default **0** - -20 to 25

Degrees the fresh-snow SLAB holds ABOVE the sand's repose angle. Dry sand tops out near 34 degrees; cohesive snow bridges its own weight and stands well past that until it fails, so a scene with BOTH needs two numbers. Until this existed the slab was handed the sand angle and could only release BELOW it, which made "sand at 34 and snow holding 45" impossible to express. IT MOVES THE SLAB MECHANISM ONLY -- the granular relaxation that settles sand takes one scalar limit straight into the GPU kernel, so that half stays shared between the two materials. 0 = the shipped behaviour, bit-identical

Full Cover Depth

`snow_cover_depth` - Float - default **0.02** - 0.0001 to 10

How deep the snow has to be before it fully HIDES the ground in the render, in height units. This is the reference the snow mask is measured against, and it used to be the field's own MIN-MAX range -- which fails at both ends: snow of the same depth everywhere has no range, so it came out as ZERO snow (measured, a 50-unit-deep pack wrote 0.000000 to the shader), and a 0.001 dusting was stretched to full white. Lower it for snow that covers quickly, raise it to keep the ground showing through longer

---

Vegetation

Plants that grow where sand is calm, die when they are buried or undermined, and hold the ground down while they live. The interesting behaviour is the feedback: plants stabilise sand, stable sand grows more plants, until a storm buries the lot.

When you need it: Semi-arid landscapes, nabkha dunes (the mounds that form around bushes), and anywhere you want the ground to look inhabited rather than sterile.

Vegetation

`veg_enabled` - Bool - default **off

Master switch for the vegetation system: plants grow on calm sand, get killed by burial/erosion, and shield the surface from the wind -- the tug-of-war that creates parabolic dunes, nabkha mounds and blowout hollows. All Vegetation settings below need this ON. Paint cover with the Vegetation brush or load a map

Initial Cover

`veg_initial` - Float - default **0** - 0 to 1

Vegetation cover the whole field starts with at frame 0 (0 = bare sand, 1 = fully grassed). High starting cover stabilises the field until burial/erosion opens gaps. Frame-0 only; needs Vegetation ON. For patterns use the brush, Nabkha or Blowout instead of a uniform value

Growth

`veg_grow` - Float - default **0.06** - 0 to 1

How fast vegetation regrows toward Max Cover on undisturbed sand each step (higher = plants recolonise faster)

Burial

`veg_bury` - Float - default **2.5** - 0 to 20

How strongly ANY bed disturbance (burial by deposition OR undermining by erosion) sets the vegetation back -- high values kill plants on active dunes. THIS IS THE CONTROL BEHIND "the dunes slide underneath the plants and the plants stand still". They do stand still, and on purpose: plants are rooted, so the cover field never travels with the sand -- it only grows and gets buried. Whether a dune passing over them KILLS them is this number, and nothing else. MEASURED over 150 steps with a ridge crossing seeded clumps: at the shipped 2.5 the cover collapsed from 0.2280 to 0.0014, while at 0.05 it stayed at 0.2280 and the plants rode over the dune untouched -- which is exactly what reads as vegetation floating above the surface

Max Cover

`veg_max` - Float - default **1** - 0.01 to 5

Ceiling the vegetation cover grows toward (the shielding saturates here)

Wind Shield

`veg_shield` - Float - default **8** - 0 to 50

How much full vegetation reduces the local wind shear (anchors the sand). Higher = plants hold the surface harder -> parabolic/nabkha dunes

Water Feeds Roots

`veg_water_growth` - Float - default **0** - 0 to 1

How much of the WETNESS MAP reaches the root zone. Until now plants drank from RAINFALL alone, so a lake, the sea or a flooded channel left the root store empty: measured on a field with standing water over 79.9% of its cells, the cover came out BIT-IDENTICAL to dry sand. 0 keeps that behaviour exactly. Raise it and vegetation thickens along shorelines, oases and wet channels the way it does in the field. Rain and standing water are combined with MAXIMUM rather than added -- a shower on a lake shore is one wetness, not two

Dieback

`veg_mortality` - Float - default **0** - 0 to 0.5

How much of the cover dies back each step for no outside reason -- old age and generational turnover. Until now plants in this model NEVER died: measured, a stand held 0.500000 for 60 steps with no water and no moving sand, because the only loss term was burial. 0 keeps that exactly. Raise it and the cover settles at a BALANCE between growth and loss instead of climbing to the ceiling, which is what gives a scene a carrying capacity below Max Cover. Proportional to the cover, so it thins a stand rather than clearing it

Drought Dieback

`veg_die_dry` - Float - default **0** - 0 to 0.5

How fast plants die where the root zone is DRY, scaled by how dry it is: full rate on bare-dry ground, nothing on saturated ground. Reads the same root-zone water the growth term drinks from, so rain, a lake and a wet channel all count. Turns a shoreline stand into something that RETREATS when the water drops, instead of a permanent green ring. Does nothing on a scene that has no wetness at all -- no moisture map is not the same statement as drought

Drought Tolerance

`veg_drought_tolerance` - Float - default **1** - 0 to 1

Wetness below which plants START to suffer. 1.0 means they suffer as soon as the ground is anything but saturated, which is the shipped curve; 0.15 is a succulent that does not care until the ground is nearly bone dry. Drought Dieback then scales with how far BELOW this the wetness has fallen, reaching full rate at zero wetness. One curve for all vegetation was the same simplification as one entrainment threshold for sand and gravel

Lifespan (steps)

`veg_lifespan` - Float - default **0** - 0 to 2000

How many steps a stand takes to reach full old age. 0 turns the AGE off entirely and Dieback stays what it was: a fixed fraction of the cover lost every step, so a freshly seeded patch and a century-old stand die at exactly the same rate. That is a model of HOW MUCH is lost, not of WHAT dies. With a lifespan the cover carries an age, every bit of new growth DILUTES it (new shoots are young), and Senescence turns that age into mortality

Senescence

`veg_senescence` - Float - default **0** - 0 to 0.5

Extra dieback at FULL old age, on top of Dieback. Needs a Lifespan above 0. This is what makes a stand behave like a generation instead of a fixed leak: it establishes, thickens, ages and thins, and the gap it leaves is young again because growth dilutes the age

Age Slows Growth

`veg_age_growth` - Float - default **0** - 0 to 1

How much a stand SLOWS DOWN as it ages. Until now Plant Age fed only the dying side: a century-old stand put on new cover exactly as fast as fresh seed, which is a model of "how much is lost" and not of "what is growing". At 1.0 a fully mature cell adds nothing; at 0.0 age does not touch growth at all and the field computes bit for bit what it did before. NEEDS Plant Lifespan above 0 -- without it there is no age field to read, and this term does nothing rather than guessing one

Second Species

`veg_species_mix` - Float - default **0** - 0 to 1

Put a SECOND plant next to the first. Drought Tolerance used to be one number for the whole scene, so a succulent and a grass could not stand side by side -- which is the commonest arrangement on a real dune. Above 0 the tolerance becomes a FIELD: patches of species A and species B, mixed by a deterministic map seeded from Seed. At 0 the scalar is used and nothing changes. The map only matters through the two tolerances -- set them equal and the result is identical to the bit

Tolerance (Species B)

`veg_drought_tolerance_b` - Float - default **0.35** - 0.001 to 1

Drought threshold of the SECOND species, read the same way as the first: the moisture below which this plant starts to suffer. A succulent sits near 0.2, a grass near 0.8. Only used when Second Species is above 0

Species Patch Size

`veg_species_size` - Float - default **8** - 1 to 64

How many species patches span the domain. Small numbers give a few broad stands, large numbers give a fine mosaic. The layout is deterministic in Seed, so the same scene always grows the same species map. Only used when Second Species is above 0

Seeds Ride The Wind

`veg_seed_wind` - Float - default **0** - 0 to 1

Make Veg Spread carry seed DOWNWIND instead of spreading it evenly in every direction. The shipped neighbourhood is symmetric, so a patch GROWS but never TRAVELS -- which is why vegetation never walks across a field the way it does in life. Above 0 the cell takes its seed from upwind, so the downwind side of a patch grows faster than the upwind side. IT BIASES GROWTH; IT DOES NOT CARRY COVER, and the difference matters if you are expecting a patch to march. MEASURED on a single square patch in an empty field, as the cover gained in the empty column BEHIND it against the one IN FRONT: 1.00x at 0 (exactly symmetric), then 1.49x, 2.19x and 5.10x at 0.25, 0.5 and 1.0 -- so the bias is strong. The patch CENTRE, however, moves only 0.045 of a cell over 40 steps and 0.063 over 120, because the upwind side still grows too and the bias fades as the patch edge softens. Expect a lopsided patch, not a migrating one. NEEDS Veg Spread above 0: the spread term is what carries it. At 0 the symmetric neighbourhood is used, exactly as before

Frost Kills

`veg_frost_kill` - Float - default **0** - 0 to 0.5

How fast hard frost KILLS the cover, rather than merely pausing its growth. Needs the Temperature model. Until now -30 C only stopped growth: measured, a stand sat at 0.500000 for 60 steps in that freezer. Ramps from no effect at 0 C to the full rate at -10 C and below -- a ramp, not a switch, so the edge of a frost pocket is a gradient rather than a drawn line. With the temperature FIELD on, it acts cell by cell: green survives on the warm slope and dies in the frost hollow

Plants Drink

`veg_transpiration` - Float - default **0** - 0 to 8

How much faster the ROOT ZONE dries under plant cover -- the water the plants actually use. This closes the rain-plant loop the other way: rain and standing water already feed the roots, and until now the roots gave nothing back (measured difference against bare sand: exactly zero). NOT a contradiction with Cover Shades Ground: that one slows evaporation from the SURFACE store, this one empties the DEEP store. Two different reservoirs. Raise it and a dense stand can dry out its own ground and thin itself

Wind Drives Transpiration

`veg_transp_wind` - Float - default **0** - 0 to 8

How much the WIND increases what plants drink, measured against the reference shear rather than the local one -- normalising by the local stress would make the term blind to wind speed, which is the mistake the rain model already paid for once. Real evapotranspiration climbs steeply with both wind and temperature; until now it was a function of plant cover alone. The temperature half needs no slider: it goes through the same gain that drives ground evaporation, so the two cannot drift apart

Nabkha

`nabkha` - Float - default **0** - 0 to 200

Seed scattered vegetation clumps at Initialize; each clump traps drifting sand into a NABKHA -- a hummocky plant-anchored mound with a downwind tail. Higher = more/denser clumps. Needs Vegetation ON and some sand supply; frame-0 seeding only. The slider stops at 30, where the clumps are still separate hummocks; type a higher value (up to 200) and they merge into near-continuous cover

Blowout

`blowout` - Float - default **0** - 0 to 15

Start from FULL vegetation cover with bare gaps punched in it; the wind scours the unprotected gaps into BLOWOUT hollows while the vegetated surroundings hold -- the classic coastal-grassland erosion look (often evolving into parabolic dunes). Higher = more/larger gaps. Needs Vegetation ON; frame-0 seeding only. Combines with Nabkha: the gaps are punched into the cover first and the clumps are then seeded on top of it

Patch Size Variation

`veg_patch_size_var` - Float - default **0** - 0 to 1

Spread of clump/gap SIZES. At 0 every patch is exactly the same radius, which is the giveaway that they were stamped rather than grown; at 1 they range from tiny to double. The mean AREA is held constant as you open this, so it changes the variety and not how much of the field is covered. Applies to Nabkha and Blowout

Patch Edge Noise

`veg_patch_edge_noise` - Float - default **0** - 0 to 1

How RAGGED the outline of each patch is. At 0 they are perfect circles -- measured, the seeded field scored the same circularity as a synthetic field of ideal discs. Raising this deforms the outline with three harmonics of the angle, so a patch stays one connected blob with a lobed, plant-like edge instead of fraying into speckle. Applies to Nabkha and Blowout

Patch Clustering

`veg_patch_cluster` - Float - default **0** - 0 to 1

Whether patches scatter EVENLY or grow in STANDS. At 0 the positions are a uniform random scatter, which is what the solver has always done and is not how plants colonise ground -- they spread from where they already are. Raising this draws the patches around a handful of parent points (a Thomas process), tightening from field-wide to a few cells across. Applies to Nabkha and Blowout

Stands

`veg_patch_cluster_count` - Int - default **6** - 1 to 64

How many separate STANDS the patches gather into when Patch Clustering is above 0. Few stands = a couple of dense thickets with bare ground between them; many = a loose mottle. Ignored while clustering is 0

Spread From Plants

`veg_spread` - Float - default **0** - 0 to 1

How much growth needs plants ALREADY NEARBY. 0 is the shipped behaviour, and the original model too: every cell gains cover at the same rate, so bare sand a hundred metres from the nearest clump greens exactly as fast as ground touching one -- which is why a field with Initial Cover 0 and a few Nabkha seeds turns green ALL OVER instead of spreading out from the seeds. 1 means a cell can only gain in proportion to the cover around it, so the vegetation advances outward from what is already there and bare ground stays bare. 0.6-0.9 reads as colonisation without stalling completely on an empty field

Min Slope

`veg_slope_min` - Float - default **0** - 0 to 1.5708

Shallowest ground plants will colonise. Only GROWTH is gated -- cover that already exists stays and burial still removes it, so tightening this does not make established plants vanish

Max Slope

`veg_slope_max` - Float - default **1.5708** - 0 to 1.5708

Steepest ground plants will colonise. A slip face at the repose angle avalanches constantly, so set this a few degrees below it (around 30) to keep the active faces bare and let the plants hold the flanks and the interdune floors

Min Height

`veg_height_min` - Float - default **0** - 0 to 1

Lowest ground plants will colonise, as a FRACTION of the height range of the terrain (0 = its lowest point, 1 = its highest). A fraction rather than metres, so "the top third" means the same thing on a 3 m field and a 300 m one

Max Height

`veg_height_max` - Float - default **1** - 0 to 1

Highest ground plants will colonise, as a fraction of the terrain height range. Set it low to keep the crests bare and the hollows vegetated, which is what a real dune field looks like

Invert Limits

`veg_limit_invert` - Bool - default **off

Flip the slope and height windows: plants grow everywhere EXCEPT inside them. Turns "only on gentle low ground" into "anywhere but gentle low ground" without re-entering four numbers

Ragged Limit Edge

`veg_limit_edge_noise` - Float - default **0** - 0 to 1

Break up the line the slope and height limits cut. At 0 a limit is exactly a contour -- vegetation stops dead along a mathematically smooth curve, which reads as a mask rather than as a plant community. This wanders the

THRESHOLD from cell to cell instead (up to about 10 degrees of slope and a tenth of the relief at 1.0), so the boundary frays and interlocks the way a real vegetation limit does. The pattern is fixed by the scene Seed and correlated over a few cells: it does not shimmer between steps, and it does not depend on how many steps you ran. The terrain itself is untouched -- only the limit moves. 0 = the hard line, exactly as before

Limits Apply to Seeding

`veg_limit_seeding` - Bool - default **off

Make Initial Cover, Nabkha and Blowout obey the slope and height windows above. THEY DO NOT BY DEFAULT, and that surprises everyone who meets it: the windows gate GROWTH, while the three seeders write cover directly at frame 0 -- which is why Initial Cover 1 carpets the slip faces however tight the Max Slope is. OFF is the old behaviour exactly, so no existing scene or preset moves; switch it on and a seeding lands only where plants are allowed. It never REMOVES cover that is already there -- a limit stops plants arriving, it does not kill them

Keep Off Sand Source

`veg_avoid_sand_source` - Bool - default **off

Stop Nabkha and Blowout seeding on a painted Sand Source. A source is ground where sand is ARRIVING, which is about the least likely place on the map to hold a plant, but the seeders scatter over the whole grid and land on it happily. Reads the same source mask the supply uses -- either a painted one or the rasterised Source object -- so it follows whatever fed the sand before Initialize. IT ONLY GATES THE SEEDING, AND THE HEAD START IS TEMPORARY: growth is untouched, so ordinary spread colonises the source afterwards and the gap closes. MEASURED on a nabkha field with Veg Spread 0.85, as cover on the source against cover off it: at the moment of seeding 0.00 with this on against 0.86 with it off, then 0.01 vs 0.15 after 20 steps, 0.05 vs 0.09 after 60, and 0.09 vs 0.10 after 150 -- by then the two are the same picture. Turn Veg Spread down if you want the bare source to last. OFF is the old behaviour

Keep Off Imported Sand Map

`veg_avoid_sand_map` - Bool - default **off

Stop Initial Cover, Nabkha and Blowout seeding on sand that came from the imported Sand Map. An imported dune field arrives fully formed at frame 0, and the seeders scatter over the whole grid, so the plants land on top of dunes that have never moved a grain -- a mature cover on brand new sand. GROWTH IS NOT BLOCKED: plants colonise the imported dunes during the run exactly as they colonise generated ones, they simply do not start there. Needs Use Sand Map on; with no map loaded this changes nothing. OFF is the old behaviour

Sand Map Threshold

`veg_sand_map_thresh` - Float - default **0.02** - 0 to 1

How much imported sand counts as "on the map", as a FRACTION OF THE MAP PEAK. At 0.02 anything the map put down blocks the seeding, which is usually what you want; raise it to let plants start on the thin aprons and keep them off the dune bodies only. A fraction and not a depth in metres on purpose: Sand Map Depth, Gamma, Offset and Feather all rescale the field, so a number in metres would change meaning every time you touched one of them. Only read when Keep Off Imported Sand Map is on

Growth Has Its Own Limits

`veg_growth_own_limits` - Bool - default **off

Give GROWTH a separate slope and height window from the one the seeding uses. They are the same window by default, and for a lot of scenes that is right. It stops being right when you want a landform to START somewhere narrow and then be allowed to creep: nabkha seeded only in the interdune hollows, say, but free to climb a little way up the flanks afterwards. OFF keeps one window for both

Growth Min Slope

`veg_growth_slope_min` - Float - default **0** - 0 to 1.5708

Shallowest ground plants may SPREAD onto, when Growth Has Its Own Limits is on. Ignored otherwise

Growth Max Slope

`veg_growth_slope_max` - Float - default **1.5708** - 0 to 1.5708

Steepest ground plants may SPREAD onto, when Growth Has Its Own Limits is on. Set it a few degrees under the angle of repose (around 30) and the active slip faces stay bare while the flanks and hollows green over

Growth Min Height

`veg_growth_height_min` - Float - default **0** - 0 to 1

Lowest ground plants may SPREAD onto, as a FRACTION of the terrain height range, when Growth Has Its Own Limits is on

Growth Max Height

`veg_growth_height_max` - Float - default **1** - 0 to 1

Highest ground plants may SPREAD onto, as a fraction of the terrain height range, when Growth Has Its Own Limits is on. Keep it low and the crests stay bare while the hollows fill in

---

Interactive Brush

Direct manual control: paint sand in or out, paint the wind, paint where sand may not go. The simulation carries on from whatever you painted.

When you need it: Art direction. When the shot needs a dune exactly THERE and physics will not be talked into it.

Paint

`brush_field` - Enum - default **SAND

Which solver field the brush edits

Option	Meaning
Sand Height	Sculpt the sand surface up/down
Bedrock	Raise/lower the solid non-erodible floor
Erodibility	Paint how easily sand erodes (0 = protected, 1 = free)
Non-Erodible Rock	Paint bare rock that never erodes but can still be buried (paint toward 1 = rock)
Wind Gain	Paint local wind strength (gust / lull zones)
Vegetation	Paint vegetation cover that anchors the sand
Snow Depth	Paint the depth of FRESH loose snow, in metres, exactly as Sand Height paints sand. Painting it arms the Snow model, and before Initialize it is staged on the mesh and loaded automatically. This is the powder that drifts and can be blown off crests; for the wind-packed crust use Snow Crust
Snow Crust	Paint how HARD the snow is, 0 = loose powder, 1 = wind slab or glaze ice. Separate from Snow Depth on purpose: the crust is what decides whether a slab releases and which end of the snow colour ramp the surface takes, so a sastrugi field can be tuned without also moving the fresh powder
Slab Weakness	Paint a WEAK LAYER in the snow. The painted cells release their slab at a lower angle -- by up to Slab Weakness Drop -- so you can put the fracture line where you want it instead of waiting for one. Like the Avalanche Trigger it lowers a THRESHOLD and adds no snow, and it needs fresh snow and a crust to act on
Avalanche Trigger	Paint where slopes let go EARLY. The painted cells drop their repose angle by up to Avalanche Trigger Drop, so a slope that was standing quietly starts to slide -- for forcing a cascade where you want one instead of waiting for the wind to build it. It lowers the THRESHOLD, it never adds sand: an avalanche here moves material, it does not create it. At 0 everywhere the simulation is bit-for-bit what it was

Option	Meaning
Snow Source	Paint where snow FALLS. Works like Sand Source but feeds fresh powder: the painted cells get Snow Source Rate of snow every step, on top of the global Snowfall. Needs Snow Source Rate above 0 to do anything
Sand Source	Paint where fresh sand blows in each step
No-Deposit	Paint ground where sand may NOT settle. Deposition only: whatever sand is already there still blows away, so a painted slab or road clears itself instead of freezing. Paint = block, Shift = allow again
Water / Wet	Paint wet / submerged ground (locked from wind erosion, deposition still allowed; enables the Water system)
Freeze Mask	Paint areas that stop evolving (locked)
Flow Strength	Paint how fast the SOLVED wind runs here. Needs the Flow 2D wind model. 1.0 leaves the solve alone, below 1 slows the air, above speeds it up -- and Flow Paint Mode decides whether the paint SCALES the solve or REPLACES it
Flow Direction	Paint WHICH WAY the wind goes, by dragging: the direction of your stroke is what gets written into the cells under the brush. Needs the Flow 2D wind model and Flow Paint Mode = Painted Speed + Direction, otherwise nothing reads it. Direction is stored as a vector, not an angle, so overlapping strokes average the way you expect instead of flipping to the opposite heading; an untouched cell keeps the solved flow. Shift+drag erases back to unpainted
Wind Gizmo	Drag in the viewport to aim the global wind from the drag vector (paints no field)

Stamp

`brush_stamp` - Enum - default **NONE

Freehand sculpt, or click-stamp a pre-shaped dune that then evolves

Option	Meaning
Sculpt (drag)	Freehand raise/lower/smooth
Barchan Stamp	Click to drop a crescent barchan seed
Dome Stamp	Click to drop a rounded dome seed
Ridge Stamp	Click to drop a transverse ridge seed
Scatter Clumps	Scatter small clumps of the field under the brush -- ideal for vegetation patches
Image Stamp	Stamp a bitmap as a height relief (its brightness = height); load any image below

Mode

`brush_mode` - Enum - default **ADD

What a stroke does to the field under the cursor. Add, Smooth and Set are the everyday three; Grab, Slope and Erase are drag-aware and read the direction you move the mouse

Option	Meaning
Add	Add to the field (Shift to subtract)
Smooth	Blur/relax the field under the brush
Set	Flatten the field toward the Target Value
Flatten	Level toward the average height under the brush
Roughen	Add procedural bumps (break up smooth areas)
Sharpen	Push values away from the local average (crisp ridges)
Grab	Drag the sand mass along with the cursor -- pull dunes around (height fields)
Slope	Paint a linear ramp tilted along the drag direction (height fields)
Erase to Bedrock	Scrape the sand down to the bedrock floor, exposing bare rock

Radius

`brush_radius` - Float - default **6** - 0.1 to 1000

Brush radius in world units (mouse wheel to resize while painting)

Strength

`brush_strength` - Float - default **0.5** - 0 to 1

Stroke intensity 0..1. For ADD/height modes it scales the height moved per dab; for mask fields a full deliberate pass reaches about half the field range at 0.5. Shift inverts the direction (subtract / un-paint)

Falloff

`brush_falloff` - Enum - default **SMOOTH

How a stroke fades from the centre of the brush to its edge. Constant gives a hard-edged stamp, which is what you want for a mask meant to be exact

Option	Meaning
Smooth	Soft smoothstep edge
Sharp	Tight, peaked falloff
Constant	Hard-edged flat disc

Target Value

`brush_value` - Float - default **1** - 0 to 10

Level the SET brush mode pushes the field toward under the stroke: for height fields it is a height in solver units (flatten a plateau at an exact level), for mask fields use 0..1 (e.g. Set 0 on Erodibility = paint full protection)

Scatter Count

`brush_scatter_count` - Int - default **6** - 1 to 200

How many small clumps the Scatter stamp drops per dab

Scatter Clump Size

`brush_scatter_size` - Float - default **0.18** - 0.02 to 1

Radius of each scattered clump as a fraction of the brush radius

Stamp Image

`brush_image_path` - String - default **//

Image whose brightness is stamped as height (Image Stamp). White = high, black = low; Shift carves it in

Symmetry

`brush_symmetry` - Enum - default **NONE

Mirror every stroke across the middle of the terrain, so a painted mask stays symmetric without painting it twice

Option	Meaning
None	No mirroring
X	Mirror strokes across the domain centre X
Y	Mirror strokes across the domain centre Y
XY	Mirror across both axes (4-way)

Show Painted Mask

`brush_show_overlay` - Bool - default **on

Tint the terrain live while painting a MASK field (erodibility, source, wind gain...) so you see where the stroke landed; the final mask overlay refreshes when the stroke commits. Height fields (Sand/Bedrock) show as geometry instead and ignore this. Viewport only

Also Show

`brush_peek_fields` - Enum

Other masks to show UNDERNEATH the one you are painting, each in its own legend colour. Painting a mask blind against the ones it has to line up with is guesswork -- a sand source belongs where the rock is not, vegetation where the water is. Viewport only, and it never touches the fields themselves. The mask being PAINTED always stays on top

Option	Meaning
Erodibility	Show the erodibility mask underneath
Rock	Show the non-erodible rock mask underneath
Vegetation	Show the vegetation mask underneath
Water	Show the water / wet mask underneath
Wind Gain	Show the wind-gain mask underneath
Sand Source	Show the sand-source mask underneath
Snow Depth	Show the fresh-snow depth underneath
Snow Crust	Show the snow hardness underneath
Snow Source	Show the snow-source mask underneath
Avalanche	Show the avalanche trigger map underneath
Slab Weakness	Show the slab weak-layer map underneath
Freeze	Show the freeze mask underneath

Peek Strength

`brush_peek_strength` - Float - default **0.55** - 0 to 1

How strongly the other masks show through. Low keeps them as a hint under the stroke; 1 makes them as loud as the mask you are painting, which is useful for lining two up exactly and useless for painting

Track Curve

`track_curve` - Pointer

Curve object whose path is pressed into the sand as a track/rut

Track Style

`track_mode` - Enum - default **RUT

Whether the curve leaves one continuous rut or a line of separate footprints

Option	Meaning
Continuous rut	One unbroken channel with berms along the curve -- a tyre, a sled, a dragged branch. This is what the tool has always done
Footprints	Discrete prints stepped along the curve, alternating left and right of the line. THE POINT IS SCALE: a dune with nothing on it could be two metres or two hundred, and a line of prints settles the question for the viewer instantly

Print Shape

`print_kind` - Enum - default **BOOT

Which animal left the trail

Option	Meaning
Boot / human	An ellipse longer along the walk, a little wider at the front
Hoof	Two crescents split by a cleft -- horse, camel, deer
Paw	A pad with four toes ahead of it -- dog, fox, cat
Bird	Three toes forward and one back. The most readable at small sizes because it is the most ANISOTROPIC shape of the four

Print Length

`print_size` - Float - default **0.28** - 0.02 to 2

Length of one print, in metres -- 0.28 for an adult boot, 0.12 for a large bird, 0.15 for a camel toe pair. MEASURED LIMIT, AND IT IS THE ONE THAT DECIDES WHETHER THIS TOOL IS USABLE AT ALL: the print needs about SIX CELLS of length to read as a shape, four to read as a dot, and below three it does not appear at all. At 0.28 m that means a cell of 0.047 m, i.e. a small domain or a very high Solve Res. The panel does this arithmetic for you and says so

Stride

`print_spacing` - Float - default **0.75** - 0.05 to 20

Distance between consecutive prints along the curve, in metres. A walking human is about 0.75, a running one 1.5-2.0, a camel about 1.1, a small bird 0.1-0.2. Stride is the strongest scale cue in the whole trail -- more than the print itself

Track Width

`print_stagger` - Float - default **0.18** - 0 to 3

How far left and right of the curve the prints alternate, in metres. 0 puts every print on the centre line, which reads as a machine rather than an animal. A human walks about 0.18 apart, a four-legged animal more

Track Width

`track_width` - Float - default **2** - 0.05 to 100

Width of the carved rut in world units (e.g. tyre/foot/sled)

Track Depth

`track_depth` - Float - default **0.4** - 0 to 50

How deep the rut is carved (solver height units); the sim then reworks it

Berm Height

`track_berm` - Float - default **0.2** - 0 to 50

Height of the pushed-up ridges along each side of the rut (0 = none)

Surface Shader

How the surface is textured -- and specifically how the texture is PROJECTED onto a surface that is changing shape every frame. This is a bigger problem than it sounds: an ordinary UV map stretches and swims when the geometry moves underneath it.

When you need it: As soon as the camera is close enough to see the surface rather than the silhouette.

Scan Set A

`shader_texture_set_a` - Enum

First scanned sand surface. The list is whatever is in the add-on textures/ folder -- drop your own folder in (albedo.jpg, normal.jpg, rough.jpg, height.jpg) and it appears here. Both slots are sampled and blended across the terrain by Scan Blend, so the ground changes character instead of being one repeated surface

Set A Tint

`shader_set_a_tint` - FloatVector - default **1.0, 1.0, 1.0** - 0 to 2

Colour multiplied into Scan Set A ALBEDO only -- its normal, roughness and height are untouched, so the surface keeps its relief and only changes hue. This is how one bundled scan serves a red desert and a grey one without a second texture set, and how you make the two slots read as different MATERIALS rather than as two photos of the same sand. White is the shipped look and costs NOTHING -- at white the tint node is not built at all. Values above 1 brighten

Scan Set B

`shader_texture_set_b` - Enum

Second scanned sand surface, blended against Scan Set A by a large patch mask

Set B Tint

`shader_set_b_tint` - FloatVector - default **1.0, 1.0, 1.0** - 0 to 2

Colour multiplied into Scan Set B ALBEDO only. Same rule as Set A: relief untouched, white is free and builds no node. Tinting the two slots apart is what makes Scan Blend read as a change of ground rather than a change of photo

Texture Scale (all)

`shader_texture_scale` - Float - default **8** - 0.05 to 50

ONE multiplier over every texture size at once -- the scanned sand, the macro blotching, the grain, the ripples, the dunelets, the bombing patches, the scan mask, the splash noise and the bedrock tiling (its layers inherit it). Bigger = a LARGER pattern repeating FEWER times. Every layer still has its own size in metres below; this scales all of them together, which is what you want when the whole surface reads as too fine rather than one layer being wrong. At 1.0 the shipped 200 m terrain carries 167 repeats of the scanned sand and over a thousand of the grain -- breaking up the repetition (bombing, hex) hides the SEAM but never the SCALE. The default is set for that terrain; the sizes are in WORLD METRES, so a terrain scaled up gets more repeats rather than a bigger pattern, and this is the knob to answer that with. THE DEFAULT WAS CHOSEN FROM THE PICTURES, not from a number: detail energy rises monotonically all the way to 36x and the blotchiness measure never turns over, so no metric here can pick an optimum. By eye, 1x is a smooth creamy surface, 8-12x reads as sand, and past ~24x the grain starts reading as scratches

Sand (fresh)

`shader_sand_color` - FloatVector - default **0.76, 0.62, 0.4** - 0 to 1

Colour of freshly moved, dry sand -- what a slip face that just avalanched looks like. Blended toward Sand (weathered) by the sandAge mask, so this colour shows exactly where the solver says the surface was recently disturbed

Sand (weathered)

`shader_sand_old_color` - FloatVector - default **0.6, 0.51, 0.38** - 0 to 1

Colour of long-undisturbed sand: crests and interdune floors that have not moved for a while. Driven by the sandAge attribute (Export panel -> Sand Age), which the solver resets wherever it erodes. With Sand Age off, the surface stays at the fresh colour

Base Roughness

`shader_roughness` - Float - default **0.62** - 0 to 1

Roughness of dry sand. Bare rock is automatically rougher and wet sand automatically smoother -- this sets the middle of that range

Grain Strength

`shader_grain_strength` - Float - default **0.15** - 0 to 1

Bump strength of the fine grain. Its depth also follows Height Scale. COST, MEASURED: the grain octave is the most expensive layer in this shader -- around 1.25-1.31x of the whole frame -- and unlike the ripple and dunelet octaves it earns almost none of it back on screen: at 15 cm per pixel it changes nothing, and even at 0.9 cm per pixel it moves under 1% of pixels by more than 0.01. Ripples cost 1.04x and carry 37% of the pixels; dunelets cost 1.19x and carry 16%. Turning this slider DOWN does not make it cheaper -- a shader cannot branch, so the layer is evaluated whatever the value. If you never see grain at your camera distance, the layer is pure cost

Detail Source

`shader_detail_mode` - Enum - default **BOTH

Which detail the surface is built from. It is a shortcut, not a second setting: it writes Detail Balance and the mask, and both stay editable afterwards (the panel then shows Custom)

Option	Meaning
Both	Procedural octaves and scanned maps together, mixed by Detail Balance and its mask
Procedural Only	Only the procedural octaves (dunelets, micro-ripples, grain). Sets Detail Balance to 0 and clears the mask
Scanned Only	Only the scanned sand maps. Sets Detail Balance to 1 and clears the mask. To make the scan block genuinely free instead of just silent, switch Build Scanned Sand off -- a shader cannot branch, so an unused block still costs on every pixel

Scan Amount

`shader_photo` - Float - default **0.55** - 0 to 1

How strongly the scanned sand shows. It goes on as an OVERLAY, so everything the simulation decided -- sand age, wetness, snow, coarse lag, plants -- survives underneath and the scan adds its grain and colour break-up on top. Gated to sand, so bedrock and snow are not covered in beach sand

Macro Variation

`shader_macro` - Float - default **0.25** - 0 to 1

How strongly the macro variation darkens/lightens the sand. 0 = flat colour

Snow (fresh)

`shader_snow_color` - FloatVector - default **0.9, 0.93, 0.97** - 0 to 1

Colour of FRESH snow -- the soft end of the winter ramp. The snow HARDNESS mask blends from this toward Ice below, so sastrugi and wind slab read differently from new drift without you masking anything by hand. Needs the Snow model on (it writes the layerMask / modulation attributes)

Ice (packed / glaze)

`shader_ice_color` - FloatVector - default **0.74, 0.8, 0.88** - 0 to 1

Colour of WIND-PACKED snow, firn and glaze ice -- the hard, cold end of the winter ramp. The snow hardness the solver computes (sintering, wind slab, rain-on-snow crust) is what mixes between Snow and this, so the harder the pack the more of this you see. Denser ice is bluer and darker because thicker clear ice absorbs red: pull it toward blue for glacier glaze, toward white for a chalky old drift. Needs the Snow model on

Shader Preset

`shader_preset_path` - String - default **//tydunes_presets/shader_preset.json

JSON file for Save/Load Shader Preset. It holds ONLY the look -- colours, scan sets, rock layers, amounts -- so loading one onto another scene changes how the sand reads without touching a single physics setting. The load path filters the file as well as the save path writes it, so a hand-edited or foreign file cannot slip a simulation setting in

Concavity Shading

`fast_concavity` - Float - default **0.45** - 0 to 2

How much the HOLLOWS are darkened in the Fast Surface. Read from the tydShape attribute the solver already writes (curvature and cavity), so it costs NO texture sample at all -- which is the whole point of the fast variant. 0 = flat photo, 1 = strong sculpted look. This is what keeps a dune reading as a dune once the procedural layers are gone

Convexity Shading

`fast_convexity` - Float - default **0.35** - 0 to 2

How much the RIDGES and crests are brightened in the Fast Surface, from the same tydShape curvature. Sand crests really do catch the light -- the grains there are loose and dry -- so a little of this reads as sunlit rather than as a highlight

Hollow Tint

`fast_concave_color` - FloatVector - default **0.42, 0.34, 0.26** - 0 to 1

Colour multiplied into hollows and cavities in the Fast Surface. A damp, slightly cooler sand is the usual choice; pure grey reads as dust rather than as shade

Ridge Tint

`fast_convex_color` - FloatVector - default **1.0, 0.94, 0.82** - 0 to 1

Colour screened onto ridges and crests in the Fast Surface. Warm and near-white is what dry, wind-polished sand looks like

Roughness Min

`fast_rough_min` - Float - default **0.55** - 0 to 1

Where the scanned roughness map lands at its DARKEST. Sand is rough everywhere, so the fast material remaps the map into a band rather than using it raw -- a raw map often reaches 0 and turns patches of the dune into a mirror

Roughness Max

`fast_rough_max` - Float - default **0.95** - 0 to 1

Where the scanned roughness map lands at its BRIGHTEST

Normal Strength

`fast_normal_strength` - Float - default **0.8** - 0 to 4

Strength of the scanned normal map in the Fast Surface. This is the ONLY fine detail the fast variant has -- there are no procedural ripples or grain in it -- so it carries more of the look here than it does in the full shader

Keep Masks Updated

`shader_auto_masks` - Bool - default **on

When adding the shader, also switch on Auto Material Masks so the attributes it reads are refreshed during playback. Without this the shader shows the masks as they were when it was created

Generate Shape Attributes

`surface_geo_attrs` - Bool - default **off

Keep the generated curvature / cavity / slope / height attributes refreshed during playback, at the same cadence as the material masks. Switched on automatically when you add the Surface Shader. Costs a small per-refresh CPU pass, so it stays off for scenes that do not use them

Bedrock

`shader_rock_color` - FloatVector - default **0.29, 0.25, 0.22** - 0 to 1

Colour of bare, sand-free ground. The shader switches to it where the sand-depth mask falls below ~0.12, i.e. exactly where the solver has scoured down to the non-erodible floor -- so rock appears and disappears as the dunes migrate, with no hand painting

Coarse lag / pavement

`shader_coarse_color` - FloatVector - default **0.44, 0.38, 0.31** - 0 to 1

Colour of the coarse-grain lag deposit ("desert pavement") that the Multi-Fraction model leaves where fine sand has been blown away. Only visible with Multi-Fraction enabled -- without it the grainCoarse mask does not exist and this layer stays off

Frost / hoar

`shader_frost_color` - FloatVector - default **0.86, 0.9, 0.95** - 0 to 1

Colour of the frost film -- hoar crystals and rime. This is a surface COATING, not a layer with thickness or mass: it has no depth to displace and it never avalanches, it only tints and roughens what is underneath. Driven by the tydFrost attribute the temperature model writes, so it needs Temperature on with enough cold and damp for frost to grow at all

Frost Roughness

`shader_frost_roughness` - Float - default **0.92** - 0 to 1

How matte the frost film is. THIS IS THE CUE THAT READS AS "frozen" rather than merely "pale": frost kills the specular sheen of damp sand or wet snow, and without that a frosted surface just looks bleached. Near 1 is a dry, powdery hoar; lower it toward 0.5 for the glassy look of rime that has partly thawed and refrozen

Vegetation

`shader_veg_color` - FloatVector - default **0.3, 0.33, 0.18** - 0 to 1

Colour of plant cover. The solver has always tracked vegetation -- it shelters the surface from shear and gets buried by deposition -- but until now nothing exported it, so the render could not see the one field that most obviously changes how ground looks. Needs Vegetation enabled

Vegetation Colour (Species B)

`shader_veg_color_b` - FloatVector - default **0.55, 0.52, 0.3** - 0 to 1

Colour of the SECOND plant, where Second Species puts it. Until 1.3.135 the two species differed in behaviour and were painted the same, so a scene with both read as "more cover" rather than as two communities standing side by side. Defaults to a drier, paler tone than the first: the tolerant species is usually the one holding the dry ground. Only visible when Second Species is above 0 -- with it off the map is zero and this colour is never mixed in

Slab Scar

`shader_slab_scar` - Float - default **0** - 0 to 1

Show the BED SURFACE a slab avalanche peeled back to. The solver has accumulated this footprint since the slab model shipped and exported it every step, but nothing read it -- not the mesh bridge, not the material, not the field preview -- so an avalanche left a mark that could not be seen. The bed is the older, wind-packed layer under the slab: lighter and SMOOTHER than the debris beside it, which is what draws the crown line. It rides the spare alpha of the vegetation attribute, so it costs none of the 14 named attributes EEVEE can compile. At 0 the material is unchanged

Slab Scar Colour

`shader_slab_scar_color` - FloatVector - default **0.8, 0.84, 0.9, 1.0** - 0 to 1

Colour of the bed surface left behind by a slab release. Wind-packed old snow, so slightly cooler and brighter than the fresh slab that slid off it. Only used when Slab Scar is above 0

Nabkha Clump

`shader_nabkha_color` - FloatVector - default **0.09, 0.14, 0.04** - 0 to 1

Colour of a seeded plant CLUMP, as opposed to the spread cover around it. A hummock is a bush -- much darker and more saturated than a thin lawn of the same density -- and until now the render had no way to tell the two apart: measured, two windows of cover at the same density came out 0.000025 apart, the noise floor. Only shows where Nabkha seeded clumps, so a field without them renders exactly as before. THIS is the knob that makes clumps read, more than the strength slider: the difference you see is the distance between this colour and the Vegetation one

Nabkha Clumps

`shader_nabkha` - Float - default **1** - 0 to 2

How strongly seeded clumps take their own colour and roughness. At 0 they read as ordinary vegetation again, which is what every build before this one did. 1 blends up to 85% of the clump colour, leaving ground showing between the plants; above 1 it pushes on to a full replacement, which is what a dense thicket actually looks like. NOTE THIS DOES NOT MAKE THE SHADER CHEAPER: the chain costs 6 nodes and is built either way (measured, 262 nodes at both 0 and 1)

Texture Motion

`tex_motion` - Enum - default **WORLD

Where texture coordinates come from. The dunes are geometry and move either way -- this decides whether the SURFACE PATTERN travels with the sand or stays pinned to the world. Only matters when dunes migrate HORIZONTALLY: for sand simply piling up the three modes look the same, because the XY projection reads world X and Y, which do not change

Option	Meaning
World (base)	Texture coordinates are the WORLD position -- the shipped behaviour. A dune that migrates slides UNDER a pattern that stays put. Costs nothing and never distorts
Follow Sand (bounded)	The pattern travels with the sand but FORGETS after Texture Memory steps, so the distortion has a ceiling by construction. MEASURED at 256 cells over 400 steps: the pattern keeps up with 47% of the crest travel early on and 10% late, and the local stretch stays healthy (median 0.993, zero seams)
Advect (faithful)	The pattern is carried with the sand for as long as the sim runs -- physically the honest one. MEASURED: it keeps up with 70-90% of the crest travel, but the coordinate flattens where the sand moves, so the texture SMEARS ALONG THE WIND. On a 6 m checker after 300 steps the pattern went from 33 edges across the field to 8 -- four times too big, and the cross-wind rows are all that survive. Use it for SHORT runs, not for a long evolution

Texture Memory

`tex_memory` - Float - default **60** - 1 to 2000

How many steps the FOLLOW mode remembers. The offset settles at roughly speed x this, so a bigger number follows the sand further and distorts more; a smaller one stays crisp and lags. No effect in World or Advect

Texture Rotation (all)

`shader_texture_rotation` - Float - default **0** - -6.28319 to 6.28319

Turn EVERY texture layer at once -- the scanned sand, the macro blotching, the grain, the ripples, the dunelets, the bombing patches, the scan mask, the splash noise and the bedrock tiling. The surface pattern rotates; the DUNES do not, because they are geometry. Use it when a scanned set has a visible grain direction that fights the wind, or to move a repeat seam off camera. At 0 the result is identical to before this control existed

Macro Variation Size (m)

`shader_macro_size` - Float - default **8** - 0.25 to 200

Size IN METRES of the large-scale colour blotching that keeps a big sand sea from reading as one flat tone. World-space: scaling the terrain object does NOT stretch it, so 8 m stays 8 m

Grain Size (m)

`shader_grain_size` - Float - default **0.15** - 0.01 to 5

Size IN METRES of the fine surface grain (bump only). Below the solver cell by design -- at 200 m / 1024 cells one cell is 0.2 m, so this is the detail the simulation cannot represent. The 10-20 cm band is what reads as sand up close; raise it above ~0.4 and it starts to look like gravel instead

Grain Specks

`shader_grain_specks` - Float - default **0.45** - 0 to 1

Blend the grain octave from smooth fBm noise (0) towards a Voronoi cell field (1). Plain noise reads as soft LUMPS; the cell field gives discrete specks, which at 10-20 cm is what individual grains of sand look like. Around 0.4-0.6 keeps both -- lumpy drifts carrying visible grain. Bump only, no extra attribute. If the result reads as a cell pattern rather than as sand, that is what Speck Stretch and Speck Softness are for

Speck Stretch

`shader_speck_stretch` - Float - default **0.6** - 0 to 1

Draw the grain specks out ACROSS the wind, the way the ripple crests and the dunelets run. 0 = round isotropic cells, which is the main reason a cell field reads as "Voronoi" rather than as sand; 0.6 measures 2.1x elongated, 0.9 about 3.8x. Follows Wind Direction

Speck Softness

`shader_speck_soft` - Float - default **0.55** - 0 to 1

Round off the hard crease along every cell wall. It drives BOTH the Voronoi Smoothness and a domain warp that bends the walls -- measured, the Smoothness alone barely moved the edge metric because it rounds the cell CENTRES, and bending the walls is what actually stops them reading as polygon edges. Past about 0.7 the warp starts adding gradients of its own and the result measures HARDER again, so the useful band is roughly 0.4-0.7

Bake Resolution

`bake_detail_res` - Int - default **2048** - 256 to 8192

Pixels across for the baked detail map. The grain octave is the finest thing in it -- Grain Size defaults to 0.15 m -- so on a 200 m terrain 2048 gives about 10 cm per texel, which resolves the dunelets and ripples but NOT individual grains. Raise it if you shoot close-ups, lower it if you only ever see the field from a distance. Memory is 4 bytes per pixel: 2048 is 16 MB, 8192 is 256 MB. NOTE, measured: resolution is NOT what separates the baked look from the procedural one -- 1024 to 8192 gave the same difference to four decimals. What did separate them was a bug (the optimised material was never given your settings), and with that fixed the remaining difference is small: mean 0.00545 of a frame over 22% of its pixels, from a Bump reading a filtered texture instead of the function itself

Set Tiling (m)

`splat_set_scale` - Float - default **1.5** - 0.05 to 50

Size IN METRES of one tile of the scanned sets. The splat, colour, AO, normal and roughness maps always cover the terrain ONCE; only the scanned surfaces repeat, and this is how often

Cavity

`splat_cavity` - Float - default **0.55** - 0 to 1

Darken the hollows, read from the solver's own `tydShape` attribute. Costs NO texture sample and no noise -- the shape is already on the mesh, so this is the cheapest realism in the add-on. Needs Surface Geometry Attributes on

Convexity

`splat_convexity` - Float - default **0.35** - 0 to 1

Brighten the crests, from the same attribute. Curvature 0.5 is flat, so only the convex half is used -- ridges catch the light, hollows are left to Cavity. Also free of any texture sample

Build Frost

`shader_build_frost` - Bool - default **off

Render the solver's surface FROST as a white, rough film on the sand. Frost is not thin snow and is deliberately not rendered as any: it has no thickness worth displacing and no mass, so it only touches the colour and the roughness -- rendered as a layer it would avalanche. Needs Use Temperature: the frost field only exists when the air can freeze the ground, and with the temperature model off this adds nothing to the material. Rebuilds the material

Frost Strength

`shader_frost_strength` - Float - default **1** - 0 to 4

Gain on the frost field before it whitens the surface. A gain rather than a solver setting on purpose: wanting a heavier rime in the render should not mean keying the physics that grew it. The result is clamped, so past the point where the field already reaches 1 this stops doing anything

Build Scanned Sand

`shader_build_photo` - Bool - default **on

Include the scanned-sand layer (12 image samples) in the material. Off leaves the procedural octaves only, which is much cheaper. Rebuilds the material

Build Displacement

`shader_build_displace` - Bool - default **off

Drive real DISPLACEMENT from the bundled height maps. Each texture set ships a height map that nothing sampled before, so this is detail already sitting in the package. It samples the height through the SAME bombing taps and the same set blend as the colour and the normal, so the displaced geometry lines up with them. Costs 4 more image samples (12 -> 16), and a shader cannot branch, so the block is only BUILT when this is on -- leaving it in at Depth 0 would cost the same as using it. Needs Build Scanned Sand, which is what loads the maps. Rebuilds the material. IN CYCLES, this together with Build Rain Impacts is the one pair the reference shader cannot compile -- it overflows the SVM stack and the terrain renders BLACK (measured: 690 nodes refused, mean RGB 0.580 -> 0.052; either flag alone is fine). Bake Procedural Detail fixes it: the same pair on the optimised material renders at 695 nodes, because the stack counts values live at once and one texture sample replaces three noise chains that stayed live across the whole surface chain. EEVEE has no SVM stack and is unaffected

Displace Depth

`shader_displace_depth` - Float - default **0.02** - 0 to 2

How far the height map moves the surface, in METRES of world space -- not a fraction, so it survives scaling the terrain object and changing Height Scale, like every other size in this material. Sand grain relief is millimetres to a couple of centimetres, so 0.01-0.05 is the honest range; larger reads as gravel or rubble. 0 = no displacement (but the block is still built -- untick Build Displacement to actually save the cost)

Displace Normal Rebuild

`shader_displace_normal` - Float - default **1** - 0 to 2

Rebuild the SHADING normal from the same height map that moves the surface. Without it the geometry is displaced while the normal still comes only from the procedural bump chain, so the lighting describes a different surface from the one being rendered -- that is what makes displaced sand read as flat or streaky. In EEVEE it matters even more: EEVEE has NO true displacement, so before this the scanned height contributed nothing at all and the relief was simply unlit. The bump distance is driven by the SAME Displace Depth that scales the geometry, so the two cannot drift apart; this is the artistic multiplier on top. 1.0 = normal matches the displacement, 0 = the pre-1.2.10 behaviour

Build Rain Impacts

`shader_build_rain` - Bool - default **off

Include the rain-impact layer in the material. MEASURED: it is 341 of the 603 nodes and the full graph renders about 40x slower than a plain material -- and a shader cannot branch, so leaving it in with Rain Drops at 0 costs exactly as much as leaving it at 1. Switch it on only for scenes that actually rain. Changing this REBUILDS the material, which takes a fraction of a second. IN CYCLES, rain TOGETHER WITH Build Displacement overflows the SVM stack on the reference shader and the terrain renders BLACK (measured: 690 nodes refused, mean RGB 0.580 -> 0.052; rain alone compiles). The cure is Bake Procedural Detail: the same two flags on the optimised material render at 695 nodes, with the rain still fully procedural. EEVEE has no SVM stack and is unaffected either way

Snow Set

`shader_snow_texture_set` - Enum

Scanned surface for the SNOW, from the add-on textures/snow/ folder -- drop your own in (albedo.jpg, normal.jpg, rough.jpg, height.jpg) and it appears here. WITHOUT ONE THE SNOW WORE THE SAND SCAN: measured on a fully snow-covered render, swapping the sand set moved the picture by 0.1218, against 0.2570 on bare sand -- so roughly half the beach was showing through the drifts. The sand scan is now cut off the snow whether or not you pick a set here; picking one gives the snow its own grain instead of a flat colour

Snow Scan

`shader_snow_scan` - Float - default **0.6** - 0 to 1

How strongly the scanned SNOW shows. It goes on as an OVERLAY, so Snow Colour stays the control that decides blue-white against grey and the scan only supplies the grain. 0 = flat colour, the behaviour before this existed

Snow Texture Size

`shader_snow_texture_size` - Float - default **1.2** - 0.02 to 50

Metres across one tile of the snow scan. Its own size and not the sand one, because snow reads at a different scale: drift grain is finer than dune sand at the same distance

Snow Break-up

`shader_snow_tile_method` - Enum - default **HEX

How the repetition of the snow scan is broken up. Snow reads by its NORMAL more than by its colour, so a printed grid shows in the relief and not just in the tint -- every number in these options was measured on the snow normal maps

Option	Meaning
None	One tap of the snow scan, tiled square. Cheapest -- one image sample per map -- and it prints a visible grid at the Snow Texture Size spacing: measured on an orthographic render, the frame correlates 0.974 with itself at exactly one tile
Two-Tap Blend	A second tap, offset, turned 60 degrees and rescaled, blended into the first with a large-patch noise. MEASURED AND NOT RECOMMENDED: it does not remove the grid, it MOVES it. The tile peak drops to 0.509, but the strongest repeat in the frame is then 0.573 at a two-by-three tile period -- the moire of two fixed taps at a 1.31 scale ratio. It also costs 31% of the contrast, and that part is arithmetic rather than a bug: blending two uncorrelated samples at a gives variance (a^2 + (1-a)^2), i.e. half at a = 0.5. Kept because it may suit a particular scan, and because removing a shipped option silently is worse
Texture Bombing	Two square grids offset by half a cell, each taking a random offset PER CELL, averaged with weights that fall to zero on each grid's own borders. The same code the sand and the rock layers use. ! READ THE SECOND NUMBER, NOT THE FIRST: it kills the tile repeat outright (0.974 down to 0.0072) and then leaves a grid of its OWN at 0.429, half a tile ON THE DIAGONAL, which is exactly the offset between its two grids -- and that one is plainly visible in a render. Keeps 82.7% of the contrast. Two image samples per map
Hex-Tiling (Mikkelsen 2022)	Three samples on a triangular lattice, each hexagonal tile taking a random offset AND a random rotation, blended by barycentric weights sharpened to the 7th power and pushed through an S-curve that puts the contrast back. MEASURED on the same render against the other two: tile repeat 0.0021, and its strongest repeat ANYWHERE is 0.026 at 0.07 of a tile -- which is the snow's own grain, not a period (a plain grey reference frame reads 0.005 there). Keeps 94.3% of the contrast against bombing's 82.7% and the two-tap blend's 69.0%. THE PRICE IS REAL: 302 to 428 nodes and three samples per map. It shares Hex Rotation and Hex Contrast with the sand

Snow Break-up Amount

`shader_snow_breakup` - Float - default **1** - 0 to 1

How far the snow break-up may move each patch, for Texture Bombing, or how strongly the second tap shows, for the Two-Tap Blend. Hex-Tiling does not use it -- its per-tile offset is the whole method. AT 0 THE BREAK-UP IS NOT BUILT AT ALL, and that is deliberate: bombing with no jitter still blends the texture with a copy of itself half a tile across, which costs contrast (measured 0.82 of the source) and buys nothing

Scan Blend

`shader_photo_blend` - Float - default **0.5** - 0 to 1

0 = Set A only, 1 = as much Set B as the patch mask allows. The two are mixed by a large noise so the surface changes across the terrain rather than everywhere at once

Scan Patch Size (m)

`shader_photo_patch` - Float - default **9** - 0.5 to 200

Size in METRES of the patches over which the two sets swap

Procedural / Scan

`shader_detail_balance` - Float - default **0.5** - 0 to 1

Balance between the PROCEDURAL detail (dunelets, micro-ripples, grain) and the SCANNED sand. 0 = procedural only, 1 = scan only, 0.5 = both at full strength. The two ramps cross at the middle, so it is a mix control rather than a fade to nothing

Procedural/Scan Mask

`shader_scan_mask` - Float - default **0** - 0 to 1

Make Detail Balance a MAP instead of one number: patches of the terrain lean procedural, others lean scanned, so the surface is not the same recipe everywhere. 0 = the single global ratio, exactly as before. 1 = the mask swings the balance across its whole range. Use it with Detail Balance, which stays the CENTRE the mask pushes either side of

Mask Patch Size

`shader_scan_mask_size` - Float - default **6** - 0.1 to 200

Size of the procedural/scan patches, in METRES of world space (so it survives scaling the terrain and changing solver resolution). Small values read as noise in the material; large ones as regions of different ground. Only used while Procedural/Scan Mask is above 0

Mask Contrast

`shader_scan_mask_contrast` - Float - default **0.5** - 0 to 1

How abruptly the mask crosses from one to the other. 0 is a long soft gradient, 1 a near-hard split. Never a true step -- a mask with no gradient at all reads as aliased edges rather than as a boundary. Only used while Procedural/Scan Mask is above 0

Mask Noise

`shader_scan_mask_noise` - Enum - default **FBM

Shape of the procedural/scan mask. It is a property of the noise NODE, not a socket, so switching it re-shapes the mask live without rebuilding the material

Option	Meaning
fBM	Even, cloud-like patches -- the neutral choice
Multifractal	Uneven: some areas much busier than others, good for a terrain that changes character across it
Hybrid Multifractal	Flat lowlands with detailed highs -- reads like weathering that only took hold in places
Ridged Multifractal	Sharp veins and ridges between the two materials rather than blobs
Hetero Terrain	Detail that grows with height -- ties the mix to elevation

Displacement Method

`shader_displace_method` - Enum - default **BOTH

How the height map is applied. Set on the MATERIAL, so it also shows up in Material Properties > Settings

Option	Meaning
Bump Only	No geometry moves. Works in EEVEE and Cycles and costs nothing -- but it also IGNORES Displace Depth and Displace Midlevel, so those two are greyed out here. MEASURED: depth 0 -> 0.5 moves 4.64% of the pixels under True Displacement and exactly 0.000000 under Bump Only. The reason is this material: it drives the BSDF Normal socket itself, and an explicitly linked Normal overrides the bump Cycles derives from the Displacement output. Use Photo Normal to set the bump strength in this mode
True Displacement	Cycles moves the actual vertices at render time. The simulation mesh is already dense, so this gives real relief per grid cell without any subdivision; add a Subdivision modifier with adaptive subdivision for detail finer than a cell. EEVEE has no true displacement and falls back to flat
Displacement And Bump	True displacement for what the mesh can carry plus bump for the detail below one vertex. The usual choice

Displace Midlevel

`shader_displace_mid` - Float - default **0.5** - 0 to 1

Which grey level counts as "not moved". A height map is 0..1 with the surface around the middle, so 0.5 pushes the bright parts out and pulls the dark parts in by equal amounts. Lower it to make the map only ADD material (everything moves outward), raise it to make it only carve

Scan Size (m)

`shader_photo_size` - Float - default **1.2** - 0.05 to 20

World size in METRES of one tile of the scanned surface

Scan Follow Wind

`shader_scan_wind_align` - Float - default **0** - 0 to 1

Turn the scanned sand maps onto the WIND heading, the same angle the procedural micro-ripples already use. At 0 the photo is tiled square to the world axes, so its own grain runs at an angle to everything the solver built -- which is most of what makes a scan read as wallpaper rather than as ground. 1 lays it fully along the wind. Costs nothing: it rotates the sampling coordinate

Scan Follow Slope

`shader_scan_slope_align` - Float - default **0** - 0 to 1

Turn the scanned maps onto the LOCAL downslope direction, taken per pixel from the surface normal. Where Follow Wind gives the whole terrain one heading, this makes the grain wrap each dune flank separately -- the same reason the ripple crests bend around a dune. Combine them: a little of each usually reads better than all of either. 0 = off

Scan Normal

`shader_photo_normal` - Float - default **0.7** - 0 to 2

Strength of the scanned normal map. It sits UNDER every procedural bump, so dunelets, ripples and grain still stack on top of it

Scan Roughness

`shader_photo_rough` - Float - default **0.6** - 0 to 1

How far the scanned roughness map replaces the computed one

Use Packed Maps

`shader_packed_maps` - Bool - default **on

Sample the scanned NORMAL, ROUGHNESS and HEIGHT from one packed RGBA file instead of three separate images. The normal keeps only X and Y -- Z is rebuilt on the unit sphere -- which leaves exactly four channels, so a texture set costs 2 image fetches per tap instead of 4. The packed file is LOSSLESS, so nothing is given up: measured

max error 0.0000, and Blender hands the bytes back exactly. Falls back per SET, so a folder without a packed file still uses its four separate maps. Run pack_textures.py to build them. Changing this REBUILDS the material

Build Texture Bombing

`shader_build_bombing` - Bool - default **on

BUILD the second sampling grid at all. This is the cost switch, not the Texture Bombing slider above: with it OFF the second tap is never created, which HALVES the image samples of every scanned map -- albedo, roughness, normal and height alike. WHAT YOU GIVE UP: the scan tiles visibly at the Photo Size grid, because hiding that repetition is the only thing bombing does. Worth turning off for viewport work, for terrain seen from far enough that the tile is below a pixel, and whenever the scan is mostly hidden under snow or vegetation. Changing it REBUILDS the material

Texture Bombing

`shader_photo_bombing` - Float - default **1** - 0 to 1

Random per-cell offset applied when the scan is sampled, so a 1K tile stops repeating. Two offset grids are averaged with weights that fall to zero on each grid own cell borders, so neither grid shows a seam. 0 = plain tiling, and the repetition comes back. NOTE THIS DOES NOT MAKE THE SHADER CHEAPER: at 0 the two grids sample the SAME coordinate and the graph still pays for both. To drop the cost, switch Build Texture Bombing off instead. It does nothing for the PROCEDURAL layers -- those do not tile

Hex Rotation

`shader_hex_rotation` - Float - default **1** - 0 to 1

How far each hexagonal tile may be turned, as a fraction of a half-turn. This is the part texture bombing cannot do at all, and it is what stops recognisable features -- pebbles, bedding, streaks -- from pointing the same way across the whole surface. 0 keeps every tile square to the others: measured, that alone takes the leftover correlation from 0.276 up to 0.308. Costs nothing extra; the rotation is built either way

Hex Contrast

`shader_hex_contrast` - Float - default **0.7** - 0.5 to 0.95

Sharpness of the blend between neighbouring hex tiles (the paper's r). 0.5 is a plain blend and the tiles fade into each other, softening the surface; higher values tighten the seam and keep the contrast, at the price of the hexagons themselves starting to show on some textures. The paper recommends 0.65 to 0.75, which is where the default sits. It is a live slider, so a shader cannot skip the curve: 0.5 turns it off in effect, not in cost

Triplanar Scanned Sand

`shader_scan_triplanar` - Bool - default **off

Project the scanned sand on all three planes and blend them by the surface normal instead of tiling flat from above. A slip face stands at 34 degrees and a flat projection stretches the grain down it, which is what makes a steep dune flank read as smeared. It costs THREE image reads per map instead of one and MULTIPLIES with texture bombing (three planes, two taps each = six samples per map), so it is off by default and the panel prints the measured node count. It cannot be combined with Hex-Tiling: that path computes one weight set from the first map and shares it, and three lattices would be about 330 nodes before the first sample -- with Hex selected this is ignored and the panel says so

Triplanar Blend

`shader_rock_triplanar_blend` - Float - default **4** - 1 to 16

ONE exponent for every triplanar in this material -- the rock, the scanned sand and the plant cover -- because the weights come from the surface normal and nothing else, so all three can share the same 15-node block. How sharply the three projections hand over. Low values cross-fade broadly, which is soft but slightly washed on 45-degree ground; high values snap to whichever plane faces the surface, which is crisp but can show a seam on the diagonal. 4 is the usual compromise

Dunelet Size (m)

`shader_dunelet_size` - Float - default **0.7** - 0.1 to 8

Wavelength IN METRES of the DUNELETS -- the octave between the simulated megaripples and the centimetre grain. The physical ripple solver measures its wavelength in grid CELLS, so on a normal terrain it cannot go below a couple of metres; this layer fills the half-metre-to-metre band that the eye reads as small dunes crawling over the big ones. Bump only, no extra attribute

Dunelet Strength

`shader_dunelet_strength` - Float - default **0.45** - 0 to 1

Bump strength of the dunelets. 0 = off. Their depth follows Dunelet Size and Height Scale, so relief stays honest when either changes

Dunelet Sharpness

`shader_dunelet_sharp` - Float - default **0.65** - 0 to 1

How narrow the dunelet crest is. 0 = soft mounds, 1 = a sharp brow with a wide flat trough. This is what separates a dune SHAPE from symmetric noise: the wave already runs long on the windward side and drops on the lee, and this narrows the top of it

Dunelet Break

`shader_dunelet_break` - Float - default **0.55** - 0 to 1

Break the dunelet ridges into separate crescents. 0 = continuous corrugation running across the whole frame, 1 = scattered individual dunelets with bare sand between them. Real small dunes come and go along a crest line rather than running unbroken

Dunelet Stretch

`shader_dunelet_stretch` - Float - default **0.6** - 0 to 1

How far the dunelets are drawn out ACROSS the wind. 0 = round blobs, 1 = long ridges perpendicular to Wind Direction. Lower it towards barchan-like patches, raise it for transverse ripple trains

Micro-Ripple Size (m)

`shader_ripple_size` - Float - default **0.12** - 0.02 to 2

Wavelength IN METRES of the wind-aligned micro-ripples. REAL wind ripples are 5-30 cm, which is below one solver cell on any normal terrain -- so the physical ripple solver inherently works at MEGARIPPLE scale and this shader layer is the only place the centimetre ripples can come from. Aligned to Wind Direction and gated to exposed sand, so they follow the same wind as the sim

Micro-Ripple Strength

`shader_ripple_strength` - Float - default **0.25** - 0 to 1

Bump strength of the micro-ripples. 0 = off

Micro-Ripple Stretch

`shader_ripple_aniso` - Float - default **0.86** - 0 to 1

How elongated the ripple crests are ACROSS the wind. High values give long continuous crest lines (what a steady wind makes), low values break them into a cellular pattern (gusty / turning wind)

Micro-Ripple Sand Gain

`shader_ripple_sand_gain` - Float - default **6** - 1 to 16

How readily THIN sand cover counts as rippled. The ripple gate is the sand mask times the wind-exposure mask, and the sand mask is sand DEPTH normalised against the field own range -- a relative rank, not "is there sand here". MEASURED on a 256 dune field: the gate averages 0.03 and its MEDIAN is 0.00, reaching 0.1 on only 10 percent of the surface, while sand at least 5 cm deep covers 25 percent -- so a cell carrying a whole metre of sand still ripples at about 7 percent strength. Real ripples are a 5 cm surface feature, so 20 cm and 4 m of sand ripple the SAME. Raise this to saturate the sand term and let every genuinely sandy cell ripple at full strength; the shipped 6.0 covers a normal

dune field and saturates soon after. Set 1.0 for the pre-1.2.9 look bit for bit, where the whole micro-detail family was calibrated with the old collapsed gate in place

Curvature Radius (m)

`curvature_radius` - Float - default **1.5** - 0.05 to 50

The neighbourhood size IN METRES used to generate curvature and cavity: the height is compared against its own average over this radius. Small = tight creases only; large = whole dune flanks read as convex or concave. Being a length rather than a cell count is what makes the result identical at any solver resolution

Terrain Shade

`shader_terrain_shade` - Float - default **0** - 0 to 1

How much the terrain sits in its OWN shade. Cycles and EEVEE already cast the real geometric shadow of one dune onto the next; this is the other half -- the ambient occlusion an enclosed, low spot gets from the terrain around it, which a sun lamp alone cannot give. Built from the generated height and cavity channels, so it needs Surface Shape Attributes and costs no new attribute. 0 = off and bit-identical; 0.2-0.4 deepens interdune hollows without flattening the sunlit crests

Cavity

`shader_cavity` - Float - default **0.55** - 0 to 1

How much hollows and crevices darken and roughen. Driven by a CAVITY attribute generated from the heightfield over a neighbourhood measured in METRES (Curvature Radius), so a 1.5 m hollow stays 1.5 m whether the grid is 256 or 2048 -- unlike Blender own Geometry > Pointiness, which is per-vertex, changes meaning with the grid, and does not exist in EEVEE. Also dents the surface normal, not just the colour

Convexity

`shader_convex` - Float - default **0.35** - 0 to 1

How much convex ridges read as wind-swept: slightly brighter, drier and coarser than the flats. Convex and concave come from the same generated CURVATURE attribute (0.5 = flat, above = ridge, below = hollow), so the two are always consistent

Slope Tint

`shader_slope_tint` - Float - default **0.3** - 0 to 1

Tint by the surface ANGLE, from a generated slope attribute (degrees / 90). Independent of the Slip Faces layer, which uses the geometric normal and only fires past the repose angle -- this one shades the whole gentle-to-steep range and keeps working when the mesh is drawn at a lower Display Resolution

Height Tint

`shader_height_tint` - Float - default **0.15** - 0 to 1

Subtle tint by RELATIVE height (0 = the field lowest point, 1 = the highest), from a generated attribute. Useful for bleaching crests or keeping interdune floors darker and damper-looking

Wind Exposure

`shader_exposure` - Float - default **1** - 0 to 2

How much the wind-exposure mask bleaches windward faces. This is the solver shear, so the bright side always faces the wind the simulation is actually using, including wobble and schedules

Lee Shadow

`shader_lee` - Float - default **1** - 0 to 2

How much the lee-shadow mask darkens sheltered ground behind crests and obstacles. Driven by the separation bubble the solver computes, so it tracks the real recirculation zone

Slip Faces

`shader_slipface` - Float - default **0.6** - 0 to 1

How strongly avalanche faces read as FRESH sand. Any face steeper than the repose angle has just avalanched, so it is loose, uncompacted and lighter than the packed windward back -- and it also loses its wind ripples, which this drives too (ripples do not survive on a slip face). The threshold follows the solver Repose Angle, so it stays honest if you change the sand

Sand Cover Depth

`sand_cover_depth` - Float - default **0.06** - 0.0001 to 20

How deep the sand has to be before the surface reads as fully SAND rather than bedrock, in height units. This is the reference the sand mask is measured against -- the mask that gates the scanned albedo, the ripples, the dunelets, the grain and the Bedrock Threshold. It used to be the field's own MIN-MAX range, which fails at both ends: measured on the shipped preset, a dune field 0 to 1.75 m deep sent 49.8% of the terrain to the shader as "no sand", and a UNIFORM 2 m layer came out as 0.000000 everywhere. Lower it if too much bedrock shows through; raise it to expose the rock wherever the dunes have thinned

Bedrock Layer

`shader_rock` - Float - default **0** - 0 to 1

How much bare BEDROCK is allowed to show through where the solver reports no sand. OFF by default on purpose: with the standard Terrain Is Sand = off, the bedrock IS the surface you started from, so "no sand above bedrock" happens everywhere the dunes have simply migrated away -- switching rock on there turns an ordinary dune field almost black. Raise it when your scene really does scour down to rock (imported rock terrain, a bedrock object, or a painted non-erodible floor)

Bedrock Threshold

`shader_rock_threshold` - Float - default **0.03** - 0 to 0.5

How little sand counts as BARE ROCK. Read this before touching it: the sand mask is NORMALISED to the field own minimum and maximum, not an absolute depth -- so an interdune floor with 20 cm of sand under a 3 m dune reads as about 0.07, not as "deep". Set this too high and a perfectly ordinary sand sheet renders as a rock plain. Raise it only when your scene really does scour down to bedrock

Rock Tiling (m)

`shader_rock_tiling` - Float - default **4** - 0.05 to 200

Size IN METRES of one tile of the Rock Colour Map. Measured from the terrain origin in world space, so the rock does not slide when the terrain object moves. Does nothing without a map loaded

Triplanar Rock

`shader_rock_triplanar` - Bool - default **off

Sample the bedrock maps on all three planes and blend them by the surface normal, instead of projecting flat from above. A flat projection SMEARS on slopes -- and rock is mostly slope, because that is where the sand ran off it -- so a steep face gets the texture stretched down the fall line as vertical streaks. Costs THREE image reads per map instead of one, which is why it is off by default: this shader lives near the Cycles node ceiling and the panel prints the measured cost

Rock Normal Strength

`shader_rock_normal_strength` - Float - default **1** - 0 to 2

How strongly the Rock Normal Map bends the surface normal. Does nothing without that map loaded

Rock Blend Mode

`rock_blend_mode` - Enum - default **WEIGHTED

How rock layers combine where their masks overlap

Option	Meaning
Weighted	Every enabled layer contributes its own mask as a WEIGHT, and the weights are normalised. Boundaries are soft and no layer ever vanishes completely where another one is also present -- this is the mode in which blending the masks against each other actually means something
Stack	Layer 4 over 3 over 2 over 1, each using its mask as alpha. Predictable and splat-like, but the upper layer always wins where two masks overlap, so mixing masks only softens edges

Rock Bombing

`shader_rock_bombing` - Float - default **0.35** - 0 to 1

How far each stamped patch is shoved when a rock layer has Bombing on. 0 puts every patch back on the plain grid, which is the tiling the bombing was built to hide, and it still pays for both samples -- turn the layer's Bombing checkbox off instead. This is the same quantity as Photo Bombing and drives the same taps; it is a separate number only because rock tiles in metres and the sand scans in centimetres

Rock Bake Resolution

`rock_bake_res` - Int - default **2048** - 256 to 8192

Pixels across for the three baked rock maps. Unlike the layer textures, which tile, this one covers the whole terrain once -- so on a 200 m terrain 2048 is about 10 cm per texel and the rock grain WILL be softer than the live layers up close. 4096 costs four times the memory (3 maps x 64 MB) and halves the texel. Bake at the size you actually render

Use Baked Rock

`rock_baked_active` - Bool - default **off

Let the baked rock maps drive the bedrock INSTEAD of the live layer stack. This is what makes the bake stick: with it on, the layer checkboxes, their masks and their twelve image slots build NOTHING into the shader, and the rock still looks like what you baked. Turn it off to go back to the live layers and carry on editing -- the baked maps are kept, so it is a switch between two inputs, not a destructive step. Does nothing until a Baked Rock Colour exists

Sand Age

`shader_age` - Float - default **1** - 0 to 2

How strongly sand age shifts fresh sand toward the weathered colour

Protected Zones

`shader_protected` - Float - default **0.5** - 0 to 1

How visibly non-erodible (protected) ground reads as crusted. Driven by the erodibility mask, so anything you painted with the Freeze/Erodibility brush or rasterised from an object shows up

Coarse Lag

`shader_coarse` - Float - default **1** - 0 to 2

Strength of the coarse-grain pavement layer (Multi-Fraction only)

Strata in Rock

`shader_strata` - Float - default **0.8** - 0 to 1

How strongly the cross-bedding laminae show in the BEDROCK. Strata are a rock feature, so they are multiplied into the rock layer only and never into the loose sand. Needs Strata enabled

Wetness

`shader_wet` - Float - default **1** - 0 to 2

How strongly the water-table mask darkens AND smooths the surface. Wet sand is both darker and more specular, so this drives colour and roughness together. Needs the Water model on

Snow Cover

`shader_snow` - Float - default **1** - 0 to 2

Strength of the snow layer (fresh cover + hardness tint). Needs the Snow model

Vegetation Cover

`shader_veg` - Float - default **1** - 0 to 2

How strongly plant cover shows. It also roughens the surface and SUPPRESSES the micro-ripples, because plants anchor the sand -- a vegetated patch does not ripple

Mask Break-up

`shader_mask_breakup` - Float - default **0.5** - 0 to 1

Hide the STAIRCASE where vegetation meets sand and where bedrock meets sand. Those boundaries are drawn from per-VERTEX values at the solver resolution, and the mesh interpolates them across whole triangles -- so the edge is polygon-shaped, with straight facets and visible steps, however fine the textures on either side are. This adds a fine noise to the mask itself, so the boundary wanders at TEXTURE scale instead of at mesh scale. It costs nine shader nodes and nothing in the simulation. MEASURED on a deliberately coarse 96-cell field, share of the frame that changes: 0.15 -> 0.06%, 0.30 -> 8.9%, 0.60 -> 29.5%. So 0.3-0.6 is the working range; below 0.2 it is doing almost nothing, and far above 0.6 it starts eating small patches whole. DEFAULTS TO 0.5, the middle of that range: the staircase is a defect rather than a look, so hiding it is the sensible starting point and the knob is there to tune or switch off. 0 = off, and the mask is then exactly as the solver wrote it

Break-up Size (m)

`shader_mask_breakup_size` - Float - default **0.35** - 0.02 to 20

How coarse the break-up noise is, in metres. It has to be SMALLER than a solver cell or it cannot hide a cell edge -- it would just add a second, larger wobble on top of the staircase. A few tens of centimetres suits a field where cells are a metre or two across. Only used while Mask Break-up is above 0

Flatten Relief Under Plants

`shader_veg_normal_fade` - Float - default **0** - 0 to 1

How much of the surface RELIEF the plants cover up, as a percentage. The rock normal is mixed in on the rock/sand boundary, which knows nothing about plant cover -- so a vegetated cell on bare rock reads as green paint on stone, complete with the stone's every crack. This fades the composed normal back toward the plain geometric one where the cover is, and it is a percentage rather than a switch because a thin lawn really does show the ground through it. 0 = off, and at 0 it builds NO nodes at all

Vegetation Set

`veg_texture_set` - Enum

Give the plant cover its own scanned surface instead of a flat colour. Same list as the sand and the rock layers -- drop a folder into textures/ (albedo, normal, rough) and it appears here. Sampled on the same world-space metres as everything else, at Vegetation Texture Size, and shown only where the cover is

Triplanar Vegetation

`shader_veg_triplanar` - Bool - default **off

Project the vegetation maps on all three planes and blend them by the surface normal. The plants smear on slopes WORSE than the rock does: scrub tiles at well under a metre, so the same 34 degrees of dune flank stretches it several times further than a 4 m rock tile. Three image reads per map instead of one. It shares the Triplanar Blend exponent and the weight block with the rock and the sand, so the second and third triplanar in a material cost only their own samples

Vegetation Texture Size (m)

`shader_veg_texture_size` - Float - default **0.6** - 0.02 to 50

Size in METRES of one tile of the vegetation maps. Scrub and grass read at a much finer scale than the sand under them, which is why this is its own number and not the shared Photo Size. It is multiplied by the master Texture Scale like every other size

Rain Impacts

`shader_rain_drops` - Float - default **1** - 0 to 1

Master for raindrop impacts. WHERE and WHEN they appear is not a shader guess: the solver exports the field of rain that is falling right now, so impacts follow the storm cells across the terrain and stop the moment the cell has passed. With Rain off that field is empty and this layer costs you nothing visible

Water Film At

`shader_water_film` - Float - default **0.8** - 0 to 1

How wet the surface has to be before a drop lands in WATER rather than on ground. Above this the impact is a ring spreading across a film; below it, a splash crater in damp sand. Lower it to get rings over merely damp ground, raise it so only genuinely flooded areas ripple

Impacts From Particles

`shader_impacts_from_particles` - Bool - default **off

Draw the rings and craters WHERE DROPLETS ACTUALLY LANDED instead of on a procedural lattice. Off, the shader invents drop sites from a hash of the surface and they have nothing to do with the particles; on, the particle pool records each landing into a small map and the graph reads it, so a ring appears under a droplet you can watch fall. Needs Splash Particles and Rain. It is a BUILD switch, not a fade: a shader has no branching, so an unused block still costs on every pixel and the only way to make one free is to leave it out -- flipping this rebuilds the material

Impact Grid

`impact_grid_res` - Int - default **256** - 16 to 4096

Resolution of the impact map, in cells across the terrain. It also sets how far apart drop sites can be: one impact is remembered per cell, so a coarse grid merges drops that landed close together while a fine one keeps them apart and costs more memory (RGBA float: 256 = 1 MB, 1024 = 16 MB). At 256 on a 200 m terrain a cell is 78 cm. Only used while Impacts From Particles is on

Impact Memory

`impact_life` - Float - default **24** - 1 to 400

How many simulation steps a recorded impact keeps drawing before it is forgotten. It is the ring lifetime: the ring expands over exactly this span and then the cell goes empty again. Short = sparse, sharp, individual rings; long = many overlapping rings and a busier surface. Only used while Impacts From Particles is on

Drop Spacing

`shader_drop_spacing` - Float - default **0.35** - 0.02 to 5

Average distance between impact points, in METRES, so it keeps its meaning when the terrain is scaled or the resolution changes. Small = dense drizzle, large = fat scattered drops

Triplanar Drops (Experimental)

`shader_drops_triplanar` - Bool - default **off

EXPERIMENTAL -- KNOWN DEFECT IN CYCLES. With this on, Cycles can render patches of the terrain solid BLACK: on a perfectly flat test terrain it blackens 7.46% of the surface, and the black comes specifically from the Y (XZ-plane) projection. EEVEE is unaffected to seven digits. The cause is not known -- eight separate investigations ruled out the refactor, the geometry, the node count, value spikes in both the ring and the splash chains, the lattice coordinate and the expression-tree width, and the black follows the CONNECTION rather than anything the lattice

computes. It is left in because it is opt-in and off by default; if you see black patches in Cycles, turn it off and the render returns bit-for-bit to the single-projection result. WHAT IT DOES: project the rain-drop lattice from all three axes and blend by the surface normal, instead of straight down from world XY. WHY: the flat projection lays a cell boundary every Drop Spacing metres of HORIZONTAL distance, and one metre along a slope is only cos(angle) metres horizontally -- so on the surface the drops are stretched along the slope by 1/cos, i.e. 1.21x at the 34 degree sand repose and 1.70x on 54 degree rock. COST: the whole nine-tap block is built three times, so expect roughly 3x the rain layer and check it against your own scene. On flat ground it changes nothing. Ignored while Impacts From Particles is on, because that map is one texel per terrain cell and only exists top-down. HOW MUCH IT ACTUALLY BUYS: blending three projections BOUNDS the stretch at sqrt(2), it does not remove it -- and at the 34 degree repose the top projection on its own (1.21x) is already better than any blend, so the gain is real only on rock steeper than 45 degrees. Rebuilds the material

Drops Per Second

`shader_drop_rate` - Float - default **3** - 0.05 to 30

How often each impact site is struck, per second of scene time. It drives the animation directly, so it is also what sets how long a ring lives: one impact lasts 1 / this many seconds

Ring Speed

`shader_ring_speed` - Float - default **0.9** - 0 to 10

How fast a ring spreads across standing water, in metres per second. Together with Drops Per Second this sets how big a ring gets before it dies: radius = speed / rate

Ring Width

`shader_ring_width` - Float - default **0.025** - 0.002 to 0.5

Thickness of the ring crest in metres. Thin reads as clean water, thick as a soupy or muddy surface

Ring Waves

`shader_ring_waves` - Float - default **0.55** - 0 to 1

How many trailing waves each impact leaves. A drop on water does not make one line -- it makes a capillary PACKET: a leading crest, a trough right behind it, then a weaker second crest. 0 is a single crest (the old look, which read as an etched groove); higher values add the trailing train

Ring Noise

`shader_ring_noise` - Float - default **0.35** - 0 to 1

Wobble the rings off a perfect circle. Real rings are disturbed by the surface they travel over; drafted circles are the giveaway that this is a shader. Scaled by Ring Width, so it stays proportionate at any ring size

Water Chop

`shader_water_noise` - Float - default **0.3** - 0 to 1

Fine chop on the water surface while rain is falling. Rain does not only leave rings, it roughens the whole surface. Gated to the water film (dry sand does not chop) and to rain actually falling, so it goes away with the shower

Splash Size

`shader_splash_size` - Float - default **0.05** - 0.005 to 1

Radius in metres of the crater a drop digs when it lands on ground rather than water. Unlike a ring it does not travel -- a drop hitting damp sand makes a pit with a rim, it does not send a wave anywhere

Impact Depth

`shader_drop_depth` - Float - default **0.018** - 0 to 0.2

How deep the impacts dent the surface normal, in metres. Follows Height Scale like every other length in the graph, so it stays physically sized when the terrain is stretched

Wet Spot

`shader_drop_darken` - Float - default **0.55** - 0 to 1

How much a fresh impact darkens the ground where it lands. Only applies where there is NO standing water -- on open water there is nothing to darken, so this fades out as the film builds

Lee Smoothing

`shader_lee_smoothing` - Float - default **0.18** - 0 to 1

How much SMOOTHER the ground is in the wind shadow. Fine dust settles where the wind cannot reach, so a lee streak is not only darker -- it is glossier than the swept crest beside it. The shadow already tinted the COLOUR before this existed; what it did not touch was roughness, and roughness is the half that makes a streak come alive when the sun moves across it. Two surfaces of the same colour and different roughness look identical in a still frame and completely different in motion. 0 leaves the roughness chain bit-identical

Ripple Anisotropy

`shader_ripple_anisotropy` - Float - default **0.3** - 0 to 1

Stretches the specular highlight ACROSS the ripple crests, the way any grooved surface reflects -- brushed metal and corduroy do the same thing. The tangent comes from the SAME Wind Angle the ripples themselves use, turned 90 degrees, so the highlight cannot end up lying at an angle to the grooves it is supposed to come from. Gated to where ripples actually are (bare sand, off the slip face, off plants) -- anisotropy on bare rock would describe grooves that are not there. Strongest under a low sun and on wet sand. 0 = the isotropic highlight of every earlier build

Grain Sparkle

`shader_sparkle` - Float - default **0.15** - 0 to 1

How many grains catch the sun as a MIRROR. Sand carries flakes of mica and quartz that reflect specularly and pointwise, and that glitter is one of the strongest cues that a surface is real sand rather than a brown material. THIS IS NOT SHEEN. Sheen is a broad grazing-angle scatter along the silhouette; this is individual grains flashing. They stack. IT RIDES THE COAT LAYER, with its own normal, and that is the whole reason it works: an earlier build perturbed the SURFACE normal instead, which bought every flash with a dark speck underneath -- measured 167 darkened pixels per lit one, and the contact sheet read as grainy dirt rather than as sparkle. DEFAULT 0.15 IS MEASURED, not guessed: at 0.35 the speckle starts showing in shadow (0.157% of the frame darkened against 0.045% at 0.15), and past 0.7 it reads as noise. The slider moves the THRESHOLD, so it controls how MANY grains light up, not how bright each one is -- a real flash is either there or it is not. 0 = off, and the block still builds; clear it from the graph by leaving it at 0 if you need the nodes back

Sparkle Grain

`shader_sparkle_size` - Float - default **0.004** - 0.0002 to 0.05

Size of one flashing grain, in METRES. 0.004 is a 4 mm cell, which is coarser than a real sand grain on purpose: a flash the size of an actual 0.25 mm grain is far below one pixel at any normal camera distance, so it cannot be seen -- only aliased. It is a world length like every other size in this shader, so it survives scaling the terrain

Sparkle Fade

`shader_sparkle_fade` - Float - default **35** - 1 to 500

Distance at which the glitter drops to HALF strength. THIS IS PART OF THE EFFECT, NOT A PERFORMANCE OPTION. A flash is sub-pixel by nature, so past a certain distance it falls under the sampling rate and turns into crawling noise instead of sparkle -- the same failure every snow shader has. Raise it for a close-up, lower it for a wide landscape where the far dunes should read as smooth

Sheen

`shader_sheen` - Float - default **0.25** - 0 to 1

Grazing-angle sheen. Dry wind-blown sand genuinely has one: the grains are fine, rounded and dust-coated, so they scatter forward and the surface goes bright when you look along it toward the sun. Driven DOWN by wetness and by

vegetation, because a water film fills the interstices and plants break the grain carpet -- so it reacts to the simulation instead of being a constant. Needs Blender 4+ (Principled v2 sheen)

Sheen Roughness

`shader_sheen_rough` - Float - default **0.35** - 0 to 1

How tight the sheen highlight is. Low = a narrow rim right at the silhouette (very fine, well-sorted sand); high = a broad soft wash over the whole lit side (dusty, poorly sorted)

Sheen Tint

`shader_sheen_tint` - FloatVector - default **0.94, 0.9, 0.82** - 0 to 1

Colour of the sheen. Slightly warmer and lighter than the sand reads as airborne dust catching the light; neutral white reads as clean quartz

---

Surface Detail

The layer that lives ABOVE the dune: ripples, micro-relief, the texture the wind writes on the sand. These are geometry driven by the wind field, so they wrap around the dune and turn where the wind turns.

When you need it: Close-ups. At distance the silhouette carries the shot; up close this does.

Show Render Detail

`detail_enabled` - Bool - default **off

Master switch for the RENDER-ONLY detail layer written onto the mesh: wind ripples, fine sand grain, crest sharpening and the edge-aware Polish smoothing. The physics never sees any of it -- toggle freely at any time, no re-Initialize. Costs some ms per mesh update at high resolution; with the Displacement Preview active it streams into the preview image instead. Caution with Terrain Feedback: reading the detailed mesh back can absorb the ripples as real sand

Detail Feeds Sim

`detail_feeds_sim` - Float - default **0** - 0 to 1

Turn the RENDER detail layer into real sand so it EVOLVES, instead of sitting on the surface like a decal. Ripples, dunelets, grain and the micro-detail pass are drawn on top of the solver height and nothing moves them -- the wind does not, a dune migrating through does not rework them, and baking the heightfield loses them. This hands back a fraction of that relief every step. Adds TEXTURE, not mass. Keep it SMALL: the detail is regenerated each frame, so a large value keeps piling roughness on. Needs Show Render Detail. NOTE, measured: the visible field is the detail layer AND the Polish/Smooth Edge pass, so what comes back carries the smoothing too -- with Polish high the net effect can be a SOFTER solver surface rather than a rougher one. Turn Polish down if you want this to add texture only. 0 = the shipped behaviour, unchanged

Ripple Style

`ripple_mode` - Enum - default **DUNELETS

Which pattern the render-detail ripples use. This is the TEXTURE layer and can be drawn at any size; the Physical Ripple Solver beside it is a separate, simulated layer limited by the grid

Option	Meaning
Dunelets	Organic broken mini-dunes without obvious sine lines
Classic Waves	Legacy wind-aligned sine ripples
Carved Sastrugi	Erosional wind-carved grooves for snow

Ripple Amp

`ripple_amp` - Float - default **0.4** - 0 to 3

RENDER-ONLY ripple height (solver height units) of the procedural ripple texture layered on the dunes. Needs Show Render Detail; 0 disables the ripple layer. Does not change the physics (for physically simulated ripples use Physical Ripples below)

Ripple Length

`ripple_len` - Float - default **5** - 0.5 to 80

Wavelength of the render ripple texture in SOLVER CELLS (crest-to-crest). Bigger = wider-spaced, calmer ripples

Ripple Length Follows Grain

`ripple_grain_link` - Float - default **1** - 0 to 1

How much the ripple wavelength follows Grain Diameter instead of the fixed Ripple Length above. 0 = off and the shipped behaviour, where the two are unrelated -- Ripple Length is typed in and nothing computes it. THE PHYSICS IT RESTORES: a ripple wavelength is set by the REPTATION HOP, the short ballistic bounce a grain makes when a saltating one hits it, and that hop scales with the grain. Coarse sand therefore carries longer, coarser ripples and fine sand tight ones -- which is why a gravel lag surface and a dune crest never look the same close up. At 1.0 the wavelength is fully proportional to the diameter, so 500 um doubles it and 125 um halves it, against the 250 um reference. This changes the DETAIL LAYER only; ripples are centimetres and solver cells are metres, so they are sub-grid by two orders of magnitude and could never have been resolved by the dune solver itself

Ripple Warp

`ripple_warp` - Float - default **0.6** - 0 to 8

How strongly the ripple crests bend/wander (0 = dead-straight sine crests, higher = organic, broken crests). Render-only

Slope Fade

`ripple_slope_fade` - Float - default **0.35** - 0 to 1

Fade the render ripple texture out on steep slopes (0 = ripples everywhere, 1 = strong fade). Real slip faces are smooth flowing sand -- without the fade the ripple lines pack together on steep faces and read as noisy stripes. Render-only

Soft Shadow Edge (render)

`shadow_soft` - Int - default **0** - 0 to 8

RENDER-ONLY: feather the wind-exposure field by N blur passes so the ripple/detail cutoff behind crests and obstacles is a soft gradient instead of a razor line (the hard line also visibly SNAPS when the wind direction changes). Physics untouched; 0 = off (exact 3ds Max look)

Dunelet

`dunelet` - Float - default **0** - 0 to 1

Amount 0..1 of broken mini-dune patches mixed into the ripple texture (DUNELETS mode look): 0 = continuous ripple field, higher = scattered crescent patches. Render-only

Detail Grain

`detail_amp` - Float - default **0.12** - 0 to 2

RENDER-ONLY fine sand grain amplitude (solver height units) -- the smallest speckle layer, visible up close. Dusts even bare ground lightly, unlike ripples which follow sand cover and wind exposure

Sharpness

`sharpness` - Float - default **0.67** - 0 to 1

Ripple crest profile 0..1: low = soft sine-like waves, high = sharp crests with flat troughs (wind-worn look). Render-only

Polish

`polish` - Int - default **2** - 0 to 60

RENDER-ONLY smoothing passes that clean small numeric noise off the open windward faces while deliberately SKIPPING lee cells, so slip-face edges stay crisp (SmoothSkipLee). 1-3 passes usually suffice; high counts soften genuine small dunes. Strength per pass comes from Smooth Edge; needs Show Render Detail

Smooth Edge

`smooth_edge` - Float - default **0.2** - 0 to 20

Edge-aware smoothing strength of the render POLISH pass (0 = off). This is the RANGE sigma of a bilateral filter, so it must be of the order of the NOISE amplitude, not of the terrain relief: neighbours that differ by much more than this are treated as a different surface and left out of the average, which is what keeps an edge. MEASURED (fine noise 0.02 across a 2-unit wall, 3 passes): noise removal saturates by 0.05 and the wall survives at 100 percent all the way to 0.35, then falls to 99.6 at 0.6, 85.1 at 1.0 and 60.2 at 2.0. The shipped 1.0 therefore sat PAST the knee and gave away 15 percent of every edge for no extra cleaning, against the promise in this very tooltip; 0.2 sits in the middle of the flat 100 percent stretch. Raise it only if your noise is much coarser

Render Subdivision

`render_subdiv` - Int - default **1** - 1 to 4

Bicubically upsample the solver grid to a finer render mesh (Mitchell-Netravali). 1 = render at solver resolution; 2-4 = smoother high-detail render from a coarser/faster solver. Rebuilds the terrain mesh to solver_res x subdiv (heavy at high resolution)

De-facet Crests

`crest_defacet` - Float - default **0** - 0 to 1

RENDER-ONLY cure for the grid look: a unidirectional near-axis wind lets the solver grid show through as straight horizontal or vertical zigzag crest segments (grid anisotropy, inherent to grid dune solvers). This smooths the dune crests ALONG their length so they read as flowing lines, while the cross-crest profile and slip faces stay sharp. 0 = off (mesh byte-identical), 0.4-0.7 = natural. The dune physics never feels it. Free alternative in the sim itself: Star or Reversing wind, or more Wobble / Jitter

De-facet Passes

`crest_defacet_passes` - Int - default **3** - 1 to 10

How many crest-aligned smoothing passes De-facet Crests runs. More = straighter zigzags erased but wider dune crests softened. 2-4 is usually enough. Only used when De-facet Crests is above 0. Render-only

Physical Ripple Solver

`ripple_solver_enabled` - Bool - default **off

Add a PHYSICAL wind-ripple micro-layer on top of the dunes (anisotropic Kuramoto-Sivashinsky). Unlike the noise-texture ripples above, these are a real migrating instability: correct wavelength, they drift downwind, form Y-junctions and merge. Costs one extra light GPU pass per frame. Use INSTEAD of (or with) Ripple Amp. THESE are the crosswise bands on the dunes -- a different control from Show Render Detail, which is why turning that off does not remove them. Every preset switches this ON as part of its look; switch it off by hand once and presets will leave it alone from then on

Ripple Height

`ripple_solver_amp` - Float - default **0.06** - 0 to 2

Height at which the physical ripples are composited onto the render mesh (solver height units); 0 = the ripple sim still runs but is invisible. Render-only -- the dune physics never feels the ripples. Keep small (0.03-0.1): real ripples are centimetres on metre-scale dunes

Ripple Wavelength

`ripple_solver_wavelength` - Float - default **7** - 0.01 to 64

Crest-to-crest spacing of the PHYSICAL ripples, in CELLS of the ripple grid -- so its size in metres is this times the cell size (a 200 m terrain gives about 0.4 m per cell, i.e. 7 = roughly 3 m). Measured limits: below 5 the pattern collapses into grid-scale noise (at 4 nearly two thirds of its energy sits at the grid), 6-7 is the finest CLEAN result, and 10+ reads as megaripples (pair those with Megaripple Height in Multi-Fraction). REALITY CHECK: true wind ripples are 5-30 cm, which is far below one cell on a normal terrain, so this layer always works at MEGARIPPLE scale -- for centimetre ripples use the texture layer (Show Render Detail), which is not limited by the grid. The ripple field is a real migrating simulation composited onto the render mesh; the base dunes are untouched

Ripple Detail

`ripple_solver_detail` - Float - default **1** - 0.5 to 4

How fine the ripple GRID is, as a multiple of the solver grid. This is the control to reach for when the ripples are too big: Ripple Wavelength is measured in CELLS and cannot go below 5 without collapsing into grid noise, so the only way to smaller ripples in METRES is smaller cells. 2 halves the ripple size, 4 quarters it. Cost grows with the AREA (2 = 4x the ripple grid), though the wavelength in cells is unchanged so the internal substep count is not affected. The panel shows the resulting size in metres

Ripple Growth

`ripple_solver_growth` - Float - default **0.6** - 0.05 to 2

How fast the physical ripple pattern develops and sharpens: low = ripples take many frames to appear and stay soft, high = a crisp field forms quickly. Any value is numerically SAFE (above ~1 the solver substeps automatically -- denser substeps = a bit slower). Too high can look busy/harsh -- back off or raise Crest Smoothing if the field turns noisy

Ripple Migration

`ripple_solver_migrate` - Float - default **0.35** - 0 to 2

Downwind drift speed of the physical ripple crests: 0 = a frozen pattern, higher = ripples visibly crawl downwind between frames -- the living-surface detail that sells animation close-ups. Keep moderate for stills (drift does not matter) and for motion-blur renders (very fast crawl can strobe)

Ripple Crest Smoothing

`ripple_solver_smooth` - Float - default **0.3** - 0 to 1

Across-wind straightening of the physical ripple crests: high = long continuous parallel crests (steady-wind look), low = short broken wavy segments with many Y-junctions (gusty look). Purely a style dial on the simulated ripple field

Crest Sharpness

`ripple_solver_sharp` - Float - default **0** - 0 to 1

Cusp the physical ripple crests: 0 = the raw smooth sine-like profile, higher = narrow SHARP crest lines standing on wide gently rounded troughs -- the classic raking-light sand-ripple look. Render-only shaping of the simulated field: coordinates and wavelength never change, so no whorl artifacts. Try 0.5-0.8

Patchiness

`ripple_solver_variety` - Float - default **0** - 0 to 1

Break the too-even ripple carpet into natural patches: a static low-frequency field makes some areas ripple strongly while others stay calm, like real sand sheets. Scales AMPLITUDE only (never the wavelength). 0 = uniform carpet, 0.5-0.8 = clearly patchy. Deterministic per Seed

Ripples On Dunes

`ripple_solver_expose` - Float - default **0.7** - 0 to 1

How strongly the physical ripples ride the ACTIVE windward sand and fade in the lee shadow / bare interdunes -- so they WRAP the individual dunes instead of blanketing the whole terrain in one uniform set of transverse lines. 0 = ripples everywhere (uniform); 0.7 = strongly follow the dune faces (recommended)

Ripples Follow Dunes

`ripple_follow` - Float - default **0.5** - 0 to 1

Option B: bend the ripple crests so they wrap AROUND the dune flanks (the wind deflects around dunes, so real ripples curve with the contours). 0 = straight global-wind rows; 1 = strongly bent. Only warps coordinates, never the wavelength, so no fingerprint whorls

Ripple Sand Cutoff

`ripple_sand_cutoff` - Float - default **0.05** - 0 to 2

Sand depth (solver units) below which a cell counts as BARE and carries no ripples at all. This is the single biggest reason ripples disappear from parts of a terrain: measured, it zeroes 8.3% of a sandy field and 73.2% of a rock-based one, and it accounts for essentially all of the dead area. Lower it to let ripples run over thin sand, raise it to keep them on the deep drifts only

Ripple Sand Fade

`ripple_sand_soft` - Float - default **0** - 0 to 2

Width (solver units) of the fade above the cutoff. 0 = a hard STEP, which is what shipped and is bit-identical -- but it draws a visible cut edge where the sand thins. Set it to roughly the cutoff value, or a few times it, and the ripples fade out as the sand runs out instead of stopping on a line

Fine Sand Depth

`micro_detail_depth` - Float - default **0.3** - 0.01 to 10

Thickness of the fresh fine-sand layer laid over the frozen big dunes for the Micro-Detail pass. Thicker = larger/taller small dunes on top (in solver height units)

Fine Dune Size (Ls)

`micro_detail_ls` - Float - default **1.5** - 0.5 to 20

Saturation length for the Micro-Detail pass: the SIZE of the small dunes grown on the frozen base. Keep it well below the main Dune Size (Ls) so the two scales read as a hierarchy

Micro Steps

`micro_detail_steps` - Int - default **60** - 1 to 500

How many solver steps to grow the fine micro-dune layer

Slump Allowance

`micro_detail_relax` - Float - default **0** - 0 to 20

How deep BELOW the old surface the pass freezes the rock, as a multiple of Fine Sand Depth. Freezing exactly AT the surface is what made a single pass add facets: the avalanche may only move sand that lies above the rock, so under a thin veneer a face steeper than the repose angle has nothing it is ALLOWED to move and stays rigid at whatever angle it had -- the surface turns into a polyhedron of frozen faces. Giving it a little depth to work with lets those faces slump to repose while the large shapes stay held. MEASURED on a real 1024 scene through the operator itself, at three different points in its evolution (share of the sloping surface that is a flat face): 4.43 -> 4.18, 6.26 -> 6.06, 8.82 -> 8.53 -- a small, consistent improvement, with the relief unchanged. It is NOT a cure for faceting in general: on that scene the pass itself did not raise the flat-face share at all. 0 = the shipped behaviour, bit for bit

Micro Octaves

`micro_detail_octaves` - Int - default **1** - 1 to 4

How many times to repeat the pass, each at a smaller dune size than the last and each freezing the one before it. 1 = the single pass this has always been. This is the ONLY way the solver can build a hierarchy: running extra transport at a finer scale inside one simulation does the opposite -- small dunes are unstable and MERGE, so the field just coarsens (measured). Separating the scales in time, with the previous octave held as rock, is what makes small-on-big stable. Each octave costs another Steps run

Octave Falloff

`micro_detail_falloff` - Float - default **0.55** - 0.2 to 0.95

How much smaller each octave is than the one before -- it scales both the dune size and the sand laid down for it, so a finer octave is genuinely a thinner, finer layer rather than the same amount of sand in smaller shapes. 0.55 gives roughly halving. Below about 2 grid CELLS the transport is at grid scale and makes spikes, not detail, so an octave that would go under that is clamped and the operator says so

Octave Wind Shift

`micro_detail_wind_shift` - Float - default **0** - -90 to 90

Degrees to turn the wind for each successive octave. 0 = every octave uses the same wind, so the small forms are a scaled copy of the big ones lying in exactly the same direction -- which is the main thing that makes a micro-detail pass read as a texture instead of as bedforms. Real small dunes are built by a wind that is not quite the prevailing one; 10-30 degrees is plenty

Seed Roughness

`micro_detail_seed_noise` - Float - default **0.55** - 0 to 3

How rough the fresh layer is when it is LAID DOWN, as a fraction of its own depth. This is what makes the pass detail rather than smooth. The layer used to go on perfectly flat, and dune growth is an AMPLIFIER -- with nothing to amplify, a short run at a small saturation length can only even the layer out, which measured as a LOSS of fine relief at every setting tried. The perturbation is correlated over a few cells rather than being per-cell white noise: one-cell noise is below the scale the transport can carry and is erased before it can grow. 0 = the old, flat layer

Crispness

`micro_detail_sharpen` - Float - default **0** - 0 to 3

Sharpen the fine layer after the pass -- an unsharp mask on the THICKNESS, so it crisps ripple crests and hollows alike without touching the dune underneath. This is the dial to reach for when the detail reads as NOISE rather than as bedforms: raise Seed Roughness and the layer starts from more relief, raise this and what the transport made of that relief gets harder edges. 0 = the pass exactly as before

Crests

`micro_detail_crest` - Float - default **0** - 0 to 3

Build up the RIDGES only, leaving the troughs where they are. Where Crispness cuts both ways -- peaks up and hollows down -- this lifts only what already stands above its surroundings, which is what makes a ripple field read as crested rather than as a rough surface. It ADDS material, so the mean thickness rises; Crispness does not. 0 = off

Limit Spikes

`micro_detail_normalize` - Bool - default **on

Cap the micro layer so the fine pass cannot pile a runaway spike on an already-pointy crest (it otherwise keeps feeding the peak). Clamps the added height to Spike Cap x Fine Sand Depth

Spike Cap

`micro_detail_clamp` - Float - default **2.5** - 0.5 to 10

Max micro-layer height as a multiple of Fine Sand Depth (with Limit Spikes on)

Pre-Init Detail

`micro_detail_preinit_mode` - Enum - default **KEEP

What happens to pre-Initialize detail when the simulation is reset. Keep is the one that lets you detail a raw heightmap and then run on it

Option	Meaning
Keep (survives Reset)	The detailed surface becomes the new START terrain. Reset comes back HERE, and a later Initialize reads it as the ground it starts from. Start From is set to Mesh so the main Initialize cannot regenerate the terrain and throw the detail away
Preview (Reset discards it)	Run the pass and leave the solver alive so you can look at it. Nothing is committed, so Reset puts the smooth terrain back. For trying settings before deciding which one to keep

Sand Haze

Airborne sand: the sheet of material that lifts off the crests and drifts downwind. It is what tells the audience there is wind, in a still frame.

When you need it: Any shot where the wind is supposed to be felt rather than deduced.

Sand Haze

`haze_enabled` - Bool - default **off

Blowing-sand HAZE: a lightweight renderable mesh of wind-borne streaks generated from where the solver actually moves material -- the low drifting veil that sells wind in stills and animation. Motion-blur friendly; tune with the P* settings below and cap cost with Max Grains. Works for snow spindrift too

Haze Object

`haze_object` - Pointer

Generated DunesForge haze mesh

PSpeed

`p_speed` - Float - default **0.6** - 0 to 5

How fast the haze grains race downwind -- also stretches their streak triangles longer (speed = streak length). Low = lazy floating dust; high = driving sandstorm streaks

PHop

`p_hop` - Float - default **3** - 0 to 30

Random vertical skipping of the haze grains (saltation hops): 0 = grains glide in flat sheets, higher = they bounce and jump, giving a livelier, more turbulent veil

PLift

`p_lift` - Float - default **1** - 0 to 30

Base hover height of the haze above the terrain (solver height units). Real saltation stays within ~1-2 units of the surface -- keep it low for a ground-hugging sheet, raise it for an airborne dust cloud look

PGrain

`p_grain` - Float - default **0.5** - 0 to 10

Size and positional jitter of the individual haze grains/streaks: small = fine sharp grains for close-ups, large = soft blurry puffs that read at distance. Purely the look of the haze mesh

PDensity

`p_density` - Float - default **0** - 0 to 1

Where the haze appears: 0 = an even sparse veil across the whole terrain, 1 = grains concentrate strongly where the solver is actually moving sand (active crests smoke, calm interdunes stay clear) -- the physically honest and usually

prettier setting

Haze Update Every

`p_every` - Int - default **1** - 1 to 16

Advance the haze pool only every Nth frame, and move it further each time so the grains keep the same speed. PROFILED at 256x256: the pool costs 6.9 ms per frame at 5000 grains, 13.9 at 20000 and 26.4 at 50000, and 96% of that is the physics update -- building the render geometry is under 3 ms even at 50000. So this is the lever that matters, not the material or the transparency. What you lose is the smoothness of each path between updates, which on a haze is invisible at 2-3 and starts to strobe past 6

Max Grains

`haze_max_grains` - Int - default **3000** - 0 to 200000

Hard cap on the haze particle pool (each grain = one streak triangle). The main cost/quality dial of the haze: a few thousand is plenty for mid shots; tens of thousands only for dense storm close-ups (heavier viewport and renders)

Haze Render

`haze_render_mode` - Enum - default **POINTS

How the sand-haze pool is turned into renderable geometry. Both modes keep a CONSTANT point count between frames, which is what Cycles deformation motion blur needs, and both collapse dead grains to nothing. Particles is the flexible one: the node group is a normal modifier you can open and extend

Option	Meaning
Particles	Draw each grain as a real particle: the carrier becomes a POINT CLOUD and Geometry Nodes instances a stretched sphere (or your own object) on every point. Editable downstream like any other Geometry Nodes setup
Streak Triangles	The original layout: one flat triangle per grain, rendered directly. Cheapest, but up close the grains read as triangles

Particle Size

`haze_point_size` - Float - default **1** - 0.05 to 20

Multiplier on the grain radius the solver computes (Particles mode only). 1.0 matches the width of the old streak triangles; raise it for a thicker, dustier veil, lower it for fine sharp grains in a close-up

Particle Stretch

`haze_point_stretch` - Float - default **1** - 0 to 6

How far a grain is stretched along its own motion (Particles mode only). 0 = every grain is a round speck; 1 = it is as long as the streak the old triangle drew; higher exaggerates the smear for a driving sandstorm. A stopped grain stays a sphere either way

Particle Detail

`haze_point_detail` - Int - default **1** - 0 to 4

Ico-sphere subdivisions for the instanced grain (Particles mode only). 1 is right for grains a few pixels across; raise it only for extreme close-ups, since the cost is multiplied by Max Grains

Custom Particle

`haze_instance_object` - Pointer

Use this object as the grain instead of the built-in sphere (Particles mode only) -- a leaf, a snowflake card, a pebble. It is instanced, not copied, so thousands cost little; it is still aimed along the grain motion and scaled by the grain size. Leave empty for the default sphere

Freeze Terrain (haze flies)

`haze_freeze_terrain` - Bool - default **off

Hero shot: hold the dunes perfectly static while the sand haze keeps blowing. During playback the solver stops advancing the terrain, but the wind still turns and the particle haze keeps advecting and settling -- static dunes with live blowing sand

---

Dust Devils

Dust devils -- travelling vortices that pick material up, carry it and drop it. They can read the ground under them, so a devil walking onto bare rock weakens, and one over fresh snow picks up snow.

When you need it: Desert and steppe shots that need something alive in the middle distance.

Dust Devils

`dust_enabled` - Bool - default **off

Wandering swirling vortices of lifted sand/snow (dust devils). Each is a rising helix of particles; they drift with the wind, are deflected around high terrain (following the relief) and wander, with the column riding the surface. Renderable, motion-blur-ready mesh. Switching it OFF clears that mesh and hides the object, so the devils really leave the scene instead of freezing in mid-air

Render As

`dust_render` - Enum - default **POINTS

How the dust devils are built. POINTS writes a point cloud carrying dust_size / dust_life / dust_strength / dust_height / dust_skirt / dust_age / dust_index, and the DunesForge_DustDevils modifier instances DunesForge_DustMote on it -- point Object Info at your own object to change the look without touching the simulation. MESH is the older triangle build, kept so existing scenes can go back to exactly the previous look

Option	Meaning
Points (Geometry Nodes)	A point cloud with per-particle attributes, with a Geometry Nodes modifier instancing a swappable object on every point
Triangles (legacy)	The fixed-topology triangle mesh used before 1.3.96

Dust Devils Object

`dust_object` - Pointer

Generated DunesForge dust-devil particle mesh

Count

`dust_count` - Int - default **4** - 0 to 200

How many dust devils wander the terrain at once

Particles / Devil

`dust_density` - Int - default **60** - 3 to 2000

Particle count in each swirling column (denser = thicker vortex)

Size / Radius

`dust_size` - Float - default **3** - 0.3 to 60

Vortex radius in solver cells (bigger = wider devils)

Column Height

`dust_height` - Float - default **6** - 0.5 to 200

How tall the swirling dust column rises (solver height units)

Grain Size

`dust_grain` - Float - default **0.12** - 0.01 to 2

Rendered particle size in the dust column. In POINTS mode this is a COVERAGE multiplier: the base size is derived from the column itself (its radius divided by the cube root of the particle count), so the particles overlap into continuous dust. The 0.12 default maps to 1.0 -- raise it for denser, more solid columns, lower it for a thinner, grainier haze. In the legacy MESH mode it is the triangle size directly

Dust Density

`dust_density_gain` - Float - default **0.25** - 0.02 to 2

How much light ONE particle takes out. Kept low on purpose: the particles overlap, so a column of k layers ends up at 1-(1-a)^k -- the sense of a thick column comes from the NUMBER of layers, not from any single one being opaque. Push it up for a heavy sandstorm, down for a thin haze

Edge Softness

`dust_softness` - Float - default **2.5** - 1 to 6

How fast a particle thins out towards its rim. 1.0 is the exact profile of a UNIFORM sphere and still shows an edge; higher values describe a puff whose density falls off outward, which is what real dust does -- it has no edge at all. Raising this also thins the column, so Dust Density may want to follow

Forward Scatter

`dust_forward_scatter` - Float - default **0.72** - 0 to 1

How much light goes THROUGH the grain rather than bouncing back. This is the reason dust glows when you look into the sun and goes flat and grey when the sun is behind you -- grains this big scatter mostly forward. 0 makes it a dull powder, 1 makes it almost pure backlight

Shape Irregularity

`dust_shape_noise` - Float - default **0.55** - 0 to 1

Breaks the particles out of being spheres. Noise is subtracted from the density profile, so the rim moves in and out instead of staying a circle. 0 gives perfectly round puffs; high values give torn, wispy shreds

Shape Detail

`dust_shape_scale` - Float - default **3.2** - 0.5 to 20

Size of the features in Shape Irregularity. Low values give a few big lobes per particle, high values give a fine, frayed edge. Each particle gets its own offset into the noise, so they do not repeat

Variation

`dust_color_var` - Float - default **0.35** - 0 to 1

How much particles differ from each other in tint and density. Half of the variation comes from the individual grain, half from the vortex it belongs to, so whole columns read differently AND no two grains match. 0 makes every particle identical

Base Colour

`dust_color_base` - FloatVector - default **0.58, 0.47, 0.35** - 0 to 1

Colour at the foot of the column, where the layer is thickest and the grains are coarsest. Darker and warmer than the tip

Tip Colour

`dust_color_tip` - FloatVector - default **0.93, 0.87, 0.78** - 0 to 1

Colour at the top of the column. Airborne grains scatter many times and are seen from every side, so they read much lighter than the same sand lying on the ground

Skirt Colour

`dust_color_skirt` - FloatVector - default **0.44, 0.34, 0.25** - 0 to 1

Colour of the ground-hugging debris skirt around the base. This is coarser material than the column carries, so it is darker

Drift Speed

`dust_speed` - Float - default **0.5** - 0 to 5

How fast the devils are carried by the wind

Wander

`dust_wander` - Float - default **0.4** - 0 to 3

Random meander of the vortex path (0 = straight downwind, higher = erratic)

Terrain Deflect

`dust_deflect` - Float - default **1** - 0 to 8

How strongly the devils steer AWAY from uphill (follow the relief around dunes)

Swirl Speed

`dust_spin` - Float - default **0.6** - 0 to 4

How fast the dust column turns. The core spins faster than the rim (angular speed ~1/r), so the column shears into a spiral instead of rotating as a rigid body. 0 = no swirl and the devils read as drifting smudges. Until 1.3.76 this was pinned at 0.6 with no way to change it

Needs Loose Sand

`dust_ground_sand` - Bool - default **off

A dust devil lives off LOOSE material, so fade the vortex out where the wind has scraped the bed down to rock. Strength follows sand depth (surface minus bedrock) up to Loose Sand Depth: a devil that wanders onto bare stone shrinks and dies instead of spinning on rock. Needs a bedrock -- with none set the whole column counts as sand and nothing is damped

Loose Sand Depth

`dust_sand_ref` - Float - default **0.15** - 0.001 to 5

Sand depth at which a devil reaches FULL strength. Below it the vortex shrinks in proportion and at zero sand it disappears entirely. Raise it to make devils pickier about deep sand

Needs Moving Sand

`dust_ground_tau` - Bool - default **off

Raise dust only where the wind is actually above the threshold of motion: strength follows shear stress over the transport threshold. In sheltered pockets and inside lee bubbles there is nothing to lift, so the devil fades out there

Damped by Wet and Plants

`dust_ground_moist` - Bool - default **off

Wet sand does not fly and grass holds the surface down, so fade the vortex out over moisture and vegetation. Each field only counts once its own feature is enabled -- with neither on, nothing is damped

Needs Fresh Snow

`dust_ground_snow` - Bool - default **off

Snow variant: drive the vortex from FRESH (unpacked) snow depth instead of sand, so it blows where new snow lies and dies over wind-packed crust. Needs snow layers enabled

Respawn On Loose Ground

`dust_respawn_weighted` - Bool - default **on

When a vortex reaches the end of its life it is reborn somewhere else. With this on it is reborn where there is something to lift: several candidate spots are drawn and the one with the most loose material wins. Without it a devil on a half-rocky scene has an even chance of being born on bare stone, where it is invisible, and the scene reads as having fewer devils than you asked for. Has NO effect unless one of the ground couplings above is enabled

Fresh Snow Depth

`dust_snow_ref` - Float - default **0.1** - 0.001 to 5

Fresh-snow depth at which a snow devil reaches FULL strength. Below it the column shrinks in proportion; over bare crust it disappears

Bake Frames

`dust_bake_frames` - Int - default **100** - 1 to 5000

How many frames of dust-devil motion "Bake Dust Sequence" simulates from the CURRENT frame and stores as per-frame shape keys on the dust object. The baked animation plays and renders standalone (with motion blur), no live simulation needed. Memory ~= frames x particles; keep Count x Particles moderate for long bakes

---

Crest Backlight

A thin ribbon of light along the dune crest, for backlit shots.

When you need it: Sunrise and sunset framing, where the crest line is the composition.

Crest Backlight

`crest_glow` - Float - default **0** - 0 to 1

Make dune CRESTS light up when the sun is BEHIND them, the way a backlit sand ridge does. A thin sheet is built standing ON the crest line and given a forward-scattering shader, so sunlight arrives on its far face and is carried straight through to the camera. The effect is strong against the sun and nearly absent with the sun behind you, because that is what transmission does. Only SAND gets one: slopes steeper than the angle of repose are rock or a stamped object, carry no veil of airborne grains, and are skipped together with their immediate surroundings. At 0 no sheet and no material are built at all

Crest Sheet Height

`crest_glow_height` - Float - default **0.4** - 0.001 to 20

How far the glowing sheet rises above the crest, in SIMULATION METRES. What glows on a real backlit crest is the veil of airborne grains streaming off it, not the sand itself: dry quartz passes light for a millimetre or two, and a millimetre is a hundredth of a pixel on a 200 m dune field. Measured on the demo scene at 0.15 m per pixel, so 0.40 m reads as about three pixels of rim

Crest Rise

`crest_glow_rise` - Float - default **0.1** - 0 to 50

How far a cell must stand above its own neighbourhood before it counts as a crest, in SIMULATION METRES. A LENGTH and not a percentage on purpose: the generated curvature channel is normalised by the field's own 98th percentile, so it is a RANK and cannot judge size. Measured on the demo scene, a smooth 1.8 cm noise field puts MORE terrain over "curvature > 0.55" (33.0 %) than real dunes do (31.3 %). Raise this on a big dune field, lower it on a small one; 0 marks every local high. THIS IS NOT THE HEIGHT OF THE DUNE: it is the rise above the mean of the Curvature Radius neighbourhood, so a broad 0.3 m ridge 3 m wide stands only 0.09 m above its own surroundings

Crest Backlight Tint

`crest_glow_tint` - FloatVector - default **1.0, 0.72, 0.38** - 0 to 1

Colour of the light carried through the crest sheet. Sand lit from behind reads far warmer than sand lit from the front, because the short blue end is scattered out of the path first

Crest Backlight Object

`crest_glow_object` - Pointer

Generated DunesForge crest-backlight sheet

Multi-Fraction (Megaripples / Pavement)

Grains of more than one size, and what happens when the wind sorts them. Fine grains blow away, coarse ones stay behind and armour the surface -- that is desert pavement, and megaripples are the same mechanism at a larger scale.

When you need it: When the ground needs to look like it has a history rather than a texture.

Transport Two Grain Sizes

`bifraction_enabled` - Bool - default **off

Carry the coarse grains as their OWN layer and blow them with their OWN physics, instead of book-keeping a coarse fraction on top of one sand. This is the real thing and it is not the same switch as Multi-Fraction Grain below: that one derives a coarse RATIO from how much the bed moved and feeds it back as an erodibility multiplier, so nothing is ever transported separately and nothing can SORT. This gives the coarse layer a thickness, a higher entrainment threshold and a longer saturation length, then runs the whole transport a second time for it. MEASURED: the coarse share then RISES wherever the bed deflates and falls where it builds -- correlation +0.85 between erosion and coarse enrichment -- which is a desert pavement forming by itself rather than by a coefficient. With both grain sizes set equal the same correlation is -0.03, i.e. the split alone sorts nothing. COSTS a second transport pass: measured +41% of a sand step at 1024 and +64% at 256

Coarse Share at Start

`bifraction_initial` - Float - default **0.2** - 0 to 1

How much of the starting bed is COARSE, by thickness. 0 leaves the second fraction empty, and the run is then bit-identical to the single-grain solver (measured: exactly 0.000e+00 difference) -- which is the honest way to switch the feature off without switching the code path off. Real desert sands run roughly 0.05-0.3 coarse before any winnowing has happened

Coarse Grain Size

`grain_diameter_coarse_um` - Float - default **700** - 100 to 2000

Diameter of the COARSE mode, in microns, against the fine mode set by Grain Size. THE WHOLE PHYSICS OF THE TWO-FRACTION MODEL IS THIS ONE NUMBER: it sets the coarse threshold and saturation length through the same expressions the fine grain uses, so the pavement, the coarse ripple crests and the sorting all follow from a grain diameter rather than from a tuning knob. 700 um is granule sand, the classic megaripple armour; equal to Grain Size means the two fractions are the same material and nothing sorts

Coarse Rolls Further

`bifraction_runout` - Float - default **0.35** - 0 to 1

How much further a COARSE grain runs than a fine one once a slip face lets go. This is why a real dune has a band of grit along its TOE and clean sand up the face: an avalanche does not just move sand down, it SORTS it on the way. 0 makes the avalanche blind to grain size -- sand slumps and both fractions keep the proportion the column already had, which is what 1.3.18 did and is bit-identical to it. Every gram taken out of a cell is handed to a lower neighbour, so this moves grains around without creating or destroying any. Needs Transport Two Grain Sizes

Coarse Share Blowing In

`bifraction_inflow` - Float - default **0.2** - 0 to 1

Coarse share of the sand arriving at an OPEN upwind boundary. Only does anything with a non-periodic domain and Inflow Flux above zero -- a periodic domain has no upwind edge, so this is an exact no-op there. Set it below Coarse Share at Start to feed the scene clean fine sand and let a lag build

Multi-Fraction Grain

`multifraction_enabled` - Bool - default **off

EXPERIMENTAL, opt-in. Model TWO grain sizes instead of one: a fine mobile fraction that builds the dunes, and a coarse fraction that lags behind. Where the wind strips the fines away, the coarse grains concentrate into a LAG that armours the bed -- giving deflation pavement (reg / desert pavement: flat, erosion-proof gravelly surfaces) and megaripple-scale roughness (with Physical Ripples). Works for sand AND wind-blown snow (coarse = old wind-packed grains). Pure surface bookkeeping -- OFF = the heightfield is unchanged

Coarse Fraction

`coarse_initial` - Float - default **0.15** - 0 to 1

Starting fraction of coarse (hard-to-move) grains mixed through the sand/snow (0 = all fine, 1 = all coarse). Higher = pavement forms sooner and over more of the terrain. Only active with Multi-Fraction Grain ON

Winnowing

`coarse_winnow` - Float - default **0.6** - 0 to 1

How selectively erosion removes the FINES and leaves the coarse grains behind. Higher = the coarse lag concentrates faster as a patch deflates, so pavement builds up more quickly (0 = both fractions blow away together, no lag). Only active with Multi-Fraction Grain ON

Lag Armouring

`coarse_armour` - Float - default **0.8** - 0 to 1

How strongly the coarse lag PROTECTS the bed from further erosion (1 = a fully coarse surface is erosion-proof pavement; 0 = the lag is cosmetic and does not slow erosion). This is what freezes deflation flats while the fine sand keeps drifting around them. Only active with Multi-Fraction Grain ON

Coarse in Drift

`coarse_deposit` - Float - default **0** - 0 to 1

Coarse fraction carried in freshly DEPOSITED sand (0 = drifts are pure fines, so deposition strongly dilutes the surface back to fine -> maximum contrast between mobile dunes and coarse pavement). Raise a little if you want some coarse grains to travel with the dunes. Only active with Multi-Fraction Grain ON

Megaripple Height

`megaripple_amp` - Float - default **0** - 0 to 2

RENDER-only: extra Physical-Ripple height on the coarse/lag surfaces (needs Multi-Fraction Grain AND Physical Ripples both ON). Coarse-armoured patches carry taller, longer ripples -- real megaripples -- while the fine dune faces keep their small ripples. 0 = off. Raise the Physical Ripple Wavelength too for a clear megaripple scale

Fast Ripple Solver (wavelength field)

`ripple_etd_field` - Bool - default **off

Integrate the ripple equation with IMEX-ETDRK when the wavelength varies across the grid (megaripples, grain coupling). Without it the substep count is set by the COARSEST cell and grows as wavelength squared, so one patch of long ripples slows the whole grid down -- measured 87 substeps where this needs 2, and 4.4x faster overall. It is a DIFFERENT integrator, so the result changes: measured on the same scene the wavelength and amplitude match, and there is LESS high-frequency noise, but it is not bit-identical. With a uniform wavelength nothing changes at all

Megaripples in the Ripple Solver (SLOW)

`ripple_solver_megawave` - Bool - default **off

Give the PHYSICAL ripple solver a per-cell wavelength, so the coarse-armoured ground grows genuinely LONGER waves instead of taller short ones. Everything else only scales the amplitude, and a megaripple is defined by its wavelength, so this is the only control that makes one in the solved ripples. MEASURED WORKING: a field split fine/coarse grew 8.35 and 21.33 cells side by side, stayed finite, and put 0.8% of its energy at the grid scale (a failed one puts ~99% there and looks granular). STILL OFF BY DEFAULT, AND HERE IS THE HONEST COST. It began at 241x slower per ripple tick, because a wavelength field could not use the fused CUDA kernels at all. A per-cell twin of the kernel now exists (`ripple_step_bfield`, gated to compute EXACTLY what the reference does -- measured 0.000e+00 difference), which brought it to roughly 16x at 256 and 58x at 512. What is left is not fixable by a kernel: the substep count grows as the LONGEST wavelength SQUARED and substeps are global, so one coarse cell slows the whole grid. The procedural detail layer already does megaripples for free; use this when you need them in the SOLVED ripples

---

Field Maps (Load / Save)

Load and save the solver's own fields as images: heights, masks, sand sources, wind gain.

When you need it: Round-tripping with other tools, or reusing a hand-painted mask across shots.

Map Field

`map_field` - Enum - default **ERODIBILITY

Which solver field a loaded image map drives

Option	Meaning
Sand Height	Load as the sand surface
Bedrock	Load as the non-erodible floor
Erodibility	Load 0..1 erosion mask (dark = protected)
Non-Erodible Rock	Load a 0..1 bare-rock mask: bright (>0.5) never erodes, but can still be buried by drifting sand
Water / Wetness	Load a 0..1 water mask: bright = wet/submerged (locked from wind, ripples fade)
Wind Gain	Load a local wind-strength multiplier
Sand Source	Load where sand blows in each step
Vegetation	Load a vegetation-cover map
Snow Depth	Load a starting SNOW map: the image becomes the depth of fresh powder in metres, exactly as Sand Height loads a sand surface. Map Amplitude is the depth the brightest pixel means. Arms the Snow model
Snow Crust	Load a 0..1 snow HARDNESS map: bright = wind slab or glaze ice, dark = loose powder. Separate from the depth, so an imported sastrugi field can carry its own crust
Avalanche Trigger	Load a 0..1 map of where slopes let go EARLY. Bright cells drop their repose angle by Avalanche Trigger Drop; it lowers a threshold and adds no sand
Slab Weakness	Load a 0..1 map of the weak layer in the snow. Bright cells release their slab at a lower angle, so an imported fracture line is where the slab breaks

Map Image

`map_path` - String - default **//

Image file (PNG/EXR/JPG/PGM/TXT) loaded by the Load Map button: its luminance is stretched onto the whole solver grid and drives the field chosen in Map Field (erodibility, source, water...). Scaled by Map Amplitude and Map Offset

Map Amplitude

`map_amplitude` - Float - default **1** - -1000 to 1000

Multiplier applied to the loaded map before it is written into the solver field. Mask fields expect a 0..1 result (1 = neutral scale); for Sand Height / Bedrock the value is in solver height units, so it sets the terrain relief of the map. Negative values invert the map

Map Offset

`map_offset` - Float - default **0** - -1000 to 1000

Constant added to the loaded map AFTER Map Amplitude: shifts the whole field up/down (heights) or biases a mask toward 0 or 1. E.g. Amplitude -1 with Offset 1 = a fully inverted 0..1 mask

Mask Export Dir

`mask_export_dir` - String - default **//tydunes_masks

Folder where Export All Masks writes every simulated field -- sand depth, moisture, frost, hardness, vegetation, wind flow, snow layers, grain size and the rest -- one image per mask plus a JSON listing each map's range. // means next to the .blend file

Save Resolution

`map_save_res` - Enum - default **SOLVER

Resolution the maps are resampled to when saving

Option	Meaning
Solver Res	Save at the current solver resolution (exact sim detail)
Terrain Res	Save at the terrain mesh resolution
Custom	Save at an explicit resolution (e.g. paint low, bake high)

Custom Res

`map_custom_res` - Int - default **1024** - 16 to 8192

Square pixel resolution used when Save Resolution is set to Custom -- e.g. paint masks on a low solver grid and export them at 4096 for texturing, or downsample a heavy sim for a light proxy map

---

Export / Bake

Getting results out: meshes, caches, and baked maps for your shading pipeline.

When you need it: Handoff. Also when the simulation is finished and you want the cost to stop.

Checkpoint

`checkpoint_path` - String - default **//tydunes_checkpoint.npz

File for Save/Load Checkpoint: the EXACT solver state (heights, masks, wind clock, random state) in one .npz. Loading resumes the run bit-for-bit -- save before risky experiments so you can always return to a known good dune field

Heightmap

`heightmap_path` - String - default **//tydunes_heightmap.txt

File path used by the Import/Export Heightmap buttons (TXT or PGM heightfield). Export saves the current solver heights for external tools; Import replaces the heights from the file (scaled by Import Amp/Offset). // means next to the .blend

Mask Export Dir

`mask_export_dir` - String - default **//tydunes_masks

Folder where Export All Masks writes every simulated field -- sand depth, moisture, frost, hardness, vegetation, wind flow, snow layers, grain size and the rest -- one image per mask plus a JSON listing each map's range. // means next to the .blend file

Mask Format

`mask_export_format` - Enum - default **EXR

File format for Export All Masks

Option	Meaning
EXR (32-bit float)	Real values, signed and unbounded. This is the one to use if you are building your own material: height stays in metres, temperature keeps its sign and flow keeps its direction. A 16-bit PNG cannot hold any of that without rescaling
PNG (16-bit)	Smaller and openable anywhere, but the format is 0..1 only -- so anything outside that range has to be normalised, and the scale lives in the exported JSON instead of in the file

Raw Values

`mask_export_raw` - Bool - default **on

Write the numbers the solver actually holds instead of stretching each map to 0..1. NORMALISING THROWS THE SCALE AWAY: moisture and height both come back as 0..1 and nothing in the file says whether 0.5 means half-wet or six metres. Either way the exported tydunes_masks.json carries every map's min, max and mean, so a normalised map can still be rescaled -- value * (max - min) + min. Turn this OFF only when you want maps to LOOK at rather than compute with (PNG cannot hold negatives at all)

Import Amp

`heightmap_import_amp` - Float - default **1** - -1000 to 1000

Multiplier applied to the imported heightmap values (Import Heightmap button): imported_height * Amp + Offset, in solver units. Use it to match the scale of an external map to the sim

Import Offset

`heightmap_import_offset` - Float - default **0** - -1000 to 1000

Constant added to the imported heightmap after Amp (solver units) -- raises/lowers the whole imported terrain

Displacement Map Size

`displace_bake_res` - Int - default **2048** - 256 to 8192

Pixel size of the height map baked by Bake To Displacement. This is what sets the RENDER detail, and it is independent of Solver Resolution -- a 4096 map on a 512 sim gives finer render geometry than a 2048 mesh while the scene stays light. Bigger costs only disk and render subdivision, not simulation speed

Map Format

`displace_format` - Enum - default **EXR

File format for the baked displacement height map

Option	Meaning
OpenEXR (float, exact)	Full-float heights: the displaced surface matches the simulated one exactly, no quantisation. Larger files. Recommended
PNG 16-bit	Heights normalised into 16-bit greys, with the range written into the material. Smaller and readable by any app, but quantised into 65536 steps -- on a very flat field that can show as faint terracing

Render Subdivisions

`displace_subdiv` - Int - default **6** - 1 to 10

Subdivision level of the light render plane when ADAPTIVE subdivision is not available (EEVEE, or Cycles without the Experimental feature set). With Cycles Experimental the operator switches the modifier to adaptive instead and this is ignored -- adaptive dices to the pixel size, which is what makes the map resolution the only detail limit

Hide Sim Terrain In Render

`displace_hide_source` - Bool - default **on

Hide the heavy simulation mesh from renders once the light displacement object exists, so the two do not overlap. The sim terrain stays visible in the viewport and keeps simulating

Displacement Wears Terrain Shader

`displace_with_shader` - Bool - default **off

Give the displacement object a COPY of the material the terrain wears, with every mask the shader reads from mesh attributes baked to an EXR texture (displace_masks/ next to the height map) and the Attribute nodes swapped for Image Texture nodes -- inside a private copy of the node group, so the terrain material is not touched. The look then matches the simulated terrain at the moment of the bake. Masks are read straight from the terrain mesh, so write them first (Auto Masks or Write Material Masks). One frame: for an animated look use the Bake Viewer with Viewer Wears Terrain Shader. Off keeps the plain sand-coloured displacement material, exactly as before

Overlay

`diagnostic_overlay` - Enum - default **HEIGHT

Solver array used for the diagnostic color attribute

Option	Meaning
Height	Current solver height
Shear	Computed shear stress
Flux	Transport flux
Shadow	Lee shadow / bubble mask
Erodibility	Effective erodibility mask
Non-Erodible Rock	Painted/loaded bare-rock mask
Placement	Initial placement / hold mask
Wind Gain	Local wind-gain multiplier
Vegetation	Vegetation cover
Hardness	Snow hardness
Fresh Snow	Fresh snow depth
Coarse Grain / Lag	Multi-fraction coarse-grain lag (pavement) fraction
Moisture / Wetness	Water-table wetness (1 = submerged, 0 = dry)

Color Attribute

`diagnostic_color_attr` - String - default **DunesForge_Overlay

Name of the vertex color attribute the diagnostic Overlay writes onto the terrain mesh -- select it in the viewport shading / your material to see the chosen solver field as color. Change it to keep several overlays side by side

Material Masks

`material_mask_mode` - Enum - default **ALL

Material mask attribute set written to the terrain mesh

Option	Meaning
RGB Combo	Write color = exposure/sand/slipface preview
Named Channels	Write dirt, sand and flowMask grayscale attributes
Snow Channels	Write snow tint, hardness and fresh-powder channels
All Masks	Write every material/debug channel

Preview Attr

`material_color_attr` - String - default **color

Vertex color attribute that receives the RGB material preview (exposure / sand / slip-face tint). Point your shader or the viewport color at this attribute; rename to avoid clashing with existing color layers on the terrain

Auto Masks

`auto_material_masks` - Bool - default **off

Keep the material mask attributes (dirt/sand/flow, snow channels...) refreshed automatically every Step and during playback, so shaders driven by them stay live. Costs extra mesh writes per frame at high resolution -- throttle with ColorEvery or leave OFF and refresh manually before renders

Track Sand Age

`sand_age_enabled` - Bool - default **off

Track how long each patch of sand has sat undisturbed (grows while it rests/accretes, resets where the wind cuts it away) and write it to a "sandAge" colour attribute -- drive a fresh-vs-weathered shader from it (fresh sand light/dry, old sand darker; valleys read as damp)

Sand Age Rate

`sand_age_rate` - Float - default **0.02** - 0 to 1

How fast resting sand matures toward fully "old" in the sandAge attribute (higher = the fresh-vs-weathered contrast develops in fewer steps). Tune it to your shot length so active crests stay visibly fresh while calm interdunes darken. Needs Track Sand Age ON

Strata / Cross-Bedding

`strata_enabled` - Bool - default **off

Render-only: write a "strata" colour attribute = geological laminae anchored in the GROUND. Because the bands depend on height, wind erosion EXPOSES deeper layers -- deflated hollows and scoured lee faces reveal cross-bedding, like a real eroded dune outcrop. Drive a shader from the "strata" attribute

Layer Thickness

`strata_thickness` - Float - default **0.4** - 0.02 to 20

Vertical spacing of the laminae in solver height units (thinner = more, finer bands)

Bed Dip

`strata_tilt` - Float - default **1.5** - 0 to 20

How steeply the layers dip across the terrain (0 = flat horizontal beds, high = steep cross-beds)

Bed Waviness

`strata_warp` - Float - default **0.3** - 0 to 3

Wavy undulation of the layers so they look natural, not perfectly straight

Band Contrast

`strata_contrast` - Float - default **0.4** - 0 to 1

Sharpness of the layer boundaries (0 = soft gradient, 1 = hard distinct bands)

ColorEvery

`color_every` - Int - default **1** - 1 to 1000

Throttle for Auto Masks: refresh the material mask attributes only every Nth step/frame. The masks lag a little between refreshes but playback stays fast -- raise it at high resolution where the color writes get expensive

Normal Strength

`ripple_normal_strength` - Float - default **1** - min 0.05

Slope exaggeration baked into the exported ripple NORMAL map: higher = deeper-looking ripples in the material without any extra geometry. Adjust to taste and re-export

Pebble Density

`scatter_pebble_density` - Float - default **40** - min 0

Pebbles per square meter at full coarse-lag coverage (the Geometry Nodes scatter multiplies this by the local coarse mask, so pebbles concentrate on the deflation pavement). Re-run Setup Scatter after changing

Grass Density

`scatter_grass_density` - Float - default **200** - min 0

Grass blades per square meter at full vegetation coverage (multiplied by the local vegetation mask -- tufts appear where the sim grew plants). Re-run Setup Scatter after changing

Format

`export_anim_format` - Enum - default **ABC

Cache format for the recorded-evolution mesh export

Option	Meaning
Alembic (.abc)	Alembic mesh cache -- imports into Maya, Houdini, Max, Cinema 4D, Unreal and back into Blender
USD (.usd/.usdc)	Universal Scene Description -- Omniverse, Houdini Solaris, USD-based pipelines

Animation Path

`export_anim_path` - String - default **//tydunes_anim.abc

Output file for the animated mesh cache export (.abc for Alembic, .usd/.usdc for USD). The recorded evolution is replayed through the Bake Viewer and cached frame by frame

Bundle Dir

`bundle_export_dir` - String - default **//tydunes_bundle

Folder the one-click Export Bundle writes into: heightmap, masks and settings together -- handy for support requests, backups or handing a setup to another artist. // means next to the .blend

Bake Dir

`bake_dir` - String - default **//tydunes_bake

Folder where Bake Height Sequence and the Evolution Recorder write their per-frame heightfield files (f#####). Replay them with Build Bake Viewer + Follow Timeline without re-simulating. // means next to the .blend file

Run ID

`record_run_id` - String - default **run_001

Subfolder of Bake Dir that THIS recording goes into, so two runs do not overwrite each other. Without it every run writes the same f0001.npz and the same height.0001.png, the second silently replaces the first, and the viewer replays

a mixture of the two. Every reader uses the same rule, so switching this back to an older id loads that run again. Blank = write straight into Bake Dir, which is how it behaved before

Record Evolution

`record_enabled` - Bool - default **off

FLIGHT RECORDER: save the visible heightfield into Bake Dir as compact binary f####.npz (~2 MB per 1024 frame, a few ms -- safe to leave on while testing). The moment you switch it ON it captures the CURRENT state under the current timeline frame (works right after Initialize, while paused, and with the Displacement Preview active); then every simulated playback frame is recorded, and a "Run Steps" batch records its final state under the current frame. Replay/scrub/render afterwards with Build Bake Viewer + Follow Timeline (no re-simulation). Frames overwrite on re-record

Lossless Recording

`record_lossless` - Bool - default **off

Store recorder frames in full float32 instead of the compact float16 (roughly 4 MB instead of 2 MB per 1024 frame). float16 keeps about 3 significant digits, which can show as faint stepping on very flat areas of a replay -- switch this ON for final-quality replays and renders from the recording

Record Masks Too

`record_masks` - Bool - default **off

Write the simulated MASKS alongside the heightfield on every recorded frame, numbered as an image SEQUENCE (wetness.0042.exr) so a compositor or a shader can read them straight back. The recorder has always saved the terrain per frame and nothing else, so a mask that drove the look could not be reproduced later at all. Costs disk: one file per selected mask per frame

Masks To Record

`record_mask_fields` - Enum - default **height, moisture, vegetation

Which masks the recorder writes each frame. Every field the exporter knows would be seventeen maps per frame and fills a disk quickly, so this is a choice -- the one-shot Export All Masks button still writes everything

Option	Meaning
Height	The heightfield itself
Moisture	Surface wetness
Vegetation	Plant cover
Nabkha Clumps	Which of that cover is a seeded clump
Erodibility	How freely the bed erodes
Shear Stress	Wind shear on the bed
Lee Shadow	Wind shadow / separation bubble
Sand Age	Fresh against weathered sand
Coarse Lag	Deflation pavement
Coarse Lag (transported)	The coarse fraction as actually carried by the two-fraction transport and sorted by the slip face -- the real desert pavement, not the ratio inferred from how much the bed moved. Empty unless Transport Two Grain Sizes is on
Sand Flux	How much sand is moving
Water Temperature	Temperature of the liquid in degrees Celsius. Empty unless Water Thermal Inertia is above 0 -- with it at 0 the water is at air temperature by definition and the map would carry no number the air map does not already have
Ice Thickness	Thickness of the ice sheet in metres. Empty unless Ice Thickness is above 0 -- the stateless ice has no thickness to export, only a cover

Option	Meaning
Water Depth	Depth of the water column in metres: the free surface minus the bed the field CARVED (terrain + dz), clamped at 0
Water Surface	Free surface height eta in world units -- the level the water settled to, hollow by hollow
Water Flow U	Current, X component, metres per second. Weighted by wetness so the dry-front spike does not read as a 60 m/s current
Water Flow V	Current, Y component, metres per second. Same wetness weighting as the U channel
Water Vorticity	Curl of the current, 1/s: where the water spins -- eddies behind rocks and along the channel walls
Water Silt	Suspended sediment as a volume fraction: where the water is actually carrying sand, not one murkiness value for the scene
Water Bed Change	Cumulative bed change dz in metres: what the water dug out or dropped, negative where it scoured

Simulate While Rendering

`step_during_render` - Bool - default **off

Let the simulation ADVANCE during an F12 animation render, so a sequence can be rendered straight from a running sim with nothing baked first. Off by default and deliberately: the frame handler normally only steps while PLAYBACK is armed, so a still render or a re-render of one frame cannot move the dunes under you. With it on, every rendered frame advances the solver exactly once -- which also means re-rendering a frame gives a DIFFERENT picture, and rendering backwards does not rewind. Use the recorder if you need the same frames twice

Bake From

`bake_from` - Int - default **1** - -100000 to 100000

First timeline frame of the Bake Height Sequence range (inclusive). Baking SIMULATES from the current state through the range and writes one heightfield file per frame into Bake Dir

Bake To

`bake_to` - Int - default **120** - -100000 to 100000

Last timeline frame of the bake range (inclusive). Must be >= Bake From; the solver state advances through the bake (it does not rewind after)

BakeRes

`bake_res` - Int - default **256** - 16 to 8192

Pixel resolution of the baked heightfield files (clamped to the solver resolution -- baking cannot invent detail above it). Lower = smaller, faster files for previz; match the solver resolution for full-quality displacement

Bake Visual Detail

`bake_visual_detail` - Bool - default **on

Bake the VISIBLE surface -- with render ripples, grain and polish baked into the heightfield -- instead of the raw solver heights. ON for final displacement/render use; OFF for clean physics data you plan to re-detail or post-process elsewhere

Bake Viewer

`bake_viewer_object` - Pointer

Mesh object used to preview baked heightfield frames

Viewer Frame

`bake_viewer_frame` - Int - default **1** - -100000 to 100000

Which baked/recorded frame the Bake Viewer object displays when you load manually. For scrubbing, enable Follow Timeline instead and the viewer tracks the playhead by itself

Follow Timeline

`bake_follow_timeline` - Bool - default **off

Make the Bake Viewer automatically load the baked/recorded file matching the current timeline frame -- scrub or render the whole evolution from disk with zero simulation cost. Frames missing from Bake Dir simply keep the last loaded state

Viewer Wears Terrain Shader

`bake_viewer_shader` - Bool - default **off

Give the Bake Viewer the SAME material the simulated terrain wears -- the same datablock, so every setting you changed on it comes along -- and feed it the recorded mask maps of each baked frame, so the look ANIMATES with the geometry when you scrub or render with Follow Timeline. Needs the masks recorded (Record Mask Fields on the recorder or the height bake); frames without a mask file keep the last masks loaded. Off leaves the viewer as a bare mesh, exactly as before

Viewer Placement

`bake_place_mode` - Enum - default **ORIGINAL

Where Build Bake Viewer parks the replay mesh. Changing this re-parks an existing viewer straight away, so you can flip between the two without rebuilding

Option	Meaning
On The Original	Park the bake viewer exactly where the terrain is, sharing its transform. The two surfaces occupy the same space, so A/B is a matter of toggling one of them off in the outliner -- the comparison nothing else can give you, because any offset at all changes the lighting on both
One Whole Terrain Away	Push the bake viewer a FULL TERRAIN WIDTH to one side, so the replay stands beside the live sim edge to edge instead of inside it -- the same contact-sheet idea as Snapshot Terrain, for a baked sequence

Offset Direction

`bake_offset_dir` - Enum - default **X_PLUS

Which way One Whole Terrain Away pushes the viewer. The step is measured along the TERRAIN OWN axes, so a rotated terrain still gets a neighbour that lines up with its grid instead of one marching off at an angle

Option	Meaning
X+	One terrain toward +X (terrain-local, so a rotated terrain still lines up)
X-	One terrain toward -X
Y+	One terrain toward +Y
Y-	One terrain toward -Y

Offset Gap

`bake_offset_gap` - Float - default **0** - min 0

Extra space between the terrain and the offset viewer, on top of the full terrain width. 0 puts them edge to edge, which is what "one whole terrain" means -- raise it if you want a visible seam between the two

Copy Maps Into The Run

`export_run_copy_maps` - Bool - default **on

What Export Run Bundle does with the baked look maps. ON copies them next to the recorded frames, which is the whole point of the bundle: the folder then replays and renders on its own, on another machine, after a Reset, or in

another pair of hands. It costs disk -- three rock maps plus the detail map at 4096 is a few hundred MB per run. OFF writes only their paths into the manifest, which is free and fine while the run stays on this machine beside this blend, and breaks the moment the run folder travels without them

---

Performance

The speed/quality trade-off, stated honestly. Grid resolution is the main lever, and this panel is where you decide what to spend.

When you need it: When it is too slow, or when it is fast enough and you want more detail.

Trace Steps (diagnostic)

`trace_steps` - Bool - default **off

Write the PHASE of every simulation step to the diagnostic log (separation, transport, settle, avalanche events, snow, rain). The normal log records one line per 25 steps, which at 1024 is a 10-50 second blind spot -- and a NATIVE death (a CUDA illegal address, a driver reset, an out-of-memory kill) unwinds no Python at all, so no traceback is written and that one coarse line is everything that survives. With this on, the last line names what was actually running. Costs a flushed disk write per phase, so arm it only to reproduce a hang, then switch it off

Give The CPU Back

`blocking_sync` - Bool - default **on

Wait for the GPU in the operating system scheduler instead of polling it in a loop. CuPy spins by default, which pins one CPU core for the whole of every step -- MEASURED on a 2048 grid: 213 of 216 ms per step, 99% of a core, doing nothing but asking the GPU whether it is done yet. That is a real part of why a big simulation makes the whole machine feel dead, and it has nothing to do with GPU priority (there is no such setting: CUDA stream priorities only order streams inside one process and do not yield to the desktop). MEASURED with it on: the same 216 ms per step and 11 ms of CPU -- 95% of the core handed back for no wall-clock cost. Results are bit-identical either way; this only changes HOW the wait happens. The small-grid worry has since been MEASURED and does not happen: interleaved A/B at 64 / 128 / 256 / 512 / 1024 gives wall-clock ratios of 1.00 / 0.99 / 0.95 / 1.12 / 1.05 (no trend, and the card was busy with other work), while the core handed back grows with the grid -- at 64 it gives nothing back (98% of a core either way, the step is over before the thread could sleep), then 60% / 39% / 13% / 5% of a core still used at 128 / 256 / 512 / 1024. So it is free on small grids rather than harmful, and the benefit is real from 128 up

Fast 32-bit Solver

`fast_precision` - Bool - default **off

PERFORMANCE vs precision. Run the whole solver (heights, shear FFT, transport, avalanche) in 32-bit float instead of 64-bit. On consumer NVIDIA GPUs (e.g. RTX 3090) 64-bit runs at a FRACTION of 32-bit speed, so this is up to ~2x faster there (measured 49.97 -> 23.14 ms/step at 1024, i.e. 2.16x) and halves memory. It is the single biggest speed win that keeps every physics option on. The RENDER layer follows: the ripple field, the exposure and the finished visual heightfield are 32-bit too, so the per-frame mesh data halves (33.6 -> 16.8 MB at 2048) and the visual pass runs ~1.3-1.45x faster there. The cost is lower numerical precision -- over very long runs the dunes may drift slightly from the 64-bit result, but it is visually near-identical (measured correlation 1.0000, mean difference 0.00% of the height range). Takes effect after Reset + Initialize (all state is allocated at this precision -- toggling mid-run does nothing). Leave OFF for the exact reference result

Wind Sweep

`wind_sweep_mode` - Enum - default **QUANTIZED_8DIR

How the transport sweep follows the wind. Exact Bilinear takes any angle and, on the GPU, costs the SAME as the 8-direction mode since the fused scan, so there is no reason to accept 8-facet banding there. On the CPU it degrades to Quantized above a work limit

Option	Meaning
Exact Bilinear	True any-angle wind sweep: no direction snapping and no 8-facet banding. On GPU the fused kernels make it the SAME SPEED as Fast 8-Direction, so it is the best choice there. On CPU it is capped/slow (see Allow Slow Exact Sweep)
Fast 8-Direction	Wind snapped to the nearest of 8 directions (up to 22.5 deg error): skew winds can migrate along the wrong axis and the field can band into 8 facets -- enable Dither 8-Dir Wind to hide that. Mainly useful on CPU; on GPU Exact Bilinear costs the same

Dither 8-Dir Wind

`wind_dither` - Bool - default **off

Fast 8-Direction only: instead of snapping a skew wind to the single nearest of 8 directions (up to 22.5 deg error, dunes migrate along the wrong axis and band to 8 facets), alternate between the two neighbouring directions over successive steps so the time-averaged wind matches the true direction. Same speed, much better direction fidelity. Best with Iterations/Frame > 1

Allow Slow Exact Sweep

`allow_slow_exact_sweep` - Bool - default **off

CPU-ONLY escape hatch: run the original serial Exact Bilinear sweep at ANY resolution on the CPU, removing the normal 256 cap. It gets VERY slow above 256 -- expect long per-frame waits. On GPU the exact sweep is always the fast parallel scan and this option does nothing; use it only when you must have exact any-angle wind without a CUDA GPU

Preview Physics

`preview_physics` - Enum - default **OFF

FULL-RESOLUTION preview speedup (no proxy, no grid change): temporarily relaxes the substep-accuracy knobs while you iterate, on the SAME terrain at the SAME resolution. Switch back to Off for the final run -- the toggle changes solver pacing only, never the terrain, masks or render detail. Overrides Max Substeps / Avalanche Every Substep / Shadow Every Substep while active

Option	Meaning
Off (Final Quality)	Full-quality physics: every CFL substep runs the shadow and the avalanche (bit-exact final result)
Balanced (~2x)	Same terrain, same resolution: caps CFL substeps at 8 and relaxes the avalanche once per step instead of every substep. Measured ~2x faster at 1024; dunes form the same shapes and sizes, slip faces are marginally softer while sand is moving
Fast (~3x)	Caps substeps at 4, avalanche once per step, lee shadow once per step. Measured ~3x faster at 1024; dune character and scale match the final look but exact crest positions drift and moving faces are visibly softer -- lookdev/layout mode
Extreme (~4-5x)	Caps substeps at 2: the fastest full-resolution preview. Dune size and overall motion stay readable, but expect soft faces, crest drift and possible faint transverse stepping at 512+ (the presets Settle Slopes bursts mask most of it). Blocking and layout only -- never judge final detail here
Draft (fastest)	Everything Extreme does, plus HALF the repose-relaxation passes (3 instead of your setting). The avalanche is the single most expensive part of a step -- measured 16 ms of 59 at 1024 -- so this is the fastest the solver goes at full resolution. What you lose: slip faces are noticeably rounder and slopes sit above the true repose angle while sand is moving, so silhouettes are only approximate. Scrubbing the timeline and framing shots only

Auto Substep

`auto_substep` - Bool - default **on

Automatically split each frame into enough transport substeps to stay numerically stable at the current resolution (CFL limit). LEAVE ON: turning it OFF at 512+ produces regular transverse "staircase" ribbing across the field -- a numerical

artifact that looks like fake dense dunes, not physics. If you need speed, cap Max Substeps or use Preview Physics instead of disabling this

Max Substeps

`max_substeps` - Int - default **16** - 1 to 32

PERFORMANCE vs QUALITY. Upper limit on CFL substeps per frame. The solver needs ~1/dx substeps for stability (e.g. ~11 at 512 on a 50-unit terrain). Lowering this is FASTER at high resolution but REDUCES accuracy -- dunes migrate in coarser jumps and can staircase or wobble; with Auto Substep OFF at 512+ the shortfall shows as regular transverse staircase ribbing. 16 = full quality; try 4-8 for a fast preview (or just use Preview Physics)

Shadow Every Substep

`substep_shadow` - Bool - default **on

Recompute the lee wind-shadow on every CFL substep (accurate, the default). Turning it OFF computes the shadow once per step instead, so the shadow lags the sand within a frame. NOT a speed knob: measured at 1024 f32 it is 49.97 ms/step ON vs 51.45 ms OFF -- the separation scan is not the bottleneck, so switching it off costs accuracy and buys nothing. Leave it ON. For real speed use Fast Precision, Min/Max Substeps or Avalanche Every Substep

Min Substeps (force)

`min_substeps` - Int - default **1** - 1 to 64

HARD floor on substeps per step -- forces AT LEAST this many, winning over Max Substeps, Preview Physics and the automatic CFL count. Use it to densify the sim when sudden wind turns (Reversing / Star schedules, big keyed direction jumps) spike the terrain at bare-CFL pacing, or to truly FORCE a substep count (note: with Auto Substep OFF the solver otherwise runs a single substep and Max Substeps is ignored). 1 = no forcing. Cost scales linearly with the count

Boost Substeps On Wind Turns

`turn_substep_boost` - Bool - default **off

When the wind heading jumps more than ~20 degrees in a single step (a Reversing or Star phase flip, a keyed direction jump), automatically DOUBLE the substep count for exactly that step -- sudden turns dump transport in a new direction all at once and can spike steep terrain at bare-CFL pacing. Zero cost while the wind is steady. Presets with scheduled winds switch this ON

Avalanche Every Substep

`ava_substep` - Bool - default **on

Run the repose avalanche relaxation on every CFL substep (the 3ds Max original does this -- accurate slip faces, the default). Turn OFF to relax only once at the end of each step: a BIG speed win at high resolution (the avalanche dominates the step cost), with slightly softer slip faces while sand (or snow) is in motion. QUALITY knob -- the settled dunes look nearly identical

Fused GPU Avalanche

`fused_avalanche` - Bool - default **on

Run the avalanche relaxation as one fused CUDA kernel per sweep direction instead of many small GPU operations. The result is IDENTICAL bit-for-bit (same math, same order) -- this is a pure speedup on NVIDIA GPUs and changes nothing on CPU. Disable only for debugging or comparison

Avalanche Quality

`avalanche_quality` - Enum - default **EXACT

How exactly the avalanche sweep is solved. It is worth a switch because the avalanche is the single most expensive part of a step -- MEASURED 54% of a 1024 step and 69% at 2048 -- and four independent LOSSLESS speedups were tried and closed by measurement, so anything further has to trade something. Note the ceiling: even an infinitely fast avalanche can only take a whole step 2.1x (1024) or 3.4x (2048)

Option	Meaning
Exact	The shipped sweep, bit-identical to the 3ds Max original. Use for finals
Fast (32-bit sweep)	Run the avalanche in 32-bit even when the simulation is 64-bit. MEASURED ~2.6x on the avalanche and 1.5-1.7x on a whole 64-bit step, for a relative error around 1e-7 per call -- relief and slope statistics come out the same. Does nothing extra when Fast Precision is already on, since the field is 32-bit anyway
Draft (over-relaxed, 3 passes)	Fast, plus a relaxation factor of 0.40 instead of 0.25 and three passes instead of six. The higher factor converges about twice as fast TO THE SAME FIXED POINT, so the repose angle does not move: MEASURED ~5x on the avalanche and 1.8-2.1x on the step, for relief +0.3-1.4% and the steepest slope about 3 degrees over. For blocking and layout

Fused GPU Wind Scan

`fused_scan` - Bool - default **on

Run each doubling level of the Exact Bilinear wind sweep (flux saturation scan + lee-shadow scan) as ONE fused CUDA kernel instead of dozens of small GPU operations. The result is IDENTICAL bit-for-bit (same math, same order) -- a pure speedup that makes the Exact (any-angle) sweep usable at high resolution; changes nothing on CPU or in the Quantized 8-dir mode. Disable only for debugging or comparison

Proxy Simulation

`proxy_sim` - Enum - default **OFF

LOSSY preview speed mode: run the SOLVER on a coarser grid than the terrain mesh and bicubically upsample the result. The big dune shapes, wind behaviour and masks match the final run (enable Res-Independent Dune Size so proportions are preserved); fine ripples and small dunes are softer/approximate. Requires re-Initialize to take effect. Switch back to Off + Initialize for the final-quality run. If you want a speedup WITHOUT losing resolution, use Preview Physics instead (same grid, relaxed substepping)

Option	Meaning
Off (full res)	Solver runs at the full Solve Resolution (final quality)
Half res (~4x faster)	PREVIEW: solver runs at 1/2 the resolution, the visual is bicubically upsampled to the full mesh. With Res-Independent Dune Size the dune-to-terrain look matches the final; fine detail is softer
Quarter res (~10x faster)	FAST PREVIEW: solver runs at 1/4 the resolution, upsampled to the full mesh. Same dune proportions with Res-Independent Dune Size; small-scale detail is approximate

Viewport Update Every N

`viewport_update_every` - Int - default **1** - 1 to 250

Refresh the terrain mesh / render detail only every Nth playback frame. The simulation still steps EVERY frame (the result is exact); only the viewport display skips frames. THE NUMBERS: a skipped frame saves the render-detail pass, the mesh write AND -- the big one -- Blender rebuilding the draw buffer for the changed geometry, measured on an RTX 3090 at ~7 ms per frame at 512, ~36 ms at 1024 and ~210 ms at 2048. At 2048 that single item is about two thirds of the frame, so N=5-10 makes playback dramatically faster; with Display Resolution it is cheaper still. Haze, dust and auto material masks refresh on the same cadence. 1 = update every frame (default). Together with Preview Physics this is the biggest playback win at 1024+

Display Resolution

`display_res` - Enum - default **FULL

VIEWPORT-ONLY decimation: draw the terrain on a COARSER mesh than the solver computes. The simulation itself is untouched -- same grid, same physics, bit-identical results -- only the preview geometry is smaller. WHAT YOU LOSE: the displayed surface is area-averaged down, so ripples and fine crest detail are smoothed or gone in the viewport (they are still in the solver, and Bake, Bake To Displacement, heightmap/mask export and F12 renders always use the FULL resolution). WHY: after every geometry change Blender rebuilds the whole draw buffer, and that

cost is pure vertex count -- measured on an RTX 3090 at ~7 ms (512), ~36 ms (1024) and ~210 ms (2048) per changed frame, on top of the physics. Half at 1024 or Quarter at 2048 buys that time back. Auto material masks / sand-age colours follow the display grid too

Option	Meaning
Full (1:1)	Preview mesh matches the solver grid exactly -- no change, no loss (default)
Half (1/2)	Preview mesh at half the solver grid = 1/4 of the vertices. At 1024 that is the 512 redraw cost (~36 ms -> ~7 ms per changed frame)
Quarter (1/4)	Preview mesh at a quarter of the solver grid = 1/16 of the vertices. At 2048 that is the 512 redraw cost (~210 ms -> ~7 ms per changed frame)
Eighth (1/8)	Preview mesh at an eighth of the solver grid = 1/64 of the vertices. Fastest scrub; only the big dune shapes survive

Pipelined Playback (1 frame behind)

`pipelined_playback` - Bool - default **off

Overlap the simulation with Blender's viewport redraw. Normally the frame handler WAITS for the GPU, returns, and only then does Blender rebuild the terrain's draw buffer -- so the card sits idle for the whole redraw. With this on the handler starts an asynchronous download and returns while the GPU is still working, so the redraw overlaps the tail of the frame. WHAT YOU LOSE: the viewport runs exactly ONE FRAME BEHIND the solver during playback (it catches up the moment you pause; Bake, export and F12 renders are never affected, and the physics is bit-identical either way). MEASURED gain, RTX 3090: 1.30x at 1024 full-res with Steps/Frame 1, 1.50x at 2048 full-res -- but only ~1.01-1.09x once Display Resolution is on or Steps/Frame is high, because those already remove the cost this hides. Automatically inactive with the Displacement Preview, the Evolution Recorder or De-facet Crests (render-only) enabled, and on the CPU backend

Shadow Reach (0=Full)

`shadow_reach` - Int - default **0** - 0 to 8192

Maximum lee-shadow length in solver cells. 0 = FULL (the shadow can span the whole domain -- accurate, but the scan cost grows with resolution). A finite value (e.g. 128) is FASTER but TRUNCATES the long wind shadow behind tall dunes on fine grids -- lee sides then get too much sand. QUALITY knob; leave 0 unless chasing speed

---

Debug / State

What the solver currently holds, in numbers.

When you need it: When something is not doing what you expect and you want to see the state rather than guess at it.

Check Steps

`evolution_check_steps` - Int - default **24** - 1 to 10000

How many trial steps the Evolution Check button simulates on a THROWAWAY copy of the state to show where the field is heading -- the real sim is untouched. More steps = a further look ahead but a longer wait

Terrain Feedback

`terrain_feedback` - Bool - default **off

Read the terrain mesh back into the solver as SAND before every step: sculpt strokes, shape keys or modifier results become real material the wind then reworks -- live art-direction of a running sim. CAUTION: it reads whatever is on the mesh, so combined with Show Render Detail it can absorb the cosmetic ripples as real sand and coarsen the field -- prefer feedback with the detail layer off. Costs a mesh read per step

Feedback Strength

`terrain_feedback_strength` - Float - default **1** - 0 to 1

Blend factor for Terrain Feedback: 1 = the solver snaps fully to the edited mesh each step (edits land instantly and hard), lower values ease the edits in over several steps so the wind smooths them as they appear -- softer, more natural integration

OldDunes

`old_dunes` - Bool - default **off

Legacy 3ds Max look: disable strict erosion conservation so the field GAINS material -- dunes bloat taller and rounder over long runs. This OVERRIDES Conserve Erosion: with OldDunes ON that checkbox has no effect. The gain is NOT gradual -- it feeds back on itself once bare rock appears. MEASURED on a deep bed: 0.0000% at 120 steps, +15% at 480, +156% at 1920, peak height 12x. Use only to match old renders made with the original solver; for physically honest results leave OFF and keep Conserve Erosion ON

Extra Phenomena

Extra physical phenomena that are not part of the core dune model: frost, ice, thermal effects and the more specialised processes.

When you need it: Specific shots that need them. Everything here is off by default.

Rain Seeds Growth

`rain_veg_seed` - Float - default **0** - 0 to 0.5

Let RAIN grow plants on its own, instead of only speeding up growth that was already happening. Growth From Rain above is a MULTIPLIER, so with Growth at 0 no amount of rain does anything: measured on a still bed, a nabkha seeding covering 12.94% of the field finished at 12.94% both with and without heavy rain, identical to the digit. This is an added growth rate driven by the root zone. IT OBEYS VEG SPREAD, and that is what decides whether the cover creeps out of the clumps or appears everywhere: measured over 60 wet steps from the same 12.94% of seeded clumps, at Spread 1.0 it reached 15.8 / 22.1 / 24.8 / 39.6 / 53.1% for rates 0.001 / 0.003 / 0.01 / 0.03 / 0.05, while at Spread 0.9 -- which leaves a tenth of the growth rate on bare ground -- rate 0.003 already covered 93% of the field. Set Veg Spread to 1.0 if you want it to spread FROM the nabkha. 0 = off

Plants Shade The Ground

`veg_dry_shade` - Float - default **0** - 0 to 4

Damp ground under plant cover dries more slowly than the bare sand beside it. This is the RETURN half of the rain-plant loop: rain already feeds growth, and this lets the growth hold on to the water that made it. Lengthens the surface drying time in proportion to the local cover. 0 = off

Frost Binds The Surface

`frost_bind` - Float - default **0** - 0 to 1

Let surface FROST resist the wind. Frost is otherwise a render-only field: measured across the whole coupling matrix, switching it off or capping it moved nothing at all except frost itself, in both wind models. NOT the same rule as Freeze Wet Ground -- that one keys on air temperature and moisture and bonds any cold damp cell, while this follows the frost FIELD, so it works on a field seeded with Initial Frost and no temperature model. 0 = off

Playa Crust

`playa_enabled` - Bool - default **off

Ground that dried out sets into a hard crust broken by a polygonal crack network -- a takyr or playa floor. The crust stops the wind lifting anything, so deflation runs ALONG the cracks and leaves the plates standing proud. Needs a source of wetness (Water or Rain); with neither, nothing ever dried and nothing crusts

Drying Rate

`playa_rate` - Float - default **0.05** - 0 to 1

How fast dry ground sets, per step. Flooding it again dissolves the crust at twice this rate, so a tide or a rain cycle really does rework the surface

Plate Size

`playa_plate` - Float - default **12** - 2 to 256

Average width of one mud plate, in GRID CELLS. Smaller gives the fine crazing of a thin crust, larger the metre-wide slabs of a deep dried lake bed

Crack Openness

`playa_crack` - Float - default **0.6** - 0 to 1

How much of the wind the crack lines let through. 0 seals them and the whole crust is inert; 1 makes the cracks as erodible as bare sand, so they scour out into channels between the plates

Wind Erases Tracks

`track_fade_enabled` - Bool - default **off

Let the wind rub out fine surface detail -- footprints, tyre ruts, brush marks -- faster where it blows harder. Transport alone erases them only as fast as it moves sand, which at low supply is far slower than a real print disappears. Mass conserving: a sheltered print survives, an exposed one does not

Erase Rate

`track_fade_rate` - Float - default **0.15** - 0 to 1

Strength per step. Measured on a 25-step run: a rut ended 27 % shallower at 0.30 than with the option off. Push it high and dunes themselves start to soften, which is why it is not on by default

Obstacle Porosity

`obstacle_porosity` - Float - default **0** - 0 to 1

Treat blockers as POROUS -- a snow fence or a windbreak hedge rather than a wall. A solid face drops a short steep drift against itself; something that passes part of the flow lays a long drift downwind instead, which is the whole reason snow fences are built with gaps. 0 = solid, exactly as before

Sublimation

`sublimation_rate` - Float - default **0** - 0 to 0.2

Snow lost straight to vapour, without ever melting -- why a snowfield thins on a clear cold windy day. Exposure driven: it bites where the shear is highest and not at all in a lee, so it sculpts as well as removes

Surface Hoar

`hoar_rate` - Float - default **0** - 0 to 0.2

The same process running backwards in the sheltered cells: feathery crystals grow on the surface. They add depth, but the point is that they are MECHANICALLY WEAK -- this lowers hardness, and a buried hoar layer is what later releases a slab

Snow Layer Stack

`snow_layers_enabled` - Bool - default **off

Track the snowpack as a stack of dated layers instead of one "fresh" number. A season lays a layer per storm, each hardens at its own pace, and a slab breaks at the WEAKEST INTERFACE -- which may be buried under a metre of later snow. Without this, every release has to guess from the surface alone

Layers

`snow_layer_max` - Int - default **6** - 2 to 16

How many layers the stack keeps. When it is full the oldest two are merged, so memory is fixed (layers x grid) instead of growing with the length of the run

New Layer At

`snow_layer_new` - Float - default **0.05** - 0 to 5

Depth the top layer must reach before the stack is pushed down and a fresh one starts -- in effect, how big a snowfall counts as its own storm

Sinter Rate

`snow_layer_harden` - Float - default **0.02** - 0 to 1

How fast a layer sinters per step. BURIED layers harden faster because they carry the load above them, which is what lets an old layer become the strong one and a recent one the weak one

Weak Layer Coupling

`snow_layer_couple` - Float - default **0.5** - 0 to 1

How much the WEAKEST buried layer sets the hardness the avalanche model reads. 0 keeps the old surface-only behaviour; 1 makes the buried weak layer decide entirely, so a slab releases over a hoar layer the surface alone would have called safe

Depth Hoar Every N Layers

`snow_hoar_period` - Int - default **0** - 0 to 64

Turn every Nth layer that gets buried into DEPTH HOAR: a weak layer that never hardens, so it stays a failure plane under any amount of snow. 0 is off and the pack behaves exactly as before. WHY THIS EXISTS, MEASURED: the slab depth the model reports can never exceed New Layer Threshold, because the search takes the snow lying above the weakest BURIED layer and hardening grows with burial depth -- so the weakest buried layer is always the youngest one, with only the surface layer on top of it. Neither Max Layers (2 against 16) nor Layer Hardening (0.001 against 0.05) moved it by a digit. On a 38 degree slope the releases also die out completely: 7 358 seed cells in the first 20 steps and ZERO after step 100, while the snow keeps falling. With a layer that does not harden the slab reaches 8.6 m and the releases keep coming (47 199 seeds after step 100). Small N means frequent weak layers and a jumpy pack; a hoar layer lives Max Layers minus 2 pushes before the oldest layers merge it away

Depth Hoar Hardness

`snow_hoar_hardness` - Float - default **0.05** - 0 to 1

How weak a layer becomes the moment it is marked as depth hoar. The marking both RESETS the hardness to this value and stops the layer hardening afterwards -- depth hoar forms by turning snow that is already lying into faceted crystals, so a layer that had packed down as the surface must lose that hardness when it becomes hoar. Without the reset a marked layer kept whatever it had gained on the surface (measured: 0.8442) and was hoar in name only. Only used when Depth Hoar Every N Layers is above 0

Snow Density

`snow_density_enabled` - Bool - default **off

Give snow a DENSITY, so the conserved quantity becomes MASS instead of depth. Off by default and then bit-identical: with it off the solver reproduces every existing scene exactly. WHY IT MATTERS: 10 cm of fresh powder (about 100 kg/m3) and 10 cm of settled snow (about 400) are the SAME NUMBER to a depth-only solver and four times apart in reality. Density is read off the hardness the layer stack already tracks, so nothing new is stored and the slab release gets a real density contrast to break on

Fresh Snow Density

`snow_rho_fresh` - Float - default **100** - 30 to 300

Bulk density of newly fallen snow, in kg/m3, i.e. the density at hardness 0. Literature puts fresh snow at 50-150; 100 is the usual figure for a calm, cold snowfall. Wet or windy snowfall lands denser -- raise this to about 200 for a maritime

storm

Packed Snow Density

`snow_rho_packed` - Float - default **500** - 200 to 700

Bulk density of fully hardened snow, in kg/m3, i.e. the density at hardness 1. This is the OTHER end of the ramp and it sets how much a drift can compact: settled snow is 200-300, wind slab 300-400 and firn 500-550. The ratio to Fresh Snow Density is what the physics actually uses, so 100 against 500 means a drift can shrink to a FIFTH of its fallen depth without losing a gram

Drifted Snow Density

`snow_rho_drift` - Float - default **350** - 30 to 700

Bulk density of snow the WIND has moved, in kg/m3. Snow does not land the way it fell: saltation shatters the crystals and they pack tighter, so a drift is 300-400 where the snowfall that fed it was 50-150. THE SOLVER USES THE RATIO, not the number: at the 100/350 defaults, 5 cm of blown powder arrives as about 1.4 cm of drift carrying the same mass -- which is why a drift on the lee of a fence is dense and shallow rather than a heap of loose powder. Set this equal to Fresh Snow Density to switch the effect off exactly

Slab Avalanche on the Weak Layer (SLOW)

`snow_slabfail` - Bool - default **off

Release a whole SLAB when the shear stress from its own weight beats the strength of the weakest BURIED interface the layer stack found. This is a different mechanism from the granular avalanche, not a stronger setting of it. The granular one is a SLOPE rule: neighbouring cells over the repose angle pass their excess along, so it trickles and it can only ever fire on ground that is already too steep. A slab fails on stress AT DEPTH -- weight of everything above the weak layer, resolved along the bed -- so a 30 degree face that every repose rule in this solver calls safe lets go as one piece once a hoar layer is buried under it. It leaves a sharp CROWN where it tore and a deposit at the base. NEEDS Snow Layer Stack: the weak interface and the slab depth are what that stack computes, and without it there is nothing to fail on. OFF BY DEFAULT AND HERE IS THE COST: the crack flood and the runout routing are both iterative, so a step that releases pays for up to a few hundred whole-grid passes. Steps with nothing failing pay for ONE gradient and one compare

Weak Layer Strength

`snow_slabfail_strength` - Float - default **1** - 0.01 to 50

Shear strength of the buried weak layer, as a multiplier on its hardness. The slab releases where depth *sin(slope)* cos(slope) exceeds this times the weak layer hardness, so LOWER releases more easily and HIGHER holds. Turn it up to make a season that only ever slides after a big storm; turn it down for a pack that goes on every loading. Note that the slope term PEAKS at 45 degrees and falls off again -- genuinely steep faces sluff continuously instead of building slabs, which is in the physics and not a limitation

Crack Propagation

`snow_slabfail_propagate` - Float - default **1.6** - 1 to 8

How far past its own failure point a crack keeps running. Starting a fracture needs the full stress; a crack that is ALREADY running needs less, because its tip concentrates stress ahead of itself -- so it unzips along the weak layer through snow that would never have failed on its own. This is what makes the release collective instead of a scatter of pits. 1.0 switches propagation off entirely (only cells that individually fail let go, which is the control this feature is measured against); 2.0 runs the crack into snow at half the failure stress and takes the whole face

Min Slab Depth

`snow_slabfail_min_depth` - Float - default **0.02** - 0 to 5

Thinnest slab allowed to release, in scene units of snow depth. A dusting over a weak layer is arithmetically releasable but nothing about it reads as an avalanche, and letting it fire turns the whole slope into permanent noise. Raise it if you want only the deep storm slabs

Runout Angle

`snow_slabfail_runout` - Float - default **12** - 0 to 45

Degrees of slope below which the sliding debris stops and piles up. The released mass is routed downhill on the current surface and deposits where the ground flattens past this, which is what puts the debris fan at the BASE of the path instead of leaving it on the face. Lower = the slide runs further out onto flat ground; higher = it stops on the slope. 0 makes it run until it finds a genuine hollow

Slab Runs Out (dynamic)

`snow_slabfail_dynamic` - Float - default **0** - 0 to 1

Let the released slab TRAVEL instead of being spread downhill. The shipped runout is a ROUTING rule: it pushes debris while the local drop stays above Runout Angle and drops it where the ground flattens -- so on a flat valley floor it stops at the foot of the slope, because there is no drop left to route along. Above 0 the debris is handed to the depth-averaged flow instead: it accelerates, runs onto the flat and stops by FRICTION. MEASURED on a 35-degree slope with a flat floor: routing leaves 18.0% of the deposit below the foot of the slope, the moving flow leaves 70.8%. At 0 the routing path runs unchanged, to the bit

Slab Friction

`snow_slabfail_mu` - Float - default **0.3** - 0 to 2

Coulomb friction of the moving debris -- the physical definition of the angle it stops at: below a slope of arctan(this) it will not move at all. Dry snow sits near 0.2-0.3, wet debris higher. MEASURED, share of the deposit reaching the flat floor: 0.15 -> 75.6%, 0.30 -> 58.1%, 0.60 -> 22.7%. Only used when Slab Runs Out is above 0

Turbulent Friction (xi)

`snow_slabfail_xi` - Float - default **0** - 0 to 20000

The SECOND half of Voellmy friction. Coulomb friction (mu) does not depend on speed, so it sets how FAR the debris runs; this term is g*|v|^2/xi and it caps how FAST it ever goes -- in steady state on a slope, v_max = sqrt(xi * S). It was held back from 1.3.136 to 1.3.150 because it was being judged by RUNOUT, which it does not control: those sweeps read 50/50/50/46 m and looked dead. MEASURED by peak speed instead (35 degree slope, 1.5 m slab, mu 0.30): 17.66 m/s at xi 4000, 16.79 at 1500, 14.91 at 600, 12.31 at 250 and 9.63 at 100 -- monotonic, and at xi 250 the measured 12.31 m/s lands on the theoretical 11.97 within 3%. Literature puts flowing avalanches at 1000-2000. 0 = off, and the run is then bit-identical to Coulomb friction alone

Entrainment (m per m of path)

`snow_slabfail_entrain` - Float - default **0** - 0 to 0.05

A real avalanche does not carry only the slab that let go -- it scours the snow it runs over and grows. This is that term: for every metre the flow travels it lifts this much loose snow off the bed, capped by what is actually lying there. MEASURED on a 34-degree slope with 0.74 m of powder (the depth the AVALANCHE preset settles at), the snow lying on the path is worth 5.8x the released slab when the weak layer is a band and 22.7x when it is scattered patches -- so there is real material to pick up. With a uniformly weak layer the whole slope goes at once and the ratio drops to 0.53, which is why this needs a layer stack that varies. Mass is conserved: what the flow lifts is removed from the bed before the deposit is laid down. Typical values 0.001-0.01; at 0 nothing here runs and the avalanche is computed exactly as before

Carry Momentum

`snow_slabfail_momentum` - Float - default **0** - 0 to 1

Let the debris KEEP the speed it has built up, instead of losing it the moment the flow slows down. MEASURED, and the control is one nothing can talk its way out of: on a FLAT bed with friction switched OFF, in a periodic domain, no force may take momentum away -- yet at 0 the total momentum falls to 4.8% in 2000 steps while the mass stays exact to 5e-16. The cause is that each cell face is carried by two one-way pipes clamped at zero, so every slowdown DELETES momentum instead of handing it to the other side. At 1 the face carries one signed flux and the clamp only splits it losslessly: the same control then measures 100.0% of the momentum kept. What you see: debris that runs up the far side of a valley about TWICE as far (measured 0.35/0.53/1.23/2.63/5.25 m against 0.18/0.35/0.70/1.58/2.45 m

at speeds of 2.5 to 40 m/s). It does NOT give textbook ballistic run-up -- that also needs momentum to travel WITH the mass, which this solver does not do yet. Costs a second pass over the fluxes, so it is off by default. Only used when Slab Runs Out is above 0

Momentum Rides Along

`snow_slabfail_advect` - Float - default **0** - 0 to 1

Let the speed TRAVEL WITH THE DEBRIS instead of staying on the cell boundary it was created at. Carry Momentum stopped the solver from DELETING momentum; this is the other half -- the missing d(h u^2)/dx term of shallow water, carried by the same conservative upwind transport the ice sheet already uses. What it buys, MEASURED as run-up on a 15 degree slope against the frictionless head v^2/2g: at 5, 10 and 20 m/s the debris now climbs 42%, 53% and 55% of that head, against 42%, 21% and 11% with Carry Momentum alone and 0%, 11% and 5% with neither -- and it now scales with the SQUARE of speed, which is what a body trading speed for height does. It is still about half the textbook value: an upwind scheme diffuses, and the wetting front genuinely cannot hand on more than the cell holds. Needs Carry Momentum above 0 -- without it there is no signed face flux to carry. Off by default

Sharper Transport

`snow_slabfail_hires` - Float - default **0** - 0 to 1

Stop the transport from BLURRING the flow. Momentum Rides Along carries speed with the debris using a first-order upwind step, and that step has a built-in smearing of about half a cell per step; this replaces it with a limited second-order reconstruction, which does not add physics -- it just stops losing what is already there. MEASURED, run-up as a share of the frictionless head v^2/2g, first order -> second order: on a 15 degree slope at 20 m/s 55% -> 92%, on 5 degrees at 20 m/s 51% -> 102%, on 30 degrees at 30 m/s 58% -> 68%. IT ONLY WORKS IN THE FAST REGIME, and that is measured too, not guessed: at 5 and 10 m/s every fixture gave exactly 1.00x -- no change at all. Below roughly 20 m/s the first-order smearing is not what limits the result, so this knob has nothing to recover. It also shifts the momentum budget on a flat bed from +3.8% to -4.0%: it swaps a small excess for a small deficit rather than removing either. Needs Momentum Rides Along above 0. Off by default

Momentum Crosses Lanes

`snow_slabfail_cross` - Float - default **0** - 0 to 1

Carry speed SIDEWAYS as well as forwards. Momentum Rides Along moves the along-flow part; the full equation has a second term, d(h u v)/dy, which does nothing at all in a straight chute and everything where neighbouring lanes of debris move at different speeds -- a flow rounding a spur, splitting round an outcrop, or shearing against a slower edge. MEASURED on a shear fixture -- lanes of different speed plus a steady 2 m/s crosswise flow, so after 9 seconds the pattern should have travelled exactly 9 cells: at 0 it moves 0.00 cells, at 1 it moves 9.00, with its amplitude kept at 98.8% and total momentum at 100.00% in BOTH axes. The pattern is carried at the crosswise flow speed to the cell, which is what this term is. Needs Momentum Rides Along above 0. Off by default

Crown Line

`snow_slab_crown` - Float - default **0** - 0 to 1

Draw the CROWN -- the line the slab tore along -- instead of the whole release area. The scar mask has always been the FOOTPRINT: every cell that let go reads 1, so the material paints a patch where a mountain shows a sharp edge above bare bed. This fades the INTERIOR and keeps the rim at full strength. MEASURED share of a release that is rim: 9.8% for a 40x40 slab, 19.0% at 20x20, 36.0% for a 6x50 band and 67.8% for a 3x60 sliver -- which is why this FADES the middle rather than replacing the mask with its own edge: on a narrow slab the two are nearly the same set, and replacing would make the knob act backwards. 0 = the footprint exactly as before, bit for bit

Slab Block Size

`snow_slabfail_block` - Float - default **0** - 0 to 20

Break the DEBRIS into blocks instead of laying it down as a smooth sheet. A slab avalanche shatters into lumps and that is its recognisable look; the shallow-water runout returns a smooth depth field, correct in mass and reach but reading like spilled bedding. Mass is conserved WITHIN EVERY BLOCK, not just overall, so the lumps cannot wander down the slope and this setting cannot change the RUNOUT DISTANCE -- reach stays what the friction law says. Roughly the size of a single lump; 2 to 5 m looks like a dry slab. At 0 the deposit is smooth, bit-for-bit as before

Weak Layer Patch Size

`snow_slabfail_patch` - Float - default **0** - 0 to 80

Give the buried weak layer PATCHES instead of one uniform strength, roughly this size across. Buried hoar settles in flat plates with fairly sharp edges, not as smooth noise, and that unevenness is what makes a slope fail in ONE place rather than all over. MEASURED on a 38 degree face near the failure threshold: without patches the release covers 19776 cells and the deposit travels 62.5 m; with 20 m patches it covers 17508 cells and travels 79.8 m. A SMALLER slab runs FURTHER, because a release that starts everywhere has nowhere left to run. The pattern comes from an integer hash, so the same slope fails the same way in every run and a render is reproducible. Only bites when the pack is near its threshold -- at a strength far below critical everything fails regardless. At 0 the layer is uniform, bit-for-bit as before

Slab Gravity

`snow_slabfail_g` - Float - default **9.81** - 0.01 to 30

Gravity the moving debris falls under, in m/s^2. Stated here and NOT taken from the planet preset on purpose: the sand solver, in its own words, "contains no gravity" -- there is no value to inherit, and quietly inventing one is the kind of half-wiring this add-on has been bitten by. Set it yourself for another world

Slab Flow Substeps

`snow_slabfail_substeps` - Int - default **200** - 10 to 20000

How long the debris is allowed to run before it is laid down. The flow is resolved to REST inside one solver step rather than carried between steps, so this is a budget, not a speed: too few and the debris is deposited while still moving. MEASURED on the Avalanche Into A Valley terrain (512 cells, 0.39 m): 1500 substeps is about 3 s of flow, and a flow with all four momentum controls on needs about 7 s just to reach the counter-slope and 11 s to run up it -- so on that scene 1500 lays the debris down mid-flight. Pair a large budget with Slab Flow Rest Speed so the run ends when the flow actually stops. Only used when Slab Runs Out is above 0

Slab Flow Rest Speed

`snow_slabfail_rest` - Float - default **0** - 0 to 10

End the debris run EARLY, once the bulk of the flow is slower than this, in m/s. 0 keeps the whole substep budget every time, exactly as before. The speed is the mass-weighted mean of the moving debris -- NOT the per-cell velocity, which on a thin front reports hundreds of m/s and would never read as stopped. Checked every 50 substeps after the first 200, because each check is a GPU sync. MEASURED on the Avalanche Into A Valley terrain at 0.05 m/s: the run ends after 4450 substeps (10.8 s) with momentum off and 8650 (17.6 s) with the four momentum controls on, against a 12000 budget. Only used when Slab Runs Out is above 0

Runout Sees Obstacles

`snow_slabfail_walls` - Bool - default **off

Let the dynamic debris run be STOPPED by obstacle objects. The relaxation and the routing have treated obstacles as walls for a long time, but the momentum solver under Slab Runs Out never did and debris went straight through a fence. MEASURED on a 128 grid with a barrier halfway down a 38 degree wall and a 1.2 m slab: routing held the debris (89 per cent below the barrier line without it, 45.5 per cent with it) while the dynamic run pushed 85 per cent of the mass THROUGH the barrier. A cell counts as a wall only while the obstacle STANDS ABOVE the surface, the same rule the avalanche uses, so a buried plate stops being a wall instead of becoming a permanent dam. OFF by default because turning it on changes any existing scene that has both obstacles and the dynamic run, and it costs the fast water path. Only used when Slab Runs Out is above 0

Canyon Channelling

`wind_channel` - Float - default **0** - 0 to 8

Speed the wind up where the terrain SQUEEZES it -- the Venturi effect that scours sand out of a canyon narrows and drops it where the walls open again. NOT a reflection and not a flow solver: the wind model here is an analytical shear with a lee shadow, and a true bounce needs a 2D flow solver. Confinement is measured ACROSS the wind and needs walls on BOTH sides, so a single cliff face does not speed anything up. 0 = off, shear untouched

Channel Width

`wind_channel_reach` - Float - default **8** - 1 to 512

How far to each side, in GRID CELLS, the terrain is checked for a wall. Set it to about half the width of the gaps you want to read as canyons: too small and only slot canyons register, too large and an open basin counts as confined

Tides

`tide_enabled` - Bool - default **off

Move the water level up and down on a cycle. The damp band migrates with it, so the beach is alternately locked and released and the dunes behind it advance in steps -- the mechanism that builds real coastal dune ridges. Needs Water enabled

Tide Range

`tide_amplitude` - Float - default **0.5** - 0 to 100

How far the level swings each way from Water Level, in solver units. The wet band sweeps this whole distance, so it also sets how wide the damp foreshore is

Tide Period

`tide_period` - Float - default **240** - 2 to 100000

Length of one full tidal cycle in simulation STEPS. Keep it long compared with Steps/Frame or the tide will run faster than the sand can respond to it

---

Demo scenes

66 complete scenes, built by **Presets > Demo Scenes > Create Demo Scene**. Each one builds terrain, lights, camera and materials, and turns on the settings that make its subject visible. They are the fastest way to see a feature working, and the settings are yours to copy.

Use **Remove Demo Scene** to take one back out; it leaves your own objects alone.

Scene	What it demonstrates
Grand Scene: Desert Expedition	A 400 m landscape with dunes, grains, rock shelters, sources, tracks and airborne dust
Grand Scene: Living Oasis	A 400 m rain-fed basin combining wet shores, plant species, growth and surface weathering
Grand Scene: Winter Storm	A 400 m snowfield with layered drifts, density, slabs, shelter and snow-fed vortices
Grand Scene: Dynamic Coast	A 400 m coast with Flow 2D wind, shallow water, waves, sediment, rain and vegetation
Desert: Dunes + Obstacles	Classic sand dune field with rock obstacles casting wind shadows, pebble scatter, camera, sun and a sand material. Pre-runs the simulation so dunes are already formed
Snow: Drifts + Fence	Wind-packed snow drifting behind a slatted fence, with a snow material and cool daylight. Shows the snow solver and obstacle wind shadows building lee tails
Oasis: Water + Valley Masks	Sand placed in valleys around a shallow water table with a translucent water plane: wet shores lock in place while dry sand keeps migrating around them
Water: Pools That Flow	The SAME oasis as above, with the water given a depth field instead of one global level -- so every hollow settles to its own height, water runs downhill between them and the shore is wherever the ground says. Built as a direct A/B against the Oasis demo: same terrain, same warmup, one switch different
Water: Wind On The Lake	The flowing field with a 30 m/s wind gripping the surface. The water piles up on the DOWNWIND shore at a slope of exactly tau/(rho g h), so the shoreline moves and the pools lean. MEASURED on this terrain at 30 m/s against no wind, after 3 s of water time: 7699 cells cross between wet and dry -- 4010 flooded and 3689 drained -- and the surface stands 13.7 mm apart at most. The wind here is the ANALYTIC one, so it is the same everywhere; switch Wind Model to Flow 2D and give the flow an Air Layer to have crests and lee pockets push the water differently

Scene	What it demonstrates
Snow: Slab Avalanche	A weak layer buried under a wind slab lets go on a 38 degree face and the debris RUNS OUT onto the flat below. This is the one scene where Slab Dynamics is on: without it the released mass is only ROUTED downhill, which moves it DOWN THE SLOPE rather than out across the flat. MEASURED on this scene, the two models differ by 10.7 % of the released mass: turning Dynamics on shifts 266 units off the upper face and lays them on the runout, concentrated around 75-87 % of the way across. Both models DO reach the valley floor and both stop at the same 95th-percentile distance -- the difference is WHERE the debris piles up, not how far it gets, and in a wide shot it is only 0.12 % of pixels. Look at the runout zone, not at the whole slope. MEASURED, and it is why the scene sculpts its own mountainside rather than using noise: on the stock demo terrain the 95th-percentile slope is 0.103 against a release threshold of tan(34) = 0.675 -- six times too flat, and 220 steps released nothing at all. Try Slab Dynamics 1 -> 0 and watch the pile move UP the runout toward the foot of the face
Snow: Avalanche That Scours Its Path	The same 38 degree face and the same storm as Snow: Slab Avalanche, with Entrainment ON (0.007 m per m of path) and Turbulent Friction set to 400. A real avalanche does not carry only the slab that let go: it scrapes the snow it runs over and grows. MEASURED on this scene, over the same storm: the flow picked up 35 033 units of snow off its path against 9 137 released by the slabs themselves -- 3.8 times the slab -- and 54 342 with the turbulent term on top, because a slower flow lives longer inside its step budget and scrapes further. What you see is the SCOURED TRACKS down the face and a fuller pile at the foot: 5.0 % of the pixels change against Slab Avalanche at the 1 % level, where the two release models of that scene differ by 1.7 %. The turbulent term is here for the SPEED the entrainment law reads, not for the runout: on this 90 m flat it does NOT move the deposit -- 0.432 of the debris ends on the flat at xi 400, 0.432 at 0 and 0.437 at 10, MEASURED -- while the mass-weighted speed drops from 70 to 54 m/s. Try Entrainment 0.007 -> 0 and watch the tracks on the face fill back in
Snow: Avalanche Into A Valley	The same 38 degree face and the same storm as Snow: Slab Avalanche, but the flat is SHORT (30 m) and a 22 degree counter-slope rises behind it -- and all four momentum controls (Carry Momentum, Momentum Rides Along, Sharper Transport, Momentum Crosses Lanes) are ON. Momentum is what lets debris CLIMB the far side; with the four at 0 the flow is hydrostatic and stops at the foot. MEASURED on this valley (512 cells, a 0.5 m slab off the face, friction 0.12): with the four on the front reaches the counter-slope after about 7 s, climbs to 174 m along it (22 m up) by about 11 s, then part slides back and 11 % of the debris settles on the counter-slope; with the four off the flow stops on the flat after about 10 s and never touches it. TIME IS THE TRICK: at 1500 substeps the run is cut after about 3 s and both variants deposit mid-flight on the flat with no run-up at all, so this scene runs 12000 substeps with Slab Flow Rest Speed 0.05 m/s. Try Carry Momentum 1 -> 0: the other three go dead with it and the debris stays on the flat
Sand: Slide Under Gravity	No wind. A rock face sculpted to 45 degrees, steeper than the Repose Angle (34), with a 0.6 m strip of sand and two thicker patches painted on it. The repose relaxation runs every step whether or not the wind blows, so the sand slides on its own and stops where its surface reaches 34 degrees. MEASURED on this scene (512 cells): 91 % on the face when the demo opens, 73 % after 20 steps and at rest from 40 steps on at 69.5 %, with 30.5 % at the foot as talus, mass kept to the last digit; with Avalanche Passes 0 nothing moves at all. Try Repose Angle 34 -> 25 (rest at 49 %) or 40 (85 % and still creeping)
Snow: Sluffs Off A Rock Face	The same rock face at 50 degrees, no wind, and a storm falling straight down. Snow landing above the repose limit sluffs to the foot as it falls; with Avalanche Trigger 6 it holds a few degrees past repose and lets go in events. MEASURED: 30-40 % of what fell on the face is still there when the storm ends, the drops come in steps of up to 0.18 points per step against 0.04 with continuous relaxation. Snow Repose Offset does NOT change this -- it belongs to the slab model (measured 0 / +12 / +20 identical). Try Avalanche Trigger 6 -> 0

Scene	What it demonstrates
Snow: Slab Avalanches Held Back By Gates	The Slab Avalanche face with three plates standing across it at 30, 40 and 50 % of the way down; the storm keeps falling while you Play, slabs keep letting go and the debris is ROUTED downhill (Slab Runs Out 0 on purpose: only the routing sees obstacles). The plates are KEYFRAMED to lift at frames 30, 70 and 110; each lift hands what it held to the next gate. MEASURED on a fixture: without a plate 89 % of the debris ends below its line, with it 45.5 % and the upper half stays put; with Slab Runs Out 1 the plate is ignored (85 % passes). Loose snow would not do: a uniform layer does not slide in the local repose rule, it only retreats from the top edge
Water: Reservoir Behind A Sluice Gate	A rock plateau with a 6 m basin, a dam with a 16 m gap and a valley below. The basin holds 1.5 m of water above the sill; a red gate in the gap is an ordinary obstacle that, with Water Sees the Rocks on, is part of the water bed and holds the water back. It is KEYFRAMED to lift 24 m from frame 20: press Play and the reservoir pours down the valley as a shallow-water flood. MEASURED on a fixture of this geometry: without the gate 7.8 % of the water is below the dam after 20 s and 34.8 % after 120 s, with the gate 0.0 % for 120 s, gate removed at 40 s -> the same curve shifted by 40 s; mass 1.0000. Try Water Sees the Rocks off: the flood starts at once
Water: Ripples Carried By The Current	The Pools That Flow basin with a DEAD SWELL pumping it, so the current never settles -- and that is the whole condition of this scene. Flow Ripples makes the fine chop read the FLOW FIELD instead of an independent noise: the pattern stretches along the current and twists where it turns. MEASURED on this scene: straight after the warm-up the basin is in EQUILIBRIUM -- peak speed 0.0 and 0.0 % of cells moving -- so the knob changes 0.000 % of the pixels, because there is no flow to read. With the swell on: peak 36.15, mean 0.218, 70.4 % of cells alive, and the same knob changes 4.69 % of the pixels on the built scene (4.84 % on the gate fixture) against 0.000 % render noise. Try Flow Ripples 2.5 -> 0: the chop goes back to an independent noise that does not know where the water is going. Try Swell Height 0: the basin settles, the current dies, and the knob stops mattering -- which is the same trap the Ice: Pack scene documents
Ice: Pack, Ridges and Leads	The ice mechanics of 1.3.134-1.3.138 in one scene: a frozen basin kept ALIVE by a steady inflow, so the pack never settles. The current carries the sheet, piles it against the shore into RIDGES where it converges and pulls it open into LEADS where it diverges -- all three out of one conservative flux, with no extra rule. MEASURED, and this is why the demo exists: on the Pools That Flow terrain the water reaches equilibrium and the ice sampled a LIVE current in only 2 of 20 steps, so none of it could be seen. With the inflow on it is 20 of 20. Try Ice Yield Strength 20 -> 0 and watch the ridge turn into an even thickening
Plants: Two Species Side By Side	A succulent and a grass on the same dune, which is the commonest arrangement on a real one. Drought Tolerance used to be ONE number for the whole scene, so the two could not stand together; above 0 Second Species turns it into a FIELD of patches, deterministic in Seed. MEASURED on the gate fixture, cover of the tolerant plant against the sensitive one: 0.4973 / 0.7078 with one species (a +0.2105 gap that is just the two thresholds) against 0.5087 / 0.7811 with the species map on -- the gap widens to +0.2725 because each plant now sits where it can. The two colours are what makes it readable: Vegetation Colour is the grass, Vegetation Colour B the succulent. Try setting Tolerance (Species B) equal to Drought Tolerance: the map stops mattering and the result goes back to being identical to the bit
Snow: A Thaw That Leaves One Patch	The drift field and the same fence, run through a THAW. Snow lies at Initialize (Initial Snow Depth 0.5), the winter builds the drifts, and then the air goes to +8 C with Melt Rate on -- and the field loses ALL of it. MEASURED over 30 steps: mass of the fresh layer 20 584 with no thaw against 0 with it, cover 98.2 % against 0.0 %. One patch survives, because a snow EMITTER keeps feeding it: Snow Source Rate 0.05 over a box, feathered by Snow Source Smoothing, holds 2 842 of mass and 4.9 % cover when everything around it is bare. MEASURED and the reason this scene keeps the DunesForge surface shader instead of the flat snow material the other snow demos wear: on flat white paint the same thaw changed frame brightness from 0.7232 to 0.7200 and the renders looked identically white -- only the drift relief moved. With the snow mask driving the surface, the thaw repaints 57.1 % of the pixels and the emitter patch another 4.4 %. Try Melt Rate 0.01 -> 0 with the air still warm: melting is Melt Rate PLUS the temperature term, so the warm air alone still takes it, just more slowly. Try Snow Source Rate 0.05 -> 0: the last white patch goes with it

Scene	What it demonstrates
Another Planet: Mars	The same ORGANIC dune field as the other scenes with ONE switch different: Other Planet, set to Mars. Martian air is sixty times thinner than ours (0.020 against 1.20 kg/m3) and the saturation length divides by air density, so the same physics builds dunes far larger on the same ground. Turn Other Planet off and the scene rebuilds itself at Earth scale, which is the whole point of the comparison; scale the terrain object up if the Martian dunes outgrow the default 200 m domain
Snow: Fresh, Packed and Drifted	The same fence and the same wind as Snow: Drifts + Fence, with Snow Density switched ON -- so the pack stops being a depth and starts being a MASS. A depth-only solver cannot tell 10 cm of powder from 10 cm of settled snow; MEASURED on the gate fixture those are 10.00 and 50.00 kg/m2, five times apart at the same depth, and across this field the mass runs 0.37 to 17.42 kg/m2. Drifted snow is denser than the snow that fell, because the grains shatter in saltation and pack tighter, so the drift behind the fence settles SHALLOWER and HARDER than the fresh fall around it -- measured mean hardness 0.1204 at Drifted Density 350 against 0.8125 at 100. Try Drifted Snow Density 350 -> 100: the mechanism switches itself off bit for bit and the pack deepens again
Rock + Sand Source	Start Terrain = Rock workflow: a rocky FBM terrain frozen as bedrock, with sand fed only from an upwind source zone so drifts grow from nothing over bare rock
Rain: Wet Sand + Plants	Storm cells sweep a formed dune field: the dunes are built DRY first, then the rain arrives, so you can see what it changes. Damp patches go dark and matte and stop moving, the lee of the rocks stays dry in the rain shadow, and vegetation greens up where the root zone holds water and then pins the sand down
Rain: Monsoon Shutdown	The heavy end of the rain model: near-total coverage, slow cells and strong infiltration, so almost the whole field goes wet and almost all of it stops moving. Shows what OUTLIVES the rain -- a drying crust plus the vegetation it fed, which keep holding the sand after the weather has gone
Rain: Wet Oasis	Water table AND rain at the same time: two independent sources feeding one wetness mask, each with its own hold strength. Plants ring the standing water where the root zone never dries out. Good for checking that the two systems combine rather than fight
Rain: After The Storm	The field is soaked and then the rain STOPS. Wind Drying makes exposed crests dry first while sheltered lee slopes stay damp, so the dunes wake up unevenly instead of all at once -- the clearest demonstration of the wind/moisture coupling
Rain: Drops On Water	A close-up of the rain IMPACT layer, framed on a shoreline so you can see both behaviours at once: rings spreading where a drop lands in standing water, and splash craters with dark wet spots where it lands on the shore. The camera sits low and near on purpose -- an impact is about 5 cm across, so from the usual demo distance every one of them would be a single pixel
Terrain: Mountain Range	Ridged fractal peaks with sand collecting in the valleys -- the classic case where the rock is the scenery and the dunes are what moves through it
Terrain: Canyon	A terraced, deeply-cut canyon frozen as bedrock with sand fed along the floor, so drifts snake between the walls and pile against them
Terrain: Cellular Mesas	Turbulence terracing gives flat-topped mesas with steep sides; sand streams around them and builds lee tails behind each
Terrain: Ready Dune Field	Wind-aligned ridges as the starting field, so a mature dune sea exists from frame 0 instead of forming over hundreds of steps
Terrain: Beach + Buildings	A shallow shore with a water table, a row of buildings behind the dune line, and sand drifting up against them
Terrain: Buried Town	A block of buildings on an open plain being swallowed by sand: each one casts a wind shadow and grows its own lee drift
Terrain: Crater Basin	A steep-walled basin that traps everything blown into it -- a sand trap, and a good stress test for avalanching on rock
Terrain: Rolling Hills	Soft FBM hills with a full sand cover: gentle relief where the dunes and the ground shape each other
Terrain: Stepped Plateau	Hard terraces frozen as bedrock, with sand only on the steps -- shows the non-erodible rock path and how sand cascades between levels

Scene	What it demonstrates
Terrain: Islands + Water	Scattered high ground in a shallow sea: wet locked shores, dry migrating crests, and rain to tie the two together
Showcase: Object Masks	Three object slots doing three different jobs in one scene: a source box feeding fresh sand, a wind-gain box acting as a gust lane, and rock obstacles casting wind shadows. Every one of these masks can also be painted with the Interactive Brush
Showcase: What Objects Tell The Sand	Three slabs on one moving sand sheet, each holding the ground to a different rule, with every number below measured on the scene itself. BEDROCK is a floor the sand cannot sink below; this one sits above the sheet, so it stands as a rock sill the sand flows around, 1.43 m proud of the field. ERODE is the quiet one: the ground there stops moving altogether, roughness 0.0025 against 0.30 for the sheet beside it, so dunes travel past a patch that never changes. NO-DEPOSIT is the one people mix up with it: nothing new lands, while whatever sand is there is carried off as usual, so the footprint SINKS -- 0.12 m after the warm-up and 0.38 m after 300 steps more. Erode holding still while No-Deposit digs itself out is the whole point
Showcase: Grain Systems	The grain-scale models together: Multi-Fraction winnowing fines out into an armouring coarse-lag pavement, Strata painting cross-bedding on cut faces, Sand Age reading fresh on scoured slip faces, and the physical ripple solver at megaripple scale. Ships with the Surface Shader so all of it is visible
Phenomena: Playa Crust	A dried lake bed: the crust sets and cracks into plates, and the wind deflates along the cracks while the plates stand
Phenomena: Wind Erases Tracks	A rut imprinted across the field, fading faster where the wind is strongest and surviving in the lee
Vegetation: Nabkha Hummocks	Scattered plant clumps on a sand-fed plain, each one trapping a mound with a downwind tail -- the nabkha (coppice dune) landform. Built with the patch-shape controls open, so the clumps are ragged, come in a range of sizes and grow in stands instead of being identical circles on a uniform scatter, and with the clumps taking their own darker colour so a hummock reads as a bush and not as a green stain
Vegetation: Rain-Fed (nothing grows on its own)	The only scene where NOTHING grows by itself: Veg Growth is set to zero, so every plant you see was put there by rain. Growth From Rain is a multiplier and cannot help at zero growth, so Rain Seeds Growth -- which adds its own rate from the root zone -- is the only term that can raise a single stem, which makes the scene its own control. Spread sits at 1.0 so growth happens only where plants already are: clumps thicken and creep at their edges while two thirds of the ground stays bare, and that bare sand is what lets you see the second half of the loop, Plants Shade The Ground, holding moisture under the cover that the rain grew
Plants: Valleys Only (Slope and Height Windows)	The blowout field again, with the vegetation kept OUT of the top 40 % of the relief by Height Max 0.6, Limits Apply to Seeding ON and Edge Noise 0.5. The window is what keeps a real field from greening over: plants hold the hollows and the crests stay bare sand. MEASURED on this scene after the 150 step warm-up, cover on the top third of the relief: 0.992 with no window against 0.040 with it, and 0.000 at frame 0 because Limits Apply to Seeding never lays it there; without that switch seeding covers the crests first and burial has to clear them. The relief is 8.0 here, ten times the Blowouts scene, MEASURED: at 0.8 the dunes rebuild the height field during the warm-up, the crests at the end are not the crests the seeding obeyed, and the window keeps only 0.676 of them bare (2.0 -> 0.292, 4.0 -> 0.098, 8.0 -> 0.040). Burial is 0.5, not 2.0, because at 2.0 the crests are bare with or without a window and there would be nothing to show. MEASURED and fixed in this version: Edge Noise used to jitter the OPEN limits too, so a height-only window refused 45.3 % of a perfectly flat floor; it now frays only the limit you set. Try Invert Limits: the crests green and the hollows go bare. Try Edge Noise 0.5 -> 0 and the tree-line turns into a contour
Vegetation: Blowout Hollows	The opposite starting point: a fully vegetated coastal grassland with bare gaps punched in it. The wind scours the unprotected gaps into deflation hollows while the anchored surroundings hold, which is how a blowout -- and eventually a parabolic dune -- gets going

Scene	What it demonstrates
Vegetation: Green Retreats	A field that starts GREEN EVERYWHERE and loses it: drought dieback takes the cover off the dry ridges while the plants ringing the water hold, so the green pulls back to the shoreline instead of simply fading. Built the way round it is for a measured reason -- with the usual near-bare start the same physics moves 41% of the dry-zone cover and changes 1% of the rendered pixels, because 0.068 and 0.040 both read as bare sand. Starting green makes the loss an AREA that shrinks. MEASURED on this scene after 90 steps against the same scene with Drought Dieback at 0, counting only VISIBLE LAND (ground above the waterline -- cover under an opaque water surface is not something you can see): the strongly green share of the land falls from 78.4% to 11.0%, and mean cover on land from 0.549 to 0.165. The pool itself is half a percent of the frame while the wetness reaches 77% of it, so the green ring sits on LAND rather than under water. Turn Drought Dieback to 0 and the green stays put
Temperature: Cold Front	One cloud over a warming scene: the same storm lays snow while the air is below zero and wets the ground once it is above
Temperature: Spring Thaw	A snowfield under a rising sun -- melt on degree-days, so it stays put until the air crosses zero and then goes quickly
Rain on Snow: Crust	Warm rain falling on fresh snow: it takes depth off the pack and leaves what is left bonded into a crust
Rain on Snow: Loaded Slab	The same crust with fresh snow dumped on top of it -- the layered setup a slab releases over
Freeze: Locked Shoreline	Wet sand along a water table, frozen solid: the damp band stops eroding entirely while the dry dunes behind it keep migrating
Freeze: Thawed Shoreline	The same shoreline above freezing -- the control that shows the lock is the ice and not the water
Frost: Cold Dawn	Surface frost whitening the damp hollows while the dry crests stay bare sand
Frost: Hoar on Snow	A cold, sheltered snowfield growing surface hoar -- the weak layer a later slab will release over
Flow 2D: Canyon Jet	Two rock walls with a gap. The solved wind speeds up through the gap because the same air has to pass a narrower opening -- a result of mass conservation, not of a correction term
Flow 2D: Recirculating Wake	A single bluff rock. Behind it the flow REVERSES and turns back on itself, which the analytic wind cannot do at all -- it has a lee shadow but no momentum
Flow 2D: Wind Off a Wall	An angled barrier that turns the wind sideways. Tracers and the direction overlay are on, so the bounce is visible as motion and as colour
Phenomena: Snow Fence	A POROUS blocker: the long downwind drift a real snow fence builds, next to the short steep pile a solid wall makes
Phenomena: Snow Layer Stack	A season of storms as dated layers, with a buried weak layer that decides where a slab releases
Phenomena: Sublimation + Hoar	Snow leaving as vapour on the exposed crests while surface hoar grows in the sheltered hollows
Phenomena: Tidal Coast	A moving shoreline: the damp band sweeps up and down the beach and the dunes behind it advance in steps
Showcase: Surface Detail	The layer that lives ABOVE the dune, shot from close enough to see it -- from the usual demo distance it is smaller than a pixel. The Micro-Detail pass has been run once already, freezing the dunes as bedrock and running transport again at a finer scale over them, with Crispness, Crests and Slump Allowance set; those three do nothing until the pass is RUN, so press the button again after changing them. De-facet Crests takes out the straight grid zigzag that a one-direction wind leaves along the crest lines, Edge Rounding takes the corner off the brink but not off the flats, and Detail Feeds Sim hands a little of the detail back to the solver so the ripples are sand rather than a decal the wind ignores

Scene	What it demonstrates
Showcase: Surface Shader	How the surface texture is PROJECTED, rather than what it is. A flat top-down projection stretches the grain down every slope and a dune is mostly slope, so this scene switches the scanned sand and the bedrock onto a triplanar projection and turns up the four map-side controls: the scan follows the wind heading and the local downslope, the procedural/scan balance becomes a map instead of one number, and the terrain sits in its own ambient shade. Note the trade it has to make: triplanar and Hex-Tiling cannot both run on the sand, so the scene uses Texture Bombing -- put Hex back and the triplanar tick silently stops doing anything
Showcase: Airborne + Dressing	Everything that is not the heightfield: swept sand-haze particles emitted off exposed crests and settling in the lee, wandering dust devils, a curve imprinted as a rut with berms, and pebble scatter driven by the simulation masks